

Helios: Historical Development Catalogue

Checked components and deferred execution machinery

Pramaana research development

15 September 2026

Abstract

This catalogue preserves the mathematical interfaces, counterexamples and component validation records from the development of the concrete Helios reduction and its deferred machine-execution implementation. The accompanying [formal reference](#) states the current symbolic and computational secrecy results and their assumptions. In particular, polynomial-time implementation of the complete computational reductions is an explicit external assumption; the execution fragments collected here do not discharge it.

Historical provenance. The material is extracted from the development recorded at source revision 9273cdc. The component results remain checked at their original source paths. Local validation counts describe their recorded test campaigns. Section order follows the construction of those components; it does not prescribe further work or qualify the current secrecy theorem. Each section’s interface and exclusions are local to that component. The native-machine construction remains supporting evidence, separate from the external efficiency argument in the formal reference.

Contents

1	Historical concrete proof and execution components	4
1.1	Historical component: Random-oracle programming prerequisites	5
1.2	Historical component: The explicit ballot hash oracle	6
1.3	Historical component: One-proof programming distance	7
1.4	Historical component: Adaptive honest-proof simulation	8
1.5	Historical component: Replay extraction for selected live targets	9
1.6	Historical component: From the selector to actual verification	10
1.7	Historical component: Proof targets, weeding and honest rejection	11
1.8	Historical component: Replayable programmed-cache interpretation	12
1.9	Historical component: Sampled honest ballots and actual requests	13
1.10	Historical component: Actual oracle submissions and the honest prefix	14
1.11	Historical component: Actual prefix rejection and simulation loss	15
1.12	Historical component: Selected-proof extraction after actual submission	16
1.13	Historical component: Retaining the first complete submission	17
1.14	Historical component: Joint-witness contract and the scalar modulus	18
1.15	Historical component: Joint replay from one original submission	19
1.16	Historical component: Quantitative joint replay	20
1.17	Historical component: Finite live-cache implementation	21
1.18	Historical component: Finite programmed shadow cache	22
1.19	Historical component: Finite saved replay tapes	23
1.20	Historical component: Finite logging in every replay run	24
1.21	Historical component: Replay storage and complete key records	25
1.22	Historical component: Retained shadow cache and request history	26
1.23	Historical component: Submission output and tape operands	27
1.24	Historical component: Canonical binary uniform operands	28

1.25	Historical component: Historical scalars and complete saved bit tapes	29
1.26	Historical component: Explicit prime scalar and nonce sampling	30
1.27	Historical component: Explicit nonces in the complete submission source	31
1.28	Historical component: Joint replay for the explicit source	32
1.29	Historical component: Finite fair-bit sampling boundary	33
1.30	Historical component: Actual extra-bit modulo sampler	34
1.31	Historical component: Adaptive transfer to the joint extractor	35
1.32	Historical component: Public group coordinates and canonical records	36
1.33	Historical component: Complete key and finite-cache bit records	37
1.34	Historical component: Complete finite interpreter data	38
1.35	Historical component: Complete outputs and public observations	39
1.36	Historical component: Concrete cache execution primitives	40
1.37	Historical component: Concrete parsing of binary-natural prefixes	41
1.38	Historical component: Canonical binary decrement for field reading	42
1.39	Historical component: Linked execution of length-prefixed fields	43
1.40	Historical component: Outer-state preservation and one record iteration	44
1.41	Historical component: Original-frame accumulation and workspace reuse	45
1.42	Historical component: Complete counted-record execution	46
1.43	Historical component: Restoring the query between comparisons	47
1.44	Historical component: Executing the original entry header and key	48
1.45	Historical component: Complete entry framing and retained comparison	49
1.46	Historical component: Complete lookup in source-generated caches	50
1.47	Historical component: Writing original field and count prefixes	51
1.48	Historical component: Guarded insertion in source-generated caches	52
1.49	Historical component: Source request scheduling through executed cache operations	53
1.50	Historical component: Chronological log append and scalar-prefix execution	54
1.51	Historical component: Parsing the actual cached answer	55
1.52	Historical component: Guarded subtraction for scalar range reduction	56
1.53	Historical component: One proof oracle for the sampled election	57
1.54	Historical component: The actual prefix after earlier oracle queries	59
1.55	Historical component: Full-source replay and the actual submitted ballot	60
1.56	Historical component: Actual acceptance and joint replay probability	62
1.57	Historical component: Trustee simulation after reachable prior queries	63
1.58	Historical component: The original private nonce schedule	64
1.59	Historical component: Composing the actual trustee finish	65
1.60	Historical component: Key and honest-ballot simulation in the actual prefix	66
1.61	Historical component: Complete prepared-election proof simulation	67
1.62	Historical component: Replayable full-source correspondence	68
1.63	Historical component: Joint extraction from complete programmed elections	70
1.64	Historical component: DDH ciphertext pairing and the statement interface	72
1.65	Historical component: Actual honest submissions from challenge ciphertexts	73
1.66	Historical component: Extraction before tally publication	74
1.67	Historical component: Repeated extraction retaining the original execution	76
1.68	Historical component: Complete real-challenge comparison	77
1.69	Historical component: Finishing from extracted ballot nonces	79
1.70	Historical component: Complete source interaction costs	80
1.71	Historical component: Public rejection in the DDH games	81
1.72	Historical component: Effective integer extraction parameters	83
1.73	Historical component: Random-mask secrecy of the complete comparison	84
1.74	Historical component: Finite reduction of the original prepared election	85
1.75	Historical component: Known repair control under the shared oracle	87
1.76	Historical component: Fixed finite-control oracle interface	88
1.77	Historical component: Actual oracle-port word movement	88
1.78	Historical component: Finite source execution in the assembled loop	90
1.79	Historical component: Common-clock execution and handler transport	91
1.80	Historical component: Quantitative reuse of the stack-to-tape translation	92
1.81	Historical component: Finite tape runs and canonical presentation	93
1.82	Historical component: Physical outgoing query-tape transfer	94
1.83	Historical component: Physical answer loading and response execution	95

1.84	Historical component: Native oracle dispatch and complete call execution	96
1.85	Historical component: Finite source execution in the tape loop	97
1.86	Historical component: Source-derived polynomial tape clocks	99
1.87	Historical component: Primitive costs through the existing compiler	100
1.88	Historical component: Finite execution in the primitive oracle loop	101
1.89	Historical component: Finite target control and primitive source clocks	102
1.90	Historical component: Physical initial raw-input loading	102
1.91	Historical component: Local standard-tape computation	103
1.92	Historical component: Physical input through native local execution	104
1.93	Historical component: Native oracle reference and private frames	105
1.94	Historical component: Conditional asymptotic completion prototype	106
1.95	Historical component: Fair-bit transport of the exact DDH reductions	107
1.96	Historical component: Complete loaded-word binary remainder	108
1.97	Historical component: Executed chronological coin loading	110
1.98	Historical component: Executed coin-to-remainder sampler	111
1.99	Historical component: Serialization of the executed scalar	111
1.100	Historical component: Uninterrupted coin-to-prefix execution	112
1.101	Historical component: Cache requests with executed sampling	113
1.102	Historical component: Complete adaptive callers with executed sampling	115
1.103	Historical component: Serialized operands and the complete physical sampler	115
1.104	Historical component: Sampler execution from retained caller storage	118
1.105	Historical component: Complete cache requests in finite control	119
1.106	Historical component: Serialized input to a complete cache request	121
1.107	Historical component: Bounds at original request boundaries	123
1.108	Historical component: Resident caller entry and guarded return	125
1.109	Historical component: Executed nonzero-nonce response	127
1.110	Historical component: Complete historical nonzero-nonce sampling	128
1.111	Historical component: Executing the historical nonce-pair caller	130
1.112	Historical component: Loaded public-coordinate multiplication	132
1.113	Historical component: Loaded exponentiation for encryption	134
1.114	Historical component: Executing both ciphertext coordinates	136
1.115	Historical component: From raw constructor input to the first ciphertext	137
1.116	Historical component: Four programmed-transcript scalar draws	139
1.117	Historical component: First simulated commitment from raw input	141
1.118	Historical component: Zero-branch commitment pair from raw input	143
1.119	Historical component: Executed canonical challenge difference	144
1.120	Historical component: Executed adjusted ciphertext operand	145
1.121	Historical component: All simulator commitments from raw input	146
1.122	Historical component: Executed simulator hash-key serialization	147
1.123	Historical component: Recovering the original programmed state	149
1.124	Historical component: Executing the first programmed-state update	151
1.125	Historical component: Encoding the returned proof and saved state	153
1.126	Historical component: Continuing with the retained nonce pair	155
1.127	Historical component: The second proof's first commitment	157
1.128	Historical component: Completing the second proof's commitment computation	159
1.129	Historical component: The second proof's original hash key	160
1.130	Historical component: One complete programmed proof request	162
1.131	Historical component: Historical reentry and the architecture assessment	163
1.132	Historical component: A bounded native adaptive compilation test	165
1.133	Historical component: General native query export	166
1.134	Historical component: One general native oracle round trip	167

1 Historical concrete proof and execution components

The interactive disjunctive proof is installed in VCVio’s Σ -protocol interface. A statement is (g, h, C) ; a witness is a bit b and nonce r with $C = \text{Enc}_{g,h}(r, b)$. The commitment contains the four group elements from the two branches. The response is (c_0, z_0, z_1) ; verification reconstructs $c_1 = c - c_0$ from the total challenge c . Honest commitment and response construction agrees exactly with the repaired strong-FS ballot constructor.

Lean: `ballotTranscript_strongProveVote`.

Theorem 1.1 (Completeness and special soundness). *Let F be a field and G an additive commutative group with an F -module structure and decidable equality. Honest transcripts verify for every valid statement and witness. Two accepting transcripts for the same statement and commitment, with distinct total challenges, yield a bit and nonce satisfying that statement’s encryption relation.*

The extractor selects a branch i with $c_i \neq c'_i$ and returns $(i, (z_i - z'_i)/(c_i - c'_i))$. No discrete-log hardness assumption or generator injectivity is required for this algebraic result. The finite-field protocol keeps three independent nonzero prover coins and a uniform full-field verifier challenge.

Lean: `ballotSigma_complete`.

Lean: `ballotSigma_speciallySound`.

Theorem 1.2 (Honest-verifier simulation, compressed interface). *Suppose additionally that F is finite, with $q = |F|$. For every valid statement and witness, the statistical distance between the actual interactive transcript and `ballotFullSimTranscript` is at most $3/q$.*

The witness-free simulator samples c, c_0, z_0, z_1 uniformly from F and reconstructs both branch commitments from the verification equations. Full-field prover coins give exact transcript equality. For the historical sampler, replacing a zero uniform draw with an independent nonzero draw has exactly the required nonzero marginal; three replacements give the bound.

Lean: `ballotSigma_full_hvzk`.

Lean: `nonzeroRefill_eq`.

Lean: `ballotSigma_hvzk`.

Retained counterexamples and boundary. Equal total challenges or changing commitments need not extract a witness. Exact equality with this full-field simulator is false: for a zero vote, its fake-branch challenge can be zero, whereas the actual transcript excludes zero. Independent multiplicative fixtures check both extraction branches and exact small-field transcript distributions.

Lean: `BallotSigmaControls.same_challenge_does_not_extract`.

Lean: `nonzero_transcript_ne_full_simulator`.

These interactive properties are prerequisites.

1.1 Historical component: Random-oracle programming prerequisites

A strong ballot-proof hash binds the complete statement $x = (g, h, C)$ and four commitment elements P . Simulation must arrange that the oracle answer at (x, P) equals the sampled total challenge. A prior query at that point can prevent this programming step. The following results bound this collision and identify the required challenge distribution; Section 1.4 derives cache budgets and composes these steps across adaptive requests.

Theorem 1.3 (Simulator commitment and prior-query bounds). *Let F be a finite field, $q = |F|$, and G an F -module with decidable equality. Fix a statement $x = (g, h, C)$ with injective $r \mapsto r \bullet g$. For every commitment P and finite set Q of statement-commitment pairs,*

$$\begin{aligned} \Pr[T.\text{commit} = P] &\leq 1/q, \\ \Pr[(x, T.\text{commit}) \in Q] &\leq |Q|/q, \quad T \leftarrow \text{ballotFullSimTranscript}(x). \end{aligned}$$

A single branch commitment coordinate is an injective translate of a uniform response scalar. Equality of the whole commitment implies equality of that coordinate. Summing over the prior query domain gives the second bound; queries containing other statements remain in that domain. No valid-witness premise is needed for these two bounds.

Lean: `ballotSim_commit_probability_le`.

Lean: `ballotSim_query_collision_le`.

Theorem 1.4 (Conditional challenge uniformity, compressed interface). *For every statement admitting a bit/nonce witness, every P and $c \in F$,*

$$\Pr[T.\text{commit} = P \wedge T.\text{challenge} = c] = \Pr[T.\text{commit} = P] / q.$$

Here T is the same full-field simulator transcript. Generator injectivity is not required for this identity.

Exact full-field honest-verifier simulation reduces the identity to the real interactive protocol, whose challenge is sampled independently after its commitment. This is VCVio's `simChalUniformGivenCommit` property for the actual ballot interface. The $3/q$ historical sampling loss remains separate.

Lean: `ballotSigma_simChalUniformGivenCommit`.

Why the premises are retained. With zero generator, key and ciphertext, every commitment coordinate is zero; the commitment probability is one. With $F = \mathbb{Z}_3$, $g = 1$, $h = 2$ and $C = (1, 1)$ in additive notation, no bit/nonce witness exists. Conditioning its simulator on four zero commitment elements forces challenge zero, refuting unconditional challenge uniformity. Both are checked counterexamples.

Lean: `BallotSigmaProgrammingControls.zero_generator_refutes_unconditional_bound`.

Lean: `BallotSigmaProgrammingControls.nonbit_refutes_unconditional_challenge_uniformity`.

These results concern each fixed prior query domain. Adaptive multiple-proof simulation also needs reachable cache budgets and composition. The next subsections supply these steps. Extraction is a separate result.

1.2 Historical component: The explicit ballot hash oracle

The ballot prover, verifier and any attacker computation can now query one shared lazy random oracle. Its input is the complete pair $((g, h, C), P)$. On a miss, it samples a uniform field challenge and caches it; on a hit, it returns the stored answer. Uniform prover randomness is forwarded separately, so reusing a hash answer does not reuse a prover coin.

Lean: `runBallotOracle_query`.

Agreement with the existing algorithms. `strongBallotProofOracle` draws the historical three nonzero proof coins, constructs the branch commitments, queries the oracle and computes the two responses. Interpreting every query through any fixed typed hash function recovers exactly the existing `strongProveVote` constructor with those same draws. Verification consumes the full original proof, including both branch challenges, and agrees with its existing validity predicate.

Lean: `strongBallotProofOracle_function`.

Lean: `strongBallotVerifyOracle_function`.

For every choice of prover coins, honest proofs verify in their resulting oracle state for every initial cache and valid bit/nonce witness; no fresh-hash-input premise is needed.

Lean: `runBallotOracle_honest_valid`.

A stateful witness-free simulator. `strongBallotSimOracle` draws the full-field simulated transcript from the previous subsection. If its input is fresh, it stores the simulated total challenge and returns the proof with a false internal collision flag. If the input is already cached, it preserves that answer and returns a true flag. The proof is still returned on a collision; no execution is conditioned away.

Theorem 1.5 (Simulator validity and answer preservation, compressed interface). *For every initial cache, every unflagged simulator output verifies against its returned cache. Every simulator output preserves all existing answers. Every reachable state of an arbitrary adaptive ballot-oracle computation also preserves all answers from its initial cache. Consequently a proof valid for a cached challenge remains accepted after such a computation.*

Lean: `strongBallotSimOracle_unflagged_valid`.

Lean: `strongBallotSimOracle_cache_le`.

Lean: `runBallotOracle_verification_preserved`.

For a finite set Q covering all initial cached inputs, generator injectivity gives collision-flag probability at most $|Q|/|F|$. This fixed-domain result assumes the cache cover; the adaptive bounds are established in later component results.

Lean: `strongBallotSimOracle_bad_probability_le`.

Controls and interface boundary. Independent modulo-23 commitments are $((3, 4), (18, 12))$. A fresh simulation verifies under challenge 5; a conflicting prior answer 6 is retained, flagged, and rejects that proof. A changed statement with the same commitments has a different hash input. The independent constructor/cache harness checks 22,000 cases and detects resetting the verifier's cache in each one.

Lean: `StrongBallotOracleControls.conflicting_proof_rejected`.

The next subsection compares real and simulated one-proof executions.

1.3 Historical component: One-proof programming distance

Let $S = (g, h, C)$ be a ballot statement, $w = (b, r)$ a bit/nonce witness and K the initial cache. Write $\text{Real}(S, w, K)$ for the actual nonzero-coin oracle prover, with a false internal collision flag appended to its output. Write $\text{Sim}(S, K)$ for the implemented full-field simulator. Each experiment returns the full proof, internal flag and final cache. Flags and cache are reduction state, not additional public election observations.

Theorem 1.6 (One-step state-inclusive distance, compressed interface). *Let F be a finite field, G an additive F -module and $q = |F|$. Assume $C = \text{Enc}_{g,h}(r, b)$, the map $x \mapsto xg$ is injective, and Q is a finite set of complete strong-hash inputs covering all populated entries of K . Then*

$$\text{TV}(\text{Real}(S, w, K), \text{Sim}(S, K)) \leq \frac{3}{q} + \frac{|Q|}{q}.$$

Lean: [strongBallot_programming_step_distance_le](#).

Proof dependencies. Replace the actual three nonzero prover draws by full-field draws in the same stateful runtime; the checked sampler bound costs at most $3/q$. Outside the event that the commitment key is already cached, this full-field execution agrees with the interactive transcript mapped through the simulator’s cache update. Exact full-field honest-verifier simulation identifies that mapped distribution with the implemented simulator. The checked query-collision bound charges at most $|Q|/q$. The triangle inequality combines the losses. Neither collisions nor the sampler discrepancy are conditioned away.

Independent controls. For the literal $q = 11$ statement, Lean checks an empty-cache distance bound of $3/11$. For a populated finite-domain cache, Lean checks that the simulator’s flag is always true and the actual prover’s flag is always false, so their state-inclusive distributions differ.

Lean: [StrongBallotProgrammingControls.empty_cache_distance_le](#).

Lean: [StrongBallotProgrammingControls.populated_distributions_differ](#).

An independent multiplicative enumeration compares exact distributions for $q = 3, 5, 11$, both bits and four initial cache regimes: 24 comparisons with zero failures or discarded cases. Empty caches expose the nonzero-sampling loss. Fully populated valid commitment domains give distance one; for $q > 3$ they refute omitting the collision term. These finite checks validate fixtures, not the universal bound.

Composition. The next subsection derives cache covers from query-bounded executions and composes the one-step result. This one-step statistical theorem does not establish computational ballot secrecy.

1.4 Historical component: Adaptive honest-proof simulation

The runtime now interleaves uniform draws, arbitrary strong-hash queries and internal honest-proof requests (b, r) . Under fixed generator g and key h , a request constructs $S = (g, h, \text{Enc}_{g,h}(r, b))$ and returns its proof. The program may choose its next request from previous proofs and hash answers. The real implementation runs the existing historical nonzero-coin constructor. The simulated implementation programs fresh points and retains old answers on collisions. Its internal collision flag stays true once set. Programs receive only proofs and hash answers; the flag and cache remain internal state.

Lean: `runBallotProofReal_request`.

Lean: `runBallotProofSim_flag_sticky`.

Derived cache budgets. A cache has budget k when some finite set of at most k complete keys covers its populated entries. Every supported output of an adaptive ballot-oracle computation with at most n hash queries has cache budget $k + n$. Uniform draws cost zero. A simulated proof adds at most one key, even on a collision. The induction constructs the covers; it does not assume them for successor states.

Lean: `runBallotOracle_cache_bound`.

Lean: `strongBallot_programming_after_queries_le`.

Theorem 1.7 (Adaptive simulation, compressed interface). *Let F be a finite field, G an additive F -module, $q = |F|$, and $x \mapsto xg$ injective. Fix a program in the request interface above with at most s proof requests and n hash-or-proof requests on every response branch. For every common initial state whose cache has budget k , the distance between its real and simulated executions, including final cache and flag, satisfies*

$$\text{TV}(\text{Run}_{\text{real}}, \text{Run}_{\text{sim}}) \leq \frac{s(3 + k + n)}{q}.$$

Lean: `strongBallot_programming_distance_le`.

Witness validity follows from constructing each requested statement by encryption. Each proof costs at most $3/q + (k + n)/q$ by the one-step theorem and reachable cache bound. Raw hash queries use the same handler in both runs. Induction over the program, conditioning on supported outputs, combines these losses through VCVio’s TV bind inequalities. Collision executions remain in both distributions, including after subsequent fresh requests.

Controls and boundary. Lean checks an adaptive two-proof example at $q = 11$ with bound $10/11$, exact equality for a hash-only program, and a fresh proof following a prior collision whose flag cannot be reset. Independent fresh commitments are $((18, 9), (8, 6))$ modulo 23. A cache with a known entry refutes budget zero.

Lean: `AdaptiveBallotControls.adaptive_two_proofs_distance_le`.

Lean: `AdaptiveBallotControls.fresh_after_collision_keeps_flag`.

The independent cache harness checks 132,860 adaptive traces with zero failures or discarded cases. It checks cache policy, not the statistical theorem or cryptographic hardness. Internal proof requests in this component model honest challenger calls. Extraction of accepted malicious ballots and the complete election reduction are separate results.

1.5 Historical component: Replay extraction for selected live targets

Let a ballot-oracle program output an arbitrary statement S and full proof π . The replay wrapper logs fresh hash queries with complete keys $(S, \text{pc}(\pi))$, including all four commitments. Its verification flag checks the original full proof against an existing live cache entry; missing entries produce false. Forgetting the miss log and Unit key tag gives exactly the original lazy ballot-oracle runtime, for every adaptive program and every initial cache.

Lean: [ballotForkRunTrace_eq](#).

Lean: [ballotFork_runtime_eq](#).

Derived replay correspondence. The selector returns the target’s first miss-log position when verification succeeds and the position is below $n + 1$. Every selected position corresponds to a real sampled challenge. Two successful contextual replays selecting it have the same complete statement and commitments. The challenges recovered from the physical replay logs are those that accepted the original proofs. These facts follow from VCVio’s shared-prefix replay machinery, not from an assumed equality of targets or numeric positions alone.

Lean: [ballotFork_target_eq](#).

Lean: [ballotFork_selected_challenge](#).

Theorem 1.8 (Selected-target extraction). *Let F be a finite field of size q and G an additive F -module. Every successful output $(S, (b, r))$ of the implemented replay extractor satisfies $S.\text{ciphertext} = \text{Enc}_{S.g, S.h}(r, b)$. If a is the probability that the trace selector returns a position, then*

$$\Pr[\text{extractor succeeds}] \geq a \left(\frac{a}{n+1} - \frac{1}{q} \right).$$

Subtraction is truncated at zero, as in the Lean ENNReal statement.

Lean: [ballotFork_extract_valid](#).

Lean: [ballotFork_extraction_probability_le](#).

The private Unit-relation adapter supplies the upstream trace interface only; its unused interactive methods and relation provide no security evidence. Extraction uses the actual ballot verification equations and special soundness. Full-proof validity justifies response normalization, including both challenges.

Controls and correspondence boundary. A real one-query honest prover at $q = 11$ has $a = 1$ and a strictly positive extraction bound. Literal challenges 5 and 6 extract nonce 3. Unrelated traces with index zero have different targets; changing the full proof’s second challenge rejects, although unchecked normalization erases that rejection.

Lean: [BallotForkControls.honest_extraction_probability_positive](#).

Lean: [BallotForkControls.unchecked_normalization_loses_rejection](#).

The independent multiplicative harness checks 2,420 extraction pairs and 242 normalization defects with no failures or discarded cases. This theorem’s a is selected live-target probability, not all ballot acceptance. The next subsection derives query coverage and includes unqueried acceptance.

1.6 Historical component: From the selector to actual verification

The live-only wrapper above assigns false to targets absent from its cache. An actual verifier queries that target and can accept a guessed challenge. To preserve this behavior, `Complete(A)` runs the arbitrary raw ballot-oracle program A , queries the returned proof's complete hash key, and returns the same statement and full proof. This is the verifier's existing query; the proof is not changed or normalized.

Exact runtime correspondence. Let `VerifyGame(A)` run A followed by the original full-proof verifier. Interpreting the completed trace's uniform queries gives exactly the same Boolean computation as running this game in the original lazy ballot oracle from the empty cache. Cache hits keep their answers; misses draw the same uniform challenge. Full-proof verification checks both original branch challenges.

Lean: [ballotFork_complete_runtime](#).

Derived query coverage. If A makes at most n hash queries on every response branch, completion makes at most $n + 1$. Uniform draws do not consume this budget. Every actual miss-log entry comes from one such query, and every populated cache key belongs to the log. The cache/log invariant is proved from the empty initial runtime. Consequently every completed verified target has a selector position below $n + 1$, including a fresh query made only by the verifier.

Lean: [ballotFork_log_bound](#).

Lean: [ballotFork_selector_of_bound](#).

Lean: [ballotFork_acceptance_probability_eq](#).

Theorem 1.9 (Extraction under actual verification acceptance). *For a finite field F of size q , an additive F -module G and any raw ballot-oracle program A with structural hash-query bound n , set $a = \Pr[\text{VerifyGame}(A) \text{ accepts}]$. Then*

$$\Pr[\text{Extract}(\text{Complete}(A), n) \text{ succeeds}] \geq a \left(\frac{a}{n+1} - \frac{1}{q} \right).$$

Subtraction is truncated at zero. Every successful output satisfies the actual bit/nonce encryption relation by the preceding extraction theorem.

Lean: [ballotFork_acceptance_extraction_le](#).

No premise says the attacker queried its final target. The $1/q$ term remains; no guessed-challenge run is discarded.

Controls and boundary. A fixed unqueried valid transcript at $q = 11$ accepts with probability $1/11$. Completion preserves this probability, while the old live-only selector gives zero. For an already queried honest prover, completion leaves the entire trace computation unchanged.

Lean: [BallotForkControls.completed_unqueried_probability_preserved](#).

Lean: [BallotForkControls.queried_completion_preserves_trace](#).

Independent multiplicative enumeration checks 5,324 verifier-completion cases and detects 242 omitted-query defects, with zero failures or discarded cases. The theorem covers arbitrary statements and full proofs in the raw ballot oracle. The board's weeding/provenance correspondence, established separately, connects prior programmed honest proof points to accepted complete ballots. Trustee publications, full election observations and the DDH/PPT reduction remain outside this result.

1.7 Historical component: Proof targets, weeding and honest rejection

Actual repaired submission acceptance entails proof validity and expanded weeding. Therefore each accepted component or aggregate statement differs from every covered statement on the previous accepted board, regardless of either proof's commitments. Both component-to-aggregate directions are included. When both honest prefix decisions accept, Lean derives the actual two board entries from execution. Generated honest bit/nonce statements agree with the corresponding ballot components and aggregate.

Lean: `repairedSubmit_accepted_target_ne`.

Lean: `repairedCastHonestPair_board_of_accepted`.

Lean: `strongHonestBallot_covered_statement`.

Theorem 1.10 (Generated-target match bound). *Fix an injective generator map $x \mapsto xg$ over a finite field of size q , a public key and either swapped vote. Draw the existing honest coin pair. Allow any probabilistic continuation to choose a static hash and submission, even given those coins. Then*

$$\Pr \left[\begin{array}{l} \text{the submission is accepted and one covered statement} \\ \text{matches a generated honest component or aggregate statement} \end{array} \right] \leq \frac{9}{q-1}.$$

Lean: `accepted_honest_target_match_probability_le`.

Such a match implies an honest-prefix rejection: if both decisions accepted, weeding would exclude the matched statement. Rejection in turn implies a covered-nonce collision, bounded by the [formal reference's concrete repair section](#). The continuation above is an auxiliary upper-bound experiment. It adds no public coin disclosure to the protocol. All rejection and continuation-failure outcomes are retained; no target freshness premise is assumed.

Why rejection matters. At $p = 23, q = 11$, use Alice's nonces (1, 2) with vote 0 and Bob's nonces (4, 2) with vote 1. Bob repeats Alice's second zero ciphertext and is rejected. A subsequent valid submission with nonces (4, 5) and vote 1 is accepted on the retained board and has exactly Bob's generated first proof target. That target is absent from every covered statement on the accepted board. Thus unconditional freshness against all generated targets is false; this counterexample concerns the proof invariant, not a demonstrated secrecy attack.

Lean: `BallotProofProvenanceControls.rejected_honest_target_can_recur`.

A fresh third ballot with nonces (6, 9) is accepted, while a component copying Alice's aggregate is rejected. Independent multiplicative enumeration checks 120,000 target/weeding cases, including 10,700 recurrences after honest rejection; none violates the rejection/collision implication.

Connection to simulation and remaining scope. A simulation step changes only keys carrying its requested statement. Hence simulation for a prior accepted ballot preserves an accepted submission's covered target entry; actual weeding supplies the statement inequality.

Lean: `strongBallotSimOracle_other_statement`.

Lean: `repairedSubmit_accepted_simulation_preserves_target`.

The following subsections give programmed-cache and honest-prefix composition. The static continuation bound and this local step alone do not establish that correspondence. The full reduction additionally composes honest rejection losses, extraction and trustee/election observations, as described in the formal reference.

1.8 Historical component: Replayable programmed-cache interpretation

The existing adaptive simulator can now run through the raw ballot oracle used by replay extraction. Its internal state contains a shadow cache C , sticky collision flag b , and list L of statements from actual proof requests. The live oracle has a separate cache R . A shadow hit returns its old answer; a shadow miss queries the live oracle and stores its answer. Honest requests lift the existing simulator’s uniform computation and record their statement, even when a collision is flagged. The program sees the original answers and proofs; the extra caches and list are internal reduction state.

Theorem 1.11 (Exact adaptive runtime interpretation). *Start both caches empty, with flag false and list empty. For every adaptive program in the existing ballot-hash/honest-proof request interface, projecting away the extra metadata gives exactly the original simulated computation, including its final shadow cache and sticky flag. Every supported result obeys*

$$R \leq C, \quad S \notin L \implies C(S, pc) = R(S, pc).$$

Here $R \leq C$ means every recorded live answer is preserved in the shadow. The invariant is derived from initialization and all query successors.

Lean: [ballotProgrammed_runtime_eq](#).

Lean: [ballotProgrammed_request_provenance](#).

This proof reuses VCVio’s state-composition and invariant-restricted projection lemmas. It establishes computation equality, rather than assuming two cache implementations realize the same distribution.

Verification and query cost. At any supported final state, the original full-proof verifier has the same Boolean computation in both caches when its statement is outside L . This retains both proof challenges and the original checksum check. A source bound of n hash-or-proof requests gives at most n live hash queries in the lowered program. Simulated proof requests use only uniform coins; this is a conservative hash-query bound, not a computational efficiency theorem.

Lean: [ballotProgrammed_verify_agreement](#).

Lean: [ballotProgrammed_query_bound](#).

Controls and correspondence boundary. The $p = 23, q = 11$ fixtures retain a fresh programmed-only answer, a conflicting live answer, and a sticky flag after later fresh programming. The full verifier can disagree at a programmed statement, so dropping statement exclusion is false. Erasing the request list or starting with inconsistent caches breaks the invariant. Independent enumeration checks 22,621 adaptive dual-cache trace prefixes, detects reset/overwrite defects, and reports no failures or gaveUp. This gate checks cache policy, not cryptographic security.

Lean: [BallotProgrammedControls.programmed_target_can_disagree](#).

The next subsection relates the sampled honest-ballot program to the historical constructor; Section 1.10 composes actual board decisions. Section 1.11 bounds rejected prefixes in that execution. Section 1.12 composes selected-proof replay with the post-prefix attacker.

1.9 Historical component: Sampled honest ballots and actual requests

The honest-ballot program now samples the original nonzero nonce pair and makes three proof requests: the first candidate component, the second zero component, and the implicit aggregate with the sum of their nonces. It returns the original two ciphertexts, full component proofs and aggregate proof. With explicit coins, interpreting the hash by any static function exactly recovers the existing historical ballot constructor.

Lean: [strongHonestBallotWithCoinsOracle_function](#).

Theorem 1.12 (Historical sampling correspondence). *For a finite field, fixed generator and public key, either honest vote, any initial live cache and any initial flag, real execution of the sampled request program has the same distribution as drawing the original historical coin record and running the explicit-coin ballot oracle constructor. Both sides include the final cache and unchanged flag.*

Lean: [strongHonestBallotOracle_real_eq](#).

The historical sampler groups witness, challenge and response triples before drawing the nonces. The request program draws nonces first, then coins proof by proof. Lean proves the independent-draw permutation, including the transpose of the nine proof coins. This gives distributional equality; the differently ordered computation trees are not asserted equal. No draw is discarded and no retry removes a nonce collision. Real request lowering also has an exact runtime equality for arbitrary request programs.

Lean: [drawHonestCoinsByProof_eq](#).

Lean: [ballotRealRequest_runtime_eq](#).

Compositional request provenance. Every supported simulated execution prepends exactly the returned ballot’s aggregate and two component statements to the prior request list, in reverse request order. This holds with prior cached answers and metadata; the list is derived from execution. It can therefore describe the next honest ballot. Cache consistency from empty initialization follows from the separate adapter invariant in Section 1.8.

Lean: [strongHonestBallotOracle_request_targets](#).

The program has a derived three-request budget. For an injective generator and initial cache budget k , its full output/cache/flag distributions satisfy

$$\text{TV}(\text{RealBallot}, \text{SimBallot}) \leq \frac{3(6 + k)}{q}.$$

Lean: [strongHonestBallotOracle_simulation_le](#).

Controls and scope. At $q = 11$, nonces $(1, 10)$ are supported by the original nonzero sampler and sum to zero. The resulting honest abstention ballot has a valid aggregate proof for the identity ciphertext. Controls reject substituting the first nonce for the sum and leaving proof coins in the wrong order. Independent multiplicative checks cover 2,048 permutations and 4,096 ballots over labelled draws $\{1, 2\}$, with no failures or gaveUp; they detect both injected defects. The next subsection composes actual verification and the two honest decisions, retaining rejection and public fields. This single-ballot correspondence does not prove adaptive malicious-ballot extraction, election privacy or efficiency.

1.10 Historical component: Actual oracle submissions and the honest prefix

The ballot verifier now queries and checks the original full proofs in order: component 0, component 1, then aggregate, stopping on failure. Submission returns `invalidProof` before considering ciphertext reuse; a valid reused ballot returns `reusedCiphertext`. Both rejections retain the board. A valid fresh ballot is appended with its voter identity. Interpreting the oracle by any fixed hash function recovers the existing repaired submission exactly.

Lean: `repairedSubmitOracle_function`.

Actual successful lazy verification supplies stored challenges and full validity for all three unchanged proofs. Honest construction derives this cached validity even when proof targets collide; no caller validity premise is needed.

Lean: `strongBallotVerifyAllOracle_true_cached`.

Lean: `strongHonestBallotOracle_real_cached`.

Theorem 1.13 (Historical honest-prefix correspondence). *For either vote and any initial cache and flag, real execution of the two sampled honest voters and their submissions has the same distribution as sampling the original honest coin pair and running the explicit-coin oracle prefix. Decisions, returned board, final cache and flag are included.*

Lean: `repairedCastHonestPairOracle_real_eq`.

Theorem 1.14 (Derived exclusion after actual acceptance). *Run the sampled honest prefix through the programmed adapter from its empty initial state. Suppose both actual honest decisions accept. Every recorded statement then belongs to a covered ciphertext of a ballot on the returned board. After any raw attacker computation and its actual accepted submission, none of the submitted ballot's three covered statements is in the recorded list. Freshness and board membership are derived from these executions.*

Lean: `repairedCastHonestPairOracle_programmed_targets_of_accepted`.

Lean: `repairedHonestPrefix_accepted_target_exclusion`.

Raw attacker and verification queries preserve the list. This discharges statement exclusion on the accepted-prefix event. It does not remove the honest-acceptance premise: a rejected honest target can recur later.

Controls and scope. Preloaded-cache fixtures retain fresh acceptance, valid cross-kind reuse, invalid-proof priority and aggregate-only failure; no empty-cache reachability is claimed by those fixtures. Independent full-proof checks cover all 16 validity/freshness cases, detecting seven priority defects and two omitted aggregate checks, with no failures or gaveUp. Static-prefix controls retain both honest acceptance and rejection. The next subsection bounds actual prefix rejection and its simulation loss.

1.11 Historical component: Actual prefix rejection and simulation loss

The accepted-case exclusion theorem retains an honest-acceptance premise. A rejected honest target can recur, so that premise cannot simply be removed. The rejected event is now bounded in the same actual oracle execution. Every returned covered first coordinate is the generator multiplied by its original nonce, including the aggregate sum. Full honest proofs validate against their stored answers even if targets collide. Thus expanded nonce separation forces both actual submissions to accept, for any initial cache.

Lean: [repairedCastHonestPairWithCoinsOracle_accepts_of_collisionFree](#).

Theorem 1.15 (Real honest-prefix rejection). *Fix an injective generator over a finite field of size q , any public key, either vote and any initial cache and flag. In the actual real sampled prefix,*

$$\Pr[\text{the two honest decisions are not both accepted}] \leq \frac{9}{q-1}.$$

Lean: [repairedCastHonestPairOracle_real_rejection_le](#).

The proof uses historical sampling correspondence and the existing nonce collision bound. It derives validity and freshness; neither is a caller premise. No rejection, zero aggregate nonce, cache state or flag outcome is discarded.

Derived request cost and simulated rejection. The source prefix makes six honest proof requests and at most six verification hash queries. Nonce draws consume no hash/proof budget. From initial cache budget k , the already checked adaptive simulation gives

$$\text{TV}(\text{RealPrefix}, \text{SimPrefix}) \leq \frac{6(15+k)}{q},$$

including final values, cache and sticky flag. The query budget is derived from the actual short-circuit program; it is not a PPT theorem.

Lean: [repairedCastHonestPairOracle_query_bounds](#).

Lean: [repairedCastHonestPairOracle_simulation_le](#).

Consequently the actual replayable programmed prefix, from empty initialization, has rejection probability at most $9/(q-1) + 90/q$. Its exact runtime correspondence transfers the simulated bound. Both nonzero-sampling and programming-collision losses remain; these upper bounds may exceed one for small fields.

Lean: [repairedCastHonestPairOracle_programmed_rejection_le](#).

Controls and composition boundary. At $q = 101$, real rejection is at most $9/100$ and programmed rejection at most $9909/10100 < 1$. Lean proves positive probability of an accepted programmed prefix. Conversely, a zero generator causes certain real rejection, refuting the claim with injectivity omitted; this is an invalid-parameter control.

Lean: [HonestPrefixRejectionControls.programmed_acceptance_positive](#).

Lean: [HonestPrefixRejectionControls.zero_generator_rejection_certain](#).

Independent multiplicative checks exhaust 40,000 nonzero-nonce/world/answer-policy cases with fixed proof coins. They retain 11,200 rejections and 7,600 zero-aggregate prefixes, with no failures or gaveUp. The next subsection composes this split with a post-prefix attacker, full-proof verifier and selected-proof replay.

1.12 Historical component: Selected-proof extraction after actual submission

The source program now runs the sampled honest prefix, gives its actual public decisions and retained board to a raw oracle attacker, and executes the original submission. It returns that view, the submitted ballot, decision and final board. This component starts empty and places attacker queries after the honest prefix. It does not model earlier queries or the complete election interface.

Derived full verification and target selection. When both honest decisions and the attacker submission accept, every covered proof validates in the live cache with its original challenges and commitments. Actual intermediate execution supplies cache consistency; submission supplies full cached validity; accepted-target exclusion supplies live/shadow agreement. No cache invariant or freshness premise is left to this theorem’s caller.

Lean: [repairedSubmissionOracle_accepted_live](#).

For a fixed component or aggregate index, the replay target retains that full proof on joint acceptance. Otherwise it returns an internal always-invalid proof. The public protocol result is unchanged, and even the invalid proof’s hash query remains. Actual live verifier acceptance is exactly joint acceptance; a valid fallback on rejected executions would falsely inflate this quantity.

Lean: [repairedSubmissionTargetOracle_acceptance_eq](#).

Theorem 1.16 (Composed selected-proof replay bound). *Fix an injective generator over a finite field of size q . Let every attacker continuation have raw hash-query bound n , and let B be its actual submission acceptance probability in this programmed execution. Put*

$$\delta = \frac{9}{q-1} + \frac{90}{q}, \quad a = (B - \delta)_+.$$

For any fixed covered index, the composed extractor satisfies

$$a \left(\frac{a}{n+16} - \frac{1}{q} \right)_+ \leq \Pr[\text{extractor returns a witness}].$$

Lean: [repairedSubmission_extract_le](#).

The source and lowered query bound is $n + 15$; verifier completion adds one. The sharper bound uses joint prefix/submission acceptance instead of a . The proof composes the existing replay theorem with derived live verification, query coverage and the actual rejection bound.

Lean: [repairedSubmissionTargetOracle_query_bound](#).

A successful witness satisfies the actual bit/nonce encryption relation and is for the selected ciphertext of some supported accepting source submission.

Lean: [repairedSubmission_extract_origin](#).

Controls and scope. Independent programmed-prefix/attacker fixtures check 20,000 cases: 2,720 joint acceptances and 2,840 valid submissions excluded after honest rejection. Lean controls retain unchanged accepted proofs, the invalid guard and its real hash query; a valid unguarded proof demonstrates the inflation defect. This is not an exhaustive replay-probability test or a computational efficiency proof. The source-origin conclusion here is existential. Section 1.13 now recovers the first complete submission from the existing rich fork path.

1.13 Historical component: Retaining the first complete submission

The statement/witness projection loses the complete source ballot. That loss can be repaired using the pinned libraries' existing data: VCVio's `contextForkWitness` retains both typed execution paths, and PolyFun's `Path.pullMap` recovers a path before pure output projection. No new trace format or cryptographic target is needed.

Exact projection and source recovery. Run the original source, append the existing completion query, and retain its source value. Mapping its logged result to the old trace gives exactly the old query tree. Pulling the rich fork's first path through that map recovers the source value from the first execution. The original full proof and its target are the ones in that first trace; choosing any supported source with the same ciphertext is insufficient.

Lean: `ballotForkSource_trace_eq`.

Lean: `ballotForkSourceFirst_trace`.

The source-retaining extractor maps the existing rich witness to this source value and the existing extracted bit/nonce witness. Dropping the source gives exactly the previous extractor, including its failure outcomes.

Lean: `ballotForkSourceExtract_project`.

Theorem 1.17 (First-submission extraction). *At the scope and under the assumptions of Section 1.12, let E_{src} return the first complete submission alongside the extracted statement and witness, and let E be the previous extractor. Then*

$$\text{map}(\text{dropSource}, E_{\text{src}}) = E, \quad \Pr[E_{\text{src}} \neq \text{none}] = \Pr[E \neq \text{none}].$$

Every supported $\text{some}(B, S, w)$ has B accepted, S equal to the selected covered statement of B 's submitted ballot, and w satisfying that statement's actual bit/nonce encryption relation. The returned B is from the first source execution retained by this fork.

Lean: `repairedSubmission_extractSource_project`.

Lean: `repairedSubmission_extractSource_valid`.

Consequently Theorem 1.16's bound holds without additional loss.

Lean: `repairedSubmission_extractSource_le`.

Controls and composition boundary. A tagged honest prover has positive extraction probability. Its recovered tag follows the first trace's full proof. Valid same-target transcripts at challenges 5 and 6 can have different tags, refuting equality of replay source values.

Lean: `BallotForkControls.tagged_first_value`.

Lean: `BallotForkControls.replay_can_change_source_tag`.

Independent multiplicative fixtures check 2,420 replay pairs and detect 440 wrong-second-tag cases; prior full-proof and completion controls remain. Section 1.15 constructs the three-witness algorithm for one original ballot. Its joint success and efficiency argument is outside this component's guarantee; separate selected-proof bounds do not establish it.

1.14 Historical component: Joint-witness contract and the scalar modulus

Joint extraction returns witnesses for both components and the implicit aggregate of the same ballot on success. The algebraic consumer of those witnesses is now checked. It uses the original encryption and aggregate operations; it does not assume that accepted proofs already reveal witnesses.

Theorem 1.18 (Consistency of three covered witnesses). *Let $w_0 = (b_0, r_0)$ and $w_1 = (b_1, r_1)$ witness the two component ciphertexts, and let $w_a = (b_a, r_a)$ witness their actual aggregate. If $r \mapsto r \bullet g$ is injective, then*

$$r_a = r_0 + r_1, \quad [b_a]_F = [b_0]_F + [b_1]_F.$$

If $2 \neq 0$ in F , the corresponding integer bit counts satisfy

$$[b_0]_{\mathbb{N}} + [b_1]_{\mathbb{N}} = [b_a]_{\mathbb{N}} \leq 1.$$

Lean: `Ballot.covered_witnesses_sum`.

Lean: `Ballot.covered_witnesses_atMostOne`.

The first ciphertext coordinate identifies the sum of nonces. Cancelling its public-key contribution in the second coordinate identifies the sum of scalar plaintexts. Checking the possible bits then gives the integer count, with abstention retained. This algebra needs no secret key or hardness assumption. For the historical scalar field $\mathbb{Z}_q$ with prime $q > 2$, the nonzero-two condition is derived from the modulus.

Lean: `Ballot.covered_witnesses_atMostOne_prime`.

Checked parameter counterexample. A field-size condition $|F| > 2$ alone is insufficient. Mathlib’s `GaloisField 2 2` has four elements, and generator 1 has an injective scalar action, but $1 + 1 = 0$. Two component votes of one can therefore have aggregate bit zero. The control constructs all three original strong proofs, proves their full validity and retains the three encryption witnesses. The general characteristic-two construction also passes original submission on an empty board.

Lean: `BallotWitnessConsistencyControls.four_element_counterexample`.

Lean: `BallotWitnessConsistencyControls.wrapped_submission_accepts`.

This refutes an overgeneralization of the abstract field model. It is not an attack on the documented large prime-order protocol parameters, and it changes no verification, hash or sampling rule.

Controls and extraction boundary. Honest and abstention witnesses remain checked. Independent multiplicative fixtures compare 10,648 aggregate candidates, find 363 consistent witness triples and detect 121 double-vote cases if the aggregate constraint is omitted. An independent four-element additive model retains 16 wrapped double votes. These finite controls check the algebraic contract, not replay probabilities. The algorithm in Section 1.15 derives these witnesses on success; its quantitative bound follows below. This algebraic result does not establish reduction efficiency.

1.15 Historical component: Joint replay from one original submission

The joint extractor samples one original execution, replays the selected query for each component and the aggregate, and retains the original complete submission. It returns witnesses only if all three conditional attempts succeed; every failure remains an explicit `none` outcome.

Theorem 1.19 (Joint extraction output contract). *For the actual programmed honest prefix, subsequent raw attacker and original submission, let J be `repairedSubmission_joint_extract`. If $g \neq 0$, every supported `some( $B, w$ )` output has B accepted and*

$$\forall i \in \{0, 1, a\}, \quad \text{coveredStatement}(B, i).\text{Witnesses}(w_i).$$

The returned B occurs in supported original programmed execution. If scalar action on g is injective and $2 \neq 0$ in F , these derived witnesses satisfy

$$r_a = r_0 + r_1, \quad [b_0]_{\mathbb{N}} + [b_1]_{\mathbb{N}} = [b_a]_{\mathbb{N}} \leq 1.$$

Lean: `repairedSubmission_joint_extract_valid`.

Lean: `repairedSubmission_joint_extract_consistent`.

Actual accepting paths have all three locations. Let $A(p)$ be joint acceptance on original path p . With nonzero generator and attacker hash bound n , each bounded selector satisfies $\text{Sel}_i(p).\text{isSome} = \mathbf{1}_{A(p)}$. The exact original live cache, actual acceptance theorem and source query bound supply the cache/log evidence. Each selected ordinal has a physical occurrence on its projected original path. No per-selection completion is appended.

Lean: `repairedSubmissionPath_selector`.

Lean: `repairedSubmissionPath_locations`.

Thus simultaneous original selection has probability exactly $\Pr[A]$, including both honest decisions and original attacker submission.

Lean: `repairedSubmissionPath_selection_probability`.

Connection to the existing fork. VCVio’s rich fork factors into a first path and conditional completion. Pure projection retains the source and answers; averaging one conditional attempt gives exactly the raw selected-proof extractor.

Lean: `ballotReplayAtPath_bind`.

Lean: `ballotReplaySourceAttempt_bind`.

Controls and the quantitative argument. A repeated honest target has positive joint replay probability, with its tag tied to the original proof even when a second completion changes it.

Lean: `BallotJointReplayControls.tagged_joint_positive`.

Independent fixtures retain 31,944 three-proof replay cases and add 60,000 original selector checks, including 8,160 enabled selections and detected omitted-miss-target defects. These finite controls are not an attacker bound. Separately positive marginals can have disjoint success sets.

Lean: `BallotJointReplayControls.disjoint_marginals_no_shared_success`.

Enabling all original selectors is not successful replay of all proofs. The common-path success bound follows in Section 1.16.

1.16 Historical component: Quantitative joint replay

The bound uses one shared original path. Threshold δ is an analysis parameter only.

Theorem 1.20 (Joint replay with retained rejection loss). *Let F be a finite field of cardinality q , with the uniform challenge oracle, and let scalar action $r \mapsto r \cdot g$ be injective. Fix the public key and honest vote. Let the subsequent raw attacker make at most n hash queries for every honest-prefix view. In the actual programmed prefix and original submission, write B for attacker-submission acceptance probability and α for joint acceptance including both honest decisions. Set*

$$Q = n + 16, \quad \ell = \frac{9}{q-1} + \frac{90}{q}, \quad a = (B - \ell)_+.$$

For every $\delta \in \mathbb{R}_{\geq 0} \cup \{\infty\}$, the actual joint extractor J satisfies, with all subtraction truncated at zero,

$$(a - 3Q\delta)_+(\delta - 1/q)_+^3 \leq (\alpha - 3Q\delta)_+(\delta - 1/q)_+^3 \leq \Pr[J.\text{isSome}].$$

This is a finite probabilistic statement for the two-candidate, post-prefix submission interface.

Lean: [repairedSubmission_joint_extract_accepted_le](#).

Lean: [repairedSubmission_joint_acceptance_loss](#).

Lean: [repairedSubmission_joint_extract_le](#).

Conditional mass and reconstruction. At a reachable ordinal, context mass is the probability that its completion selects it again. Exact split/completion reconstruction bounds originally selected contexts of mass at most δ by probability δ per ordinal. The union over three selections and Q ordinals costs at most $3Q\delta$.

Lean: [ballotReplay_bad_context_fixed_le](#).

Lean: [ballotJointReplay_bad_context_le](#).

A uniform focused answer equals the original challenge with probability at most $1/q$. On each good original path, the actual three conditional binds therefore give success at least $(\delta - 1/q)_+^3$. Averaging gives the displayed bound.

Lean: [ballotReplayAtPath_mass_le](#).

Lean: [ballotJointReplay_selection_le](#).

Controls and scope. A pure selected output with no query refutes dropping occurrence reachability. For the honest tagged control with $q = 11$, $Q = 2$ and $\delta = 1/8$, the lower bound is $27/2725888 > 0$; this control repeats one target three times.

Lean: [BallotReplayContextControls.missing_occurrence_refutes_bound](#).

Lean: [BallotReplayContextControls.tagged_joint_quantitative](#).

Oracle-interaction cost. With canonical finite scalar sampling and at most m total attacker interactions per honest-prefix view, the actual joint bound is $16(m + 19)$: at most $m + 19$ source requests, four interactions per handler and four runs for shared replay. Uniform draws and failures are included.

Lean: [repairedSubmission_joint_extract_canonical_total_bound](#).

Uniformity alone does not fix sampling cost: a checked padded sampler refutes one-query accounting. Pure local functions remain uncharged by this metric.

Lean: [RepairedSubmissionCostControls.uniform_sampler_cost_counterexample](#).

This interaction bound does not include executable local computation or encoded sizes.

1.17 Historical component: Finite live-cache implementation

The proof-facing cache is a function from full statement/commitment keys to optional scalar answers. The implementation uses Mathlib’s association list `AList`, whose entries have unique keys. Lookup interprets this finite list as the original cache; insertion implements the same semantic update.

Lean: `BallotFiniteCache.denote_insert`.

Theorem 1.21 (Exact finite-cache correspondence). *Use the same scalar sampler and decidable full-key equality in both interpreters. Let P be any raw ballot-oracle program and C any finite initial cache. With $D(C)$ denoting its lookup function, the finite interpreter satisfies*

$$\text{map}((x, C') \mapsto (x, D(C')))(\text{runFinite}(P, C)) = \text{runBallotOracle}(P, D(C)).$$

Equality is of oracle computations, retaining the output, explicit random draws and final cache. If P makes at most n hash queries on every branch, every supported finite output (x, C') satisfies $|C'| \leq |C| + n$.

Lean: `runBallotFiniteCache_eq`.

Lean: `runBallotFiniteCache_length_le`.

Connection to the actual repaired source. Applying the finite interpreter to the actual repaired source from an empty cache preserves the complete original submission, all decisions and its final live cache. The actual source query bound gives at most $n + 15$ live entries from attacker hash bound n per honest-prefix view.

Lean: `repairedSubmissionFiniteCache_eq`.

Lean: `repairedSubmissionFiniteCache_length_le`.

A hit returns the old answer without drawing; a miss samples and stores one answer. Repeated queries and new entries preserve earlier answers. Equal commitments with different full statements remain distinct keys.

Lean: `BallotFiniteCacheControls.hit_preserves_runtime`.

Lean: `BallotFiniteCacheControls.miss_samples_and_stores`.

Lean: `BallotFiniteCacheControls.full_key_separation`.

Representation and backend boundary. The following subsections supply the finite shadow cache and saved replay tape. Continuation execution and encoded costs are outside this component’s guarantee. List length is not encoded bit length or lookup time; key equality, group operations and extraction remain uncharged. The pinned optional VCVio complexity backend provides small exact machine certificates but no general oracle compiler or adequacy theorem. Its ElGamal example also stops at structural interaction accounting. Those artifacts do not establish standard-machine PPT for this extractor.

1.18 Historical component: Finite programmed shadow cache

The shadow state is $s = (C, b, L)$: a finite association list, sticky collision flag and ordered list of requested statements. Let $D(s) = (D(C), b, L)$ interpret only its cache as a lookup function. The finite handler uses the existing four-draw ballot transcript sampler. On an occupied programming key it retains the old answer and sets b to true, even for an agreeing challenge. On a fresh key it inserts the sampled challenge and retains b . Every proof request prepends its statement to L , including duplicates and flagged requests.

Theorem 1.22 (Exact finite shadow and complete runtime correspondence). *Let F be a field with the existing scalar sampler and G an additive commutative group with an F -module structure and decidable equality. For any $g, pk \in G$, proof-oracle program P and finite initial shadow state s ,*

$$\text{map}((x, s') \mapsto (x, D(s')))(\text{runFiniteShadow}_{g, pk}(P, s)) = \text{runShadow}_{g, pk}(P, D(s)).$$

Composing with the finite live interpreter, from empty initialization,

$$\text{map}(((x, s), R) \mapsto ((x, D(s)), D(R)))(\text{runBothFinite}_{g, pk}(P)) = \text{runProgrammed}_{g, pk}(P).$$

Both are equalities of oracle computations. They preserve the actual output, random draws, both caches, flag and exact history; no caller supplies a reachability or cache-consistency premise.

Lean: [runBallotFiniteProgrammed_eq](#).

Lean: [runBallotFiniteProgrammed_runtime](#).

Actual source and controls. For the repaired submission source (finite F , decidable scalar equality), projecting away internal state gives exactly the existing raw source. Its replay and probability results therefore apply to this finite representation. The full repaired runtime retains all decisions and boards, and the live storage bound remains $n + 15$ from attacker hash bound n per prefix view.

Lean: [repairedSubmissionFiniteSource_eq](#).

Lean: [repairedSubmissionFiniteProgrammed_eq](#).

Lean: [repairedSubmissionFiniteSource_length_le](#).

Literal controls retain conflicting and agreeing collisions, the returned proof's sampled challenge, duplicate history and sticky failure after a fresh request. A raw hit returns the cached answer without drawing or changing state.

Lean: [BallotFiniteProgrammedControls.conflict_retains_old_answer](#).

Lean: [BallotFiniteProgrammedControls.agreeing_hit_flagged](#).

Lean: [BallotFiniteProgrammedControls.fresh_after_bad_sticky](#).

Lean: [BallotFiniteProgrammedControls.raw_hit_preserves_runtime](#).

Implementation boundary. The next subsection supplies finite saved replay input and reconstructs the existing typed continuations. Finite caches alone do not supply continuation execution, bit encodings, local-operation costs or a machine-adequacy theorem.

1.19 Historical component: Finite saved replay tapes

PolyFun’s existing trace carrier is a finite list of tagged operation answers, $T = \text{List}(\Sigma t, \text{Answer}(t))$. The saved input contains no continuation function. The decoder traverses the source program, checks each tag before transporting its answer, and requires an empty tape at termination. The carrier already enforces answer types; byte decoding is a separate boundary.

Theorem 1.23 (Exact saved-path reconstruction and joint replay). *For a free oracle program P with decidable operation-tag equality, every typed path p and typed tape T satisfy*

$$\text{read}(P, \text{trace}(P, p)) = \text{some}(p), \quad \text{read}(P, T) = \text{some}(p) \implies \text{trace}(P, p) = T.$$

Direct tape collection equals the existing path-producing execution followed by trace erasure. For every raw ballot source P , selection function s and hash budget parameter n , using the existing scalar sampler and key equality,

$$\text{jointReplayTape}(P, s, n) = \text{jointReplay}(P, s, n).$$

This is equality of oracle computations, retaining the original result, all three attempts, selected physical occurrences, rejection and randomness. No caller supplies tape validity, path correspondence or a probability premise.

Lean: [ballotReplayReadTape_trace](#).

Lean: [ballotReplayReadTape_exact](#).

Lean: [ballotReplayCollectTape_eq](#).

Lean: [ballotJointReplayTape_eq](#).

Actual repaired source and storage. The repaired tape extractor uses the finite shadow source and equals the existing joint extractor exactly. Its witness and probability theorems transfer without added loss. Every saved tape has at most $4(m + 19)$ entries when the scalar sampler uses one primitive query and the attacker uses at most m total queries per prefix view. Canonical finite scalar sampling retains the existing $16(m + 19)$ interaction bound. Entries are not encoded bits; local decoding remains uncharged.

Lean: [repairedSubmissionTape_extract_eq](#).

Lean: [repairedSubmissionTape_saved_length_le](#).

Lean: [repairedSubmissionTape_canonical_total_bound](#).

Lean: [ballotReplayCollectTape_length_le](#).

Controls and implementation boundary. Literal controls reject mismatched tags, truncation, trailing events, reordering and dropped uniform-labelled answers. Decoding and locating the second repeated hash tag preserves its two-entry prefix, including the intervening other query.

Lean: [BallotReplayTapeControls.rejects_truncation_and_trailing](#).

Lean: [BallotReplayTapeControls.physical_second_occurrence](#).

Reconstruction still traverses the source `OracleComp` and produces the existing typed continuations. The next subsection integrates finite logging in all four runs, preserving the ordered miss log. This replay interface does not establish bit encodings, local execution costs or machine adequacy. Saving an entire branching computation tree would not follow from a polynomial path-length bound.

1.20 Historical component: Finite logging in every replay run

The finite logging state is (C, L) : the existing association-list cache and the chronological list of full-key misses. Projection $D(C, L)$ interprets cache lookup and leaves L unchanged. A uniform query forwards without state changes. A hash hit returns the old answer without drawing or logging. A miss draws one wrapped challenge, inserts the answer and appends the full key to L .

Theorem 1.24 (Exact finite logging and complete replay). *Use the same decidable full-key equality. For every raw ballot source P and finite initial state (C, L) ,*

$$\text{map}(\text{id} \times D)(\text{runFiniteLogged}(P, (C, L))) = \text{runLogged}(P, D(C, L)).$$

For the established field/module setting, scalar sampler, proof selection s and hash-budget parameter n , the finite first execution and three finite residual attempts satisfy

$$\text{finiteJointReplay}(P, s, n) = \text{jointReplay}(P, s, n).$$

These are exact equalities of oracle computations. They preserve the original result, complete proofs, physical occurrences, acceptance tests, both challenges, cache answers, miss-log order and randomness. Later attempts still execute after an earlier failure. No caller supplies a state or replay correspondence premise.

Lean: `runBallotFiniteLogged_eq.`

Lean: `ballotFiniteReplaySourceRun_eq.`

Lean: `ballotFiniteSourceAttempt_trace.`

Lean: `ballotFiniteJointReplay_eq.`

Actual repaired extractor and bounds. The actual extractor also uses the finite shadow source. It equals the established joint extractor exactly, so the witness and probability theorems transfer without loss. For finite F , an attacker hash bound n per prefix view gives at most $n + 15$ original miss-log entries. More generally the finite log grows by at most the source hash budget. Canonical finite scalar sampling retains the $16(m + 19)$ total-interaction bound from attacker total bound m .

Lean: `repairedSubmissionFinite_extract_eq.`

Lean: `repairedSubmissionFinite_log_length_le.`

Lean: `repairedSubmissionFinite_canonical_total_bound.`

Lean: `runBallotFiniteLogged_log_length_le.`

Controls and remaining costs. Literal controls preserve hit answers without draws or duplicate entries, append misses in chronological order and forward uniform queries without state changes. Reversed logs, duplicate-hit logs and merged statements are distinguished.

Lean: `BallotFiniteLoggedControls.hit_no_draw_no_log.`

Lean: `BallotFiniteLoggedControls.miss_appends.`

Lean: `BallotFiniteLoggedControls.detects_log_mutations.`

All replay runs now use finite cache state and saved tapes. Continuations are still constructed and traversed as `OracleComp`. The next subsection derives cache/log record bounds. These component equalities do not bound the rest of the state, bit encodings or local execution costs, and do not establish machine adequacy. These equalities do not discharge full election integration, DDH or computational ballot secrecy.

1.21 Historical component: Replay storage and complete key records

Let $c(C)$ count entries in the finite replay cache and $|L|$ count chronological miss-log entries. Counts refer to stored elements, with no bit encoding assumed.

Theorem 1.25 (Derived cache/log balance). *For a finite scalar field F , any source oracle computation P , finite initial state (C, L) and supported output $(a, (C', L'))$ of its finite logged execution,*

$$c(C') + |L| = c(C) + |L'|.$$

A miss inserts one entry and appends one full key; a hit or uniform query preserves both counts. Empty initialization therefore derives $c(C') = |L'|$. With scalar sampling and a source hash-query bound n , cache growth is at most n . No initial cache/log equality is a premise.

Lean: [runBallotFiniteLogged_storage_balance](#).

Lean: [runBallotFiniteLogged_empty_sizes](#).

Lean: [runBallotFiniteLogged_cache_length_le](#).

Fixed full-key layout. For statement $(g, pk, (c_0, c_1))$ and commitment pairs $(a_0, b_0), (a_1, b_1)$, the record is $[g, pk, c_0, c_1, a_0, b_0, a_1, b_1]$. It has eight group elements. Decoding this record returns exactly the key; successful decoding also implies that the input is exactly that key's record. Thus packing is injective, and inputs of other lengths reject.

Lean: [ballotKeyOfRecord_roundTrip](#).

Lean: [ballotKeyOfRecord_exact](#).

Lean: [ballotKeyRecord_injective](#).

Lean: [ballotKeyOfRecord_wrong_length](#).

Actual repaired source. For the existing two-candidate source over finite F , group G with its scalar action, any g, pk , vote and attacker with hash-query bound n on every prefix view, each supported empty-start finite logged output satisfies

$$c(C') = |L'| \leq n + 15, \quad |\text{GroupRecord}(C', L')| \leq 16(n + 15), \quad |\text{ScalarRecord}(C', L')| \leq n + 15.$$

The group record packs all cache and log keys; the scalar record retains cache answers. Its exact lengths are $8(c(C') + |L'|)$ and $c(C')$ respectively. These bounds count those projections, not shadow state, outputs or tape payloads.

Lean: [repairedSubmissionFinite_storage_le](#).

Lean: [repairedSubmissionFinite_record_cells_le](#).

Controls and boundary. An actual pure execution from a one-entry cache and empty log refutes unconditional equal counts. An actual fresh query records one key and answer. The literal key $[1, 2, 3, 4, 5, 6, 7, 8]$ fixes field order independently of round trips; swapped fields change the key, and seven/nine-field records reject.

Lean: [BallotReplayStorageControls.preloaded_refutes_equal_counts](#).

Lean: [BallotReplayStorageControls.fresh_query_records_once](#).

Lean: [BallotReplayStorageControls.literal_key_layout](#).

Lean: [BallotReplayStorageControls.detects_key_layout_mutations](#).

The next subsection bounds the retained shadow entries. This storage count does not bound encoded payloads or local execution costs and does not establish machine adequacy.

1.22 Historical component: Retained shadow cache and request history

The finite shadow state consists of an association-list cache C , sticky collision flag b and ordered proof-request history H . History retains every requested statement, including repeated or colliding requests. Let $c(C)$ count cache entries. These are entry counts rather than bit sizes or peak heap usage.

Theorem 1.26 (Additive shadow storage). *Use the actual programmed interpreter’s field F , group G , scalar action, decidable key equality and scalar sampler. For any source P over the proof oracle, total-query bound m , finite initial shadow state (C, b, H) and supported output $(a, (C', b', H'))$ of that interpreter’s execution,*

$$c(C') \leq c(C) + m, \quad |H'| \leq |H| + m.$$

Raw requests add at most one cache entry and leave history unchanged. Each proof request adds exactly one history entry and at most one cache entry. Induction on the actual source computation composes these bounds, including adaptive query arguments. No caller supplies a state invariant or equal cache/history sizes.

Lean: [runBallotFiniteProgrammed_storage_le](#).

Actual repaired source and random execution. For the existing two-candidate source over finite F , any g, pk , vote and attacker with total-query bound m for every prefix view, the previously derived source bound is $m + 19$. From the actual empty shadow state, every supported output therefore has at most $m + 19$ cache entries and $m + 19$ history entries. The same conclusion holds for the complete probabilistic execution with the finite live oracle. Pinned VCVio’s support-inclusion theorem derives the latter from the source bound: an output of the simulated live execution is an output in source support. No source-correspondence premise is assumed.

Lean: [repairedSubmissionFinite_shadow_storage_le](#).

Lean: [repairedSubmissionFinite_shadow_runtime_storage_le](#).

The bound charges retained entries per source request. It does not require a one-query scalar sampler and does not bound the sampler’s execution cost.

Controls and a validation correction. Repeated programming of the same key leaves one cache entry and two history entries, including on a collision. A subsequent fresh request leaves two and three. These states refute equality of the two sizes and history deduplication. The original retained-answer and sticky-flag controls remain in force.

Lean: [BallotFiniteProgrammedControls.storage_collision_offset](#).

Lean: [BallotFiniteProgrammedControls.fresh_storage_growth](#).

The independent dictionary/list harness retains its 69,904 transition comparisons and checks 17,472 further size-offset transitions from four initial states. A duplicate-insertion mutant initially escaped comparison because dictionary projection erased multiplicity. Comparing stored entry counts corrects this validation weakness. All four overwrite, duplicate-insertion, flag-reset and history-deduplication mutants are now detected before the main enumeration. These finite checks do not validate cryptographic transcript distributions.

Computational interface. The next subsection gives output bounds and a tape-size counterexample.

1.23 Historical component: Submission output and tape operands

Theorem 1.27 (Actual retained output bounds). *For the existing finite-field two-candidate source, any g, pk , vote and raw attacker, every supported submission output has at most two honest-prefix board entries and three final-board entries. No acceptance, collision-free or attacker-query-bound premise is required. This holds in source support and in the complete probabilistic execution through the finite shadow and live oracles.*

Lean: `repairedSubmission_output_board_lengths`.

Lean: `repairedSubmissionFinite_output_board_lengths`.

Each actual submission appends at most one entry; proof and reuse rejection retain the board. Each two-candidate ballot stores two ciphertext pairs and three proofs, each with two branches. Its group record retains 16 ciphertext/commitment coordinates; its scalar record retains 12 challenges/responses. Thus for prefix board H , submitted ballot and final board B , the projected counts are

$$16(|H| + 1 + |B|) \leq 96, \quad 12(|H| + 1 + |B|) \leq 72.$$

All three decisions and at most five scheduled voter identifiers remain in the structured output. These projections are element lists, not a binary serializer or a cost bound for evaluating finite-index ballot functions.

Lean: `submissionRecord_lengths`.

Lean: `repairedSubmissionFinite_output_cells_le`.

Literal records and rejection controls. Independent records fix group fields $1, \dots, 16$ and scalar fields $21, \dots, 32$. Changing an aggregate response changes the scalar record even when group records agree. Existing actual-oracle controls retain accepted append and both rejection cases. The independent 27-history board fixture detects appending on rejection and dropping prior entries; decisions are supplied in this finite shape test, rather than derived cryptographically.

Lean: `RepairedOutputStorageControls.literal_records`.

Lean: `RepairedOutputStorageControls.scalar_mutation_observed`.

Counterexample 1.28 (Event count does not bound binary uniform tags). *For each natural k , the computation querying `unifSpec` at index 2^k once, then returning unit, has total-query bound one. Its actual tape collection supports the one-event tape $[(2^k, 0)]$. That tag has $k + 1$ bits in Mathlib’s binary natural representation. Hence no function of event count alone bounds the binary tag sizes of all these supported tapes.*

Lean: `BallotTapeOperandControls.wide_tape`.

Lean: `BallotTapeOperandControls.no_count_only_tag_bound`.

The small coin fixture has the single tag bit `[true]`. Independent integer-width fixtures cover powers of two and their predecessors for exponents 0–256. The counterexample concerns actual tape collection and binary tag width; it does not assert a lower bound for every possible compressed representation.

Execution interface. The next subsection encodes uniform operands. Their construction and reading still require costs through the concrete attacker/machine interface.

1.24 Historical component: Canonical binary uniform operands

The actual uniform event type is a range index n and answer $a \in \text{Fin}(n + 1)$. The codec preserves that type and imposes no new range restriction. Write $b(n)$ for binary width and $\text{bits}_{LE}(n)$ for Mathlib’s little-endian digits. Define the natural record and event record by

$$N(n) = 1^{b(n)} \parallel [0] \parallel \text{bits}_{LE}(n), \quad U(n, a) = N(n) \parallel N(a).$$

Here $\parallel$ concatenates bit lists. The unary prefix records the binary width, not the natural value. Zero has record $[0]$. The reader counts prefix ones, reads that many payload bits and rejects a noncanonical binary payload.

Theorem 1.29 (Exact prefix and complete event decoding). *For every natural n and suffix s , reading $N(n) \parallel s$ returns (n, s) . Conversely, whenever reading word w returns (n, s) , $w = N(n) \parallel s$. The complete event decoder reads two such records, checks $a < n + 1$ and rejects any remaining bits. For every actual uniform event e ,*

$$\text{decode}(U(e)) = \text{some}(e), \quad \text{decode}(w) = \text{some}(e) \implies w = U(e).$$

Lean: `uniformNatRead_encode`.

Lean: `uniformNatRead_exact`.

Lean: `uniformEventDecode_encode`.

Lean: `uniformEventDecode_exact`.

The inverse uses the existing `Nat.bit` constructor, with its digit round trip proved by Mathlib binary induction. No finite-type enumeration is used. The exact bit lengths are

$$|N(n)| = 2b(n) + 1, \quad |U(n, a)| = 2b(n) + 2b(a) + 2.$$

Lean: `uniformNatEncode_length`.

Lean: `uniformEventEncode_length`.

Independent controls. The zero event encodes as 00; coin replies zero and one encode as 1010 and 101101. Checked controls retain arbitrary suffix bits at a natural-record boundary and reject truncation, padded zero, an answer above its declared range, and ignored trailing input. The supported wide-tag event $(2^k, 0)$ has exactly $2k + 4$ encoded bits and decodes back exactly.

Lean: `UniformOperandCodecControls.literal_events`.

Lean: `UniformOperandCodecControls.malformed_events`.

Lean: `UniformOperandCodecControls.supported_wide_event`.

An independent parser gate checks 2,145 valid events with range indices 0–64 and all 8,191 words of length at most 12, after detecting three deliberately permissive decoders. It validates finite fixtures, not cryptographic sampling.

Backend choice and interface. The pinned optional backend supplies unary naturals and fixed-width bit vectors, but no binary-natural constructor or general oracle-machine adequacy. The narrow implementation here uses Mathlib binary operations in the root build, without new dependencies. The next subsection completes scalar and saved-tape encoding. Section 1.32 supplies the primitive group codec; assembled state records, local costs and adequacy are outside this component’s guarantee. Exact decoding does not certify polynomial time or discharge the full election/DDH secrecy reduction.

1.25 Historical component: Historical scalars and complete saved bit tapes

For $q > 0$, a scalar $a \in \mathbb{Z}/q\mathbb{Z}$ encodes as $N(a.\text{val})$. The decoder rejects trailing bits and values at or above q , rather than reducing them modulo q . It is a round trip, every accepted word is canonical, and scalar length is at most $2b(q-1) + 1$.

Lean: `scalarDecode_encode`.

Lean: `scalarDecode_exact`.

Lean: `scalarEncode_length_le`.

The actual lowered entropy specification has uniform draws and Unit-indexed scalar challenges; its tapes contain no group keys. Define event payloads by $E(n, a) = 0 \parallel U(n, a)$ and $E(c) = 1 \parallel N(c.\text{val})$, respectively. For tape $T = [e_1, \dots, e_k]$, with $l_i = |E(e_i)|$, define

$$B(T) = N(k) \parallel \big\|_{i=1}^k (N(l_i) \parallel E(e_i)), \quad |B(T)| = 2b(k) + 1 + \sum_{i=1}^k (2b(l_i) + 1 + l_i).$$

The decoder checks tags, payload lengths, event count and complete consumption.

Theorem 1.30 (Canonical complete entropy tape). *For positive q and every actual tape T , decoding $B(T)$ returns $\text{some}(T)$. Whenever decoding an arbitrary word w returns $\text{some}(T)$, $w = B(T)$.*

Lean: `replayTapeDecode_encode`.

Lean: `replayTapeDecode_exact`.

Lean: `replayTapeEncode_length`.

Integration with the actual extractor. The direct collector performs each source query and collects encoded frames and a count. It equals encoding the old typed collector's result. The new extractor decodes this saved word and resumes all three existing finite replay attempts. For prime q , an additive group G with a $\mathbb{Z}/q\mathbb{Z}$ module action and decidable equality, the existing scalar sampler, every g, pk , vote, attacker and replay bound n , the new and existing repaired extractors are exactly equal as oracle computations. This retains outputs, randomness and failure behavior; there is no caller-supplied codec-validity premise.

Lean: `replayBitsCollect_eq`.

Lean: `repairedSubmissionBits_extract_eq`.

Controls and interface. The empty tape is 0; a single zero challenge is 1011100110. Literal controls also reject the scalar modulus, wrong counts, truncated frames and trailing bits.

Lean: `ReplayBitsControls.literal_frames`.

Lean: `ReplayBitsControls.malformed_frames`.

An independent gate checks 4,760 tapes, 14,278 malformed variants and 32,767 words through length 14, after detecting three decoder mutants. Continuations remain `OracleComp`; the next subsection addresses sampler enumeration.

1.26 Historical component: Explicit prime scalar and nonce sampling

Pinned VCVio already supplies a computable `FinEnum` instance for $\mathbb{Z}/q\mathbb{Z}$, using Mathlib’s explicit `Fin` equivalence. For prime q , the existing scalar sampler uses at most one uniform-index query. The actual repaired extractor consequently retains its bound $16(m + 19)$ when every attacker view has total query bound m , with this existing sampler and no additional sampler-cost premise. Group/module and decidable-equality assumptions are unchanged.

Lean: `primeScalarSampler_total_bound`.

Lean: `repairedSubmission_joint_extract_prime_total_bound`.

Historical nonzero nonces. The generic historical sampler chooses a field unit through a non-computable finite-type equivalence. The explicit prime-field replacement is

$$S_q : i \leftarrow \text{uniform}(\text{Fin}(q - 1)); \quad \text{return } ((i.\text{val} + 1) : \mathbb{Z}/q\mathbb{Z}).$$

The correspondence bijection maps i to the unit with value $i + 1$; its inverse is the canonical residue value minus one. Units are used in the proof only; the executed output map performs successor and cast.

Theorem 1.31 (Contextual nonce-distribution preservation). *Let q be prime. For every result type A , continuation $K : \mathbb{Z}/q\mathbb{Z} \rightarrow \text{ProbComp}(A)$ and result x ,*

$$\Pr[S_q \gg= K = x] = \Pr[\text{sampleNonzero}(\mathbb{Z}/q\mathbb{Z}) \gg= K = x].$$

The same statement holds for two independent uses of S_q , the actual historical `drawNoncePair`, and every continuation from a scalar pair.

Lean: `samplePrimeNonzero_probability`.

Lean: `samplePrimeNonzero_bind`.

Lean: `drawPrimeNoncePair_bind`.

The explicit nonce sampler has at most one uniform-index interaction.

Lean: `samplePrimeNonzero_total_bound`.

Controls and integration boundary. At $q = 5$, explicit indices enumerate residues 1, 2, 3, 4. Zero is absent; the independent pair (1, 2) has probability 1/16. Reusing one uniform nonce for both positions gives that pair probability zero.

Lean: `PrimeSamplersControls.literal_enumeration`.

Lean: `PrimeSamplersControls.independent_pair`.

Lean: `PrimeSamplersControls.reused_pair_rejected`.

An independent gate checks 38 residues and 412 ordered pairs over five small primes, after detecting four sampler mutants. The next subsection integrates the explicit sampler into the submission source. Distribution equality does not assert identical enumeration order or query trees, so this sampler lemma alone does not justify replay replacement. Both samplers still use ideal uniform-index draws.

1.27 Historical component: Explicit nonces in the complete submission source

Fix prime q , an additive group G with a $\mathbb{Z}/q\mathbb{Z}$ module action and decidable equality. The existing prime scalar sampler is used throughout. Factor the two honest nonce-pair draws in the submission source into a parameter D . All proof requests, verification, adaptive attacker calls and rejection branches use their existing definitions. Supplying the historical `drawNoncePair` recovers the original source as exact oracle-computation equality. Supplying the explicit sampler defines the new source P_{exp} .

Lean: `repairedSubmissionWithNoncePairs_original`.

Complete finite runtime. Let $R_{s,c}(P)$ execute P through the actual finite programmed interpreter and finite live-cache interpreter, starting from shadow state s and live cache c . Its result includes the submission, final shadow state and final live cache. Shadow state contains its physical cache entries, sticky collision flag and ordered proof-request history. The submission includes both honest decisions, the retained prefix, malicious ballot, submission decision and final board.

Theorem 1.32 (Contextual full-runtime preservation). *For every generator and public key, vote, adaptive raw attacker, initial states s, c , result type A , probabilistic continuation K on the complete runtime result and result x ,*

$$\Pr[R_{s,c}(P_{\text{exp}}) \gg= K = x] = \Pr[R_{s,c}(P_{\text{old}}) \gg= K = x].$$

No acceptance, cache-consistency or freshness premise is imposed on s, c . In particular the actual empty-initial-state runtime has the same complete law.

Lean: `repairedSubmissionPrime_runtime_bind`.

Lean: `repairedSubmissionPrime_runtime`.

The proof derives state transparency of uniform draws through both interpreters, then uses the nonce-pair continuation theorem at the two real sampling positions.

Lean: `drawPrimeNoncePair_finite_state`.

Lowered raw source. The computable definition `repairedSubmissionPrimeSourceOracle` lowers this source with the finite programmed handler. Its returned submission and live-cache distribution agree with the old finite source, by projection of the stronger complete-state result.

Lean: `repairedSubmissionPrimeSource_runtime`.

Controls and replay boundary. Actual nonce execution retains a prior collision, duplicate request history and both cached answers; the empty-state control remains unflagged and empty.

Lean: `PrimeSubmissionControls.nonce_state_retained`.

Lean: `PrimeSubmissionControls.empty_nonce_state`.

Independent synthetic continuation fixtures cover 23,136 nonce tuples and detect correlated pairs, state resets and discarded rejection branches. They validate that bounded continuation, not the historical cryptographic algorithms. The next subsection proves replay directly for the explicit source. Output-law equality is used for cached validity, not as a replay correspondence.

1.28 Historical component: Joint replay for the explicit source

The existing generic replay theorem is applied to the explicit source itself. Fix prime q , $g \neq 0$, the group/module structure and decidable group equality. An attacker with per-view raw hash bound n gives the actual explicit source a syntactically derived bound $n + 15$; uniform nonce draws do not consume it.

Lean: `repairedSubmissionPrimeSource_query_bound`.

The checked result/cache law transfers supported cached-proof validity. On the explicit source's own original paths, each selected proof verifies exactly on joint acceptance. The derived budget enables its selector, and generic reachability yields its physical replay occurrence. No old path is supplied.

Lean: `repairedSubmissionPrimePath_selector`.

Lean: `repairedSubmissionPrimePath_locations`.

Actual saved-bit extractor. `repairedSubmissionPrimeBits_extract` collects encoded entropy from this source and runs all three finite replay attempts. It equals generic joint replay of this same source. Each successful output contains its original accepted submission and witnesses for both components and their aggregate. For injective $r \mapsto r \bullet g$ and $q > 2$, those witnesses satisfy nonce addition and the integer at-most-one-vote constraint.

Lean: `repairedSubmissionPrimeBits_extract_eq`.

Lean: `repairedSubmissionPrimeBits_extract_valid`.

Lean: `repairedSubmissionPrimeBits_extract_consistent`.

Theorem 1.33 (Explicit-source joint extraction bound). *Assume generator exponentiation is injective and the per-view attacker hash bound is n . Let B be the historical programmed submission's actual malicious acceptance probability, $\nu_q = 1/(q - 1)$, and*

$$\ell = 9\nu_q + \text{ofReal}(90/q), \quad a = (B - \ell)_+.$$

For every $\delta \in \mathbb{R}_{\geq 0} \cup \{\infty\}$,

$$(a - 3(n + 16)\delta)_+ (\delta - 1/q)_+^3 \leq \Pr[\text{explicit-source saved-bit extractor succeeds}].$$

Subtractions and multiplication use extended nonnegative-real arithmetic.

Lean: `repairedSubmissionPrimeBits_extract_le`.

The three-selector probability is proved equal to actual joint acceptance; the historical rejection and simulation loss is retained. The proof applies the generic three-attempt bound anew, without asserting old/new replay equality. The threshold δ is not an algorithm input.

Interaction cost and remaining boundary. For per-view attacker total interaction bound m , the explicit original program, lowered source and saved-bit extractor use at most $m + 19$, $4(m + 19)$ and $16(m + 19)$ interactions. The last bound includes all three attempts and uses the existing computable scalar sampler, with no sampler-cost premise.

Lean: `repairedSubmissionPrimeBits_extract_total_bound`.

Independent nonce/state/rejection controls and the context/tape gates are retained, including physical-index, repeated-occurrence and failed-attempt cases. These are interaction bounds, not machine-time bounds. The following subsections implement finite fair-bit sampling with explicit error.

1.29 Historical component: Finite fair-bit sampling boundary

The current scalar and nonce samplers use ideal uniform finite-index draws. Their implementation cannot silently replace these by a terminating fair-bit program with the same law when the range has odd prime size.

Theorem 1.34 (Derived dyadic probabilities). *Let $P : \text{OracleComp coinSpec } A$ be any actual terminating program whose only queries return a fair Boolean. There exists a natural query bound n for P . For every such bound and every $x : A$, there exists $k \in \mathbb{N}$ satisfying*

$$\Pr[P = x] 2^n = k.$$

Thus for every prime $q > 2$ and every x , $\Pr[P = x] \neq 1/q$. The maximum coin count is derived from P , not supplied as a premise of the impossibility result.

Lean: `coinProgram_has_bound`.

Lean: `coinProgram_dyadic`.

Lean: `coinProgram_not_prime_uniform`.

Induction takes the maximum of the two continuation bounds at a coin query. If $\Pr[P = x] = 1/q$, the integer-numerator result implies $q \mid 2^n$, hence $q \mid 2$, contradicting primality and $q > 2$.

Independent controls. One fair bit is exactly uniform on two outcomes; two independently drawn bits are exactly uniform on four ordered pairs. Mapping their encoded values 0, 1, 2, 3 modulo three yields masses $(1/2, 1/4, 1/4)$, not a uniform distribution. The impossibility result specializes to every coin-only program returning Fin 3.

Lean: `CoinDenominatorsControls.uniform_two`.

Lean: `CoinDenominatorsControls.uniform_four`.

Lean: `CoinDenominatorsControls.modulo_biased`.

Lean: `CoinDenominatorsControls.uniform_three_impossible`.

Independent rational fixtures enumerate binary-tree masses through depth four and rejection cutoffs through eight attempts. They detect uniform-modulo and zero-cutoff-failure mutations. These finite checks supplement the general proof.

Representation decision. This is a representation obstruction, not a ballot-secrecy counterexample. Exact rejection sampling permits unbounded execution tails and needs a different representation. A finite rejection cutoff requires an explicit loss and fallback. The selected route uses extra fair bits followed by modulo reduction, with its statistical error retained. It returns an in-range value without adding a public abort. The next two subsections prove its law, contextual error and adaptive transfer into the actual extractor. The historical ideal sampling law and rejection behavior remain fixed.

1.30 Historical component: Actual extra-bit modulo sampler

For width w , the program draws w independent fair bits and encodes them in little-endian order with Mathlib’s computable binary-digit equivalence. It returns an index in $\text{Fin}(2^w)$ with exact point mass 2^{-w} .

Lean: `sampleFairBitIndex_probability`.

For a positive range q , the actual modulo sampler returns this index modulo q , with its range proof. It always returns a value; no public abort is added.

Theorem 1.35 (Exact residue law and sampling error). *Let $q > 0$, $M = 2^w$, $d = \lfloor M/q \rfloor$ and $r = M \bmod q$. For every $y < q$,*

$$\Pr[\text{sampleFairBitModulo}(q, w) = y] = \frac{d + \mathbf{1}_{y < r}}{M}.$$

Its total variation distance from the ideal uniform distribution U_q is at most r/M . With $w = \text{size}(q) + s$, where size is binary width,

$$\text{TV}(\text{sampleFairBitRange}(q, s), U_q) \leq 2^{-s}.$$

Lean: `sampleFairBitModulo_probability`.

Lean: `sampleFairBitModulo_tv_le`.

Lean: `sampleFairBitRange_tv_le`.

The proof refills the final incomplete block with an independent ideal draw. That proof-only distribution is exactly uniform. It agrees with the actual sampler off the final block, whose mass is r/M . The executed sampler performs modulo reduction throughout. Since $q < 2^{\text{size}(q)}$, the selected width gives the stated bound without assuming a sampler law.

Contextual meaning and query cost. For every common probabilistic continuation $K : \text{Fin } q \rightarrow \text{SPMF } A$, the same 2^{-s} bound holds after binding both distributions to K . It covers K ’s complete output, including retained state and observations. The actual sampler uses at most $\text{size}(q) + s$ fair-bit requests. Arithmetic, finite-function encoding and machine costs are separate obligations.

Lean: `sampleFairBitRange_bind_tv_le`.

Lean: `sampleFairBitRange_query_bound`.

Controls. Literal two-bit inputs fix digit order. Range four with width two is exactly uniform. Range three with slack three has masses $11/32, 11/32, 10/32$; it retains bias while meeting the five-bit and $1/8$ error bounds.

Lean: `FairBitSamplerControls.literal_digit_order`.

Lean: `FairBitSamplerControls.power_of_two_exact`.

Lean: `FairBitSamplerControls.extra_bits_still_biased`.

Lean: `FairBitSamplerControls.literal_budget`.

Independent rational fixtures check 1,377 masses, 4,590 events and 3,341 width cases, detecting exact-modulo, reused-bit, missing-high-bit and unspent-slack mutations. These finite controls supplement the general proofs.

1.31 Historical component: Adaptive transfer to the joint extractor

The interpreter replaces every requested $\text{Fin}(t+1)$ draw by the sampler with width $\text{size}(t+1) + s$. The next range may depend on previous answers. For any actual $P : \text{ProbComp } A$ with total query bound N ,

$$\text{TV}(\text{runFairBitUniform}(s, P), P) \leq N2^{-s}.$$

Independent support induction proves that every returned output was reachable in P . Boolean observations inherit the error bound and its truncated-subtraction form. Neither a range-size bound nor a correspondence is a caller premise.

Lean: `runFairBitUniform_tv_le`.

Lean: `runFairBitUniform_support_subset`.

Lean: `runFairBitUniform_event_loss`.

An actual two-request control chooses the second range from its first answer; its complete pair meets the two-step bound and retains the selected range. Six independent adaptive fixtures supplement this control.

Lean: `FairBitUniformControls.adaptive_distance`.

Lean: `FairBitUniformControls.adaptive_range_preserved`.

Actual Helios replay implementation. The explicit extractor still distinguishes uniform-index and scalar-challenge queries. Its existing entropy interpreter lowers both to finite-range draws, with exact distribution preservation and a derived one-query step bound. Thus the earlier $16(m+19)$ interaction bound applies after lowering. The new fair-bit extractor applies the interpreter above to this actual program. For prime q , the existing group/module structure and decidable group equality,

$$\text{TV}(\text{FairExtractor}_s, \text{IdealExtractor}) \leq 16(m+19)2^{-s},$$

where m bounds every attacker-prefix view's total queries.

Lean: `repairedSubmissionFairBits_extract_tv_le`.

For $g \neq 0$, every successful output still contains its original accepted ballot and all three valid witnesses. Generator injectivity and $q > 2$ retain nonce addition and the integer at-most-one-vote constraint.

Lean: `repairedSubmissionFairBits_extract_valid`.

Lean: `repairedSubmissionFairBits_extract_consistent`.

Theorem 1.36 (Fair-bit extraction success). *Assume generator injectivity, per-view hash bound n and total bound m . With a and the nonnegative extended-real δ from Section 1.28, set $E_s = \text{ofReal}(16(m+19)2^{-s})$. Then*

$$\left((a - 3(n+16)\delta)_+ (\delta - 1/q)_+^3 - E_s \right)_+ \leq \Pr[\text{FairExtractor}_s \text{ succeeds}].$$

Lean: `repairedSubmissionFairBits_extract_le`.

The historical ideal game, observations and rejection behavior stay fixed; the sampling loss remains explicit. This bound is not a PPT or privacy theorem.

1.32 Historical component: Public group coordinates and canonical records

The group representation needed by encoded keys, caches and boards now uses Mathlib's q th roots of unity in $\mathbb{Z}_p$, with the additive wrapper used by the existing protocol. Write $C(x)$ for the public residue coordinate. No discrete-log index is stored. For $q > 0$,

$$C(0) = 1, \quad C(x + y) = C(x)C(y), \quad C(r \bullet x) = C(x)^{\text{val}(r)}.$$

The coordinate map is injective; for prime p , $C(-x) = C(x)^{-1}$.

Lean: `primeGroupCoordinate_injective`.

Lean: `primeGroupCoordinate_smul`.

Lean: `primeGroupCoordinate_neg`.

For prime q and $C(g) \neq 1$, scalar action on g is injective. If p is also prime, the subgroup has exactly q elements, derived from this generator.

Lean: `primeGroup_generator_injective`.

Lean: `primeGroup_card`.

The actual encryption algorithm specializes, for prime q , to

$$(C(\text{encryptWith}(g, pk, r, m)_1), C(\text{encryptWith}(g, pk, r, m)_2)) = (C(g)^{\text{val}(r)}, C(g)^{\text{val}(m)}C(pk)^{\text{val}(r)}).$$

Lean: `primeGroup_encrypt_coordinates`.

Theorem 1.37 (Internal group records). *Let $p, q > 0$. Encode a group element by the canonical scalar record of $C(x)$. The decoder requires an in-range residue a with $a^q = 1$. Then*

$$D(E(x)) = \text{some}(x), \quad D(w) = \text{some}(x) \implies w = E(x), \quad |E(x)| \leq 2 \text{size}(p - 1) + 1.$$

Lean: `primeGroupDecode_encode`.

Lean: `primeGroupDecode_exact`.

Lean: `primeGroupEncode_length_le`.

Canonical nonmembers and out-of-range naturals reject; accepted encodings are injective. Identity coordinate one remains valid.

Lean: `primeGroupDecode_nonmember`.

Lean: `primeGroupDecode_outOfRange`.

Lean: `primeGroupEncode_injective`.

For $p = 23, q = 11, g = 2$, the checked instance has $pk = 8$ and ciphertext $(16, 4)$ for message one and nonce four. Controls accept identity and reject zero, nonmember 22, alias 24 and trailing input. Independent integer arithmetic validates the same equations and three decoder mutations on small prime groups.

Lean: `PrimeGroupControls.literal_group`.

Lean: `PrimeGroupControls.literal_encryption`.

Lean: `PrimeGroupControls.literal_records`.

Lean: `PrimeGroupControls.malformed_records`.

This is an internal codec for the existing typed subgroup. The historical raw-input policy, encoded execution costs and machine adequacy are outside this component's guarantee. Mathlib supplies modular-power algorithms; this result does not supply their bit-machine costs or prove computational ballot secrecy.

1.33 Historical component: Complete key and finite-cache bit records

For $p, q > 0$, the eight-field key layout from Section 1.21 now composes with the public-coordinate codec. The ordered cache record stores every complete key and scalar answer. Its decoder checks unique keys and rejects duplicates even when their answers agree. No deduplication occurs.

Lean: `ballotKeyBits_roundTrip`.

Lean: `ballotKeyBits_exact`.

Lean: `ballotCacheBits_duplicate`.

Let E_C, D_C encode and decode the actual finite association list. Then

$$D_C(E_C(c)) = \text{some}(c), \quad D_C(w) = \text{some}(c) \implies w = E_C(c).$$

The encoder is injective as a complete ordered cache object. In particular, different entry orders cannot share an encoding even if their lookups agree.

Lean: `ballotCacheBits_roundTrip`.

Lean: `ballotCacheBits_exact`.

Lean: `ballotCacheBits_injective`.

A shared list/pair framing layer stores field counts and payload lengths using the existing binary natural codec and requires complete input consumption. Its concrete instances discharge both canonicity laws.

Lean: `bitFieldsDecode_exact`.

Encoded lookup decodes the cache and full key, then performs the original finite lookup. Malformed input returns `none`; a valid cache miss returns `some(none)`. Encoded insertion also decodes its scalar answer and returns exactly the encoding of the original cache insertion.

Lean: `ballotCacheBitsLookup_encode`.

Lean: `ballotCacheBitsInsert_encode`.

Complete record bounds. Write $F(b) = 2 \text{size}(b) + 1 + b$ for payload framing, and set

$$\begin{aligned} L(p) &= 2 \text{size}(p - 1) + 1, & K(p) &= 9 + 8F(L(p)), \\ E(p, q) &= 5 + F(K(p)) + F(L(q)), & B(p, q, n) &= 2 \text{size}(n) + 1 + nF(E(p, q)). \end{aligned}$$

The key, entry and n -entry cache lengths are bounded by $K(p)$, $E(p, q)$ and $B(p, q, n)$ respectively, including all framing and scalar answers.

Lean: `ballotKeyBits_length_le`.

Lean: `ballotCacheEntryBits_length_le`.

Lean: `ballotCacheBits_length_le`.

For prime p, q and attacker per-view hash bound n , every supported execution of the explicit-nonce source through the finite logged interpreter has an encoded cache of length at most $B(p, q, n + 15)$. The source's own query budget derives the entry bound; no old/new replay-path correspondence is assumed.

Lean: `repairedSubmissionPrime_cache_bits_le`.

Literal controls retain all eight key fields, cache hits and misses, agreeing duplicate rejection, entry order and trailing-input rejection. Independent integer fixtures check the framing and bounds and detect three decoder defects.

Lean: `BallotCacheCodecControls.literal_lookup`.

Lean: `BallotCacheCodecControls.entry_order_preserved`.

Sections 1.34–1.35 assemble state and output records.

1.34 Historical component: Complete finite interpreter data

The logged-state codec stores its cache and chronological full-key miss log. The programmed-state codec stores the shadow cache, sticky flag and statement history in source order (most recent first), including repetitions. The both-cache codec adds the separate live cache.

For $p, q > 0$, these complete objects round-trip and admit only canonical words. Unit log tags have one possible value; Boolean records require one bit.

Lean: [ballotLoggedBits_roundTrip](#).

Lean: [ballotLoggedBits_exact](#).

Lean: [ballotProgrammedBits_roundTrip](#).

Lean: [ballotProgrammedBits_exact](#).

Lean: [ballotBothCachesBits_roundTrip](#).

Lean: [ballotBothCachesBits_exact](#).

Actual programming transition. For prime q , encoded programming decodes state and applies the original sampled-transcript transition, returning its typed proof and encoded successor. Statement/transcript inputs remain typed. Occupied keys retain old answers, set the flag and record each request, even on challenge agreement.

Lean: [ballotProgrammedBitsApply_encode](#).

Complete data bounds. With F, L, K, B from Section 1.33, write

$$P(a, b) = 5 + F(a) + F(b), \quad V(a, n) = 2 \text{size}(n) + 1 + nF(a).$$

For fixed p, q , define the statement, logged, programmed and both-cache bounds

$$\begin{aligned} S &= P(P(L(p), L(p)), P(L(p), L(p))), \\ J(c, l) &= P(B(p, q, c), V(K(p), l)), \\ T(c, h) &= P(B(p, q, c), P(1, V(S, h))), \\ U(c, h, l) &= P(T(c, h), B(p, q, l)). \end{aligned}$$

Every count, field prefix, payload and flag is included. For prime p, q and per-view attacker hash bound n , actual supported execution of the explicit replay source through the finite logged interpreter yields encoded state length at most $J(n + 15, n + 15)$.

Lean: [repairedSubmissionPrime_logged_bits_le](#).

With total bound m as well, the complete explicit-nonce probabilistic runtime yields both-cache data length at most $U(m + 19, m + 19, n + 15)$. Its own syntax and runtime support derive the shadow, history and live-cache counts.

Lean: [repairedSubmissionPrime_bothCaches_bits_le](#).

Controls decode an occupied-key transition: answer three remains when seven was sampled, with true flag and repeated history. Further controls distinguish erased flags/live caches, reversed logs and malformed flags. Independent fixtures cover fresh/occupied keys, both initial flags and every small-field challenge.

Lean: [BallotStateCodecControls.encoded_occupied_transition](#).

Lean: [BallotStateCodecControls.live_cache_not_erased](#).

Lean: [BallotStateCodecControls.chronological_log_not_reversed](#).

These records cover finite interpreter data; Section 1.35 assembles protocol outputs. Continuation execution and machine adequacy are outside this component's guarantee. Record sizes do not establish election integration, DDH or all-PPT secrecy.

1.35 Historical component: Complete outputs and public observations

For $p, q > 0$, canonical bit records now cover complete proofs, two-component ballots, ordered voter-identified boards and submission results. Both branches of every component and aggregate proof remain present. Decisions have three distinct two-bit tags; the unused tag rejects. For each codec (E, D) ,

$$D(E(x)) = \text{some}(x), \quad D(w) = \text{some}(x) \implies w = E(x).$$

Lean: `ballotBits_roundTrip`.

Lean: `ballotBits_exact`.

Lean: `ballotBoardBits_roundTrip`.

Lean: `ballotSubmissionBits_exact`.

Public prefix/result records also retain parameters, key proof, ordered candidate/voter lists, fingerprint, honest decisions, encrypted tallies, shares, decryption proofs and optional decoded tallies. Decoding failure differs from zero. Complete public-result encoding is injective.

Lean: `ballotPublicPrefixBits_roundTrip`.

Lean: `ballotPublicResultBits_roundTrip`.

Lean: `ballotPublicResultBits_exact`.

Lean: `ballotPublicResultBits_injective`.

For prime q and positive p , encoded submission inputs run the original shared-oracle verifier, preserving its queries, rejection priority and board. The finishing wrapper uses the original typed hashes and private inputs.

Lean: `repairedSubmitBitsOracle_encode`.

Lean: `finishRepairedElectionBits_encode`.

These internal interfaces leave the historical raw-input policy open. Well-typed invalid ballots decode and are still rejected by the verifier.

Lean: `BallotOutputCodecControls.representation_not_validity`.

Actual runtime data bound. For fixed p, q , use L, P, V, U from the preceding sections and abbreviate

$$\begin{aligned} C &= P(L(p), L(p)), & R &= P(C, P(L(q), L(q))), & H &= P(R, R), \\ A &= P(P(C, C), P(P(H, H), H)), & B_o(n) &= V(P(5, A), n), \\ S_o(n, k) &= P(P(P(2, 2), B_o(n)), P(A, P(2, B_o(k)))). \end{aligned}$$

For prime p, q , any g, pk , vote and per-view attacker with hash bound n and total query bound m , every supported word of the actual internal bit-result runtime has length at most

$$P(S_o(2, 3), U(m + 19, m + 19, n + 15)).$$

Actual source support derives both board counts. Decoding the bit-result program exactly recovers the original probabilistic computation, including both caches, history and flag. These private data remain internal.

Lean: `repairedSubmissionPrime_output_bits_le`.

Lean: `repairedSubmissionRuntimeBits_decode`.

Lean: `repairedSubmissionRuntimeBits_length_le`.

Independent integer fixtures detect twelve public-field changes and check 36 submission-size cases. Arbitrary natural metadata retains its own width; no uniform fingerprint bound is assumed.

1.36 Historical component: Concrete cache execution primitives

Comparison and copying use Mathlib TM2 with finite local callbacks and already-loaded inputs. Write T for the actual step, fixing halted configurations.

Complete-key comparison. The comparator uses two binary stacks and nine local states. It pops both heads, continuing only on matching nonempty heads. For arbitrary words x, y and memory v , the configuration $c = T^{|x|+1}(I(x, y, v))$ satisfies

$$c.\text{label} = \text{none} \wedge a(c) = \text{decide}(x = y),$$

where $a(c)$ compares the final saved optional heads.

Lean: [BitCompareMachine.run](#).

Lean: [BitCompareMachine.memory_card](#).

For $p, q > 0$ and arbitrary original typed keys u, v , canonical full-key injectivity and the bound $K(p)$ of Section 1.33 derive some fuel $n \leq K(p) + 1$ whose terminal answer is $\text{decide}(u = v)$.

Lean: [BitCompareMachine.key_run](#).

Source-preserving copying. Comparison consumes its inputs. Copying uses three binary stacks, two labels and optional-bit memory: move source to empty scratch, then restore source and push each bit onto destination. Let $C(\ell, x, y, z, v)$ contain label, source, destination, scratch and memory. For arbitrary x, y, v ,

$$T^{2|x|+2}(C(\text{collect}, x, y, [], v)) = C(\text{halt}, x, x ++ y, [], \text{none}).$$

Lean: [BitCopyMachine.run](#).

This conclusion preserves exact stack contents, including order and suffix. For positive p, q , a complete encoded key can therefore be copied with fuel at most $2K(p) + 2$.

Lean: [BitCopyMachine.key_copy](#).

For prime p, q , any g, pk , vote and attacker with per-view hash bound n , take an actual supported result of the empty-initialized explicit-nonce finite logged source. Its complete encoded cache has the same copy guarantee with some fuel at most $2B(p, q, n + 15) + 2$, derived from the source's query/storage bound.

Lean: [BitCopyMachine.source_cache_copy](#).

Controls and scope. Controls retain late mismatches, strict prefixes, nonpalindromic copying, suffixes, empty inputs and the unfinished first pass.

Lean: [BitCompareMachine.prefix_control](#).

Lean: [BitCopyMachine.copy_control](#).

Lean: [BitCopyMachine.first_pass_not_copy](#).

Lean: [BitCopyMachine.no_eraser_or_reversal](#).

Separate exhaustive campaigns each check 16,129 pairs of words of length at most six. They detect prefix acceptance, reversed copying and erased-source mutants before checking the correct programs, with no failures or discards.

These bounds count concrete TM2 transitions. Later sections execute binary record parsing and source-cache lookup (Section 1.46).

1.37 Historical component: Concrete parsing of binary-natural prefixes

Cache counts and field lengths use the existing natural record $E(n) = 1^{\text{size}(n)} 0 \text{ bits}(n)$, with little-endian payload. The concrete TM2 parser uses four binary stacks, three labels and optional-bit memory. It stores width markers, consumes one payload bit per marker, checks the most significant bit and reverses scratch into output. Zero has an empty payload. All local callbacks have finite domains.

Let $I(w)$ load input w with empty workspace/output and initial width control; let T be the actual step, fixing halted configurations. The readout $O(c)$ returns output bits and remaining input only after successful halt. Write D for the original `uniformNatRead`.

Theorem 1.38 (Every raw input, machine-checked). *For every bit word w , there exists $t \leq 3|w| + 3$ such that $c = T^t(I(w))$ satisfies*

$$c.\text{label} = \text{none}, \quad O(c) = \text{map}((n, s) \mapsto (\text{bits}(n), s), D(w)).$$

There is no well-formedness or successful-decoding premise.

Lean: `NatPrefixMachine.total_run`.

For every natural n and suffix s , execution on $E(n) ++ s$ with fuel $3 \text{size}(n) + 3$ reaches the complete successful configuration: input s , output $\text{bits}(n)$ and empty count/scratch stacks.

Lean: `NatPrefixMachine.encoded_run`.

The last-bit test is derived from the original codec's canonicity laws; the existing decoder and format are unchanged.

Lean: `uniformNatRead_frame`.

General rejection proofs cover absent delimiters, insufficient payload and noncanonical high-zero padding. These cases exhaust malformed inputs.

Lean: `NatPrefixMachine.no_delimiter_run`.

Lean: `NatPrefixMachine.truncated_run`.

Lean: `NatPrefixMachine.noncanonical_run`.

The actual cache count prefix. The complete cache encoder supplies its outer count field; parsing it preserves every remaining entry and framing bit.

Lean: `NatPrefixMachine.cache_header_run`.

For prime p, q , any g, pk , vote and attacker with per-view hash bound n , take a supported result of the empty-initialized explicit-nonce finite logged source. The actual cache header has a concrete parsing run with some fuel at most $3 \text{size}(n + 15) + 3$, returning its exact entry count as binary digits and the untouched remainder. Source support derives the count bound.

Lean: `NatPrefixMachine.source_cache_header`.

Independent enumeration compares all 8,191 words of length at most twelve with integer prefix decoding and checks phase counts. Canonicity-omission and truncation-padding mutants are detected first; the correct campaign has no failures or discards. Kernel controls retain six and zero with suffixes, malformed input and nonreversed output.

Lean: `NatPrefixMachine.suffix_control`.

Lean: `NatPrefixMachine.malformed_controls`.

Output is binary; integer conversion is uncharged. Later sections execute decrement, record parsing, lookup and guarded insertion (Section 1.48). Raw-word correctness covers internal framing; the historical network/group-input policy is separate.

1.38 Historical component: Canonical binary decrement for field reading

The field parser needs to consume binary lengths without converting them to an uncharged machine integer. Decrement now uses the existing four-stack layout, two labels and optional-bit memory. The counter occupies the output stack; scratch is empty initially. Input and count stacks retain the surrounding data.

Borrow consumes each low zero and stores a one on scratch. At the first one, the machine inserts a zero only if higher bits remain, then restores scratch. Thus one becomes canonical empty zero; three becomes the two-bit word $[0, 1]$. Zero remains empty and reports underflow. All local callbacks have finite domains.

Let $C(\ell, b, z, s, f, v)$ contain label, counter, scratch, input, count stack and local memory. Write T for the actual TM2 step, fixing halted configurations.

Theorem 1.39 (Canonical counter execution, machine-checked). *For every natural n , arbitrary words s, f and local memory v , there exists $t \leq 2 \text{size}(n) + 1$ such that*

$$\begin{aligned} T^t(C(\text{borrow}, \text{bits}(n), [], s, f, v)) \\ = C(\text{halt}, \text{bits}(n - 1), [], s, f, \text{some}(\text{decide}(n \neq 0))). \end{aligned}$$

Here $n - 1$ is truncated natural subtraction. The conclusion preserves the entire configuration, including both surrounding stacks.

Lean: [BitCounterMachine.run](#).

A chain of k low zeros followed by a one has a concrete run with sufficient fuel $2k + 2$. The bit transformation agrees with natural predecessor on canonical words; it is not a Nat operation inside the machine.

Lean: [BitCounterMachine.borrow_chain_run](#).

Lean: [BitCounterMachine.predBits_bits](#).

Connection to the actual prefix parser. For any word with $D(w) = \text{some}(n, s)$, the preceding parser theorem supplies canonical digits, suffix s and empty workspace. The counter theorem applies to that actual configuration after changing only its finite control label.

Lean: [BitCounterMachine.parsed_run](#).

This establishes the data interface between two concrete runs. The next subsection links them into a single-field program and charges its control transitions; this counter theorem alone does not supply that composition.

Independent fixtures compare 4,341 naturals with integer subtraction and canonical binary expansion, including borrow chains through 256 bits. They first detect retained-high-zero and lost-borrow mutants. The correct campaign has no failures or discards. Kernel controls retain $8 \rightarrow 7$, $1 \rightarrow 0$, $3 \rightarrow 2$, zero underflow, the distinct flags and unchanged surrounding stacks.

Lean: [BitCounterMachine.borrow_control](#).

Lean: [BitCounterMachine.one_control](#).

Lean: [BitCounterMachine.odd_control](#).

Lean: [BitCounterMachine.underflow_distinct](#).

Execution interface. Section 1.39 executes length-controlled field reads, rejecting exhausted input without unary expansion. Multi-field framing and the outer record-count loop still need execution proofs before the enclosing cache scan.

1.39 Historical component: Linked execution of length-prefixed fields

One finite-control TM2 program now parses a length and reads its payload. It uses the same four binary stacks and optional-bit memory. Subroutine labels are renamed and halt becomes a return jump; the proved transfer retains all data and uses no more fuel than the original run. Halted padding is removed before caller execution.

Lean: [TM2ReturnLink.run](#).

The field loop checks for exhausted input before decrement. Consumed bits accumulate on the count stack; when the binary counter reaches zero, restoration puts the field in its original order. Success preserves the unconsumed suffix and clears both work stacks. For declared count n and input s , rejection of $|s| < n$ takes at most $|s|(2 \text{size}(n) + 2) + 1$ transitions, leaving the remaining binary count and reversed partial field. There is no unary expansion of the declared length.

Lean: [FieldReadMachine.run](#).

Lean: [FieldReadMachine.truncated_run](#).

Define $F(w)$ by the existing natural-prefix decoder: if it rejects, return `none`; if it returns (n, s) , return `some(taken(s), dropn(s))` when $n \leq |s|$, and `none` otherwise. Let $I(w)$ load w with empty workspace and prefix control, T be the linked program's actual step fixing halted configurations, and O return the field/suffix only after a successful halt.

Theorem 1.40 (Every raw field input, machine-checked). *For every bit word w , there exists*

$$t \leq 3|w| + 5 + (|w| + 1)(2|w| + 4)$$

such that $c = T^t(I(w))$ satisfies

$$c.\text{label} = \text{none} \quad \wedge \quad O(c) = F(w).$$

No well-formedness, successful-decoding or numeric-length bound is assumed.

Lean: [FieldPrefixMachine.total_run](#).

The bound includes prefix parsing, field execution, return jumps and final halt. The original prefix exactness law bounds the count's bit width by input length. For records accepted by the existing codec with exactly one field, dropping the three-bit outer count header supplies this program's actual framing and yields that field with an empty suffix.

Lean: [FieldPrefixMachine.singleton_record_run](#).

Independent fixtures check 6,867 count/input cases, including 257-bit counts, and 8,191 raw words of length at most twelve. Reverse-output and truncation-acceptance mutants are detected; the correct campaigns have no failures or discards. Lean controls retain suffix/order, empty fields, actual rejection halts and immediate empty-input rejection for a huge count.

Lean: [FieldReadMachine.no_padding_control](#).

Lean: [FieldReadMachine.huge_empty_control](#).

Lean: [FieldPrefixMachine.rejection_controls](#).

Execution boundary. Multi-field framing and the outer count loop are outside this component's guarantee, followed by cache scan/update, loading, arithmetic, saved contexts and quantitative adequacy. This internal field parser does not settle historical network-input policy or full-election all-PPT secrecy.

1.40 Historical component: Outer-state preservation and one record iteration

Repeated field calls must retain the outer record count and already stored data. The reader uses all four work stacks, so two extra binary stacks hold this state. Stack relocation preserves every statement result, step and run exactly, with unchanged fuel and extra-stack contents. Saving these values inside the work stacks instead would need an additional encoding and repeated save/restore costs.

Lean: `TM2StackFrame.run`.

The six-stack program first calls the field reader in its original positions. It then relocates decrement onto the extra outer-count stack, retaining the just-read field and archive. Both calls and their return jumps execute in shared control. The complete successful workspace is derived from the reader's execution, rather than assumed from equality of its readout.

Lean: `RecordFieldStepMachine.field_complete`.

Lean: `RecordFieldStepMachine.counter_run`.

Write $S(\ell, s, c, z, b, o, a, v)$ for label, input, collected bits, scratch, field, outer count, archive and local memory. Let $I(w, o, a)$ start the linked program with input w , outer count o , archive a and empty work stacks.

Theorem 1.41 (Executed record-field iteration, machine-checked). *For every field b , suffix s , archive a and natural n , there exists*

$$t \leq 3 \text{size}(|b|) + 10 + |b|(2 \text{size}(|b|) + 3) + 2 \text{size}(n + 1)$$

such that the actual fixed-halt step T satisfies

$$\begin{aligned} T^t(I(E(|b|) ++ b ++ s, \text{bits}(n + 1), a)) \\ = S(\text{halt}, s, [], [], b, \text{bits}(n), a, \text{some}(1)). \end{aligned}$$

The initial outer count is positive; the archive is arbitrary and unchanged.

Lean: `RecordFieldStepMachine.run`.

Every raw field call also preserves the two extra stacks on rejection. If the single-field decoder rejects w , the program halts before decrement within $3|w| + 6 + (|w| + 1)(2|w| + 4)$ transitions, with a non-success flag and unchanged outer count/archive.

Lean: `RecordFieldStepMachine.rejected_run`.

Independent six-stack fixtures compare 10,220 raw-field/count/archive cases against indexed decoding, slicing and integer subtraction, including 257-bit outer counts. Scratch/archive and counter/field alias mutants are detected; there are no failures or discards in the correct campaign. Lean controls retain a complete 33-step field iteration with outer count $8 \rightarrow 7$, rejection frames, zero-counter underflow and distinct field/count data.

Lean: `RecordFieldStepMachine.iteration_control`.

Lean: `RecordFieldStepMachine.rejection_frame_control`.

Lean: `RecordFieldStepMachine.no_alias_control`.

Repeated traversal. This iteration preserves the archive. Section 1.41 records original field frames and clears output for reuse. Section 1.42 loads the initial count and proves the repeated loop, including malformed later fields and trailing input, against the original multi-field decoder.

1.41 Historical component: Original-frame accumulation and workspace reuse

Each successful input pop now copies its bit onto the archive stack. This retains the original length prefix and payload, in reverse, without a second field codec. Failed pops copy nothing. A structural check on the actual program proves that it never pushes to input or accesses archive, and that input-pop callbacks store their head in finite local memory.

Lean: [ArchivedFieldMachine.eligible](#).

For eligible code, the recorded run has exactly the original control, memory, other stacks and TM2 fuel. It derives a consumed prefix d with original input $d++$ residual and archive $\text{reverse}(d)++a$. These copies execute as finite instructions inside existing TM2 macro steps.

Lean: [TM2InputArchive.run](#).

The restriction on archive access is necessary: a checked program consumes true and then branches on archive. It chooses false with the original empty archive and true after recording. That program is checked to violate eligibility.

Lean: [TM2InputArchive.archive_dependency_control](#).

Let $R(b) = E(|b|) ++ b$ be the original field frame. Use the six-stack state notation S from the preceding subsection. The new start I calls the recorded iteration, checks its return flag, and clears output on success.

Theorem 1.42 (Archived iteration with reusable workspace, machine-checked). *For every field b , suffix s , archive a and natural n , there exists*

$$t \leq 3 \text{size}(|b|) + 12 + |b|(2 \text{size}(|b|) + 4) + 2 \text{size}(n + 1)$$

such that the actual fixed-halt step T satisfies

$$\begin{aligned} T^t(I(R(b) ++ s, \text{bits}(n + 1), a)) \\ = S(\text{halt}, s, [], [], [], \text{bits}(n), \text{reverse}(R(b)) ++ a, \text{some}(1)). \end{aligned}$$

Count, scratch and output are empty; the suffix and old archive are retained.

Lean: [ArchivedFieldMachine.run](#).

For every rejected raw field w , the program halts with non-success flag and unchanged outer count within $3|w| + 7 + (|w| + 1)(2|w| + 4)$ transitions. Its archive is the reversed consumed prefix followed by the unchanged old archive; cleanup cannot accept a rejected input.

Lean: [ArchivedFieldMachine.rejected_run](#).

Independent fixtures check 8,176 raw-field/count/archive combinations, including 257-bit counts. Indexed decoding independently determines consumed-prefix length on success and rejection. Missing-header and erased-archive mutants are detected; there are no failures or discards. Lean controls retain a complete 37-step run, an empty field's boundary, absent-input behavior and exact rejected-prefix data.

Lean: [ArchivedFieldMachine.iteration_control](#).

Lean: [ArchivedFieldMachine.empty_field_control](#).

Lean: [ArchivedFieldMachine.absent_bit_control](#).

Whole records. The next subsection composes initial count loading and repeated calls, retaining original framing and reusing the cleared workspace.

1.42 Historical component: Complete counted-record execution

One finite six-stack TM2 now parses the count into the outer binary stack, records its header, repeats the archived field operation, and restores the original word. Zero count accepts only empty residual input. Every call resets local memory and checks the return flag. Missing or malformed later fields halt in rejection; no unary expansion of the numeric count occurs.

Lean: [RecordParserMachine.count_run](#).

Lean: [RecordParserMachine.loop_run](#).

Write D for the original `bitFieldsDecode`, $I(w)$ for this machine's start with input w and empty workspace, and T for its actual fixed-halt TM2 step. Readout $O(c)$ is the output stack when c is halted with success flag, and is none otherwise. Define

$$U(B, h) = 3B + 16 + (B + 1)(2B + 4) + 2h.$$

Theorem 1.43 (All-input record framing, machine-checked). *For every binary word w , with $L = |w|$, there exists*

$$t \leq 3L + 4 + (L + 1)U(L, L)$$

such that $c = T^t(I(w))$ is halted and

$$O(c) = \text{Option.map}(\lambda_.w)(D(w)).$$

Thus successful output retains the complete original encoding, including the outer count and every field boundary.

Lean: [RecordParserMachine.total_run](#).

The loop invariant conserves remaining-input plus archive length. A successful call consumes at least the one-bit length prefix, even for an empty field. This bounds repeated calls by raw input length. The correspondence uses the original recursive field reader, whose existing definitions and exactness laws are exposed by visibility changes only.

Lean: [recordBodyResult_original](#).

For an encoded cache, the same machine validates its framing and retains its original word. In a supported explicit-source result initialized with empty cache/log, prime p, q and the attacker's per-view hash-query bound n derive $B = \text{cacheRecordBitBound}(p, q, n + 15)$. The fuel is at most $3B + 4 + (B + 1)U(B, B)$, without an added cache-size or execution certificate.

Lean: [RecordParserMachine.cache_run](#).

Lean: [RecordParserMachine.source_cache_run](#).

Independent indexed decoding checks 8,191 raw words of length at most 12 and 477 structured/directed records, including 257-bit counts. Three mutants accept trailing input, stop after one field, or omit the count header; all are detected, with zero failures or discards. Lean controls cover an empty record, two fields, a missing field, a malformed later field and trailing rejection.

Lean: [RecordParserMachine.multiple_fields_control](#).

Lean: [RecordParserMachine.missing_field_control](#).

Lean: [RecordParserMachine.trailing_input_control](#).

This result checks record framing. Section 1.46 derives lookup from source typing; Section 1.48 adds guarded insertion.

1.43 Historical component: Restoring the query between comparisons

Repeated lookup now has a comparison operation that restores the query and clears the candidate. A five-label TM2 uses three binary stacks and the field parser’s `Option Bool` memory. It moves query heads to scratch, compares candidate heads through literal-bit branches, then restores the query and clears the remaining candidate. Equality is retained in finite control.

Lean: [PreservingCompareMachine.supports](#).

Let $S(\ell, x, y, z, v)$ denote label, query, candidate, scratch and finite memory, and let T be this program’s actual fixed-halt TM2 step.

Theorem 1.44 (Restored-query comparison, machine-checked). *For every pair of binary words x, y and initial finite memory v , there is $t \leq 2|x| + |y| + 3$ such that*

$$T^t(S(\text{scan}, x, y, [], v)) = S(\text{halt}, x, [], [], \text{some}(\text{decide}(x = y))).$$

The original query is restored and candidate/scratch are empty on both outcomes.

Lean: [PreservingCompareMachine.run](#).

The scan invariant permits a consumed query prefix in scratch. Restoration prepends its reversal to the untouched query suffix, including after an early mismatch. Clearing the candidate tail prepares the next call’s workspace.

Lean: [PreservingCompareMachine.scan_run](#).

For the original canonical encodings of arbitrary full keys a, b , assuming nonzero p, q , injectivity gives the flag `some(decide( $a = b$ ))` and bound `3 keyRecordBitBound( $p$ ) + 3`.

Lean: [PreservingCompareMachine.key_run](#).

Concrete parser layout. The existing six parser stacks are extended with one query stack. Candidate uses the old output position and the comparison buffer uses scratch. The exact stack-layout theorem derives preservation of the input suffix, collected count, outer count and archive at the same fuel. No frame-preservation premise is supplied by the caller. Output and scratch are empty at return.

Lean: [PreservingCompareMachine.parser_run](#).

Lean: [PreservingCompareMachine.parser_supports](#).

Four Lean controls cover restored nonpalindromic equality, early mismatch with a longer candidate tail, both strict-prefix directions, and literal surrounding parser data. Each asserts the full halted state. Independent tuple equality and original input snapshots check all 16,129 pairs with word lengths at most six. Mutants omit query restoration or ignore a trailing candidate; both are detected, with zero failures or discards.

Lean: [PreservingCompareMachine.equal_control](#).

Lean: [PreservingCompareMachine.mismatch_control](#).

Lean: [PreservingCompareMachine.prefix_control](#).

Lean: [PreservingCompareMachine.parser_frame_control](#).

Lookup composition. Section 1.44 connects the original entry header/key parser to comparison; Section 1.45 adds complete answer framing. Section 1.46 proves whole-cache lookup with retained source typing; Section 1.48 adds guarded insertion.

1.44 Historical component: Executing the original entry header and key

One seven-stack TM2 checks the original pair count $E(2) = [1, 1, 0, 0, 1]$, executes the existing key-field parser, and calls the preserving comparator. The five header checks execute literal-bit branches; no input header is dropped outside the run. Parser return checking and final halt are also charged. Successful workspace is derived from the parser's run.

Lean: `CacheKeyFieldMachine.header_run`.

Lean: `CacheKeyFieldMachine.run`.

Use $I(x, w, o, a)$ for this program's start with query x , input w , outer count o , archive a and empty workspace. Its semantic prefix decoder D uses the original natural-prefix reader, requires count two, then uses the original field decoder to obtain key k and suffix s . For halted c , readout $O(c)$ is none when finite memory is none, and some $(b, \text{input}(c))$ when memory is some b .

Theorem 1.45 (All-raw-prefix comparison, machine-checked). *For all binary words x, w, o, a , with $L = |w|$, there exists*

$$t \leq 3L + 17 + (L + 1)(2L + 4) + 2|x|$$

such that the actual TM2 run $c = T^t(I(x, w, o, a))$ is halted,

$$O(c) = \text{Option.map } (\lambda(k, s). (\text{decide}(x = k), s)) (D(w)),$$

and query, outer count and archive remain x, o, a .

Lean: `CacheKeyFieldMachine.total_run`.

Malformed header/key-field framing returns none; a valid unequal key returns some false with its residual suffix. Wrong/truncated count headers halt within five steps. For valid framing, count, scratch and output are empty on return.

Lean: `CacheKeyFieldMachine.header_rejected`.

Lean: `CacheKeyFieldMachine.rejected_run`.

Original typed cache entries. For arbitrary full query/entry keys x, k and scalar answer r , with nonzero p, q , start on the original `ballotCacheEntryBitCodec` encoding of (k, r) . Execution returns the encoded query, equality flag `some(decide( $x = k$ ))`, empty workspace and exactly the scalar-answer frame $E(|\text{scalarEncode}(r)|) ++ \text{scalarEncode}(r)$ as residual input. The existing key codec derives, for $K = \text{keyRecordBitBound}(p)$,

$$t \leq 3 \text{size}(K) + 17 + K(2 \text{size}(K) + 4) + 2K.$$

Lean: `CacheKeyFieldMachine.entry_run`.

Independent indexed decoding and equality check 30,705 raw query/input pairs (input length at most ten, query length at most three) and 2,835 structured cases. Ignored-count and malformed-as-mismatch mutants are detected, with zero failures or discards. Four Lean controls retain matching and unequal keys, noncanonical length rejection, wrong count and the untouched answer suffix.

Lean: `CacheKeyFieldMachine.match_control`.

Lean: `CacheKeyFieldMachine.mismatch_control`.

Lean: `CacheKeyFieldMachine.malformed_control`.

Lean: `CacheKeyFieldMachine.wrong_count_control`.

Section 1.45 adds answer framing and full entry consumption. Section 1.46 proves lookup on internal typed caches. Guarded insertion follows in Section 1.48.

1.45 Historical component: Complete entry framing and retained comparison

The key operation now returns to an answer parser in the same seven-stack TM2. Match/mismatch is retained in a finite control label while local memory is reset and reused. Failed answer framing or nonempty residual input halts invalid. A successful answer parse returns the earlier comparison bit and original answer bytes. Both match and mismatch require a complete entry.

Lean: `CacheEntryMachine.answer_run`.

Lean: `CacheEntryMachine.answer_rejected`.

Let $D(w)$ be the original `bitFieldsDecode` followed by its exactly-two-fields case, returning (k, a) ; other cases return none. The sequential execution specification is proved equal to D , including rejection and full consumption.

Lean: `CacheEntryMachine.decode_original`.

Write $I(x, w, o, r)$ for start with query x , entry input w , outer counter o , archive r and empty workspace. Halted readout O is some (b, output) when memory contains some b , and none for invalid memory.

Theorem 1.46 (Complete raw entry framing, machine-checked). *For all binary words x, w, o, r , with $L = |w|$, some*

$$t \leq 6L + 26 + 2(L + 1)(2L + 4) + 2|x|$$

reaches an actual halted TM2 configuration $c = T^t(I(x, w, o, r))$ with

$$O(c) = \text{Option.map } (\lambda(k, a). (\text{decide}(x = k), a)) (D(w)),$$

retaining query x , outer counter o and archive r .

Lean: `CacheEntryMachine.total_run`.

For an original two-field encoding, the complete terminal state has empty source/count/scratch, original answer bytes in output and the correct comparison bit. For lengths $X = |x|$, $K = |k|$, $A = |a|$, define its structured bound

$$\begin{aligned} C(X, K, A) &= 3 \text{size}(K) + 3 \text{size}(A) + 26 + 2X \\ &\quad + K(2 \text{size}(K) + 4) + A(2 \text{size}(A) + 3). \end{aligned}$$

Lean: `CacheEntryMachine.run`.

For nonzero p, q , the typed entry instance returns the canonical scalar answer and correct key-equality flag within $C(K_p, K_p, A_q)$ transitions. Here $K_p = \text{keyRecordBitBound}(p)$ and $A_q = \text{groupRecordBitBound}(q)$.

Lean: `CacheEntryMachine.entry_run`.

Independent strict indexed decoding checks 57,337 raw query/input pairs (input length at most twelve, query length at most two) and 4,725 structured cases. Overwritten-comparison, ignored-tail and skipped-answer mutants are detected; failures and discards are zero. Five Lean controls retain answer/frame data, mismatch through answer success, empty versus absent answer framing, and trailing rejection.

Lean: `CacheEntryMachine.mismatch_control`.

Lean: `CacheEntryMachine.empty_answer_control`.

Lean: `CacheEntryMachine.missing_answer_control`.

Lean: `CacheEntryMachine.trailing_control`.

This is framing and comparison. Section 1.46 supplies whole-cache preservation and lookup; Section 1.48 adds guarded insertion.

1.46 Historical component: Complete lookup in source-generated caches

One eight-stack TM2 copies the original cache, reads its binary outer count, and repeats the existing field and entry operations. Relocation preserves the unprocessed tail during each entry call. A mismatch clears the answer and executes binary decrement before another iteration; a hit returns the encoded scalar. The query and entire original cache survive, including entries after an early hit. The repeated run equals the original `List.dlookup`; every successor workspace is derived from execution. Readout O distinguishes invalid none, a miss some none, and a hit some (some answer bits).

Lean: `CacheLookupMachine.loop_run`.

Write $K = \text{keyRecordBitBound}(p)$, $A = \text{groupRecordBitBound}(q)$, $E = \text{cacheEntryBitBound}(p, q)$ and use the preceding entry bound C . Define

$$F = 3 \text{size}(E) + 7 + E(2 \text{size}(E) + 3),$$

$$I(N) = F + C(K, K, A) + A + 2 \text{size}(N) + 6.$$

Theorem 1.47 (Executed typed-cache lookup, machine-checked). *Let p, q be nonzero, H a typed finite cache with N entries, and x a full query key. Let w and u be their original codec encodings, with $W = |w|$. Starting on u, w and empty workspace, some*

$$t \leq 2W + 3 \text{size}(N) + NI(N) + 8$$

reaches an actually halted configuration c satisfying

$$O(c) = \text{some}(\text{Option.map scalarEncode}(H.\text{lookup}(x))), \quad \text{query}(c) = u, \quad \text{saved}(c) = w.$$

The run includes the original copy, count parse and all control transfers.

Lean: `CacheLookupMachine.run`.

For prime p, q , actual supported repaired-source executions with per-view attacker hash-query bound n derive $N \leq n + 15$ and $W \leq B = \text{cacheRecordBitBound}(p, q, n + 15)$. The same machine then has bound

$$2B + 3 \text{size}(n + 15) + (n + 15)I(n + 15) + 8.$$

No caller supplies a frame, cache-size or execution-cost certificate.

Lean: `CacheLookupMachine.source_cache_run`.

Independent fixtures check 3,845 ordered caches/queries and four malformed frames. Missing-copy, first-entry-only and lost-query mutants are detected; failures/discards are zero. Lean controls retain head/late hits (answers five and three), nonempty/empty misses, and actual malformed rejection.

Lean: `CacheLookupMachine.early_hit_control`.

Lean: `CacheLookupMachine.late_hit_control`.

Lean: `CacheLookupMachine.miss_control`.

Lean: `CacheLookupMachine.empty_control`.

Lean: `CacheLookupMachine.malformed_controls`.

This covers internal typed caches. An early hit need not validate later raw entries; arbitrary raw typed-cache decoder agreement is not claimed. Source typing supplies group/scalar membership and unique keys; exact saved encoding preserves that cache. Section 1.48 adds guarded insertion.

1.47 Historical component: Writing original field and count prefixes

Cache insertion needs new length prefixes. The writer computes them from loaded payload bits, using the original $E(n) = 1^{\text{size}(n)}0 \text{ bits}(n)$ format. It does not receive a freely constructed complete entry.

Executed binary increment. A two-label TM2 uses the existing four binary stacks for carry and restoration. For every n , source and frame, some $t \leq 2 \text{ size}(n) + 2$ changes the counter from $\text{bits}(n)$ to $\text{bits}(n + 1)$, clears scratch and reaches halted success while preserving both other stacks. The digit-successor law is proved from the original natural bits representation.

Lean: `BitIncrementMachine.succBits_bits`.

Lean: `BitIncrementMachine.run`.

Complete framing writer. One five-stack TM2 counts collected payload bits through linked increments, preserving source, buffered payload and destination. It restores payload order, writes the little-endian counter digits, then the delimiter and width ones. All callbacks use finite local data; each returned workspace is derived.

Theorem 1.48 (Executed field construction, machine-checked). *For all payload words w and destination suffixes s , let $L = |w|$. Starting with payload w , destination s and empty workspace, some*

$$t \leq L(2 \text{ size}(L) + 5) + 3 \text{ size}(L) + 5$$

reaches a halted successful configuration with output

$$E(L) ++ w ++ s$$

and every other stack empty. The payload length is computed by the machine.

Lean: `FrameWriteMachine.run`.

The prefix entry point writes $E(n) ++ s$ from canonical $\text{bits}(n)$ in exactly $3 \text{ size}(n) + 3$ transitions, again clearing workspace.

Lean: `FrameWriteMachine.prefix_run`.

For nonzero moduli, the existing full-key/scalar codecs derive the field-writer bounds by substituting $K_p = \text{keyRecordBitBound}(p)$ or $A_q = \text{groupRecordBitBound}(q)$ for L in the monotone bound above.

Lean: `FrameWriteMachine.key_field_run`.

Lean: `FrameWriteMachine.scalar_field_run`.

Eight Lean controls cover carry, zero, nonpalindromic payload/suffix data, length-boundary behavior and the required empty-field delimiter. Independent integer framing and stack execution check 1,028 increments and 6,141 fields, including 257-bit carry chains. Broken carry, reversed payload and omitted delimiter mutants are detected; failures/discards are zero.

Lean: `BitIncrementMachine.carry_control`.

Lean: `FrameWriteMachine.field_control`.

Lean: `FrameWriteMachine.boundary_control`.

Lean: `FrameWriteMachine.empty_not_omitted`.

The finite handlers insert only after a miss. Section 1.48 composes the writes with lookup and derives freshness. General AList replacement prepends the new pair and erases the old occurrence; guarded insertion refuses occupied keys.

1.48 Historical component: Guarded insertion in source-generated caches

One nine-stack TM2 links lookup, count parsing/increment, operand copying and framing. An occupied key retains the original cache. On a miss, the machine constructs the key/scalar pair, frames it before the old tail and writes the incremented count. Lookup derives freshness and six empty work stacks.

Lean: `CacheLookupMachine.miss_run`.

Theorem 1.49 (Executed guarded insertion, machine-checked). *Let $p, q \neq 0$, H be any typed finite cache, x any typed full key and $a \in \mathbb{Z}_q$. Write $N = |H.\text{entries}|$, $W = |\text{encode}(H)|$, and use the existing key, scalar and entry bounds $K_p, A_q, E_{p,q}$. With $I(N)$ the lookup iteration bound and $J(L) = L(2 \text{size}(L) + 5) + 3 \text{size}(L) + 5$, define*

$$D(N) = 5 \text{size}(N) + 3 \text{size}(N + 1) + 2A_q + 2K_p \\ + J(A_q) + J(K_p) + J(E_{p,q}) + 21.$$

From the encoded cache/key/scalar and empty workspace, some $t \leq 2W + 3 \text{size}(N) + NI(N) + 9 + D(N)$ reaches an actually halted configuration retaining the original key and scalar, with readout

$$\begin{cases} \text{some}(\text{true}, \text{encode}(H.\text{insert}(x, a))), & H.\text{lookup}(x) = \text{none}, \\ \text{some}(\text{false}, \text{encode}(H)), & \text{otherwise.} \end{cases}$$

The bound charges all linked calls and control transfers.

Lean: `CacheInsertMachine.run`.

For prime p, q , support in the actual repaired source started at the empty cache/log and per-view hash budget n derive $N \leq n + 15$ and $W \leq \text{cacheRecordBitBound}(p, q, n + 15)$; substituting these gives the source bound without supplied freshness, frame or cost certificates.

Lean: `CacheInsertMachine.source_cache_run`.

For the actual programmed source, arbitrary state and supplied transcript, the resulting cache agrees exactly and its sticky flag is `oldBad` $\vee$ `¬inserted`. Occupied keys raise the flag even when challenges agree. This is the cache/flag projection.

Lean: `CacheInsertMachine.programmed_cache_run`.

Independent fixtures check 11,535 canonical and four malformed cases; five count, field-order, tail-loss and occupied-key mutants are detected, with zero failures/discards. Lean controls retain literal prepend order, empty insertion, disagreeing/agreeing collisions and malformed rejection.

Lean: `CacheInsertMachine.prepend_control`.

Lean: `CacheInsertMachine.head_collision_control`.

Lean: `CacheInsertMachine.agreeing_collision_control`.

Lean: `CacheInsertMachine.empty_control`.

Lean: `CacheInsertMachine.malformed_controls`.

The scalar is already loaded. Section 1.49 connects these runs to the source request schedule. Arbitrary raw typed-cache decoding is not claimed.

1.49 Historical component: Source request scheduling through executed cache operations

The driver runs lookup before requesting entropy. A hit returns the stored scalar and old cache/log words. A miss requests the original wrapped challenge, writes its scalar prefix, runs guarded insertion and executes log append. Explicit machine failure returns none. Insertion repeats lookup deterministically; it adds no request.

Executable fuel. For loaded cache word w , put $W = |w|$. Insertion uses its bound $C(p, q, W, W)$; parsed lookup uses the bound of Section 1.51 at (p, q, W, W) . Original framing derives $N \leq W$. Monotonicity and fixed halted states give exact readouts at these fuels for all typed caches and operands, with nonzero p, q . No caller supplies the entry count, freshness or fuel.

Lean: `CacheHandler.probe_encode`.

Lean: `CacheHandler.insert_encode`.

The original handler. The driver hash branch equals the original logged hash handler mapped to an optional encoded result. This equality retains the entire oracle request tree. Hits execute prefix parsing and then use the original binary-value interpretation law; misses suspend for the original challenge before the next deterministic run.

Lean: `CacheHandler.hash_eq`.

Lean: `CacheHandler.hash_hit`.

Lean: `CacheHandler.hash_miss`.

Theorem 1.50 (Every source program, machine-checked). *Let $p, q \neq 0$, A be any result type, P any original ballot-oracle program returning A , and (H, L) any typed initial cache and ordered miss log. Write E_H, E_L for their original codecs and define $\varphi(a, H', L') = \text{some}(a, E_H(H'), E_L(L'))$. Then, as an equality of oracle computations,*

$$\text{driver}(P, E_H(H), E_L(L)) = \text{map } \varphi \text{ runBallotFiniteLogged}(P, (H, L)).$$

Thus every continuation receives the original answer and exact successor state; canonical source executions cannot produce the driver's failure result.

Lean: `CacheHandler.run_eq`.

The driver uses VCVio's existing computation recursor. Uniform queries keep their original range and response. The induction derives successor encodings without a caller-supplied correspondence or cache-validity invariant.

Four Lean controls retain literal hit/miss outputs and log order, exclude eager hit resampling for every continuation, and exclude an immediate miss return for every fabricated result. Independent fixtures check 1,728 branches and 2,187 three-request sequences carrying actual returned cache words. Four sampling/log mutants are detected; failures/discards are zero.

Lean: `CacheHandler.hit_control`.

Lean: `CacheHandler.miss_control`.

Lean: `CacheHandler.hit_not_resampled`.

Lean: `CacheHandler.miss_not_immediate`.

OracleComp supplies suspension. Section 1.50 executes scalar prefix writing and log append. Loading/transfers, conversion to input digits, numeric interpretation/range checking, fuel computation and continuation execution remain host operations with unproved machine costs. This is not a whole oracle TM or a PPT theorem.

1.50 Historical component: Chronological log append and scalar-prefix execution

The log codec erases Unit tags and writes a count followed by framed full keys. One eight-stack TM2 parses and increments that count, copies/frames the new key, transfers the old body before it in order, then writes the new count prefix.

Theorem 1.51 (Executed chronological append, machine-checked). *For every list of bit words S and key word k , put $N = |S|$, $K = |k|$, $B = |\text{bitFramesEncode}(S)|$ and let J be the existing field-writer cost. Starting with k , the original record encoding of S and empty workspace, some run of at most*

$$5 \text{size}(N) + 3 \text{size}(N + 1) + 2K + J(K) + 2B + 17$$

transitions halts successfully with output $\text{bitFieldsEncode}(S + [k])$, retained key k and every other stack empty. Every linked call and transfer is included in this bound.

Lean: [LogAppendMachine.run](#).

For nonzero p, q , the typed instance uses the original log codec and key bound. For prime p, q , actual supported repaired-source runs from empty cache/log with per-view hash budget n derive $N \leq n + 15$ and log length at most the existing $\text{bitListSize}(K_p, n + 15)$. No caller supplies a log invariant or fuel.

Lean: [LogAppendMachine.log_run](#).

Lean: [LogAppendMachine.source_log_run](#).

Scalar prefix from loaded digits. For nonzero q and any $a \in \mathbb{Z}_q$, the existing prefix writer takes $\text{bits}(a.\text{val})$ and suffix s to $\text{scalarEncode}(a) ++ s$, clears workspace and halts in at most $3 \text{size}(q - 1) + 3$ transitions. No new scalar machine is needed.

Lean: [FrameWriteMachine.scalar_prefix_run](#).

Lean: [FrameWriteMachine.scalar_prefix_control](#).

Lean: [FrameWriteMachine.scalar_zero_control](#).

Driver integration and controls. The existing driver now keeps the log encoded, executes this append and runs the scalar prefix writer before insertion. Fixed-fuel readout laws preserve the full source theorem in Section 1.49.

Lean: [CacheHashHandler.append_encode](#).

Lean: [CacheHashHandler.scalarWord_encode](#).

Five append controls retain literal chronological/repeated keys, empty-field framing, nonpalindromic transfer, exclusion of prepend and count-width growth.

Lean: [LogAppendMachine.order_control](#).

Lean: [LogAppendMachine.empty_control](#).

Lean: [LogAppendMachine.order_bits_control](#).

Lean: [LogAppendMachine.not_prepend_control](#).

Lean: [LogAppendMachine.boundary_control](#).

Independent campaigns check 3,905 records, six count boundaries, 108 scalar prefixes, 1,728 driver branches and 2,187 request triples. Respectively five, two and four mutation families are detected; failures/discards are zero.

1.51 Historical component: Parsing the actual cached answer

A cache hit leaves its encoded answer in the archive stack. The complete scan now derives empty parser workspace, retaining the unread cache tail, entry counter, query and saved original cache. The existing prefix parser is relocated onto that archive in the same eight-stack program; no answer reload is used.

Lean: `CacheLookupMachine.hit_run`.

Theorem 1.52 (Executed cached-answer parsing, machine-checked). *For nonzero p, q , every original typed cache C and full query k , let $N = |C.entries|$ and $W = |\text{encode}(C)|$. Starting with their original encoded words, some run of at most*

$$2W + 3 \text{size}(N) + N \text{iterationCost}(p, q, N) + 3 \text{size}(q - 1) + 13$$

transitions halts with readout $\text{some}(C.\text{lookup}(k).\text{map}(a \mapsto \text{bits}(a.\text{val})))$, retaining the exact query and saved cache. The machine itself distinguishes hits from misses and requires full scalar-field consumption. A zero answer is a hit with empty digits.

Lean: `CacheReadMachine.run`.

For prime p, q , supported repaired-source runs from empty cache/log and per-view hash budget n derive $N \leq n + 15$ and $W \leq B(p, q, n + 15)$. Cost monotonicity also supplies the driver's fixed fuel from the actual loaded cache length.

Lean: `CacheReadMachine.source_cache_run`.

Lean: `CacheReadMachine.cost_mono`.

Driver and controls. The existing driver's probe now executes this parsing. Its typed boundary uses the original binary-value interpretation and range check; the exact source query-tree theorem in Section 1.49 remains checked.

Lean: `bitsValue_bits`.

Lean: `CacheHashHandler.scalarValue_bits`.

Four Lean controls retain nonpalindromic scalar six, zero distinct from absence, a different-query miss and malformed/trailing scalar rejection.

Lean: `CacheReadMachine.hit_control`.

Lean: `CacheReadMachine.zero_control`.

Lean: `CacheReadMachine.miss_control`.

Lean: `CacheReadMachine.malformed_controls`.

Independent fixtures check 3,845 canonical cases and four malformed scalar records, detecting five parsing/frame mutations. Updated driver fixtures retain 1,728 branches and 2,187 request triples with four detected sampling/log mutations; failures/discards are zero.

Numeric interpretation/range checking, initial digit loading, remaining physical transfers, fuel and continuations still lack machine-cost correspondence.

1.52 Historical component: Guarded subtraction for scalar range reduction

The existing fair-bit sampler applies modulus reduction as host arithmetic. Binary long division maintains $r < q$, so the next value $2r + b < 2q$ needs at most one subtraction. Repeated subtraction from the entire sampled integer would not give a polynomial bit bound. This section supplies the required local operation.

Loaded-word subtraction. For all little-endian words x, y , a six-stack finite-control program halts within $3(|x| + |y|) + 5$ transitions. If their numeric values satisfy $Y \leq X$, it returns canonical digits of $X - Y$ with success true. Otherwise it returns the exact original word x with success false. Both branches retain y and clear all other workspace. Thus noncanonical high zeros and underflow have explicit behavior; no numeric comparison is performed by a host callback.

Lean: [BinarySubtractMachine.run](#).

Theorem 1.53 (Executed remainder update, machine-checked). *For natural numbers r, q and a bit b , assume $r < q$. Starting with canonical loaded words $\text{bits}(r)$ and $\text{bits}(q)$, the same program executes the shift, including its zero case, then guarded subtraction. Some run of at most $6 \text{size}(q) + 9$ transitions halts with canonical output*

$$\text{bits}((2r + \text{toNat}(b)) \bmod q),$$

retained modulus digits, empty workspace and flag $\text{decide}(q \leq 2r + \text{toNat}(b))$.

Lean: [BinarySubtractMachine.remainder_step](#).

The proof derives borrow from a binary-value conservation identity and removes high zero bits on successful subtraction. Its flag says whether subtraction was taken, not whether the division update succeeded.

Six Lean controls retain a long borrow chain, underflow with padded input, equal numeric values of different widths, nonpalindromic output, the update $17 \bmod 11 = 6$ and the canonical zero shift.

Lean: [BinarySubtractMachine.borrow_control](#).

Lean: [BinarySubtractMachine.underflow_control](#).

Lean: [BinarySubtractMachine.equal_control](#).

Lean: [BinarySubtractMachine.order_control](#).

Lean: [BinarySubtractMachine.remainder_control](#).

Lean: [BinarySubtractMachine.zero_control](#).

Independent integer fixtures check 16,129 raw operand pairs through six bits, 20 boundary cases through 258 bits and 4,160 remainder updates with $1 \leq q \leq 64$. Five borrow/order/modulus/equality/underflow mutations are detected; failures/discards are zero.

This is one division update. This result neither changes the sampler law nor removes its statistical loss, and it is not a computational ballot-secrecy theorem.

1.53 Historical component: One proof oracle for the sampled election

The computational game now uses one oracle interface for ballot, public-key and decryption proof challenges. Its three tagged request types retain the original arguments:

$$B(s, a), \quad K(g, pk, a), \quad D(g, pk, ct, d, (a, b)).$$

Here s is the full ballot statement, a its proof commitments in the first case, and d the decryption share in the last. The tags represent the existing three typed hash functions; no concrete byte-domain-separation theorem is claimed. The public fingerprint remains its original parameter function.

Lean: `ElectionOracle.Key`.

Lean: `ElectionOracle.functionImpl`.

An attacker receives the original prefix before casting and the complete result before guessing, with private state between these callbacks. Both callbacks can query this oracle. Preparation can also query before election sampling and save state for the callbacks. The original random draw order, rejected input, retained board, encrypted tally, shares, trustee proofs and decoding failures are preserved.

Lean: `ElectionOracle.Adversary`.

Lean: `ElectionOracle.world`.

Lean: `ElectionOracle.preparedGame`.

Theorem 1.54 (Sampled-game correspondence, machine-checked). *Let F be a finite field and G a decidable additive commutative group with an F -module structure. Let h be arbitrary original proof hashes and fingerprint, $g \in G$, and A any stateful oracle attacker. Write $\mathcal{I}_h$ for fixed-function interpretation, and A_h for interpretation of both callbacks. Then, as probabilistic oracle computations,*

$$\mathcal{I}_h(\text{world}(h_{fp}, g, A, v)) = \text{repairedBallotWorld}(h, g, A_h, v),$$

and the same equality holds for the sampled guessing game. For any preparation P , the prepared game interprets to sampling i from $\mathcal{I}_h(P)$ and then executing the original repaired game with $(A(i))_h$.

Lean: `ElectionOracle.prefix_function`.

Lean: `ElectionOracle.finish_function`.

Lean: `ElectionOracle.world_function`.

Lean: `ElectionOracle.game_function`.

Lean: `ElectionOracle.preparedGame_function`.

Lean: `ElectionOracle.Adversary.interpret_ofOriginal`.

With a `SampleableType F` instance, random interpretation uses the pinned lazy oracle and one private cache. A hit returns its saved answer; a miss samples and inserts an answer at the full tagged key. Continuations receive the actual successor cache, including across preparation and election phases. `randomGame` initializes this cache once and exposes only the winning bit to the outer experiment; attacker observations remain the callback inputs.

Lean: [ElectionOracle.randomGame](#).

Lean: [ElectionOracle.run_ask](#).

Lean: [ElectionOracle.run_bind](#).

Lean: [ElectionOracle.run_repeat](#).

Lean: [ElectionOracle.run_preparedGame](#).

Independent controls. For $p = 23, q = 11, g = 2, pk = 8$, proof nonce four gives key commitment 16; the decryption request uses $ct = (3, 13)$, share 4 and commitments (16, 12). Challenge five gives response eight. Changing the response to nine violates both modular equations. Lean checks the corresponding exponent queries and transcripts, as well as hashes that distinguish changed public-key/share coordinates. These small arithmetic fixtures carry no hardness claim.

Lean: [ElectionOracleControls.key_request_control](#).

Lean: [ElectionOracleControls.partial_request_control](#).

Lean: [ElectionOracleControls.key_transcript_control](#).

Lean: [ElectionOracleControls.partial_transcript_control](#).

Lean: [ElectionOracleControls.changed_context_control](#).

Lean: [ElectionOracleControls.bad_response_control](#).

A key/decryption/key/decryption request sequence samples only its two distinct keys and retains both answers; a populated cache with answers five and seven returns $[5, 7, 5, 7]$. The empty-cache computation has multiple supported outputs and cannot be a constant return. The independent Python fixture checks both modular transcripts, eight context-deletion mutations and the same mixed query sequence, with zero failures or discards.

Lean: [ElectionOracleControls.mixed_hit_control](#).

Lean: [ElectionOracleControls.mixed_miss_control](#).

Lean: [ElectionOracleControls.miss_not_constant](#).

Component scope. This result establishes the sampled interface correspondence for the three-voter, two-candidate, one-honest-trustee schedule. It does not supply the general challenge/corruption/election-family policy, oracle-aware attack instances, full adaptive extraction, or a random-oracle secrecy reduction. The full cryptographic reduction and its external implementation assumptions are stated in the accompanying [formal reference](#).

1.54 Historical component: The actual prefix after earlier oracle queries

The ballot runtime can now be used inside the full election without resetting earlier answers. Let $\pi(C)$ project a full cache to its ballot entries, and $R(C, B)$ replace only those entries by B , retaining the key/decryption entries. For every ballot-oracle computation M and full initial cache C ,

$$\text{run}(\text{liftBallot}(M), C) = ((x, B) \mapsto (x, R(C, B)))_* \text{runBallotOracle}(M, \pi(C)).$$

This is exact computation equality, including final state. Projection, reconstruction and fresh insertion are proved; the caller supplies no empty cache, freshness or frame invariant. Only decidable group equality and the scalar sampler are required for this runtime interface.

Lean: `ElectionCache.liftBallot_run`.

Lean: `ElectionCache.replace_project`.

Lean: `ElectionCache.replace_update`.

Theorem 1.55 (Actual prefix rejection, machine-checked). *Under the finite-field/module assumptions above, let $r \mapsto r \cdot g$ be injective. The full prefix accepts both honest ballots for collision-free historical coins, for every initial cache. With the original nonzero coin sampler, any pre-election oracle computation P , any initial cache C and either challenge bit,*

$$\Pr[\text{honestDecisions} \neq (\text{accepted}, \text{accepted}) \mid \text{run}(P; \text{samplePrefix}, C)] \leq \frac{9}{|F| - 1}.$$

The generator, fingerprint and challenge bit are fixed inputs to `samplePrefix`; all secret, key-proof, decryption-proof and honest-ballot coins are sampled in the original order. Secret material in its result remains challenger-local. Exact factoring of `ElectionOracle.world` connects this prefix to the original attacker callbacks and trustee finish.

Lean: `ElectionCache.prefix_accepts`.

Lean: `ElectionCache.world_eq_samplePrefix`.

Lean: `ElectionCache.samplePrefix_rejection_le`.

Lean: `ElectionCache.preparedPrefix_rejection_le`.

Independent finite fixtures cover 729 cache states, 2,916 ballot-response branches and 1,728 mixed three-request traces. Four reset/erasure/overwrite mutations are detected, with zero failures or discards. Lean controls show that resetting a saved ballot answer changes its distribution, while a ballot miss retains the other proof domains. In the 101-element field, the full sampled prefix after an actual prior query has rejection probability at most 9/100 and strictly positive acceptance probability. The historical fixed-hash collision fixture still rejects.

Lean: `ElectionCacheControls.resetting_changes_answer`.

Lean: `ElectionCacheControls.miss_preserves_other_proofs`.

Lean: `ElectionCacheControls.rejection_nontrivial`.

Lean: `ElectionCacheControls.acceptance_positive`.

Lean: `ElectionCacheControls.original_rejection_retained`.

This closes the real-prefix cache/acceptance interface. No such correspondence, standard-PPT adequacy or secrecy theorem is assumed.

1.55 Historical component: Full-source replay and the actual submitted ballot

The full election can use the existing ballot replay engine without a second fork implementation. Every ballot request, including preparation and the final attacker continuation, remains explicit. Key/decryption proof requests instead use a local lazy cache: hits retain their answer, and misses draw ordinary uniform randomness. Auxiliary draws are not ballot fork challenges.

Let $L(M, A)$ lower a full-oracle computation M with auxiliary cache A , and let $R(A, B)$ reconstruct a full cache from auxiliary and ballot entries. With decidable group equality and the scalar sampler, for every M, A, B ,

$$((x, A', B') \mapsto (x, R(A', B'))) * \text{runBallotOracle}(L(M, A), B) = \text{run}(M, R(A, B)).$$

This is exact computation equality with final state. Its proof derives the required domain/update conditions. Lowering the entire source from empty state preserves its full returned value; the existing ballot-only sources retain exactly their query trees. No final cache substitutes for earlier history.

Lean: `ElectionReplaySource.lower_run.`

Lean: `ElectionReplaySource.source_runtime.`

Lean: `ElectionReplaySource.source_mem.`

Lean: `ElectionReplaySource.source_liftBallot.`

Theorem 1.56 (Actual-election conditional extraction, machine-checked). *Let F be a finite field with its sampler, G an additive F -module, both with decidable equality, and $g \neq 0$. Fix any fingerprint, pre-election computation P , stateful full-oracle adversary $\mathcal{A}$ and replay parameter $n \in \mathbb{N}$. Let E be `ElectionExtraction.preparedSource`, which retains the actual public result alongside the winning bit. Erasing that record gives exactly the original `ElectionOracle.preparedGame` query computation, including preparation and the final guessing continuation.*

If `ElectionExtraction.joint` returns (o, w) in its support, where $o = (V, b)$, let S_i be the covered statement for component i (or the implicit aggregate for $i = \text{none}$) of $V.\text{submission}$, with generator g and the public key in $V.\text{beforeTally}$. Then

$$\begin{aligned} &V.\text{decision} = \text{accepted}, \\ &\forall i \in \{\text{some}(0), \text{some}(1), \text{none}\}, \quad S_i.\text{Witnesses}(w_i), \\ &\exists C, \quad (o, C) \in \text{support}(\text{run}(E, \emptyset)). \end{aligned}$$

Thus all three witnesses concern one actual submitted ballot and its public key in a complete original election execution. If $r \mapsto r \cdot g$ is injective and $(2 : F) \neq 0$, their nonces sum consistently and their component bits sum, as natural numbers, to the aggregate bit and to at most one.

Lean: `ElectionExtraction.worldSource_project.`

Lean: `ElectionExtraction.preparedSource_project.`

Lean: `ElectionExtraction.joint_valid.`

Lean: `ElectionExtraction.joint_consistent.`

The single and joint algorithms reuse the existing source extractors. Rejected records select an impossible first-branch proof, so acceptance is derived from successful extraction rather than required of the caller.

Lean: `ElectionReplaySource.extract_valid.`

Lean: `ElectionReplaySource.jointExtract_valid.`

Controls and remaining probability argument. The mixed sequence queries a ballot proof, key proof, decryption proof, key proof and ballot proof. It retains both ballot requests and draws only twice for the auxiliary misses. Lean pins its exact query tree and excludes a pure constant source. Existing positive single/joint replay fixtures embed unchanged; the unqueried live-selector control remains zero. These positive fixtures cover their embedded prover programs, not arbitrary full-election attackers. Independent fixtures cover 20,736 traces and one directed five-request trace, detecting four history/cache/domain mutations with zero failures or discards.

Lean: [ElectionReplayControls.mixed_source](#).

Lean: [ElectionReplayControls.auxiliary_hit](#).

Lean: [ElectionReplayControls.mixed_source_not_pure](#).

Lean: [ElectionReplayControls.lifted_single_positive](#).

Lean: [ElectionReplayControls.lifted_joint_positive](#).

Lean: [ElectionReplayControls.unqueried_control](#).

The following subsection derives the real-source selection and extraction probability. The real source in this component samples the secret key and honest witnesses, so it is not a reduction given only a DDH challenge. The later simulation comparisons preserve the complete public view and account for their losses.

1.56 Historical component: Actual acceptance and joint replay probability

The preceding extractor now has a bound in terms of actual submission acceptance in the complete real election. The argument retains preparation, key and decryption proofs, the public result and the final guessing callback. Every full-oracle computation preserves previously cached answers. Actual submission acceptance therefore supplies three valid proofs in the final cache, and exact full-source reconstruction transfers that certificate to the replay cache. The generator is derived from the actual setup.

Lean: [ElectionAcceptance.run_cache_le](#).

Lean: [ElectionAcceptance.world_cached](#).

Lean: [ElectionAcceptance.prepared_cached](#).

Lean: [ElectionAcceptance.source_cached](#).

Theorem 1.57 (Real-election joint replay bound, machine-checked). *Let F be a finite field with its sampler, G an additive F -module, both with decidable equality, and $g \neq 0$. Fix the fingerprint, arbitrary preparation P and stateful full-oracle adversary $\mathcal{A}$. Suppose preparation makes at most n_p ballot requests, every casting callback makes at most n_c , and every guessing callback makes at most n_g . These are structural bounds over all callback inputs, not runtime or standard-PPT certificates. Define*

$$N = n_p + n_c + n_g + 15, \quad a = \Pr[V.\text{decision} = \text{accepted} \mid \text{run}(\text{preparedSource}, \emptyset)].$$

For every $\delta \in \overline{\mathbb{R}}_{\geq 0}$,

$$(a - 3(N + 1)\delta)(\delta - |F|^{-1})^3 \leq \Pr[\text{joint}(N) \neq \text{none}].$$

Subtraction is truncated nonnegative extended-real subtraction. The algorithm is the actual `ElectionExtraction.joint`, with the successful-output contract of Theorem 1.56. Neither cache correspondence nor acceptance-to-selector coverage is assumed.

Lean: [ElectionExtractionBound.joint_le](#).

The protocol contributes twelve ballot requests for the two honest constructors and their validation, and at most three for the adversarial submission. Key and decryption requests contribute zero to this predicate. Repeated ballot hits still count as requests. The full-source lowering preserves this bound.

Lean: [ElectionQueryBound.prepared_bound](#).

Lean: [ElectionQueryBound.source_bound](#).

For each original source path, all three proof selectors are enabled exactly when its actual submitted ballot accepts. Their simultaneous-selection probability is therefore exactly a , on a common first execution. The existing joint replay theorem then charges the low-context loss $3(N + 1)\delta$ and the three equal-challenge losses in the displayed expression.

Lean: [ElectionExtractionBound.path_verified](#).

Lean: [ElectionExtractionBound.path_selector](#).

Lean: [ElectionExtractionBound.selection_probability](#).

Controls and boundary. An independently checked fresh-ballot verifier fixture remains valid after arbitrary later full-domain queries; erasing its cache destroys that certificate. Its preloaded cache is not an empty-cache reachability claim. Mixed requests require two ballot queries, so a budget of one fails. A numerical control shows that the bound can be positive; it does not assert

acceptance probability one for the actual election. Independent modular fixtures cover 4,000 traces, detect three cache/history mutants and refute the rejected-proof fallback, with zero failures or discards.

Lean: `ElectionAcceptanceControls.late_queries_preserve_validity.`

Lean: `ElectionAcceptanceControls.erased_cache_not_valid.`

Lean: `ElectionAcceptanceControls.mixed_two_ballot_queries.`

Lean: `ElectionAcceptanceControls.mixed_one_is_insufficient.`

Lean: `ElectionAcceptanceControls.numerical_bound_positive.`

This is a real-source replay bound. That source uses the sampled secret key and honest witnesses; a reduction given only a DDH challenge cannot simply execute it. No honest-proof simulation has been substituted, so its statistical losses have not disappeared into this bound.

1.57 Historical component: Trustee simulation after reachable prior queries

The key and equality-of-logs proof simulators take public statements only. For base B and public key Y , sample c, z uniformly from F and set $T = zB - cY$. Return proof (T, z) and program the actual hash key to c only if it has no prior answer. A hit preserves the prior cache and sets a collision flag, even if its answer agrees. For key proofs $B = g, Y = \text{pk}$; for partial decryption $B = (g, \text{ct}_1)$ and $Y = (\text{pk}, \text{share})$. The tags retain the generator, public key, ciphertext, share and relevant commitments.

Lean: `TrusteeOracleSimulation.sim_valid.`

Lean: `TrusteeOracleSimulation.sim_cache_le.`

Theorem 1.58 (Reachable-cache trustee simulation, machine-checked). *Let F be a finite field with its sampler, G an additive F -module, both with decidable equality. Suppose $r \mapsto rg$ is injective. Let P be any full election-oracle computation with structural bound n on hash requests across all three proof domains, and (a, C) any supported result of $\text{run}(P, \emptyset)$. For every secret s and ciphertext ct , each original trustee proof step R and corresponding public-input simulator S satisfy*

$$d_{\text{TV}}(R(C), S(C)) \leq (1 + n)|F|^{-1}.$$

For the key step, $\text{pk} = sg$; for the decryption step also $\text{share} = s\text{ct}_1$. The compared output contains the proof, collision flag and complete final cache; the original step returns flag false. Each original step draws its historical nonzero nonce at the point of use.

Lean: `TrusteeReachableSimulation.key_after_queries_le.`

Lean: `TrusteeReachableSimulation.partial_after_queries_le.`

Lean: `TrusteeOracleSimulation.key_real.`

Lean: `TrusteeOracleSimulation.partial_real.`

The proof derives a cache cover of size at most n from the supported prior execution. Uniform coin queries do not consume that budget; repeated hash hits do. No cover, freshness or tag-injectivity premise remains in the displayed result. The local programming bound charges $n/|F|$ for hits and $1/|F|$ for replacing the historical nonzero nonce by a full-field nonce.

Lean: `ElectionCacheBudget.run_covered.`

Lean: `TrusteeOracleSimulation.collision_le.`

Lean: `TrusteeOracleSimulation.distance_le.`

Lean: `TrusteeSimulation.nonzero_distance.`

Pinned VCVio's Schnorr transcript theorem supplies full-field equality by mapping scalar commitments into the actual module. The point-probability bound needs only injectivity, derived for the product from its first coordinate.

Lean: `TrusteeSimulation.full_eq`.

Lean: `TrusteeSimulation.commitment_probability_le`.

Lean: `TrusteeSimulation.partial_injective`.

Controls and composition boundary. Literal key/decryption transcripts validate; changing the challenge invalidates the key proof. Checked counterexamples refute exact historical-nonce simulation and product-group surjectivity. Fresh programming stores the challenge; every hit preserves and flags the old answer. A mixed query tree requires five hash requests, refuting a bound of four. Independent multiplicative fixtures over $p = 23, q = 11$ enumerate 242 full transcripts and 242 cached cases, with exact historical distance $1/11$, maximum cached distance $2/11$, zero failures and zero discards.

Lean: `TrusteeSimulationControls.literal_key`.

Lean: `TrusteeSimulationControls.literal_partial`.

Lean: `TrusteeSimulationControls.changed_challenge`.

Lean: `TrusteeSimulationControls.nonzero_not_exact`.

Lean: `TrusteeSimulationControls.product_not_surjective`.

Lean: `TrusteeSimulationControls.fresh_programs`.

Lean: `TrusteeSimulationControls.hit_preserves`.

Lean: `TrusteeSimulationControls.mixed_five_queries`.

Lean: `TrusteeSimulationControls.mixed_four_insufficient`.

The actual election samples private decryption nonces before its honest ballots. The following result justifies moving those draws to their use points.

1.58 Historical component: The original private nonce schedule

The historical world draws the secret, key-proof nonce, both decryption nonces and honest coin records before publishing its prefix. The equivalent schedule keeps the honest records private, draws the key-proof nonce immediately before its key query, and draws each decryption nonce immediately before that proof's query. Each draw retains its original independent nonzero distribution. Submission validation, rejection, retained board and bounded decoding are the original algorithms.

Theorem 1.59 (Original-election scheduling, machine-checked). *Let F be a finite field with its sampler and G an additive F -module, both with decidable equality. Fix any generator g , fingerprint function, preparation computation P , stateful oracle adversary $\mathcal{A}$ chosen from its result, and initial full cache C . Then*

$$\mathcal{L}(\text{run}(\text{preparedSource}(P, \mathcal{A}), C)) = \mathcal{L}(\text{run}(\text{scheduled}(P, \mathcal{A}), C)).$$

Here `scheduled` is `ElectionNonceSchedule.prepared`, using the nonce-at-use order above. Both outputs contain the actual public result, winning bit and complete final cache. The equality requires neither generator injectivity nor honest acceptance, freshness or a query/runtime bound.

Lean: `ElectionNonceSchedule.prepared_eq`.

The world theorem retains the public result and guess, for either challenge bit and any initial cache. The finishing theorem commutes the two independent nonce draws past submission validation and separates them around the actual first proof query. It admits an arbitrary later full-oracle continuation. Pinned VCVio's independent bind-swap theorem supplies the distributional step. Projection then preserves the original random-oracle game's winning-bit law.

Lean: `ElectionNonceSchedule.world_eq`.

Lean: `ElectionNonceSchedule.finish_eq.`

Lean: `ElectionNonceSchedule.winning_eq.`

Controls and boundary. A checked supported nonce pair has distinct components. Reusing a single nonce changes the joint law even though its marginals agree. Sampling independent private coins preserves an earlier cached answer; resetting that answer changes its probability from one to 1/11. Independent finite query/proof fixtures cover 34,992 assignments over two initial caches, retain accepting and rejecting cases, and detect shared-nonce and cache-reset variants in each cache. They have zero failures or discards and are not a differential implementation of the full historical protocol.

Lean: `ElectionNonceScheduleControls.distinct_pair_supported.`

Lean: `ElectionNonceScheduleControls.shared_nonce_changes_law.`

Lean: `ElectionNonceScheduleControls.old_answer_preserved.`

Lean: `ElectionNonceScheduleControls.reset_answer_probability.`

Lean: `ElectionNonceScheduleControls.retained_answer_probability.`

This discharges the scheduling premise for composing the local trustee bounds. It does not replace any proof by a simulator. Equality of output laws does not identify the two source query trees or their extractor replay paths; the extraction argument must separately establish the needed properties of its chosen simulated source.

1.59 Historical component: Composing the actual trustee finish

The finishing simulator runs the original submission verifier and component weeding, then simulates the two equality-of-logs proofs in the same full cache. It takes the prefix public key and a supplied public-share function. It returns the original public fields, including rejection, board, encrypted tally, shares, proofs and bounded decoding, plus a private sticky collision flag. Both flags contribute by disjunction. Later queries receive the public result; they do not receive the flag.

Theorem 1.60 (Original finishing-phase simulation, machine-checked). *Let F be a finite field with its sampler and G an additive F -module, both with decidable equality. Fix secret s , prefix V , arbitrary submitted ballot b , and an arbitrary later full-oracle continuation K . Write $g = V.parameters.generator$ and assume*

$$V.parameters.publicKey = sg, \quad (r \mapsto rg) \text{ is injective.}$$

Let P make at most n hash requests across all proof domains and let (a, C) be a supported output of $\text{run}(P, \emptyset)$. Let R be the original finish, drawing its two independent nonzero nonces before submission validation, then running K ; append flag false to its output. Let S be `finishSim`, supplied shares $\text{shareFor}(ct) = sct_1$, followed by `observe` K . Both retain the public result, K 's return value, private flag and complete final cache. Then

$$d_{\text{TV}}(R(C), S(C)) \leq (2n + 9)|F|^{-1}.$$

No cache cover, freshness, acceptance or bound on K 's queries is assumed. The query bound on P is structural, not a runtime certificate.

Lean: `ElectionTrusteeSimulation.finish_after_queries_le.`

The pair bound from a cache covered by k keys is $(2k + 3)/|F|$. The first simulated proof inserts at most one key; the second proof's loss includes that successor cache. Actual submission validation

contributes at most three queries, using the existing verified submission bound, giving $(2k + 9)/|F|$ for the finish. The prior execution derives $k \leq n$. Data processing retains the bound through all later queries. The original schedule correspondence supplies the pre-validation nonce order in the theorem.

Lean: `ElectionTrusteeSimulation.pair_distance_le`.

Lean: `ElectionTrusteeSimulation.finish_distance_le`.

Actual prefix execution derives its generator and trustee public key regardless of acceptance or earlier cache contents. This supplies the local theorem's key relation when the prefix comes from that execution. The already checked private sampling/cache equation is now shared by scheduling and finish normalization.

Lean: `ElectionTrusteeSimulation.prefix_parameters`.

Lean: `ElectionCache.run_liftProb`.

Controls and boundary. A checked pair can return two fresh proofs with independently specified commitments. Separate controls retain a first or second collision flag; publication retains rejection and the submitted ballot. The zero-prior-query bound is strictly below one for $|F| = 11$. Independent multiplicative enumeration covers six initial-cache/ciphertext cases, including repeated proof contexts and a later adaptive query. Maximum observed distance is 49/121; all cases satisfy the pair bound. Sticky-flag reset and challenge-overwrite mutants are detected, with zero failures or discards.

Lean: `ElectionTrusteeSimulationControls.two_fresh_supported`.

Lean: `ElectionTrusteeSimulationControls.first_flag_sticky`.

Lean: `ElectionTrusteeSimulationControls.second_flag_detected`.

Lean: `ElectionTrusteeSimulationControls.rejected_publication`.

Lean: `ElectionTrusteeSimulationControls.finish_bound_nontrivial`.

This component simulates trustee *proofs given correct shares*. It does not construct those shares without the secret inside a DDH reduction. The next two subsections connect key and honest-ballot simulation to the complete source. Section 1.63 supplies extraction from the simulated source.

1.60 Historical component: Key and honest-ballot simulation in the actual prefix

The prefix simulator uses the existing key-proof and sampled honest-pair simulators. It constructs the original public parameters from the returned key proof, evaluates the supplied fingerprint function on those parameters, and retains both honest decisions and the board. The ballot phase runs in the ballot projection of the full cache; reconstruction retains every key and decryption answer. Its private initial collision flag is the key simulator's flag.

Theorem 1.61 (Actual prefix simulation, machine-checked). *Let F be a finite field with its sampler and G an additive F -module, both with decidable equality. Suppose $r \mapsto rg$ is injective. Fix any secret s , vote v , fingerprint function and prior full-oracle program P making at most n hash requests across all proof domains. For any supported output (a, C) of $\text{run}(P, \emptyset)$,*

$$d_{\text{TV}}(R(C), S(g, sg, v, C)) \leq (7n + 91)/|F|.$$

R runs the actual fixed-secret prefix with the historical honest coin records and nonzero key nonce, returning its complete public prefix, flag false and final full cache. S is the prefix simulator `sim`, with no secret-key argument. Its output has the same fields and a private sticky collision flag. The

cache cover, six proof requests and validation-query budgets are derived; no freshness or acceptance premise is assumed.

Lean: `ElectionPrefixSimulation.prefix_after_queries_le`.

Lean: `ElectionPrefixSimulation.prefix_distance_le`.

The key step contributes $(n + 1)/|F|$. Programming a key-proof context leaves the ballot projection unchanged. A finite ballot cover of size at most n is derived as the preimage of the full cover under the injective ballot tag. The existing honest-pair theorem contributes $6(n + 15)/|F|$, with six proof requests and at most six verification hashes. Mapping its result back into the full cache preserves every other domain and the incoming key flag. Thus the established pair simulation is reused without changing its historical sampling loss or rejection behavior.

The real composed runner has the actual sampled-prefix distribution, with independent draws moved past the key proof by the pinned bind-swap theorem. Exact source factoring identifies this prefix inside the original scheduled world. An arbitrary later full-oracle continuation receives the public prefix, while the flag stays private. Data processing preserves the displayed bound, retaining both the prefix and continuation result plus the final cache.

Lean: `ElectionPrefixSimulation.real_eq`.

Lean: `ElectionPrefixSimulation.world_source`.

Lean: `ElectionPrefixSimulation.continuation_after_queries_le`.

Controls and remaining integration. Checked controls retain both non-ballot proof domains and refute reconstruction that erases them. A fingerprint depending on the actual key proof changes from 12 to 14 for two literal proofs. Original rejection decisions remain observable; a later cached query preserves an earlier collision flag. The numerical bound $91/101 < 1$ is nontrivial but does not assert a zero-query full-election history. Independent finite checks cover 363 reachable mixed-cache traces and 726 flag cases, detecting lost domains, a lost key flag, reset ballot answers and constant fingerprints, with zero failures or discards. These are boundary controls, not an exhaustive distribution test of the complete prefix.

Lean: `ElectionPrefixSimulationControls.restored_other_domains`.

Lean: `ElectionPrefixSimulationControls.lost_domain_counterexample`.

Lean: `ElectionPrefixSimulationControls.fingerprint_observes_proof`.

Lean: `ElectionPrefixSimulationControls.rejection_retained`.

Lean: `ElectionPrefixSimulationControls.continuation_keeps_flag`.

Lean: `ElectionPrefixSimulationControls.prefix_bound_nontrivial`.

The next result composes the prefix and finish bounds.

1.61 Historical component: Complete prepared-election proof simulation

The composed hybrid executes the original preparation, samples the original fair challenge bit and trustee secret, runs the prefix proof simulator, then executes the original casting callback, submission validation, simulated trustee proofs and original guessing callback. It retains the complete public result, winning bit, private sticky collision flag and final full cache. The secret is still used to compute correct shares; this hybrid is not a DDH reduction.

Theorem 1.62 (Complete proof-simulation bound, machine-checked). *Let F be a finite field with its sampler and G an additive F -module, both with decidable equality. Suppose $r \mapsto rg$ is injective. Fix a fingerprint function, preparation program P and a family of adversaries indexed by its returned state. Suppose P makes at most p hash requests and every casting callback makes at most c hash*

requests, counting all three proof domains. Let R run the original `preparedSource` from the empty full cache and attach flag false. Let S be `preparedSim`, returning the fields above. Then

$$d_{\text{TV}}(R, S) \leq (11p + 2c + 126)/|F|.$$

The comparison imposes no acceptance, freshness or supplied cache-cover premise. Guessing callbacks may make arbitrary later queries. The budgets are structural query bounds, not runtime certificates.

Lean: `ElectionFullSimulation.prepared_distance_le`.

Lean: `ElectionFullSimulation.world_distance_le`.

The simulated prefix preserves its public generator/key relation on every supported branch. Its ballot cache grows by at most twelve requests. Combining that cover with the full cache after the key proof gives a conservative successor cover of $2k + 13$ from an initial k -entry cover. The actual casting callback adds at most c entries; submission verification and trustee simulation then cost $(4k + 2c + 35)/|F|$. Adding prefix loss $(7k + 91)/|F|$ gives the displayed bound. Preparation derives $k \leq p$; mapping guesses to the winning bit preserves it. All other-domain entries and earlier private flags survive the composition.

Lean: `runBallotProofSim_cache_bound`.

Lean: `ElectionPrefixSimulation.sim_covered`.

Lean: `ElectionPrefixSimulation.sim_parameters`.

Controls and residual obligations. A checked instance over $\mathbb{Z}_{257}$ makes an actual preparation query and uses casting/guessing callbacks that query the retained cache. Its loss is at most $139/257 < 1$. The guessing callback distinguishes saved states, and another control proves that an earlier prefix collision survives finishing and guessing. Independent reachable-cache enumeration adds 363 growth cases and detects a mutant that omits key-proof growth; the earlier transcript and rejection controls remain. These fixtures do not enumerate the full election distribution.

Lean: `ElectionFullSimulationControls.prepared_nontrivial`.

Lean: `ElectionFullSimulationControls.loss_below_one`.

Lean: `ElectionFullSimulationControls.guess_observes_saved`.

Lean: `ElectionFullSimulationControls.prefix_flag_survives`.

The hybrid has already interpreted its hash queries. The next subsection gives an explicit replayable source with this runtime and derives accepted live-proof validity. Section 1.63 accounts for rejection and joint extraction.

1.62 Historical component: Replayable full-source correspondence

The source retains the original preparation and attacker callbacks. Raw ballot requests pass through the existing programmed-ballot handler, which keeps a live oracle and a shadow cache. Key/decryption contexts use the auxiliary full cache. Honest proof requests record their actual statements; raw callbacks cannot add or erase that record. Trustee simulation preserves the ballot projection and carries earlier collision flags.

Theorem 1.63 (Replayable complete source, machine-checked). *Let F be a finite field with its sampler and G an additive F -module, both with decidable equality. Fix a fingerprint function, g , an arbitrary preparation program P , and an adversary family A indexed by its returned state. Let R evaluate `ElectionProgrammedSource.prepared` with empty initial source state and empty live ballot cache. Its result is $((a, s), L)$, where a contains the complete public result and winning*

bit, and s contains the full shadow cache C , private flag b , and recorded statements T . For $\pi((a, s), L) = ((a, b), C)$, suppressing the fixed fingerprint and g arguments,

$$\text{map}(\pi, R) = \text{ElectionFullSimulation.preparedSim}(P, A).$$

The equality is of probabilistic computations, not merely an equality of winning probabilities. Every supported output additionally satisfies

$$L \leq \text{project}(C), \quad \forall(stmt, c), \quad stmt \notin T \Rightarrow C(\text{ballot}(stmt, c)) = L(stmt, c).$$

No cache invariant, acceptance, freshness, query bound or target-exclusion premise is supplied to this complete-source result.

Lean: `ElectionProgrammedSource.prepared_runtime_eq.`

Lean: `ElectionProgrammedSource.prepared_inv.`

The local raw and honest-pair phase correspondences reuse the previously checked ballot runtime and auxiliary-domain lowering. Their intermediate invariants are derived by the enclosing preparation and prefix execution. The existing invariant-preservation proof now applies at those intermediate states; its empty-state corollary reuses it. The complete source composes the actual key proof, honest pair, casting, submission, trustee proofs and guessing.

Lean: `ElectionProgrammedSource.raw_runtime.`

Lean: `ElectionProgrammedSource.raw_inv.`

Lean: `ElectionProgrammedSource.ballots_runtime.`

Lean: `ElectionProgrammedSource.ballots_inv.`

Lean: `ElectionProgrammedSource.raw_requests.`

Lean: `ballotProgrammed_preserves_inv_of_inv.`

Checked counterexample and controls. A fresh raw ballot request remains exactly a live source query. Over $\mathbb{Z}_{11}$ it has the same returned-scalar law as ordinary uniform sampling. The latter has zero live hash requests; the former cannot satisfy a zero-query bound. Thus equality of output laws does not preserve query trees or available replay locations.

Lean: `ElectionProgrammedSourceControls.fresh_eq_query.`

Lean: `ElectionProgrammedSourceControls.fresh_not_zero.`

Lean: `ElectionProgrammedSourceControls.same_output_law.`

Lean: `ElectionProgrammedSourceControls.hidden_zero.`

Lean: `ElectionProgrammedSourceControls.same_law_different_source.`

Independent deterministic enumeration checks 5602 reachable mixed raw/programmed cache histories of length at most four, with adaptive query choices and both incoming flag values. Four mutants erase auxiliary entries, reset the incoming flag, hide raw requests or treat programmed shadow entries as live. All are detected, with zero failures or discards. These are cache/history fixtures; they do not enumerate complete election distributions or execute joint forks.

Accepted proofs in the complete live cache. The same source now connects actual weeding to replay verification.

Theorem 1.64 (Complete-source accepted validity, machine-checked). *Use the finite field, module and sampler of the preceding theorem. Let $((V, w), S, L)$ be a supported output of the prepared source from empty initial state and live cache. Suppose both entries of $V.beforeTally.honestDecisions$*

and $V.decision$ are accepted. Set $b = V.submission$ and take pk from its public prefix. For each component or aggregate index i , there exists c such that

$$L(b.coveredStatement(g, pk, i), b.coveredProof(i).commitment) = \text{some}(c)$$

and the full covered proof is valid for that stored challenge c .

Lean: [ElectionProgrammedSource.prepared_live](#).

Raw preparation preserves an initially empty programming record; trustee key simulation also preserves it. The honest-pair provenance theorem now permits arbitrary prior caches and flags, with an empty incoming record. When both honest decisions accept, every recorded statement belongs to a retained ballot. The actual malicious submission's expanded weeding therefore excludes each of its three targets from that record. Shadow validity follows from actual verifier acceptance; the derived invariant transfers its stored challenges to the live cache. Subsequent trustee and guessing execution preserve those live answers. No freshness, target-exclusion or cache-invariant premise is supplied to the complete-source theorem.

Lean: [repairedCastHonestPairOracle_programmed_targets_from](#).

A checked specialization includes actual preparation queries in both key and ballot domains, retaining their answers for the attacker. A negative control shows that raw execution preserves an incoming foreign record, prior key-cache entry and collision flag: raw execution cannot clear an arbitrary record. The earlier accepted-third-ballot and rejected-honest-target recurrence controls remain; both honest acceptance decisions are necessary to this argument.

Lean: [ElectionProgrammedSourceControls.prepared_live_after_queries](#).

Lean: [ElectionProgrammedSourceControls.foreign_record_survives](#).

The next subsection derives the honest-rejection probability loss, source query budget and joint extraction bound.

1.63 Historical component: Joint extraction from complete programmed elections

Let *source* be the actual prepared source of the preceding subsection, run from empty state. It retains the original public result, winning bit and private state. The extractor samples one original path and makes three conditional replay attempts on that path, for the two component proofs and aggregate proof. It returns that original output with the three witnesses. No fresh first path is chosen for an individual proof.

Theorem 1.65 (Complete programmed-source extraction, machine-checked). *Let F be a finite field with its sampler, $q = |F|$, and G an additive F -module, both with decidable equality. Fix g such that $r \mapsto rg$ is injective, a fingerprint function, a preparation program and an attacker family indexed by its saved preparation state. Suppose preparation makes at most p hash requests, every casting callback at most c , and every guessing callback at most d , counting all three proof domains. Put*

$$N = p + c + d + 15, \quad R = \frac{9}{q-1} + \frac{11p + 2c + 126}{q}.$$

The actual source has at most N live ballot-hash queries. Its probability of rejecting at least one honest ballot is at most R . Let a be the probability that the malicious submission is accepted in this same source. For every threshold $\delta \in \mathbb{R}_{\geq 0}^\infty$, the actual joint extractor with parameter N satisfies

$$\Pr[\text{joint succeeds}] \geq (a - R - 3(N+1)\delta) \left(\delta - \frac{1}{q} \right)^3.$$

Subtractions use truncated nonnegative arithmetic. On success all three witnesses concern the same retained original submission and its actual public key, and that original output has a supported complete-source execution. Both honest decisions and the malicious decision are accepted on success. If additionally $2 \neq 0$ in F , the extracted nonces and integer vote counts sum consistently, and the malicious vote count is at most one.

Lean: `ElectionProgrammedExtraction.source_bound.`

Lean: `ElectionProgrammedExtraction.prepared_rejection_le.`

Lean: `ElectionProgrammedExtraction.joint_le.`

Lean: `ElectionProgrammedExtraction.joint_valid.`

Lean: `ElectionProgrammedExtraction.joint_consistent.`

Why the protocol premises now follow. The source bound is structural. Auxiliary lowering preserves the ballot-query budget from each actual intermediate cache. Existing sampling, submission and honest-program bounds compose through the source’s original callbacks. The allowance of fifteen protocol queries is conservative; ordinary randomness and auxiliary proof simulation do not issue live ballot hashes. The theorem also permits separate, tighter ballot-only callback bounds.

Lean: `ElectionQueryBound.lower_bound.`

In the original complete election, finishing and guessing retain the sampled honest decisions. Their rejection bound is therefore the historical $9/(q-1)$ bound. The checked full proof-simulation distance transfers this event to the programmed source, accounting for the second term of R . This transfer does not assert that honest rejection has probability zero.

On paths where both honest ballots and the malicious submission are accepted, Theorem 1.64 supplies full verification in the live cache. The derived query budget supplies the actual replay locations. The simultaneous selector event is exactly this joint acceptance event on the original path. Its probability is at least $a - R$. The existing replay theorem then charges $3(N+1)\delta$ for low-success contexts on that common path and retains the $1/q$ repeated-challenge loss in each of the three conditional attempts.

Lean: `ElectionProgrammedExtraction.selection_probability.`

Controls and remaining construction. A checked instance over $\mathbb{Z}_{257}$ makes preparation queries in both key and ballot domains and gives both answers to an attacker with querying casting and guessing callbacks. Here $p = 2$, $c = d = 1$, $N = 19$, and the rejection allowance is $9/256 + 150/257$. The joint context loss is 60δ . The displayed lower bound is positive at $a = 1$ and $\delta = 1/200$; this is arithmetic nonvacuity, not a claim that actual acceptance mass is one. A separate control returns opposite guesses for saved values five and six under the same cached answer.

Lean: `ElectionProgrammedExtractionControls.rejection_after_queries.`

Lean: `ElectionProgrammedExtractionControls.joint_after_queries.`

Lean: `ElectionProgrammedExtractionControls.losses_allow_positive_bound.`

Lean: `ElectionProgrammedExtractionControls.saved_answer_changes_guess.`

Independent repair/provenance, mixed-cache and full/split replay fixtures remain, as do the positive common-path replay and disjoint-marginal counterexample. These fixtures do not establish a full-election acceptance mass or DDH hardness. The complete source in this subsection samples the election secret. The following construction supplies the needed ciphertext input laws and share identity; connecting its execution to that complete source is outside this component’s guarantee.

1.64 Historical component: DDH ciphertext pairing and the statement interface

A challenge supplies group elements A, T without their scalar witnesses. The old honest-proof handler computes its statement from a witness, so a dummy witness would program the wrong statement. The exposed statement step uses the same existing simulator. The old handler factors through it exactly, including its state. The step leaves the live cache unchanged, records the supplied statement, preserves the live/shadow invariant and makes no live hash query.

Lean: `ballotProgrammedImpl_statement`.

Lean: `ballotProgrammedStatement_runtime`.

Lean: `ballotProgrammedStatement_inv`.

Lean: `ballotProgrammedStatement_query_bound`.

Write $E(r, m) = (rg, mg + rpk)$ and $v \in \{0, 1\}$ for the challenge vote. The historical experiment swaps abstention and candidate zero. Both honest second components encrypt zero. Given known scalars t, a, b , define

$$C = (A, vg + T), \quad L = (C, E(a, 0)), \quad R = (E(t, 1) - C, E(b, 0)), \quad K = (t, a + b).$$

The output retains both ciphertext vectors and K . Here subtraction is componentwise group subtraction. No discrete logarithm of A or T is an input to this construction.

Theorem 1.66 (Challenge input laws, machine-checked). *Let F be a field and G an additive F -module. For every g, pk , vote v and scalars x, y, a, b , setting $A = xg$, $T = xpk$, $t = x + y$ gives exactly*

$$L = (E(x, v), E(a, 0)), \quad R = (E(y, 1 - v), E(b, 0)), \quad K = (x + y, a + b).$$

These ciphertexts equal those in the existing historical honest-ballot algorithm for arbitrary proof coins and hash. For finite F with decidable equality and its sampler, the joint law obtained by independently sampling x, a, t, b uniformly equals the joint law above with independent uniform x, a, y, b . Replacing all four original nonces by the historical nonzero sampler changes this law by total variation at most $4/|F|$.

For fixed g, pk, A, t, a, b and uniform $z \in F$, the output law with $T = zg$ is independent of v : the bijection $z \mapsto z - v$ absorbs its contribution. For every challenge, including one not satisfying the DH relation,

$$L_i + R_i = E(K_i, m_i), \quad (m_0, m_1) = (1, 0).$$

If $pk = sg$ and malicious ciphertexts M_i have valid witnesses (u_i, r_i) , meaning $M_i = E(r_i, u_i)$ with u_i a bit, then

$$(K_i + r_i)pk = \text{partialDecrypt}_s(L_i + R_i + M_i).$$

The left side computes the share from known sums and the malicious witness nonce; the secret appears only in the correctness statement.

Lean: `ElectionDDHConstruction.original_honestBallots`.

Lean: `ElectionDDHConstruction.paired_real`.

Lean: `ElectionDDHConstruction.paired_ciphertexts_real_eq`.

Lean: `ElectionDDHConstruction.paired_ciphertexts_nonzero_distance_le`.

Lean: `ElectionDDHConstruction.paired_ciphertexts_random_eq`.

Lean: `ElectionDDHConstruction.paired_totals`.

Lean: `ElectionDDHConstruction.tally_shares_eq`.

Controls and boundary. The independent multiplicative fixture uses the subgroup of order eleven modulo 23, generator two and public key eight. Exhaustive finite comparisons cover 29,282 real-law cases, 2,662 arbitrary challenges and 121 random-mask comparisons (the latter two with zero-component nonces two and five), with no failures or discarded cases. Six deliberate mutations are detected. Checked exponent-form controls retain a real instance, arbitrary-challenge cancellation, a zero combined nonce from two nonzero nonces, and failures from addition, omitting a malicious nonce, and using a dummy statement. For four nonzero draws the fixture’s exact distance is 4641/14641, bounded by 4/11; dropping that loss would be false.

Lean: `ElectionDDHConstructionControls.real_fixture.`

Lean: `ElectionDDHConstructionControls.zero_combined_nonce.`

Lean: `ElectionDDHConstructionControls.arbitrary_challenge_totals.`

Lean: `ElectionDDHConstructionControls.addition_mutant_changes_total.`

Lean: `ElectionDDHConstructionControls.malicious_nonce_required.`

Lean: `ElectionDDHConstructionControls.dummy_statement_differs.`

These input laws and the conditional share identity do not establish complete-election secrecy. The following subsection supplies actual honest-pair correspondence. Obtaining malicious witnesses before tally publication, charging per-original-path extraction failure and the historical key-sampling loss, and proving the full game reduction are outside this component’s guarantee.

1.65 Historical component: Actual honest submissions from challenge ciphertexts

The construction now programs a proof for each component and for their sum, then calls the existing repaired submission algorithm. It generates and submits the second ballot using the board retained by the first submission. Verification still precedes expanded weeding; an invalid proof or reused ciphertext retains the incoming board. All six proof requests occur, even if a submission rejects.

Let $H_{r,s}$ be the existing programmed honest-pair source with nonce pairs $r = (x, a)$ and $s = (y, b)$ fixed. Let $D(A, T, t, a, b, v)$ be the new challenge source using the pair of Section 1.64, retaining its known column sums. A programming state contains the shadow cache, sticky collision flag and ordered statement record; interpretation additionally retains the live cache.

Theorem 1.67 (Honest-pair source correspondence, machine-checked). *Let F be a field with its sampler, G an additive F -module, and both types have decidable equality. For arbitrary g, pk, v, x, a, y, b , the new honest-pair program with challenge (xg, xpk) and $t = x + y$ equals $H_{r,s}$ as a source computation, including all proof and raw verification requests and successor states. Retaining the private sums additionally returns $(x + y, a + b)$.*

For finite F , augment the original sampled source by returning those private sums. Projecting them away gives exactly the unchanged sampled source. For every initial programming state S and live cache L ,

$$\text{TV} \left(\text{Run}(H_{\neq 0}^+; S, L), \quad \begin{array}{l} x, a, t, b \leftarrow U(F); \\ \text{Run}(D(xg, xpk, t, a, b, v); S, L) \end{array} \right) \leq \frac{4}{|F|}.$$

Both distributions retain both decisions, the actual board, programming state, live cache and private sums. No freshness, acceptance, empty-cache or generator- injectivity premise is required for this comparison. Embedding into the existing full election state preserves this bound against its actual honest phase, projecting away only the newly retained private sums.

For arbitrary A, T , the challenge source preserves live/shadow consistency whenever it holds initially, in both the ballot and full election state. This includes either rejection branch.

Lean: `ElectionDDHSource.ballot_real`.
 Lean: `ElectionDDHSource.honest_pair_real`.
 Lean: `ElectionDDHSource.challenge_pair_real`.
 Lean: `ElectionDDHSource.sampledWithSumsfst`.
 Lean: `ElectionDDHSource.honest_pair_real_distance_le`.
 Lean: `ElectionDDHSource.ballots_real_distance_le`.
 Lean: `ElectionDDHSource.ballot_inv`.
 Lean: `ElectionDDHSource.honest_pair_inv`.
 Lean: `ElectionDDHSource.ballots_inv`.

Why the two equalities differ. At fixed nonces the ciphertext identity rewrites every requested component and aggregate statement, so the source computations agree. To compare the sampled sources, the independent second nonce pair is moved before the first ballot’s runtime. This step uses commutation only after oracle interpretation. It preserves the joint output law, including caches, but does not identify the source query trees or replay locations of the two schedules. The next prefix extractor must derive those facts for its actual challenge source.

Lean: `ElectionDDHSource.sampledWithSums_schedule`.

Controls and component scope. Six checked controls cover source equality after prior state, exact sampled-tree projection, the full-cache bound after a key query, a supported conflicting statement, actual invalid-first-proof rejection retaining any board, and nonempty incoming cache/flag/record. The independent multiplicative harness uses fixed coin tapes with cache misses, agreeing hits and conflicting hits, and both incoming flags. Seeds 17, 41 and 103 give 1,800 cases: their 3,600 decisions include 2,688 acceptances, 600 invalid-proof rejections and 312 reuse rejections. Seven deliberate defects are detected, with no failures or discarded cases. The harness checks fixed-tape behavior, not a full-election probability law.

Lean: `ElectionDDHSourceControls.real_pair_after_cache`.
 Lean: `ElectionDDHSourceControls.projection_preserves_sampled_source`.
 Lean: `ElectionDDHSourceControls.full_loss_after_key_query`.
 Lean: `ElectionDDHSourceControls.conflicting_statement_reachable`.
 Lean: `ElectionDDHSourceControls.conflicting_first_proof_rejects`.
 Lean: `ElectionDDHSourceControls.initial_is_not_empty`.

The following result supplies extraction from the actual prepared submission prefix.

1.66 Historical component: Extraction before tally publication

The reduction receives public elements g, pk, A, T . Its prefix runs the actual querying preparation, samples the vote and known nonce sums, simulates the key and honest-ballot proofs, and executes the casting callback and repaired submission. It stops before publishing tally shares or running the guessing callback. The retained output O contains the public prefix, malicious ballot, submission decision and board, preparation result, casting state, vote, known sums and full source state. Retaining preparation separately is necessary: $(adversary\ initial).guessVote$ may depend on it even when casting returns identical saved states.

Theorem 1.68 (Prepared-prefix joint extraction, machine-checked). *Let F be a finite field with its sampler and decidable equality, and let G be an additive F -module with decidable equality. Suppose $g \neq 0$. Let p, c bound live ballot-hash requests in preparation and every casting callback, respectively.*

Define $N = p + c + 9$, and let a be the probability that both honest submissions and the malicious submission accept in the actual prefix, starting with empty caches. For every $\delta \in \mathbb{R}_{\geq 0} \cup \{\infty\}$,

$$(a - 3(N + 1)\delta)(\delta - |F|^{-1})^3 \leq \Pr[\text{prefixJoint}_N \text{ returns witnesses}].$$

Subtractions are truncated at zero. The extractor uses one original prefix execution and three residual replay attempts. Every supported successful return (O, w) retains a supported original runtime output, with the supplied public parameters, all three acceptance decisions, and witnesses for all three covered statements of that original malicious ballot. If scalar multiplication by g is injective and $2 \neq 0$ in F , those witnesses have consistent component and aggregate nonces and bits, and their component bit sum is at most one.

Lean: `ElectionDDHSource.prefix_joint_le`.

Lean: `ElectionDDHSource.prefix_joint_valid`.

Lean: `ElectionDDHSource.prefix_joint_consistent`.

Derived premises. The actual source has at most six live ballot hashes for honest submissions and three for the malicious submission. Key and ballot proof programming makes no live ballot-hash requests. Preparation and key setup preserve the empty record of programmed ballot statements. Both honest acceptances therefore place every programmed target on the retained board. Accepted malicious submission excludes those targets and yields validity in its final live cache. Source consistency, public parameters and the joint-selector acceptance event are derived for this source, including prior queries; none is an additional caller premise of the extraction bound. The challenge need not be a real DDH tuple.

Lean: `ElectionDDHSource.ballot_targets`.

Lean: `ElectionDDHSource.honest_pair_targets`.

Lean: `ElectionDDHSource.ballots_targets`.

Lean: `ElectionDDHSource.ballots_bound`.

Lean: `ElectionDDHSource.prefix_inv`.

Lean: `ElectionDDHSource.prefix_parameters`.

Lean: `ElectionDDHSource.prefix_live`.

Lean: `ElectionDDHSource.prefix_bound`.

Lean: `ElectionDDHSource.prefix_selection_probability`.

Lean: `ballotJointReplay_selection_probability`.

Controls and boundary. The checked controls instantiate querying preparation and casting over $\mathbb{Z}_{257}$ at $(g, pk, A, T) = (1, 3, 2, 7)$, where $T \neq 2pk$. Their budget is ten and their context allowance is 33δ . Changing the guessing callback leaves the entire prefix source identical; a separate control checks that the callbacks differ. Another shows why preparation state cannot be recovered from casting state. At $a = 1$ and $\delta = 1/200$ the displayed lower bound is positive; this arithmetic check does not assert actual acceptance mass one. The existing independent provenance, fixed-tape honest-source and replay-history fixtures pass again.

Lean: `ElectionDDHPrefixControls.bound_after_queries`.

Lean: `ElectionDDHPrefixControls.joint_after_queries`.

Lean: `ElectionDDHPrefixControls.challenge_is_not_dh`.

Lean: `ElectionDDHPrefixControls.stops_before_guess`.

Lean: `ElectionDDHPrefixControls.guessing_suffix_differs`.

Lean: `ElectionDDHPrefixControls.preparation_changes_guess`.

Lean: `ElectionDDHPrefixControls.positive_bound_possible`.

A positive extraction probability does not bound the distance from the original election game. The following result supplies repeated-attempt failure accounting while preserving the original execution.

1.67 Historical component: Repeated extraction retaining the original execution

Let P be one sampled original prefix path. A residual joint attempt executes three replay completions selected from P . The finite loop $R_k(P)$ executes at most k such joint attempts, returning the first witness triple or failure. It never samples another original path. The complete extractor always returns $((O, L), w)$, where O is the original prefix output, L is its actual live cache, and w is an optional witness triple. It projects L from the original raw path output; residual caches never replace it.

Theorem 1.69 (Original-preserving extraction failure, machine-checked). *Use the finite-field, module, sampler and decidable-equality hypotheses of Theorem 1.68, with $g \neq 0$ and the same preparation/casting live-hash bounds p, c . Put $N = p + c + 9$. For every finite repetition count k and $\delta \in \mathbb{R}_{\geq 0} \cup \{\infty\}$,*

$$\Pr[\text{Accepted}(O) \wedge w = \text{none}] \leq 3(N + 1)\delta + (1 - (\delta - |F|^{-1})^3)^k.$$

Here **Accepted** requires both honest decisions and the malicious submission to accept; subtraction is truncated. The marginal distribution of (O, L) equals the original prefix runtime distribution exactly, including all rejections and extraction failures. There is no conditioning on success.

Every supported successful return contains witnesses for the original submission and a live cache that belongs to that same supported original runtime. Private/live consistency is derived. Scalar injectivity and $2 \neq 0$ add the component/aggregate nonce and vote-count consistency conclusions of Theorem 1.68.

Lean: `ElectionDDHSource.prefix_failure_le`.

Lean: `ElectionDDHSource.prefix_repeated_original`.

Lean: `ElectionDDHSource.prefix_repeated_valid`.

Lean: `ElectionDDHSource.prefix_repeated_consistent`.

Probability and interaction accounting. For a fixed P , let $s(P)$ be the actual joint-attempt success probability. The loop has failure probability $(1 - s(P))^k$ and no missing probability mass. Outside the existing low-context event, enabled selectors give $s(P) \geq (\delta - |F|^{-1})^3$. Averaging over the original P charges the low-context event once by $3(N + 1)\delta$. The prefix's pointwise selector condition follows from its actual trace, budget and live validity.

Lean: `replayRepeat_failure`.

Lean: `ballotJointReplayRepeatAtPath_lossless`.

Lean: `ballotJointReplayRepeatAtPath_failure_le`.

Lean: `ballotJointReplayRetained_failure_le`.

Lean: `ballotJointReplayRetained_original`.

Lean: `ballotJointReplay_selector_eq`.

Lean: `ElectionDDHSource.prefix_trace_verified`.

If the complete original source has at most m total oracle interactions on every branch, the repeated extractor has at most $(1 + 3k)m$. This includes uniform draws. The premise is a total-source bound, not the N live ballot-hash bound. This theorem assumes the total-source bound; it does not derive it from primitive sampler and callback costs or account for pure operations and standard-PPT execution. Choosing negligible δ does not automatically allow polynomially many repetitions; the eventual reduction must choose δ, k with explicit advantage and cost bounds.

Lean: [replayRepeat_bound](#).

Lean: [ballotJointReplayRetained_total_query_bound](#).

Lean: [ElectionDDHSource.prefix_repeated_query_bound](#).

Checked counterexamples and controls. Two independent fair residual attempts fail with probability $1/4$; reusing one random choice fails with probability $1/2$. Zero attempts fail even when a single attempt would surely succeed. An unextractable original tag retains its original mass $1/2$. Resampling the original instead increases the successful tag's mass to $3/4$. These exact controls exclude replacing the original execution during extraction. Two further controls instantiate the actual non-DH querying prefix's failure bound and full original-cache distribution. An independent rational-tape enumeration checks 1,800 cases over seeds 17, 41 and 103, with zero failures/discards and four detected defects. This finite experiment validates the probability decomposition, not cryptographic replay semantics or polynomial runtime.

Lean: [BallotReplayRepeatControls.two_attempts](#).

Lean: [BallotReplayRepeatControls.reused_randomness_fails](#).

Lean: [BallotReplayRepeatControls.zero_attempts](#).

Lean: [BallotReplayRepeatControls.original_tag_preserved](#).

Lean: [BallotReplayRepeatControls.resampling_bias](#).

Lean: [BallotReplayRepeatControls.prefix_failure_after_queries](#).

Lean: [BallotReplayRepeatControls.prefix_original_after_queries](#).

The following comparison composes the complete real-challenge source with the historical election. The implementable DDH finisher in the later component replaces secret-based shares by extracted witnesses and known column nonces. Source-cost derivation, the full advantage inequality and standard-PPT adequacy are outside this component's guarantee.

1.68 Historical component: Complete real-challenge comparison

The comparison experiment samples $sk, x \leftarrow U(F)$ and executes the actual public-input prefix with

$$(g, pk, A, T) = (g, sk\ g, xg, x\ pk).$$

Its finishing continuation publishes trustee proofs for the correct shares computed using sk , then runs the original guessing callback with both its preparation and casting state. This is a mathematical comparison hybrid. The implementable DDH finisher uses extracted witnesses and known nonces instead of sk .

Theorem 1.70 (Complete real-challenge comparison, machine-checked). *Let F be a finite field with its sampler and decidable equality, and G an additive F -module with decidable equality. Write $q = |F|$. Replacing the historical nonzero key and four honest nonces by the real-challenge construction changes the complete programmed-source runtime distribution by at most $5/q$. This comparison retains the final public result, winning bit, complete source state and actual live cache. It has no acceptance or empty-intermediate-cache premise.*

If scalar multiplication by g is injective, and p, c bound all proof-hash requests during preparation and every casting callback respectively, then

$$\text{TV}(\Gamma_{\text{historical}}, \Gamma_{\text{real comparison}}) \leq \frac{11p + 2c + 131}{q}.$$

Both distributions retain every public field, winning bit, private collision flag and final full cache; the original game's added flag is false. Rejection branches and the original guessing callback remain, with no bound on its later queries. The comparison world still uses the sampled secret for its shares.

Lean: [ElectionDDHSource.source_real_distance_le](#).

Lean: [ElectionDDHSource.prepared_real_distance_le](#).

Composition and source boundary. The honest-pair $4/q$ comparison survives the actual casting, repaired submission, trustee simulation, publication and guessing continuation. Replacing the key adds $1/q$. Composing these with the checked full proof-simulation allowance $(11p + 2c + 126)/q$ gives the constant 131. Independent key, exponent and honest- nonce draws commute only after oracle interpretation. The actual prefix's query positions and extraction properties continue to use their own checked source, rather than an inferred replay-tree equality.

Lean: [ElectionDDHSource.after_honest_real_distance_le](#).

Lean: [ElectionFullSimulation.prepared_distance_le](#).

The exact random-draw runtime equation preserves the complete prior source state and live cache. The actual finishing continuation preserves the supplied prefix, submission, decision, retained board, encrypted tally and correct secret-based shares in every supported final output, including after guessing queries. Neither result filters to accepted ballots.

Lean: [ElectionDDHSource.run_programmed_prob](#).

Lean: [ElectionDDHSource.raw_prob](#).

Lean: [ElectionDDHSource.finishAfterCast_publication](#).

Controls and composition. At $q = 257$, querying preparation and saved-state-dependent casting give $p = 2, c = 1$ and allowance $155/257 < 1$. Uniform key sampling returns zero with mass $1/257$, while the historical sampler never does. Additional controls preserve a nonempty cache, sticky flag and recorded target across random draws, detect cache reset, and preserve invalid-proof rejection and its retained board through actual finishing and guessing. The independent five-coordinate sampling fixture enumerates 181,258 tuples for $q = 2, 3, 5, 7, 11$, with zero failures/discards. Its exact input variation is $1 - ((q - 1)/q)^5 \leq 5/q$; at $q = 11$ it exceeds $4/q$, excluding omission of the key replacement from that input comparison. The fixture tests sampling laws; continuation correspondence is proved in Lean.

Lean: [ElectionDDHRealSourceControls.complete_comparison](#).

Lean: [ElectionDDHRealSourceControls.comparison_nonvacuous](#).

Lean: [ElectionDDHRealSourceControls.key_sampling_differs](#).

Lean: [ElectionDDHRealSourceControls.random_draw_preserves_state](#).

Lean: [ElectionDDHRealSourceControls.reset_loses_answer](#).

Lean: [ElectionDDHRealSourceControls.rejected_finish](#).

The following construction replaces the secret-based finish using actual extracted witnesses.

1.69 Historical component: Finishing from extracted ballot nonces

The reduction's inputs are g, pk, A, T , preparation and the adversary's callbacks. It runs the actual repeated prefix extractor using the pinned uniform-query sampler, retains its original prefix, complete state and live cache, then publishes trustee proofs and calls the original guessing callback. The secret is absent from this computation. No branch is conditioned on extraction success.

Source-derived board and shares. Let K_i be the prefix's retained honest column nonce sum. Supported honest ballot generation returns its supplied ciphertext fields. When both honest submissions accept, their actual board entries therefore give

$$\text{boardTally}(\text{before.board})_i = \text{Enc}_{g,pk}(K_i, \mathbf{1}_{i=0}).$$

This is derived from the actual prefix, including later casting queries and malicious submission; board presentation and freshness are not caller premises. The malicious submission appends its ballot exactly on acceptance and preserves the honest board for either rejection reason. Initial and successor consistency are derived from execution.

Lean: [ElectionDDHSource.ballots_columns](#).

Lean: [ElectionDDHSource.prefix_honest_columns](#).

Lean: [ElectionDDHSource.prefix_cast_board](#).

Lean: [ElectionProgrammedSource.raw_support](#).

For extracted component nonces r_i , finishing computes

$$D_i = \begin{cases} (K_i + r_i) pk, & \text{malicious acceptance and extraction success,} \\ K_i pk, & \text{otherwise.} \end{cases}$$

An accepted ballot without extraction uses an explicit fallback; correctness is not asserted on that branch. If both honest ballots accept and the accepted malicious ballot has extracted witnesses, these shares equal the secret-based shares for the actual retained tally. Repeated-extractor success supplies the witnesses for the original submission. The resulting finishing source, including proof generation and guessing, is exactly the secret-based continuation on these good retained executions.

Lean: [ElectionDDHSource.prefix_extracted_shares](#).

Lean: [ElectionDDHSource.extracted_finish_good](#).

Theorem 1.71 (Complete extracted finishing, machine-checked). *Let F be a finite field with its sampler and decidable equality, and let G be an additive F -module with decidable equality. Write $q = |F|$. Fix $g \neq 0$, $sk \in F$ and arbitrary $A, T \in G$; put $pk = sk g$. Suppose preparation has at most p live ballot-hash queries and every casting callback has at most c . For any $k \in \mathbb{N}$ and $\delta \in \mathbb{R}_{\geq 0}^\infty$, set $N = p + c + 9$. Let E_{ext} run k residual extraction attempts with bound N , then finish with the public-input shares above. Let E_{secret} run the same original prefix and finish using sk . If R is the actual probability that the original prefix's two honest decisions are not both accepting, then*

$$\text{ofReal}(\text{TV}(E_{\text{ext}}, E_{\text{secret}})) \leq R + 3(p + c + 10)\delta + (1 - (\delta - q^{-1})^3)^k.$$

Subtraction on the right is truncated in $\mathbb{R}_{\geq 0}^\infty$. Both outputs retain the full public result, winning bit, complete private state and original live cache. Later guessing queries are unrestricted. No acceptance, extraction success, supplied witness or board-correspondence premise occurs in this bound. Its honest-rejection term is an actual event probability, not a proved negligible allowance.

Lean: [ElectionDDHSource.extracted_finish_distance_le](#).

The proof charges honest rejection and accepted extraction failure, then uses the exact retained-original distribution. This distributional composition does not replace the separately checked source-position or extraction proofs. On bad branches the finisher still publishes the original prefix, submission, decision, board and encrypted tally. Its shares follow the explicit fallback formula; they need not be correct there.

Lean: [ElectionDDHSource.finishExtracted_publication](#).

Controls and composition boundary. At $q = 257$, $pk = 3$, $K = (5, 7)$ and malicious nonces $(6, 9)$ give accepted shares $(33, 48)$ and rejected shares $(15, 21)$. Missing extraction differs from the correct accepted share. A rejected honest ballot gives a counterexample to using the pair sum without both honest acceptances. An actual querying non-DH prefix and its original guessing callback instantiate the complete bound with $N = 10$ and context allowance 33δ .

Lean: [ElectionDDHExtractedFinishControls.accepted_shares](#).

Lean: [ElectionDDHExtractedFinishControls.rejected_shares](#).

Lean: [ElectionDDHExtractedFinishControls.missing_witness_differs](#).

Lean: [ElectionDDHExtractedFinishControls.rejected_honest_sum_differs](#).

Lean: [ElectionDDHExtractedFinishControls.finish_after_queries](#).

Independent multiplicative subgroup fixtures check 7,986 finishing columns, including malicious rejection, arbitrary challenges and missing-witness and honest-rejection counterfactuals. The extended pairing fixture detects nine mutants with zero failures/discards; its arithmetic decision histories do not claim reachable protocol execution. The separate 1,800-case honest-source fixture retains actual decision/cache checks and seven detected mutants.

The following cost theorem derives the complete source budget. The later components bound R in both challenge worlds and choose δ, k with explicit advantage and polynomial trial counts.

1.70 Historical component: Complete source interaction costs

The cost counts every uniform-index oracle request, including private proof coins and auxiliary hash misses. It counts a cached auxiliary answer as zero new draws. Pure computation, operand widths and bit-operation costs are outside this metric and remain necessary for the standard-PPT theorem.

Theorem 1.72 (Actual extracted-game interaction bound, machine-checked). *Let F be a finite field with decidable equality and a sampler using at most one uniform-index query, and G an additive F -module with decidable equality. Fix arbitrary $g, pk, A, T \in G$, preparation and the original adversary callbacks. Suppose preparation uses at most P total requests, every casting callback at most C , and every guessing callback at most D . Then the actual prefix has complete interaction bound*

$$M = 4P + 4C + 78.$$

For every selector bound n and repetition count k , the implemented extracted game, including uniform replay interpretation and final publication/guessing, has total interaction bound

$$(1 + 3k)M + 4 + 4D.$$

This theorem covers every answer branch and every rejection or extraction fallback. It assumes no complete-source cost or acceptance certificate.

Lean: [ElectionDDHSource.prefix_total_bound](#).

Lean: [ElectionDDHSource.extractedGame_total_bound](#).

The proof derives the full-election lower-handler cost from its actual cases: one forwarded request or one scalar draw on an auxiliary miss, and no draw on an auxiliary hit. The existing ballot programmer costs at most four oracle nodes per source request. Its statement simulator uses four scalar draws even when its programming target collides. These conservative bounds yield the following prefix account:

$$\underbrace{4P}_{\text{preparation}} + \underbrace{4}_{\text{vote}} + \underbrace{2}_{\text{key proof}} + \underbrace{12}_{\text{known nonces}} + \underbrace{48}_{\text{honest ballots/submissions}} + \underbrace{4C}_{\text{casting}} + \underbrace{12}_{\text{submission}}.$$

Lean: [ballotSim_total_query_bound](#).

Lean: [ElectionDDHSource.lower_total_bound](#).

Final trustee simulation uses four scalar draws, and guessing contributes the conservative $4D$ from arbitrary incoming state/live cache. The existing repetition theorem supplies the factor $1 + 3k$, with its formerly assumed source budget now derived. Interpreting the actual wrapped replay oracle with the uniform sampler preserves this cost bound.

Lean: [ElectionDDHSource.finish_total_bound](#).

For prime q , the pinned default sampler on $\mathbb{Z}/q\mathbb{Z}$ uses one uniform finite-index request. Its existing theorem instantiates the scalar cost premise, so the prime-field complete-game result takes only the stated callback query bounds. It still requires a separate bit-level cost correspondence.

Lean: [ElectionDDHSource.extractedGame_prime_total_bound](#).

Controls and component scope. The actual querying callbacks over $q = 257$ have $P = 2, C = 1, D = 1$, yielding prefix bound 90 and complete bound $(1 + 3k)90 + 8$. Controls distinguish an auxiliary miss from a hit despite both having zero live ballot hashes, and show that a discarded private draw still counts as an interaction.

Lean: [ElectionDDHSourceCostControls.prefix_after_queries](#).

Lean: [ElectionDDHSourceCostControls.complete_after_queries](#).

Lean: [ElectionDDHSourceCostControls.auxiliary_miss_cost](#).

Lean: [ElectionDDHSourceCostControls.auxiliary_hit_cost](#).

Lean: [ElectionDDHSourceCostControls.discarded_draw_cost](#).

Independent upper-account enumeration checks 78,732 DDH cases and detects five omitted-cost rules, with zero failures/discards. Its branch lengths are accounting bounds, not assertions of reachable protocol histories or exact runtime. The existing submission/replay cost controls remain. The following public test bounds honest rejection in both DDH worlds. Extraction parameter choices, the complete advantage inequality and standard-PPT adequacy are outside this component's guarantee. This interaction count alone does not make arbitrary host callbacks polynomial-time attackers.

1.71 Historical component: Public rejection in the DDH games

The public rejection test receives (g, pk, A, T) , executes the actual prepared prefix and returns whether its two honest decisions are not both accepting. It includes preparation queries and the malicious casting/submission, and uses neither a secret key nor extraction. It fits the pinned `VCVio DiffieHellman.DDHAdversary` interface directly.

The real DDH game draws $a, b \leftarrow U(F)$ and supplies $(g, ag, bg, (ab)g)$; the random game also draws independent $c \leftarrow U(F)$ and supplies (g, ag, bg, cg) . Write $R_{\text{real}}, R_{\text{rand}}$ for this test's output-true probabilities. Its two-game advantage is

$$\Delta_R = |\text{toReal}(R_{\text{real}}) - \text{toReal}(R_{\text{rand}})|.$$

This is twice the corresponding guessing advantage. All exponent sampling is from the full field, as in the pinned DDH games.

Theorem 1.73 (Actual DDH honest-rejection bounds, machine-checked). *Let F be a finite field with its sampler and decidable equality, and G an additive F -module with decidable equality. Suppose $r \mapsto rg$ is injective. Let preparation have at most p proof-hash requests and every casting callback at most c , counting all proof domains. Put $q = |F|$ and*

$$\rho = \frac{9}{q-1} + \text{ofReal}\left(\frac{11p+2c+131}{q}\right).$$

Then, in $\mathbb{R}_{\geq 0}^\infty$,

$$R_{\text{real}} \leq \rho, \quad R_{\text{rand}} \leq R_{\text{real}} + \text{ofReal}(\Delta_R) \leq \rho + \text{ofReal}(\Delta_R).$$

The first probability equals the actual honest-rejection event in the complete real-challenge comparison, including its later guessing execution. These inequalities require no acceptance, supplied freshness, negligible-rejection or DDH-hardness premise. The actual test's distinguishing advantage remains explicit.

Lean: [ElectionDDHSource.honestReject_real](#).

Lean: [ElectionDDHSource.honestReject_real_le](#).

Lean: [ElectionDDHSource.honestReject_random_transfer](#).

Lean: [ElectionDDHSource.honestReject_random_le](#).

The real bound composes the original election's nonce-collision rejection allowance with the complete $131/q$ -constant comparison. The real-game identity uses the supported finishing theorem: later proof generation and guessing preserve the original public honest decisions. The random bound uses the absolute two-game probability difference; it does not identify a random tuple with a real one or resample rejected executions.

Lean: [ElectionProgrammedExtraction.original_prepared_rejection_le](#).

If the scalar sampler costs at most one uniform-index request, and complete preparation/casting request bounds are P, C , the public test has interaction bound $4P + 4C + 78$. This derives live-interpretor and prefix costs and does not execute the guessing callback. Bit-operation costs remain separate.

Lean: [ElectionDDHSource.ballot_runtime_total_bound](#).

Lean: [ElectionDDHSource.honestReject_total_bound](#).

Controls and composition boundary. The actual querying instance over $q = 257$ has real allowance $9/256 + 155/257 < 1$, random allowance equal to that plus its actual DDH advantage, and interaction bound 90. Changing the guessing suffix leaves this public test unchanged. An independent two-world table with rejection masses 0, 1 refutes omitting the advantage or substituting half of it.

Lean: [ElectionDDHRejectionControls.real_after_queries](#).

Lean: [ElectionDDHRejectionControls.random_after_queries](#).

Lean: [ElectionDDHRejectionControls.allowance_nonvacuous](#).

Lean: [ElectionDDHRejectionControls.advantage_charge_required](#).

Lean: [ElectionDDHRejectionControls.guessing_does_not_change_rejection](#).

Lean: [ElectionDDHRejectionControls.cost_after_queries](#).

Independent exact enumeration checks 1,513 Boolean probability tables, both advantage orientations and the factor-two identity, detecting three defective transfer rules with zero failures/discards. This validates probability arithmetic; the existing joint-sampling and actual honest-source controls remain. Section 1.72 supplies effective δ, k choices. The full secrecy inequality is stated in the formal reference. Applying DDH to the constructed reductions requires their named external efficiency assumptions.

1.72 Historical component: Effective integer extraction parameters

Construction and scope. For live prefix hash budget N , field cardinality q and a positive natural accuracy request e , define

$$\delta = \frac{1}{6(N+1)e}, \quad k = 3456(N+1)^3 e^4.$$

The implemented extractor computes only natural N, k ; δ is an analysis threshold. The same fixed original execution is retained on every outcome.

Theorem 1.74 (Extraction accuracy with an executable repetition count). *For $N, q, e \in \mathbb{N}$, $e > 0$ and $12(N+1)e \leq q$, extended-real truncated subtraction gives*

$$3(N+1)\delta + (1 - (\delta - q^{-1})^3)^k \leq e^{-1}.$$

Lean: [ballotReplay_parameters](#).

Checked proof structure. The context loss is $1/(2e)$. The field-size condition implies $t = (\delta - q^{-1})^3 \geq 1/(1728(N+1)^3 e^3)$, so $kt \geq 2e$. For $0 \leq t \leq 1$, induction proves $(1-t)^k(1+kt) \leq 1$, hence $(1-t)^k \leq (1+kt)^{-1} \leq (2e)^{-1}$. Positivity and the transfer to extended-real subtraction are checked explicitly. $\square$

Actual full-election instance. Set $N = p + c + 9$, where p, c bound live ballot hashes of preparation and every casting callback. For nonzero g , secret s , arbitrary public challenge coordinates A, T and the same callbacks as Section 1.69, let X_k be the actual public-input extracted game at $pk = sg$ and Y_s the actual secret-using completed comparison source. If $e > 0$ and $12(p + c + 10)e \leq |F|$, then

$$\text{TV}(X_k, Y_s) \leq \text{Pr}[\text{honest pair rejected in the original prefix}] + e^{-1}.$$

The comparison retains the complete public result, winning bit, private source state and live cache. The rejection term is an actual event, whose real/random DDH bounds are established above. No extraction-success, accepted-board or external witness premise is supplied.

Lean: [ElectionDDHSource.extracted_finish_accuracy](#).

For the pinned prime-field sampler, complete callback interaction bounds P, C, D give that same game's uniform-index query bound

$$(1 + 3 \cdot 3456(N+1)^3 e^4)(4P + 4C + 78) + 4 + 4D.$$

This cost holds for every public (g, pk, A, T) and natural N, e ; the size and positivity conditions are needed for accuracy. Pure operations and standard-PPT execution are still outside this bound.

Lean: [ElectionDDHSource.extractedGame_accuracy_cost](#).

Controls and component scope. At $N = 0, e = 2, q = 29, k = 55,296$ gives the non-vacuous allowance $1/2$. At $q = 2$, the same threshold has zero truncated success and residual failure one: removing the size condition is false. Zero trials also fail; $e = 0$ gives an infinite allowance and is vacuous. A complete $q = 257, N = 9, e = 2$ game instantiates the bound without evaluating its 55,296,000-attempt tree. A separate querying callback fixture retains all original/replay/guess costs.

Lean: `BallotReplayParametersControls.positive_accuracy.`

Lean: `BallotReplayParametersControls.field_size_required.`

Lean: `BallotReplayParametersControls.zero_accuracy_vacuous.`

Lean: `BallotReplayParametersControls.zero_trials_fail.`

Lean: `BallotReplayParametersControls.complete_accuracy.`

Lean: `BallotReplayParametersControls.querying_cost.`

Independent rational enumeration checks 408 parameter cases and 4,225 geometric cases, detecting omitted field size, linear repetition and a missing joint cube; the existing 1,800 fixed-original replay cases remain. These finite tests have zero failures/discards. Section 1.73 proves complete random-mask secrecy for the comparison source. The parameterized theorem in the formal reference supplies its field-size and polynomial-accuracy conditions. Choosing negligible δ alone does not establish polynomial repetition.

1.73 Historical component: Random-mask secrecy of the complete comparison

Comparison and boundary. The actual `completedReal` source first runs preparation, samples the hidden vote, programs the key and both honest ballot proofs, performs honest and malicious submissions, publishes trustee shares/proofs and calls the saved-state-dependent guesser. In this mathematical hybrid, a secret s computes shares of the retained board. The executable DDH reduction must use the separately checked public-input extractor and finisher.

Theorem 1.75 (Complete random-mask comparison). *Let F be a sampleable field, G a decidable additive commutative group with an F -module structure, and let F have decidable equality. Fix any public $g, pk, A \in G$, mathematical secret $s \in F$, fingerprint function, preparation computation and family of casting/guessing callbacks with the original public interface. For any fixed source state S and live cache L ,*

$$\Pr [\text{win} \mid z \leftarrow U(F); \text{evaluate}(\text{completedReal}(g, pk, A, zg, s), S, L)] = \frac{1}{2}.$$

The displayed arguments suppress only the fixed fingerprint, preparation and callback family. No acceptance, freshness, board-shape, query-budget or cache consistency premise is required for this comparison result.

Lean: `ElectionDDHSource.completed_random_win.`

Checked source and probability argument. A private unscored continuation performs the same casting, submission, publication and guessing, returning the actual guess. An exact runtime identity recovers the original completed source by scoring that guess against the vote, retaining the full source state and live cache. For a fixed vote v , the bijection $z \mapsto z - \text{voteScalar}(v)$ preserves the uniform mask law. The paired ciphertext identity then gives equality of the entire unscored continuation, including its oracle requests and rejection paths. Independent mask and vote draws commute only in interpreted distributions. The resulting unscored output is independent of the fair vote, so the actual guess wins with probability $1/2$. $\square$

This proof does not infer replay-tree equality from semantic equality. It reasons about the actual non-replayed comparison, whose full-output distance from the extractor-based game was proved separately. Using this result in DDH requires charging that distance and the random-world rejection allowance before applying the exact hybrid equality.

Controls and finite validation. Over field 257, checked instances retain querying preparation/casting/guessing, a nonempty initial cache and source record/flag, and a further callback whose saved state depends on board length and whose later query and guess depend on published decryption shares. The paired ciphertexts distinguish the two fixed-zero-mask inputs; this is a masking control, not a complete attack bound. Uniform masks equate both input laws, and handing the hidden vote directly to a guesser makes its success one.

Lean: `ElectionDDHRandomSourceControls.querying_complete.`

Lean: `ElectionDDHRandomSourceControls.prior_cache_complete.`

Lean: `ElectionDDHRandomSourceControls.view_dependent_complete.`

Lean: `ElectionDDHRandomSourceControls.fixed_mask_ciphertext_leaks.`

Lean: `ElectionDDHRandomSourceControls.uniform_mask_both_votes.`

Lean: `ElectionDDHRandomSourceControls.leaked_vote_wins.`

The independent multiplicative fixture checks 660 mask-coupled executions on fixed tapes, extending its existing 1,800 honest-source cases. Selected replay and corruption callbacks query retained oracle answers, publish retained-board shares and trustee proofs, and guess from the view and saved answers. All three honest decisions occur; coupled full fixture records agree, and each enumerated uniform-mask table has winning mass 1/2. Fixed-mask ciphertext leakage is detected. These selected continuations validate finite arithmetic and transcript behavior; the Lean theorem supplies universal callback quantification.

Section 1.74 supplies the executable public-input DDH distinguisher and complete finite advantage inequality, including both worlds' comparison losses and the honest-rejection test's DDH advantage.

1.74 Historical component: Finite reduction of the original prepared election

Actual algorithms. Let $N = p + c + 9$ and $k = 3456(N + 1)^3 e^4$. The main DDH distinguisher $D_{N,e}$ receives only (g, pk, A, T) , runs the actual `extractedGame` with these inputs and natural N, k , and returns its winning bit. The second test D_{rej} executes the original prepared prefix and reports honest-pair rejection. Both algorithms close over the original fingerprint and attacker callbacks. Neither receives a secret, an ideal transcript or an extraction success certificate. Extraction failures and rejected boards are retained.

Theorem 1.76 (Complete finite DDH advantage bound). *In the original fixed three-voter, two-candidate, one-honest-trustee scope, let F be a finite sampleable field with decidable equality, $q = |F|$, and let G be a decidable additive commutative group with an F -module structure. Assume $r \mapsto rg$ is injective. Let preparation use at most p proof-hash queries, and every casting callback at most c , counting all three proof domains. The saved-state-dependent guessing callback is arbitrary. For natural $e > 0$ with $12(p + c + 10)e \leq q$, define*

$$L = \frac{11p + 2c + 131}{q}, \quad \Delta_g(D) = |\Pr[D(\text{DDH}_{\text{real}}) = 1] - \Pr[D(\text{DDH}_{\text{random}}) = 1]|.$$

Then the actual shared-oracle prepared election satisfies

$$\left| \Pr[\text{win}] - \frac{1}{2} \right| \leq \frac{18}{q-1} + 3L + \frac{2}{e} + \Delta_g(D_{N,e}) + \Delta_g(D_{\text{rej}}).$$

Lean: [ElectionDDHSource.prepared_ballot_secretcy_ddh_bound](#).

The source theorem expresses $1/(q-1)$ through `noncePointBound` and `toReal`. It runs the original prepared election through the existing shared-cache interpreter from an empty cache. The original nonzero key and honest nonce sampling are preserved. The actual public transcript, rejection, trustee output, prepared state and later guessing stay in the game.

Every loss is accounted for. Put $\rho = 9/(q-1) + L$. The checked chain is:

Comparison	Allowance
Original election to real secret-using comparison	L
Real comparison to actual real DDH test	$R_{\text{real}} + 1/e$
Real to random actual DDH test	$\Delta_g(D_{N,e})$
Actual random test to random secret-using comparison	$R_{\text{random}} + 1/e$
Random comparison to fair guessing	0

Here $R_{\text{real}} \leq \rho$ and $R_{\text{random}} \leq \rho + \Delta_g(D_{\text{rej}})$ are proved actual-event bounds. The real/random extraction comparisons are averaged over the pinned DDH experiments' sampled coordinates, using both directions of the event-probability bound. The original winning projection is derived from `preparedSource\_project`; no caller supplies a correspondence premise.

Lean: [ElectionDDHSource.ballotSecrecy_real_gap](#).

Lean: [ElectionDDHSource.ballotSecrecy_random_gap](#).

Interaction cost and remaining efficiency boundary. For the pinned prime-field sampler, all-request preparation/casting/guessing bounds P, C, D give the actual main distinguisher cost

$$(1 + 3 \cdot 3456(N+1)^3 e^4)(4P + 4C + 78) + 4 + 4D.$$

These are uniform-index oracle nodes. Live ballot queries are bounded by all proof hashes, so the final proof derives its live p, c premises from the stated all-proof-hash bounds. Complete call-back budgets P, C, D separately charge randomness and all proof domains. The rejection test's corresponding cost is already $4P + 4C + 78$.

Lean: [ElectionDDHSource.ballotSecrecy_prime_cost](#).

Controls and status. The $q = 65,537$, $p = 2$, $c = 1$, $e = 16$ instance retains querying preparation, board-dependent casting state and share-dependent querying/guessing. Its explicit non-DDH allowance $18/65,536 + 465/65,537 + 1/8$ is below $1/2$. This is a finite test field, with no asserted DDH hardness. Its actual reduction has the matching checked query bound. Independent probability controls require both extraction allowances, the original loss, absolute values and the two-game DDH normalization; a zero generator fails the injectivity premise.

Lean: [ElectionDDHReductionControls.complete_queried](#).

Lean: [ElectionDDHReductionControls.allowance_nonvacuous](#).

Lean: [ElectionDDHReductionControls.complete_cost](#).

Lean: [ElectionDDHReductionControls.both_extraction_losses_required](#).

Lean: [ElectionDDHReductionControls.advantage_losses_required](#).

Lean: [ElectionDDHReductionControls.zero_generator_excluded](#).

Independent exact arithmetic checks 19,683 probability chains and detects seven omitted-loss, signed-advantage and factor-two mutants with no failures/discards. These tables validate the reduction arithmetic; the Lean theorem establishes the actual-game correspondences. The formal

reference gives the parameterized theorem, its attacker family and protocol scope. It retains explicit external reduction-efficiency assumptions. The finite inequality alone does not establish that its right-hand side is negligible.

1.75 Historical component: Known repair control under the shared oracle

The repaired attack issues the full neutral-ciphertext proof query

$$H_{\text{ballot}}(g, pk, (0, 0), ((g, pk), (g, pk + g))) = c.$$

It constructs the neutral proof with challenges $c - 1, 1$ and responses $1, 1$, then copies the first accepted ballot's aggregate ciphertext and aggregate proof into the original malicious submission. The ordinary prefix and finisher execute with the same cache, including all key, ballot and trustee proof queries.

Lean: [ElectionOracle.repairedReuseSubmission_function](#).

Lean: [ElectionOracle.repairedAttackWithCoins_function](#).

Lean: [ElectionOracle.repairedAttackWorld_function](#).

Theorem 1.77 (Known repaired attack: oracle-aware rejection). *Let F be a finite field with decidable equality and the stated uniform sampling instance, and let G be an additive commutative F -module with decidable equality. For every generator g , fingerprint function, vote v and initial shared cache Q , the original nonzero-sampled repaired attack satisfies*

$$\Pr[\text{view.decision} = \text{reusedCiphertext} \mid \text{run}(\text{repairedAttackWorld}(g, v), Q)] = 1.$$

The same result holds for every fixed secret, key/trustee nonce tuple and honest coin tuple, without a collision-free or honest-pair-acceptance premise.

Lean: [ElectionOracle.repairedAttackWorld_rejected](#).

Lean: [ElectionOracle.repairedAttackWithCoins_rejected](#).

The fixed-hash interpretation equals the original complete execution and sampled world. This equality retains the public prefix, submission, rejection reason, board, encrypted tally, decryption shares, trustee proofs and bounded decoding. The proof instantiates pinned VCVio's probability-one characterization for a mixed uniform/lazy oracle: every consistent deterministic answer table satisfies the existing repair control while uniform draws remain probabilistic. This argument supplies no general quantitative transfer for the unrepaired attack.

Controls and boundary. Six Lean controls check actual cache hits/misses, the literal neutral transcript, sampled rejection, invalid-proof priority and honest acceptance with the existing component-only-weeding counterfactual. Independent multiplicative fixtures check 240 executions (seeds 17, 41, 103), including both votes and both cache cases. Both honest acceptance and rejection occur; all component-only mutants accept and all changed-response controls fail verification before weeding. The fixtures check the retained board, decoded tally and all trustee equations, with zero failures or discarded cases. They do not prove a distributional correspondence for arbitrary implementations.

Lean: [ElectionRepairControlControls.neutral_hit](#).

Lean: [ElectionRepairControlControls.neutral_miss](#).

Lean: [ElectionRepairControlControls.neutral_transcript](#).

Lean: [ElectionRepairControlControls.sampled_rejection](#).

Lean: [ElectionRepairControlControls.invalid_before_reuse](#).

Lean: [ElectionRepairControlControls.repair_is_not_universal_rejection](#).

1.76 Historical component: Fixed finite-control oracle interface

The deferred fully mechanized efficiency approach requires executable local control as well as query bounds. This interface reuses Mathlib’s TM2 stack instructions, with fixed finite stack, label and local-memory types. Unbounded words reside on stacks. A code object is fixed for the whole parameter family. Explicit hash and coin instructions suspend with a named response destination; resumption replaces that stack and retains every other stack. All-local runs preserve the original TM2 configuration exactly.

Lean: `BitOracleMachine.resume_frame`.

Lean: `BitOracleMachine.local_step`.

Lean: `BitOracleMachine.local_run`.

A raw hash instruction carries the ghost charge $1 + |request| + |oldDestination| + |answer|$; a coin replacement carries $2 + |oldDestination|$. These are the proposed port-transfer accounting rules. The cost field is proof instrumentation, not an implemented counter or a standard-machine cost theorem. The existing deterministic TM2-to-TM1 simulation does not yet implement these open-oracle transfers or give their quantitative bound.

Theorem 1.78 (Adaptive raw-port execution test). *One fixed program with four labels, four stacks and one local-memory value executes the following exact query tree for every input word x :*

$$a \leftarrow H(x); \quad b \leftarrow H(a); \quad c \leftarrow H(x).$$

After four steps it halts with stacks $[x, a, b, c]$ and derived interface charge

$$4 + 2|x| + 2|a| + |b| + |c|.$$

Equality holds as an OracleComp program, so every handler observes the same queries, responses and final configuration.

Lean: `BitOracleCanary.run_source`.

Five Lean controls check a literal miss/miss/hit run with charge 15, an occupied first entry that changes the second query and gives charge 11, observable fuel exhaustion, coin replacement and omitted transfer charges. Independent fixtures cover all binary words of lengths 0–6, four deterministic response functions and three initial-cache choices: 1,524 cases, plus 130 coin replacements. They detect forgetting the original request and dropping request/answer charges, with zero failures or discarded cases.

Lean: `BitOracleMachineControls.miss_miss_hit`.

Lean: `BitOracleMachineControls.occupied_changes_request`.

Lean: `BitOracleMachineControls.early_not_halted`.

Lean: `BitOracleMachineControls.coin_overwrite`.

Lean: `BitOracleMachineControls.transfer_costs_required`.

This raw-word test still needs the actual scalar/codec handler, malformed-message policy and a standard open-oracle embedding with realized transfer costs.

1.77 Historical component: Actual oracle-port word movement

The raw-port interface needs a physical request copy and response replacement. The existing copy routine preserves its source but prepends to the destination; a nonempty destination therefore exposes a checked mismatch with replacement. The transfer first clears that destination and reuses the checked copy program. For responses it also erases the private source after copying. Otherwise

the next request inherits an uncharged old private word. A checked stale-port control retains this insufficient workspace invariant as a failed approach.

Lean: `BitPortTransfer.append_is_not_replacement`.

Lean: `BitPortTransferControls.uncleared_port_not_free`.

The source-preserving routine takes at most $|d| + 2|w| + 4$ TM2 ticks; the consuming routine takes at most $|d| + 3|w| + 4$. Both bounds include end and return checks. Existing stack-placement and return libraries are reused. A product-memory frame preserves the caller's finite local memory at identical transition counts. The actual caller layout, incoming word, empty scratch and other stacks are derived by the adapter, rather than supplied as correspondence premises.

Lean: `BitPortTransfer.run`.

Lean: `BitPortTransfer.consume_run`.

Lean: `TM2MemoryFrame.statement`.

Lean: `TM2MemoryFrame.tick`.

Lean: `TM2MemoryFrame.run`.

Lean: `BitOraclePortTransfer.start_original`.

Lean: `BitOraclePortTransfer.start_private`.

Lean: `BitOraclePortTransfer.request_start`.

Lean: `BitOraclePortTransfer.run_request`.

Lean: `BitOraclePortTransfer.request_state`.

Lean: `BitOraclePortTransfer.run_resume`.

Theorem 1.79 (Physical port transfer for the raw interface). *For every caller configuration, selected request/destination ports and saved return address, let r be the request word and d the old destination. Starting with empty private ports, there is a request-phase execution of at most $2|r| + 4$ TM2 ticks that places exactly r on the query port. For every subsequent native answer a , the response phase halts at the exact specified **resume** state, retaining all other caller stacks and finite memory and restoring empty private ports. The total number of TM2 ticks plus the one native event is at most*

$$9(1 + |r| + |d| + |a|).$$

A native coin answer has the same successor/empty-workspace guarantee and bound $9(2 + |d|)$. Request and destination may be the same caller port.

Lean: `BitOraclePortTransfer.hash_transfer`.

Lean: `BitOraclePortTransfer.coin_transfer`.

The native hash/coin operations are actual OracleComp queries. Their handoff identifies the prepared query word and the complete response-phase start under every handler, including the saved caller state.

Lean: `BitOraclePortTransfer.hash_handoff`.

Lean: `BitOraclePortTransfer.coin_handoff`.

Six dedicated controls check order/replacement, consumed private words, saved caller data, aliased ports, stale private data and dirty scratch. Independent binary-word fixtures cover lengths 0–5: 11,907 preserving and 11,907 consuming cases, 35,721 framed inbound and 35,721 outbound cases, 3,969 composed aliased hash cases and 126 coin cases. Append, one-pass and source-erasure mutants are detected; failures and discarded cases are zero.

Lean: `BitPortTransferControls.overwrite`.

Lean: `BitPortTransferControls.consumed_private_word`.

Lean: `BitPortTransferControls.caller_frame`.

Lean: [BitPortTransferControls.aliased_request_and_answer.](#)

Lean: [BitPortTransferControls.dirty_scratch_changes_word.](#)

The native oracle supplies its answer word; these routines do not implement the cryptographic hash or its scalar codec. The next subsection assembles the open-machine loop. These bounds are not TM0 time bounds.

1.78 Historical component: Finite source execution in the assembled loop

The loop has three control forms: caller-ready, request preparation and response loading. Its unbounded data are bit stacks; saved addresses, port indices and local memory are finite. A transition executes one existing TM2 statement, selects a phase, issues a native query, or returns to the caller. It contains no residual OracleComp program and computes no word length to select execution fuel. Fuel belongs to external observation.

Request preparation stops at its first completed state before issuing a native query. Response loading derives empty private I/O and scratch before discarding those ports at return. This prevents padded halted subroutine ticks from being mistaken for loop ticks that would issue the next query. Dispatch, native events and returns increase the transfer bound to $11(1 + |r| + |d| + |a|)$ for a hash and $11(2 + |d|)$ for a coin. Request and destination may alias.

Lean: [BitOracleLoop.request_run.](#)

Lean: [BitOracleLoop.response_run.](#)

Lean: [BitOracleLoop.hash_issue.](#)

Lean: [BitOracleLoop.hash_cycle.](#)

Lean: [BitOracleLoop.coin_cycle.](#)

Lean: [BitOracleLoop.run_add.](#)

Lean: [BitOraclePortTransfer.delivery_start.](#)

For fixed code, write $U_t(s)$ for its actual t -step loop observation and $\text{ready}(c)$ for a caller boundary. The proof predicate $R(Q, p, s)$ (**Realizes**) relates a source query tree Q , already spent steps p and loop state s . It is defined by structural recursion on Q :

$$\begin{aligned} R(\text{pure}(c, C), p, s) &\iff \exists t, p + t \leq 11C \wedge U_t(s) = \text{pure}(\text{ready}(c)), \\ R(q \triangleright k, p, s) &\iff \exists t, \delta, U_t(s) = q \triangleright (\text{pure} \circ \delta) \\ &\quad \wedge \forall a, R(k(a), p + t, \delta(a)). \end{aligned}$$

Here $q \triangleright k$ is one actual typed native query with continuation k . Equality at each query boundary preserves every answer branch. The witness is constructed in the proof; it is not an executable field or a caller-supplied adequacy assumption.

Theorem 1.80 (Costed finite raw-source execution). *For every fixed raw code P , caller configuration c and finite source observation fuel n ,*

$$R(\text{sourceRun}(P, n, c), 0, \text{ready}(c)).$$

For every typed path through that source run, ending in (c', C) , there are $t \leq 11C$ and a typed path through $U_t(\text{ready}(c))$ with output $\text{ready}(c')$ and exactly the same complete query/answer log.

Lean: [BitOracleLoop.run_source.](#)

Lean: [BitOracleLoop.realizes_path.](#)

Lean: [BitOracleLoop.run_source_path.](#)

The path statement reuses pinned PolyFun paths and VCVio query logs. It assumes no raw-response length bound: observation times may depend on answers already received. It is not a

uniform-fuel probability-law theorem or a standard oracle- tape cost theorem. The next subsection supplies common-clock handler transport under explicit termination and answer bounds.

Six Lean controls include the all-word adaptive canary, literal 63-step miss/miss/hit and 52-step occupied-cache executions, the silent preparation/native event boundary, and rejection of a zero-charge witness for a changed caller. Independent fixtures compare 2,880 three-call executions: all 64 sequences from three hash-port layouts and a coin, input lengths 0–3, response lengths 0–4, and seeds 7, 19 and 41. Early-query, stale-private-word and skipped-successor mutants are detected. Failures and discarded cases are zero.

Lean: `BitOracleLoopControls.canary`.

Lean: `BitOracleLoopControls.miss_miss_hit`.

Lean: `BitOracleLoopControls.occupied_changes_request`.

Lean: `BitOracleLoopControls.preparation_is_silent`.

Lean: `BitOracleLoopControls.native_event_follows_preparation`.

Lean: `BitOracleLoopControls.zero_charge_not_arbitrary`.

1.79 Historical component: Common-clock execution and handler transport

A uniform clock requires an encoded native-answer bound. The bounded interface retains every raw request and both coin values, admitting hash answers of length at most L . Its adapter erases only the bound proof. The actual scalar handler must still derive this contract.

Write $A_L(Q)$ for interpretation through this adapter. $\text{Within}_{L,B}(Q)$ requires every allowed-answer branch to end in (c, C) with $c.\text{label} = \text{none}$ and $C \leq B$, recursively at queries. It asserts source termination and accounting, not correspondence.

Theorem 1.81 (Common-clock source and handler correspondence). *Let P be any fixed raw code, c any caller configuration and n a finite source observation fuel. If $\text{Within}_{L,B}(\text{sourceRun}(P, n, c))$, then*

$$A_L(U_{11B}(\text{ready}(c))) = (\text{ready} \circ \text{fst}) \langle \$ \rangle A_L(\text{sourceRun}(P, n, c)).$$

The equality holds for the complete query trees and is preserved by interpretation through every lawful handler of the bounded interface, including stateful probabilistic handlers. The complete caller state and handler effects are retained; only the source's ghost cost is projected away.

Lean: `BitOracleLoopBounded.realizes_run`.

Lean: `BitOracleLoopBounded.run_source`.

Lean: `BitOracleLoopBounded.run_source_handler`.

The source theorem constructs the execution witness; halted leaves absorb remaining ticks. Termination is necessary: the checked spinning-coin program makes one query in one source instruction but three in 22 loop ticks. The termination predicate rejects this padding counterexample.

Lean: `BitOracleBoundedControls.missing_halt_changes_trace`.

Lean: `BitOracleBoundedControls.spinning_not_within`.

For the existing adaptive $H(w), H(H(w)), H(w)$ canary, the actual source halt and bound $B = 4 + 2|w| + 4L$ are derived. Its loop and arbitrary-handler equalities therefore need no caller-supplied cost, termination or correspondence premise. The common clock is $11(4 + 2|w| + 4L)$.

Lean: `BitOracleBoundedCanary.within`.

Lean: `BitOracleBoundedCanary.run`.

Lean: `BitOracleBoundedCanary.run_handler`.

Seven Lean controls cover cache effects, zero-width answers, intact requests, missing termination and oversized answers. Independent fixtures check 2,880 three-call cases at 528 and 1056 ticks,

retaining complete tapes/cache/trace, halt and private workspace. Input lengths are 0–3, response limit 4, source bound 48, seeds 7/19/41. Missing-halt and early-fuel mutants are detected; failures and discarded cases are zero.

Lean: `BitOracleBoundedControls.miss_miss_hit`.

Lean: `BitOracleBoundedControls.occupied_changes_request`.

Lean: `BitOracleBoundedControls.zero_width`.

Lean: `BitOracleBoundedControls.raw_request_retained`.

Lean: `BitOracleBoundedControls.oversized_not_allowed`.

This closes common-clock transport for the bounded native-answer interface.

1.80 Historical component: Quantitative reuse of the stack-to-tape translation

Mathlib’s existing compiler represents stacks as reversed columns on one tape, with a bottom marker. Each stack operation scans outward to the selected top, performs the operation, and returns. Its correctness theorem erases execution length. The new proofs reuse that compiler and its complete `TrCfg` relation, adding counts rather than a second encoding.

An outward scan of n cells takes n TM1 ticks; return from position n' takes n' ticks. The existing top-action correctness theorem preserves all other columns. Including the operation and continuation dispatch gives $n + n' + 2$ ticks for one stack access.

Lean: `TM2TapeCost.scan_right`.

Lean: `TM2TapeCost.scan_left`.

Lean: `TM2TapeCost.top_action`.

Lean: `TM2TapeCost.roundtrip`.

Theorem 1.82 (Complete translated-statement cost). *Let $(\text{some}(\ell), v, S)$ be a source configuration, c a related tape configuration under Mathlib’s `TrCfg`, and $|S_k| \leq H$ for every stack. Let a be the maximum number of stack accesses along a branch of $M(\ell)$. There are t and c' such that*

$$t \leq 1 + a(2(H + a) + 2), \quad (\text{tapeTick}_M)^t(c) = c',$$

and c' is related to $\text{stepAux}(M(\ell), v, S)$ by `TrCfg`.

Lean: `TM2TapeCost.statement`.

Lean: `TM2TapeCost.source_step`.

The bound includes the entry tick and growth from earlier pushes in the same statement. Both branches and finite local loads are covered. The actual framed request and consuming-response programs have $a \leq 3$, giving $6H + 25$ TM1 ticks per transition, with all caller/private columns retained.

Lean: `BitOracleTapeCost.request_accesses`.

Lean: `BitOracleTapeCost.response_accesses`.

Lean: `BitOracleTapeCost.request_step`.

Lean: `BitOracleTapeCost.response_step`.

Five Lean controls construct a complete literal tape presentation, check a 16-tick peek/push, refute constant access cost, cover branch maxima and check empty peek/pop with another column intact. Independent tape fixtures cover 6,000 cases, statement depth at most 6, stack lengths 0–8 and seeds 7/19/41. Full stack/tape results and the cost bound agree; skipped-return and cross-column erasure mutants are detected. Failures and discarded cases are zero.

Lean: `TM2TapeCostControls.derived_step`.

Lean: `TM2TapeCostControls.actual_peek_push`.

Lean: `TM2TapeCostControls.constant_cost_not_enough`.

Lean: `TM2TapeCostControls.branch_bound`.

Lean: `TM2TapeCostControls.empty_peek_pop`.

These are TM1 tick bounds. The generic simulation allows arbitrary local-memory types; finite domains and reachable syntax must still justify primitive costs. The next subsection derives packing and accumulated deterministic bounds.

1.81 Historical component: Finite tape runs and canonical presentation

Canonical packing reverses each source stack and aligns the columns at their bottoms. With $H = \max_k |S_k|$, column k at position $i < H$ is $\text{reverse}(S_k)[i]?$; all later cells are blank. This constructs Mathlib's full `TrCfg` relation. Column projections also show that two related tape configurations for the same source configuration are equal.

Lean: `TM2TapeRuns.length_le_height`.

Lean: `TM2TapeRuns.columns_nth`.

Lean: `TM2TapeRuns.columns_presentation`.

Lean: `TM2TapeRuns.pack_related`.

Lean: `TM2TapeRuns.related_unique`.

Theorem 1.83 (Finite execution from canonical tape data). *Let the stack and source-label types be finite, with decidable stack equality. For source code M and configuration s , put $A = \max_\ell \text{accesses}(M(\ell))$ and $H = \max_k |s.\text{stk}(k)|$. Take empty maxima as zero. For every n , there is t such that*

$$t \leq n(1 + A(2(H + nA) + 2)),$$

$$(\text{tapeTick}_M)^t(\text{pack}(s)) = \text{pack}((\text{sourceTick}_M)^n(s)).$$

Every final source stack has length at most $H + nA$.

Lean: `TM2TapeCost.statement_height`.

Lean: `TM2TapeRuns.run`.

Lean: `TM2TapeRuns.run_packed`.

The proof derives intermediate growth, includes halt padding, and retains one persistent tape across all transitions. It requires no caller-supplied initial presentation, height bound or code-bound certificate.

For the actual port programs, $A \leq 3$. A complete transfer taking at most B source ticks therefore takes at most $B(6H + 18B + 7)$ tape ticks. The outgoing bound uses $B = |\text{oldPort}| + 2|\text{request}| + 4$; the response uses $B = |\text{oldDestination}| + 3|\text{answer}| + 4$. Here H is the maximum height of the actual starting transfer configuration. The final request tape contains the exact copied request and retained caller columns; the response returns the actual resumed caller with both private columns empty.

Lean: `BitOracleTapeCost.request_run`.

Lean: `BitOracleTapeCost.response_run`.

Five Lean controls check derived and literal growing runs, early halt and nonpalindromic packing. Four pushes take $4 + 6 + 8 + 10 = 28$ ticks; the frozen-height estimate of 20 stops inside the fourth push. Unreversed packing returns the wrong source top. Independent persistent-tape fixtures cover 4,500 two-label programs, depth at most 5, stack lengths 0–6, source fuel 0–8 and seeds 7/19/41. Both mutants are detected; failures and discarded cases are zero.

Lean: `TM2TapeRunsControls.derived_growing_run`.

Lean: `TM2TapeRunsControls.actual_growing_run`.

Lean: `TM2TapeRunsControls.frozen_height_fails`.

Lean: `TM2TapeRunsControls.packing_order_matters`.

Lean: `TM2TapeRunsControls.early_halt`.

Packing specifies data, not an executed input loader. These are deterministic TM1 regions: native oracle I/O, finite reachable control and primitive costs, scalar-handler policy and whole-algorithm PPT adequacy are outside this component’s guarantee. Generic local-memory and symbol types need not be finite in this simulation theorem.

1.82 Historical component: Physical outgoing query-tape transfer

`OracleTapeOutput.step` reads the bottom-first work column while writing leftward on a separate query tape, restoring the original word order. Four finite phases clear the old query, copy, return the work head and finish. Each transition reads current cells, writes at most one query cell and moves each head by at most one. Work cells are never written.

Write $Q(w)$ for a tape with w in ordinary list order, its head at the first bit and all other cells blank; $Q([])$ is entirely blank. Let $P(s)$ be the work tape of the existing canonical packing of source configuration s .

Theorem 1.84 (Outgoing export with physical query cleanup). *For a finite stack type with decidable equality, source configuration s , selected stack k , and any previous native word u , put $w = s.stk(k)$. Then the actual head-local transition function satisfies*

$$\text{step}_k^{|u|+2|w|+3}(\text{clear}, P(s), Q(u)) = (\text{done}, P(s), Q(w)).$$

Lean: `OracleTapeOutput.clear_run`.

Lean: `OracleTapeOutput.run`.

Lean: `OracleTapeOutput.run_packed`.

Lean: `OracleTapeOutput.run_replacing`.

Clearing takes $|u| + 1$ transitions and derives the blank query tape. Export from that blank tape takes $2|w| + 2$. The equality restores the complete work tape, including the bottom marker, head and every other column. The column and marker laws are derived from actual source data.

`BitOracleTapeOutput.request_run` connects this to the actual request preparation. With $B = |\text{oldPort}| + 2|\text{request}| + 4$ and initial maximum height H , preparation uses at most $B(6H + 18B + 7)$ TM1 ticks. The subsequent head-local transfer uses exactly $|u| + 2|\text{request}| + 3$ transitions and produces $Q(\text{request})$, preserving the complete prepared work tape. These are two derived region executions, not yet one compiled oracle-dispatch loop or a combined standard-machine cost theorem.

Lean: `BitOracleTapeOutput.request_run`.

Seven Lean controls cover exact and literal export, empty words, a longer caller column, and wrong-direction, skipped-return and stale-suffix failures. Clearing repairs the stale suffix. Independent two-head fixtures cover 5,400 cases, each with fresh and reused query tapes, 1–5 work columns, word lengths 0–20 and seeds 7/19/41. All three mutants are detected; failures and discarded cases are zero.

Lean: `OracleTapeOutputControls.derived_export`.

Lean: `OracleTapeOutputControls.actual_export`.

Lean: `OracleTapeOutputControls.rightward_write_fails`.

Lean: `OracleTapeOutputControls.return_is_required`.

Lean: `OracleTapeOutputControls.empty_export`.

Lean: `OracleTapeOutputControls.stale_query_fails`.

Lean: `OracleTapeOutputControls.stale_query_cleaned`.

The previous native tape must be $Q(u)$, including its head position; cleanup is now executed for every such prior output. Response-tape loading, native query dispatch, physical input loading, finite-control/primitive-cost adequacy for the full compiled loop and the actual scalar-handler contract are outside this component's guarantee. No protocol observation or cryptographic assumption changed.

1.83 Historical component: Physical answer loading and response execution

`OracleTapeInput.step` replaces the private work column with a native answer. Eight finite phases scan and erase the old column, scan the answer, read its bits backward while pushing them at the column top, and return both heads. Each transition reads current cells, writes at most one work cell and moves each head by at most one. The proofs reuse Mathlib's existing push/pop tape-update theorem. Other columns and the bottom marker are retained.

Let $P(s)$ be the work tape of the existing canonical source packing and $Q(a)$ the ordinary native answer tape, at its first bit with all other cells blank. Write $s[k := a]$ for replacement of only source stack k .

Theorem 1.85 (Physical answer replacement). *For a finite stack type with decidable equality, source configuration s , stack k and native answer a , the actual transition function satisfies*

$$\text{step}_k^{3|s.\text{stk}(k)|+4|a|+6}(\text{seekWork}, P(s), Q(a)) = (\text{done}, P(s[k := a]), Q(a)).$$

Lean: `OracleTapeInput.run_packed`.

The initial private word need not be empty. The conclusion identifies both complete tapes and both restored head positions. Erasure removes old suffixes when the answer is shorter or empty; backward native reading preserves source stack order. No blank-port or structural-presentation certificate is assumed.

`BitOracleTapeInput.load_request` applies the loader to the actual completed request configuration, deriving the exact work tape expected by `BitOraclePortTransfer.start`. It uses the existing delivery/layout identity to preserve every caller column and scratch port.

Lean: `BitOracleTapeInput.load_request`.

`BitOracleTapeInput.response_run` follows this with the actual consuming response program. Loading takes $3|oldPort| + 4|a| + 6$ head-local transitions; response execution takes at most $B(6H + 18B + 7)$ TM1 ticks, where $B = |oldDestination| + 3|a| + 4$ and H is the response-start maximum stack height. The final source configuration is halted, projects to the actual resumed caller, and has both private ports empty. These remain separate region executions and cost units until the compiled native-dispatch loop is assembled.

Lean: `BitOracleTapeInput.response_run`.

Six Lean controls check exact and literal shorter replacement, an empty answer, nonpalindromic order and a longer caller column. Mutants omit erasure, scan away from the answer or write to a caller column. Independent two-head fixtures cover 5,400 cases, 2–5 work columns, old/answer lengths 0–16 and seeds 7/19/41. They check full column replacement, retained answer, both heads and the exact count; all three mutants are detected, with zero failures or discarded cases.

Lean: `OracleTapeInputControls.derived_load`.

Lean: `OracleTapeInputControls.actual_load`.

Lean: `OracleTapeInputControls.old_suffix_fails`.

Lean: `OracleTapeInputControls.wrong_direction_fails`.

Lean: `OracleTapeInputControls.other_column_fails`.

Lean: `OracleTapeInputControls.empty_answer`.

The native answer must have the stated canonical word/head layout. The next section supplies native dispatch and per-call composition. Whole-source-loop composition, physical initial-input loading, finite-control and primitive-cost adequacy, and the scalar-handler contract are outside this component's guarantee. The protocol, attacker observations and cryptographic assumptions are unchanged.

1.84 Historical component: Native oracle dispatch and complete call execution

Native decoding stops at the first blank, is well-defined on the trailing-blank quotient, and satisfies $\text{readWord}(Q(w)) = w$. Hash/coin dispatch invokes the original query, retains work/query tapes and supplies the canonical answer. Whole-word interpretation belongs to the native event; local transfers execute the physical steps above.

Lean: `OracleTapeDispatch.read_wordTape`.

Lean: `OracleTapeDispatch.hash_word`.

Lean: `OracleTapeDispatch.coin_word`.

The loop combines outgoing steps, one native event, incoming steps and actual TM1 response ticks. Handoffs retain tapes and finite metadata. First-completion proofs stop pure phases before another query; the completed state is absorbing.

Fix a raw caller c with finite port/label/memory indices, request and destination ports r, d (possibly equal), next label ℓ , $w = c.\text{stk}(r)$, previous canonical query $Q(v)$ and arbitrary old answer tape. Let **begin** be export entry on the actual prepared work tape; **incoming**(a) starts loading the native reply while retaining $Q(w)$.

Theorem 1.86 (Native call boundaries). *There is $b \leq |v| + 2|w| + 5$, chosen before the answer, such that*

$$\text{run}_{\text{hash}}(b, \text{begin}) = (a \leftarrow \text{hash}(w); \text{return incoming}(a)).$$

For every raw answer a , let $B = |c.\text{stk}(d)| + 3|a| + 4$ and let H be the maximum stack height of the actual response-start configuration. There are $t \leq 3|w| + 4|a| + 8 + B(6H + 18B + 7)$ and a halted transfer configuration o with both private stacks empty and projection $\text{resume}(c, d, \ell, a)$, such that

$$\text{run}_{\text{hash}}(t, \text{incoming}(a)) = \text{return done}(\text{pack}(o), Q(w), Q(a)).$$

Lean: `BitOracleTapeCall.hash_issue`.

Lean: `BitOracleTapeCall.answer_run`.

These query-tree equalities retain every answer and complete final tapes. **received_run** allows any old private word/query tape, replacing $|w|$ by that old word's length in the bound; **coin_issue** supplies its bit reply.

Lean: `BitOracleTapeCall.received_run`.

Lean: `BitOracleTapeCall.coin_issue`.

hash_prepared_issue derives the export entry from the actual request program's TM1 run, with its existing bound $B_0(6H_0 + 18B_0 + 7)$, where $B_0 = |\text{oldPort}| + 2|w| + 4$ and H_0 is its initial maximum stack height. No caller supplies a presentation or execution certificate.

Lean: `BitOracleTapeCall.hash_prepared_issue`.

Lean: `BitOracleTapeCall.run_add`.

Seven Lean controls check preparation, silent/issue boundaries, actual hash/cache-hit/coin runs and first-blank termination. Independent fixtures cover 2,880 three-call cache interactions (seeds

7/19/41, lengths 0–8) and 2,000 prefix cases. They check physical transfers and source cache/log results, not full TM1 timing. The blank-skipping mutant is detected; failures/gaveUp=0.

Lean: `OracleTapeCallControls.hash_call`.

Lean: `OracleTapeCallControls.occupied_reply`.

Lean: `OracleTapeCallControls.coin_call`.

Lean: `OracleTapeCallControls.blank_terminates`.

Finite source-loop composition is proved next.

1.85 Historical component: Finite source execution in the tape loop

`BitOracleTapeLoop` retains explicit work/query/answer tapes through ordinary computation, request preparation and the existing call transitions. Returns change finite caller label/memory only. To regain control after one ordinary statement, the adapter saves its resulting label and halts; goto stores `some label`, source halt stores `none`. Existing relocation and TM1 execution proofs derive the source result and private frame.

Lean: `BitOracleTapeCompute.run`.

Let $F_n(c)$ be the original source run for n instructions, projected to its caller result (discarding ghost charge). Write $\text{ready}(c, Q(v), A)$ for its canonical work tape, including both empty private columns, and the stated native tapes. The query-tree relation R means:

$$\begin{aligned} R(\text{return } c, s) &\iff \exists t, v, A. \text{run}(t, s) = \text{return ready}(c, Q(v), A), \\ R(\text{query}(q, k), s) &\iff \exists t, \kappa. \text{run}(t, s) = \text{query}(q, \text{return} \circ \kappa) \wedge \forall a. R(k(a), \kappa(a)). \end{aligned}$$

Theorem 1.87 (Finite source correspondence). *For every fixed raw source code with finite port, label and memory types, source caller c , finite fuel n , previous query word v and arbitrary old answer tape A ,*

$$R(F_n(c), \text{ready}(c, Q(v), A)).$$

Every path of $F_n(c)$ occurs in an actual finite tape run with the same complete query/answer log and final caller, including its canonical work presentation and empty private columns.

Lean: `BitOracleTapeLoop.run_source`.

Lean: `BitOracleTapeLoop.realizes_path`.

The proof derives all successor presentation and execution witnesses. Source fuel may stop at a live caller; the stopping times may depend on answers. This statement establishes neither a common clock nor polynomial runtime.

Each ordinary statement has derived loop bound $3+a(2(H+a)+2)$, with a its stack-access count and H its framed initial height. Hash dispatch, two-step coin dispatch and complete reply/return retain their actual native events and region bounds. Their initial states follow from source dispatch; pure-region lifting reuses the call proofs before returning to the outer loop.

Lean: `BitOracleTapeLoop.compute_run`.

Lean: `BitOracleTapeLoop.hash_issue`.

Lean: `BitOracleTapeLoop.coin_issue`.

Lean: `BitOracleTapeLoop.reply_run`.

Eight Lean controls check original source expectations, actual mixed local/ three-call execution, occupied caches, aliased ports, coins and goto/halt failures. They inspect complete caller/private words, native words and traces. An independent source/finite-return interpreter campaign covers 3,600 cases, seeds 7/19/41, depths 0–6, 1–3 ports and word lengths 0–8; both return mutants are detected, with zero failures/gaveUp. This campaign does not measure TM1 timing.

Lean: `OracleTapeLoopControls.mixed_run`.

Lean: `OracleTapeLoopControls.occupied_run`.

Lean: `OracleTapeLoopControls.goto_not_halt`.

Lean: `OracleTapeLoopControls.halt_clears_label`.

Common-clock transport is supplied next.

1.86 Historical component: Source-derived polynomial tape clocks

The finite path result is strengthened to one clock for every permitted answer branch. Let P be fixed source code, c its initial caller, f source fuel, L a hash-answer length bound and B a source charge bound. Write H for c 's maximum word length, V for the previous native query's length, and $Q(w)$ for the canonical native word tape. The old answer tape A is arbitrary. The existing adapter α_L admits every raw request, both coin values and exactly hash answers of length at most L . The premise $\text{Within}_L(B, \text{Source.run}(P, f, c))$ requires every allowed branch to halt with total source charge at most B .

Theorem 1.88 (Common-clock source correspondence; machine-checked). *Under that source premise, put $T = 650B(H + B + V + 1)^2$. In compressed notation,*

$$\begin{aligned} \text{Obs} < \$ > \text{simulate}(\alpha_L, \text{Tape.run}(P, T, \text{ready}(c, Q(w), A))) \\ &= (\text{some} \circ \text{fst}) < \$ > \text{simulate}(\alpha_L, \text{Source.run}(P, f, c)), \quad |w| = V. \end{aligned}$$

Here **Obs** interprets every original caller word at a ready boundary and returns **none** in unfinished phases. This equality of oracle computations also preserves all effects of every lawful bounded-oracle handler.

Lean: [BitOracleTapeLoop.run_source_bounded](#).

Lean: [BitOracleTapeLoop.run_source_handler](#).

The observer is proof-side interpretation, not a free runtime decoder or an additional protocol disclosure. Its canonical correctness is proved. The cap $C = H + B + V$ bounds current height plus remaining source budget and previous-query length at successors. Actual region allowances fit $650(C + 1)^2$ per charged source transition. The accumulated execution witnesses are derived; source halt justifies padding to the common clock.

Lean: [BitOracleTapeLoop.observe_ready](#).

Lean: [BitOracleTapeLoop.run_costed](#).

For the adaptive $H(x)$, $H(H(x))$, $H(x)$ canary, halt and $B = 4 + 2|x| + 4L$ are derived without caller-supplied cost or compiler certificates. Six controls cover cache misses/hits, occupied caches, zero-width answers, height growth, nonzero halt cost and extra queries caused by padding live code.

Lean: [BitOracleTapeBoundedCanary.run](#).

Lean: [OracleTapeBoundedControls.frozen_height_fails](#).

Lean: [OracleTapeBoundedControls.live_padding_adds_queries](#).

Independent fixtures check 12,000 regional inequalities and 3,600 sampled word-transition paths (seeds 7/19/41; caps 0–64; 1–4 ports; word lengths 0–8; up to 16 active transitions, then halt). Three mutants are detected; failures and gaveUp are zero. These fixtures neither execute TM1 nor prove the all-branch source premise. The clock counts loop transitions containing TM1 steps. Finite reachable syntax/primitive-cost transfer, physical initial loading, standard-PPT attacker coverage, actual scalar policy and protocol/reduction runtime bounds are outside this component's guarantee.

1.87 Historical component: Primitive costs through the existing compiler

Mathlib’s TM1-to-TM0 compiler translates each statement into physical single moves/writes. It folds loads and branches on the finite head symbol and local memory into its transition table. Goto and halt emit a final identity write. For statement q , let $a(q)$ count the maximum moves/writes plus that final write along a syntactic branch. Let $w(q, v, T)$ count the actual branch on memory v and tape T .

Theorem 1.89 (Primitive statement and supported-run bounds; machine-checked). *For every TM1 program M , statement q , memory v and tape T ,*

$$w(q, v, T) \leq a(q),$$

$$\text{tick}_0(\text{tr}(M))^{w(q, v, T)}((\text{some } q, v), T) = \text{trCfg}_M(\text{stepAux}_1(q, v, T)).$$

If finite S supports M and the initial caller label belongs to $\text{insertNone}(S)$, put $K = \max_{\ell \in S} a(M(\ell))$. Then every n -tick TM1 run has some $t \leq nK$ with exact compiled final state:

$$\text{tick}_0(\text{tr}(M))^t(\text{trCfg}_M(c)) = \text{trCfg}_M(\text{tick}_1(M)^n(c)).$$

Ticks stutter after halt. Support includes the entry label and is closed under all statement jumps. The displayed notation compresses the pinned translations.

Lean: [TM1PrimitiveCost.statement](#).

Lean: [TM1PrimitiveCost.work_le](#).

Lean: [TM1PrimitiveCost.run](#).

For the actual returning computation and request/response programs, both pinned compiler support proofs derive finite successor-closed primitive control from finite source labels and local memory. Canonical source entries derive support membership. Thus each prior complete region bound is multiplied by a constant K determined solely by its fixed code, with no caller support/cost certificate. All work-tape columns and saved memory agree; response execution still yields the resumed caller and both empty private columns.

Lean: [BitOraclePrimitiveCost.finite_support](#).

Lean: [BitOraclePrimitiveCost.compute_run](#).

Lean: [BitOraclePrimitiveCost.request_run](#).

Lean: [BitOraclePrimitiveCost.response_run](#).

Six literal Lean controls cover complete writes/moves, finite load/branch selection, goto and halt. A supported one-tick halt program contains an excluded label whose statement takes four primitive ticks: omitting initial support membership invalidates the bound. Independent tree/flat-code interpreters agree on 4,800 generated statements (seeds 7/19/41, depths 0–7, initial tape window $[-8, 8]$), with dropped-write/zero-halt mutants detected and zero failures/gaveUp.

Lean: [TM1PrimitiveControls.physical_writes](#).

Lean: [TM1PrimitiveControls.support_is_required](#).

Finite primitive oracle-loop refinement is supplied next. Common-clock transport and complete target-control finiteness are outside this component’s guarantee. The fixtures contain no native events, and the region theorems do not close computational secrecy.

1.88 Historical component: Finite execution in the primitive oracle loop

The primitive target retains the existing physical output/input operations and native oracle dispatch. Computation, preparation and response use the pinned primitive compiler; phase returns inspect its halt control and saved caller memory. Lowering ℓ_P preserves every tape and flattens the old terminal pure call dispatch into its ready state, allowing a zero-step match there. The primitive wrapper is proved equal to the pinned compiler tick.

Lean: [BitOraclePrimitiveLoop.primitiveTick_eq](#).

Write $S_P(c)$ for membership of each old deterministic residual label in its compiler-derived finite support. Other phases already have finite tags and memory. The fixed factor G_P is one plus the sum of code-factor maxima over source labels, request ports and response ports. Ready states derive S_P ; every old-loop successor preserves it for every raw native answer, and each supported action matches within G_P primitive-loop transitions.

Lean: [BitOraclePrimitiveLoop.step_supported](#).

Lean: [BitOraclePrimitiveLoop.run_supported](#).

Lean: [BitOraclePrimitiveLoop.supported_step_match](#).

The relation $C_P(o, B, s)$ exposes complete query-tree refinement:

$$\begin{aligned} C_P(\text{return } c, B, s) &\iff \exists t \leq B. \text{run}_P(t, s) = \text{return } \ell_P(c), \\ C_P(\text{query}(q, k), B, s) &\iff \exists t \leq B, \kappa. \text{run}_P(t, s) = \text{query}(q, \text{return} \circ \kappa) \\ &\quad \wedge \forall a. C_P(k(a), B - t, \kappa(a)). \end{aligned}$$

Theorem 1.90 (Bounded finite primitive refinement; machine-checked). *For every fixed source program P , old tape-loop state c and finite fuel n ,*

$$S_P(c) \implies C_P(\text{Tape.run}(P, n, c), nG_P, \ell_P(c)).$$

For a canonical source ready entry, support is derived automatically. The conclusion retains all native query/answer branches and complete lowered leaf states, with the accumulated bound nG_P on each branch.

Lean: [BitOraclePrimitiveLoop.run_tape](#).

Lean: [BitOraclePrimitiveLoop.run_ready](#).

Eight Lean controls include a general arbitrary-fuel/word instantiation and literal primitive runs with mixed local computation, adaptive/cache-hit calls, aliased ports and coins. Full caller/private words and native tapes agree with independent source expectations. Zero-clock and actual live-goto controls exclude premature halt. Existing independent campaigns pass 4,800 pure compiler cases, 2,880 three-call physical/cache cases and 2,000 native-prefix cases; these campaigns do not execute the complete primitive loop.

Lean: [OraclePrimitiveLoopControls.mixed_run](#).

Lean: [OraclePrimitiveLoopControls.goto_returns_live](#).

Finite observation fuel may end live: this upper bound does not authorize padding. The next result derives common-clock transport under source termination and proves the complete target-control invariant.

1.89 Historical component: Finite target control and primitive source clocks

The target invariant now covers intermediate residual instructions in the pinned compiler's finite supports. Every primitive/native successor preserves it for arbitrary raw answers. Erasing only the three tapes defines a control projection π ; a code-derived finite set F_P contains $\pi(c)$ for every reachable target state c , independently of input size, answer size and fuel. Reconstruction proves that no other state was dropped:

$$\text{withTapes}(\text{tapes}(c), \pi(c)) = c.$$

Both work and native alphabets are finite. These projections are proof-side representation facts, not new runtime operations.

Lean: `BitOraclePrimitiveFinite.reachable_control`.

Lean: `BitOraclePrimitiveFinite.reconstruct`.

Lean: `BitOraclePrimitiveFinite.alphabets_finite`.

Theorem 1.91 (Derived primitive source clock; machine-checked). *Fix source code P , initial caller c , source fuel f , answer bound L and source charge bound B . Assume the original all-allowed-branch contract $\text{Within}_L(B, \text{Source.run}(P, f, c))$: every permitted source leaf halts with charge at most B . With H the initial maximum word length, previous native query w of length V , arbitrary old answer tape A , and fixed code factor G_P , put $T = 650G_PB(H + B + V + 1)^2$. In compressed notation,*

$$\begin{aligned} \text{Obs} < \$ > \text{simulate}(\alpha_L, \text{Primitive.run}(P, T, \ell_P(\text{ready}(c, Q(w), A)))) \\ &= (\text{some} \circ \text{fst}) < \$ > \text{simulate}(\alpha_L, \text{Source.run}(P, f, c)). \end{aligned}$$

The unchanged adapter α_L allows every raw request, both coin values and hash answers of length at most L . The equality preserves every lawful bounded-oracle handler's complete effects.

Lean: `BitOraclePrimitiveBounded.run_source_bounded`.

Lean: `BitOraclePrimitiveBounded.run_source_handler`.

The proof derives both primitive correspondence and halt of the intermediate tape run; neither is a caller certificate. Padding is justified only after halt. `Obs` interprets all original caller words at ready boundaries; it is not a free executed decoder or an added protocol disclosure. The adaptive $H(x), H(H(x)), H(x)$ canary derives halt and $B = 4 + 2|x| + 4L$ itself.

Lean: `BitOraclePrimitiveCanary.run_handler`.

Eight controls check adaptive/cache-hit and zero-width effects, arbitrary-fuel control membership, retained labels/memory and unrestricted native-word storage. A literal spinning program emits extra queries under 2,000 primitive ticks; live padding remains refuted. Common-clock controls apply the general theorem, not a benchmark of its large symbolic clock. Retained compiler/cap/physical-I/O fixture campaigns remain bounded region evidence, with zero failures/gaveUp.

Lean: `OraclePrimitiveContractControls.live_padding_adds_queries`.

Target finiteness and primitive common-clock transport are discharged. Physical initial loading is supplied next.

1.90 Historical component: Physical initial raw-input loading

Fix code P , an original input port i , initial label and finite memory. The initial caller c_x has the raw word x on port i and every other port empty. The executable prelude starts with blank working storage and a native input tape $Q(x)$. It writes the bottom-marker cell, reuses the physical input

loader on i , and hands its actual tapes to the primitive loop using finite control. Canonical packing occurs only in the proof statement.

Theorem 1.92 (Physical startup and source transport; machine-checked). *For every raw bitstring x , there exists $u \leq 4|x| + 8$ such that*

$$\text{Startup.run}(P, u, \text{blank}, Q(x)) = \text{pure}(\text{running}(\text{ready}(c_x, Q(\square)), Q(x))).$$

Here *ready* denotes the canonical full work tape with both private columns empty; the input tape and head are restored. Assume additionally $\text{Within}_L(B, \text{Source.run}(P, f, c_x))$, the original source halt/charge contract for all allowed answers. Put $T = 650G_P B(H + B + 1)^2$, where H is the maximum initial source word length. For a derived $u \leq 4|x| + 8$, in compressed notation,

$$\begin{aligned} \text{Obs} < \$ > \text{simulate}(\alpha_L, \text{Startup.run}(P, u + T, \text{blank}, Q(x))) \\ &= (\text{some} \circ \text{fst}) < \$ > \text{simulate}(\alpha_L, \text{Source.run}(P, f, c_x)). \end{aligned}$$

This equality transports every lawful bounded-oracle handler's complete effects. The derived startup time is shared by the entire query tree, independently of oracle answers. No presentation or intermediate correspondence premise is added.

Lean: `BitOracleInitialInput.load`.

Lean: `BitOracleInitialInput.run_running`.

Lean: `BitOracleInitialInput.run_source_bounded`.

Lean: `BitOracleInitialInput.run_source_handler`.

Six controls include empty input (8 ticks), a non-palindromic word on port 1 (20 ticks), zero clock, an incomplete prefix, a subsequent actual hash request, and an already halted entry label. The raw input becomes the first request without an input-side oracle event. The general result bounds startup time; it does not assert exact equality $u = 4|x| + 8$ for every word.

Lean: `OracleInitialInputControls.nonpalindromic_input`.

Lean: `OracleInitialInputControls.loaded_request`.

Lean: `OracleInitialInputControls.before_handoff`.

The boundary accepts arbitrary bits, including malformed encodings. Parsing, rejection and distribution of decoded operands remain charged source work. It does not grant free loading of arbitrary predecoded stacks. Existing physical transfer fixtures pass 5,400 replacement and 1,800 empty-column cases; full startup execution is checked by the Lean controls.

1.91 Historical component: Local standard-tape computation

The converse local-computation obligation uses the pinned TM0 model with alphabet $\{\perp, 0, 1\}$ and a fixed nonempty finite set of labels. Two existing bit stacks store the cells on either side, nearest the current head first. Each stored cell has two bits: $\perp \mapsto 00$, $0 \mapsto 10$, $1 \mapsto 11$. A three-valued finite register stores the current cell. Missing cells mean blank, not the data bit zero. Every native tape has such finite representatives; choosing them is proof-side existence, not executed input conversion.

Lean: `BitTapeCoverage.represents_every_tape`.

Lean: `BitTapeCoverage.cells_length`.

Theorem 1.93 (Native local computation in bit source; machine-checked). *Fix a native program M and a represented configuration c . Let $E(c)$ be its bit-source representation, P_M its compiled source program, and N_M the native transition with an explicit terminal label. At a live label, N_M*

agrees with *Mathlib*'s `TM0.step`; when it returns no successor, N_M records halt and retains the final tape. For every f ,

$$\exists a \leq 7f, \quad \text{Source.run}(P_M, f, E(c)) = \text{pure}(E(N_M^{[f]}(c)), a).$$

Every finite run preserves both full tape sides and the current cell. Each compiled statement has source charge at most seven. If $N_M^{[f]}(c)$ halts, this derives the original source contract with $B = 7f$. The existing primitive loop then returns $E(N_M^{[f]}(c))$ at clock $650G_{P_M}B(H + B + 1)^2$, with H the initial source height and empty initial native query/answer tapes. The primitive result is observed at the ready boundary using the same proof-side full-caller observation as before.

Lean: `BitTapeCoverage.native_step`.

Lean: `BitTapeCoverage.source_run`.

Lean: `BitTapeCoverage.charged_run`.

Lean: `BitTapeCoverage.local_cost`.

Lean: `BitTapeCoverage.within`.

Lean: `BitTapeCoverage.primitive_run`.

Six controls check a complete write/move run, its live prefix, independent pinned TM0 execution, motion into blank storage, blank-versus-zero branching and the derived cost. The complete fixture has blank at the head, zero one cell left and one two cells left. Independent sparse-tape and encoded-stack interpreters agree on 1,200 paths and 48,000 transitions (seeds 7/19/41, initial lengths 0–12, 40 actions, 54 cells checked on either side). Missing-cell-as-zero and dropped-push mutants are detected; failures and gaveUp are zero.

Lean: `BitTapeCoverageControls.native_run`.

Lean: `BitTapeCoverageControls.blank_is_not_false`.

This covers ordinary deterministic local computation only. The next result connects physical raw input conversion to native execution. Native coin/hash suspension, query/answer transfers and successor framing are outside this component's guarantee. Full intended standard-PPT attacker coverage, actual scalar-oracle policy and complete algorithm costs are outside this component's guarantee. No all-PPT secrecy claim follows from this component.

1.92 Historical component: Physical input through native local execution

The input prelude uses the same two bit stacks and three-valued cell register. It reverses raw bits onto the left workspace, inserts present-cell tags while moving them back, then extracts the current head. This takes exactly $2|x| + 3$ source transitions for every raw word x . The left stack ends empty; the head is the first bit (blank for empty input), and the right stack contains the remaining cells in their original order. No encoded cells are supplied.

Lean: `BitTapeInput.run`.

The existing return linker connects the three conversion phases to arbitrary fixed native code. A separate halt label precedes the relocated native labels, so a native return cannot restart conversion. Conversion statements have source charge at most five; the linked program's bound is seven. The generic deterministic charge induction also supplies the preceding native coverage result, replacing its duplicate specialized induction.

Lean: `BitTapeStart.convert_run`.

Lean: `BitTapeStart.charged_run`.

Lean: `BitOracleMachine.compute_run_cost`.

Theorem 1.94 (Physical raw input to native result; machine-checked). *Fix a TM0 program M over blank/false/true cells with a nonempty finite label set, entry label q , and raw input x . Let c_x be the ordinary native input configuration with head on the first bit. Assume $N_M^{[f]}(c_x)$ halts. For the linked source code P , put*

$$B = 7(2|x| + f + 4), \quad T = 650G_P B(|x| + B + 1)^2.$$

There exists $u \leq 4|x| + 8$ such that, in compressed notation,

$$\begin{aligned} \text{Obs} &< \$ > \text{simulate}(\alpha_L, \text{Startup.run}(P, u + T, \text{blank}, Q(x))) \\ &= \text{pure}(\text{some}(E_P(N_M^{[f]}(c_x)))). \end{aligned}$$

Here E_P retains the complete represented native tape and relocates its labels into P . This holds for every bounded-answer adapter α_L . The linked source is deterministic and its complete query tree is pure. The original native halt bound derives source termination, charge, initial presentation and conversion correctness; none is a separate caller certificate.

Lean: `BitTapeStart.physical_run`.

Seven controls check conversion and its prefix, linked source execution, the separate native return, empty input, an actual 1,024-step physical run, and a native halt bound for every input of the test machine. A false input head becomes true under native execution; its tail remains unchanged. Independent conversion fixtures pass on 3,000 inputs (seeds 7/19/41, lengths 0–64), detecting reversed-cell and skipped-head mutants; failures and gaveUp are zero.

Lean: `BitTapeInputControls.physical_literal`.

Lean: `BitTapeInputControls.native_return`.

This closes raw entry for local computation.

1.93 Historical component: Native oracle reference and private frames

The native reference machine has three ordinary tapes over $\{\perp, 0, 1\}$: work W , query Q and answer A , with independent heads. Fixed code receives only a finite label and the three current cells. A local command selects at most one move/write on each tape, or leaves it alone. Native commands issue the existing raw hash/coin events and return to a finite label. Both the transition-table input and command domains are finite. Returned configurations are internal states, not additional public election observations.

Lean: `NativeOracleTape.heads_finite`.

Lean: `NativeOracleTape.commands_finite`.

Theorem 1.95 (Native event and successor agreement; machine-checked). *Let a live configuration c select a hash command with return label r . Writing $\text{word}(Q)$ for the contiguous query word from its current head, the reference transition is exactly*

$$\text{step}(P, c) = \text{hash}(\text{word}(Q)) \gg= (a \mapsto \text{pure}(r, W, Q, Q(a))).$$

Here $Q(a)$ denotes the canonical answer tape containing a at its origin. The coin equation instead queries the actual coin and writes $Q([b])$. Every supported native successor has the selected label, complete unchanged work/query tapes and a canonical replacement answer tape. No canonical-query or caller-supplied frame premise is required.

Lean: `NativeOracleTape.hash_step`.

Lean: `NativeOracleTape.coin_step`.

Lean: `NativeOracleTape.oracle_successor`.

These are the existing dispatcher events. Its work component is unused, so a semantic adapter retains the reference work tape directly while reusing its query/answer operation. This introduces no runtime work conversion. Complete bounded runs compose, and halted states retain all tapes without issuing calls.

Lean: `NativeOracleTape.run_add`.

Lean: `NativeOracleTape.run_halted`.

Six controls cover embedded query blanks and stale cells, moved query heads, private work, replacement of a displaced answer, both coin-driven branches and halt. A checked counterexample excludes aliasing work and answer: their visible word prefixes can agree while their private cells to the left differ. This is an insufficient-representation counterexample, not a cryptographic attack.

Lean: `NativeOracleTapeControls.work_answer_alias_fails`.

Lean: `NativeOracleTapeControls.moved_query`.

Independent fixtures pass 3,000 native events (seeds 7/19/41, sparse cells at -4.4 , heads -3.3 , hash reply lengths 0–12), detecting work-alias and query-reset mutants; failures and gaveUp are zero. They do not test a source compiler. These component fixtures do not justify normalization of conventional PPT algorithms to the native reference machine.

1.94 Historical component: Conditional asymptotic completion prototype

This historical prototype checks the asymptotic assembly at the level of oracle programs. It uses the original prepared election and both exact DDH tests from the finite reduction. Its DDH-security premises are explicit. This prototype does not establish reduction efficiency; the current theorem carries the named external efficiency assumptions described in the formal reference. No protocol observation, rejection branch or attacker callback is changed.

Family and accuracy. At parameter n , let F_n be the existing finite scalar field, G_n its module, with the same sampling and decidable-equality instances. A `PreparedFamily` supplies the generator, fingerprint, preparation, saved-state types and attacker callbacks. It includes generator/scalar injectivity and hash-query bounds $p(n), c(n)$ on preparation and every casting callback. It contains no local-runtime or attacker-coverage certificate. Write $q(n) = |F_n|$ and let $b(n)$ be the absolute winning bias of this original prepared shared-oracle game. For each fixed $k \in \mathbb{N}$, set

$$e_k(n) = (n + 1)^{k+1}, \quad N(n) = p(n) + c(n) + 9.$$

Let $M_k(n)$ be the actual main DDH advantage at $(N(n), e_k(n))$, and $J(n)$ the actual honest-rejection DDH advantage. These names abbreviate the two algorithms in the finite theorem; neither is an independently supplied bound.

Theorem 1.96 (Conditional prepared-family secrecy; machine-checked). *In the family above, assume a polynomial $P \in \mathbb{N}[X]$ bounds $p(n) + c(n)$ for every n . Assume $(q(n) - 1)^{-1}$ is negligible and*

$$(\forall k \in \mathbb{N}, M_k \text{ is negligible}) \quad \wedge \quad J \text{ is negligible.}$$

Then b is negligible. Negligibility uses pinned `VCVio`'s definition, with real nonnegative advantages embedded by `ofReal`.

Lean: `ElectionSecrecyPrototype.prepared_negligible_of_ddh`.

The proof derives the finite theorem’s applicability rather than assuming it: for each fixed k , eventually

$$12(p(n) + c(n) + 10)e_k(n) \leq q(n).$$

The actual replay count $3456(N(n) + 1)^3 e_k(n)^4$ is bounded by a polynomial which may depend on P and k . This bounds the number of trials and does not bound their local runtime.

Lean: [ElectionSecrecyPrototype.accuracy_admissible](#).

Lean: [ElectionSecrecyPrototype.replay_bounded](#).

To expose the quantifier order, fix k . After multiplying the finite advantage bound by n^k , its extraction allowance satisfies

$$n^k \frac{2}{e_k(n)} \leq \frac{2}{n+1} \rightarrow 0.$$

The scaled algebraic and DDH errors also tend to zero under the stated premises. Repeating this argument for every fixed k establishes negligible bias. It does not choose one exponentially accurate reduction.

Lean: [ElectionSecrecyPrototype.negligible_of_polynomial_accuracy](#).

Controls and scope. Four controls check an exact replay count, scaled rational slack, failure of using the same linear accuracy for every degree, and the stronger fact that $2/(n+1)$ is not negligible. Existing finite queried-election and repair controls are retained. Independent arithmetic fixtures pass 3,000 cases (seeds 7/19/41; $n = 0..128$, $k = 0..8$, budget 0.64), detecting fixed-accuracy and linear-replay mutants; failures and gaveUp are zero. This tests arithmetic, not cryptographic hardness.

Lean: [ElectionSecrecyPrototypeControls.polynomial_accuracy_fixture](#).

Lean: [ElectionSecrecyPrototypeControls.fixed_accuracy_not_negligible](#).

The next section proves probability-law sampler transport in both DDH worlds. The formal reference states the current attacker coverage theorem, protocol scope and external assumptions connecting the reduction to a machine-level DDH hypothesis.

1.95 Historical component: Fair-bit transport of the exact DDH reductions

The completion prototype’s sampling obligation can be discharged at the probabilistic-program level. For an actual DDH reduction A , define A_s^{bits} by replacing its private uniform-range queries with the existing fair-bit modulo sampler at slack s . The pinned `uniformSampleImpl` expresses this coin-only computation in the `ProbComp` DDH interface with exactly the same output law. The real and random DDH challenger distributions are unchanged.

Lean: [ElectionDDHFairBits.bits_law](#).

Suppose every execution of A on a fixed public generator and arbitrary challenge coordinates has uniform-index query bound B . The complete output law in each DDH world changes by at most $B2^{-s}$. Consequently the actual two-game advantages satisfy

$$|\text{Adv}_{\text{DDH}}(A_s^{\text{bits}}) - \text{Adv}_{\text{DDH}}(A)| \leq 2B2^{-s}.$$

Both world errors are retained. Setting $s = n$ gives negligible error for any polynomial bound on B , using Mathlib’s polynomial-over-exponential limit and VCVio’s negligible-loss closure.

Lean: [ElectionDDHFairBits.real_distance](#).

Lean: [ElectionDDHFairBits.random_distance](#).

Lean: [ElectionDDHFairBits.advantage_distance](#).

Lean: [ElectionDDHFairBits.negligible_of_bits](#).

Theorem 1.97 (Prepared-family secrecy from fair-bit DDH tests; machine-checked). *Let D be the original prepared-election family over prime scalar fields $\mathbb{Z}_{q(n)}$, with its generator injectivity and pointwise preparation/ casting hash bounds. Assume $p + c$ has a natural polynomial bound and $(q(n) - 1)^{-1}$ is negligible. Suppose complete preparation, casting and guessing query bounds $P(n), C(n), J(n)$ hold for every callback state, with $P + C + J$ bounded by a natural polynomial. Assume the exact fair-bit main DDH reduction is secure for every fixed accuracy degree k , and the exact fair-bit rejection DDH reduction is secure; both use sampling slack n . Then the original prepared-election winning bias is negligible.*

Lean: [ElectionSecrecyFairBits.prepared_negligible_of_fair_bits](#).

The full main query count is derived from the existing bound $(1 + 3t)(4P + 4C + 78) + (4 + 4J)$, where t is the actual replay count. The rejection test uses $4P + 4C + 78$. Polynomial domination of both counts and the sampling losses are proved, then composed with the preceding prototype. No separate reduction-budget certificate is assumed.

Five controls check adaptive tests in both DDH worlds, their combined advantage allowance, retained modulo bias, dependence on the given public input, and a counterexample to dropping one world's loss. Independent enumeration passes 750 cases (seeds 7/19/41; fields 3, 5, 7; slack 0–6; two adaptive private queries), detecting erased-bias, changed-challenger and one-world-loss mutants; failures and gaveUp are zero. The toy fields carry no hardness assertion.

Lean: [ElectionDDHFairBitsControls.actual_modulo_bias](#).

Lean: [ElectionDDHFairBitsControls.one_world_allowance_fails](#).

This transport theorem concerns the exact fair-bit reductions. It does not establish efficient modulo, field/group or callback execution, operand-width bounds or attacker coverage. The probability-law transport does not implement those operations or complete the physical scalar sampler.

1.96 Historical component: Complete loaded-word binary remainder

Its finite control uses the existing subtraction stacks plus raw and pending input stacks. It reverses the input by stack moves, consumes one most-significant bit at a time, and calls the checked update through the existing stack-frame and return-link libraries. Two transfer passes move each result back into the remainder stack without reversing its little-endian order. The program's instructions do not call host modulo, list reversal or conversion to numbers.

Lean: [BinaryModuloMachine.supports](#).

Theorem 1.98 (Complete loaded-word division; machine-checked). *Let w be any little-endian Boolean word, including high zero padding, and let $q > 0$. Initially load w on the raw input stack and canonical q digits on the modulus stack; the remainder, pending input and four subtraction scratch/output stacks are empty. Then there exists an execution length*

$$u \leq |w|(8 \text{ size}(q) + 13) + 2$$

of the actual TM2 program which halts with canonical digits of $\text{bitsValue}(w) \bmod q$ on the remainder stack, the exact original modulus retained, all input/scratch stacks empty and local memory reset. The result has at most $\text{size}(q)$ bits.

Lean: [BinaryModuloMachine.run](#).

Lean: [BinaryModuloMachine.result_width](#).

The loop invariant is $r < q$. Each incoming bit b forms $2r + b < 2q$, so the existing single guarded subtraction suffices. Its derived bound is $6 \text{ size}(q) + 9$. Reading the bit and both return-transfer passes raise this to at most $8 \text{ size}(q) + 12$ per bit. Initial reversal and final halt give the displayed

total. This counts TM2 ticks, with fixed finite instruction bodies; it does not yet claim the full physical sampler's runtime. List reversal and modulo in this explanation interpret the executed stack operations rather than serving as uncharged instructions.

Seven controls retain complete state: nonpalindromic input/output, padded input, modulus one, empty input, a live startup prefix, and the general modulus-three instance. In particular, $6 \bmod 5 = 1$ detects reversed input, and $2 \bmod 3 = 2$ detects reversing the returned digits into one.

Lean: [BinaryModuloMachineControls.input_order](#).

Lean: [BinaryModuloMachineControls.return_order](#).

Lean: [BinaryModuloMachineControls.live_prefix](#).

Independent fixtures pass 16,352 exhaustive cases (raw width 0–8, modulus 1–32) and 1,500 seeded cases (7/19/41; width 0–128, modulus 1–65,536). They compose the previously validated subtraction interpreter with explicit driver transfers and compare with integer modulo and binary strings. Input- reversal and return-transfer mutants are detected; failures and gaveUp are zero. Connecting the coin loader below to division, modulus loading and scalar digit handoff remain, together with physical execution/efficiency and standard-PPT coverage.

1.97 Historical component: Executed chronological coin loading

The fixed program `CoinWordLoader.code` has five Boolean stacks (width, response, accumulator, output and retained modulus), four labels and three local memory values. It pops one width token, queries a coin, moves the response into its accumulator and repeats. It then reverses the accumulator by stack moves. This returns the first queried bit first, as required by the original sampler’s little-endian encoding.

Theorem 1.99 (Loaded-width coin execution; machine-checked). *Let W, Q be arbitrary Boolean words and $w = |W|$. Start at the counting label with zero local memory, width stack W , modulus stack Q , and empty response, accumulator and output stacks. The actual program’s run for $4w + 2$ source transitions equals the following oracle computation: query w coins in order, let b be their chronological response word, and return the halted configuration with zero local memory, output b , retained modulus Q , every other stack empty, and exact source charge $15w + 9$. This is equality of oracle query trees, before selecting a handler.*

Lean: [CoinWordLoader.run_source](#).

Consequently, interpreting the output word by `bitsValue` agrees with `sampleFairBitIndex` after mapping its coin events to the raw coin port. For every positive q , observing the value modulo q agrees with the original `sampleFairBitModulo` value under the same mapping. These equalities retain the query tree, including effects of any common handler. Numeric interpretation in the statements is an observation; it adds no free conversion or modulo instruction to the program.

Lean: [CoinWordLoader.word_index](#).

Lean: [CoinWordLoader.run_modulo](#).

The initially proposed charge $15w + 8$ is refuted by the zero-width case, whose charge is nine. The source cost model charges the maximum branch cost, including five for the final empty accumulator branch. Five controls pin full-state bit order, arbitrary width payload and padded modulus retention, zero-width no-query behavior, the failed charge claim, and a live response prefix.

Lean: [CoinWordLoaderControls.charge_eight_fails](#).

Lean: [CoinWordLoaderControls.live_response](#).

An independent finite interpreter passes 1,023 exhaustive words of width 0–9 and 1,500 seeded cases (7/19/41; width 0–256, modulus width 0–128). It detects omitted reversal, an extra zero-width query and stale response storage; failures and `gaveUp` are zero.

Width and modulus are loaded inputs here. The following result connects these actual output stacks to binary division. Operand loading and delivering scalar digits to the handler/writer are outside this component’s guarantee; the complete physical sampler and standard-PPT efficiency are separate obligations.

1.98 Historical component: Executed coin-to-remainder sampler

One fixed oracle program now relocates the existing coin-loader instructions and enters the existing division algorithm on the same storage. An executable finite-coordinate translation changes division's enumerated stacks, labels and local memory into `Fin` indices. It preserves every intermediate configuration and transition, with local source charge at most 32.

Lean: `BinaryModuloCode.tick`.

Lean: `BinaryModuloCode.run`.

Lean: `BinaryModuloCode.local_cost`.

Theorem 1.100 (Combined coin sampling and division; machine-checked). *Let W be any width-token word, $w = |W|$, and $q > 0$. Start the combined program at label 15 with zero local memory, width W on port two, canonical q digits on port one, and every other stack empty. Define*

$$T(w, q) = (4w + 2) + (w(8 \text{size}(q) + 13) + 3),$$

$$B(w, q) = 15w + 9 + 32(w(8 \text{size}(q) + 13) + 3).$$

There exists a derived charge function c on response words, with $c(b) \leq B(w, q)$ whenever $|b| = w$, such that the actual run for $T(w, q)$ source transitions equals the following oracle query tree: query w coins in order, let b be their chronological word, and return charge $c(b)$ and the halted configuration with zero local memory, canonical digits of $\text{bitsValue}(b) \bmod q$ on port zero, retained q digits on port one, and all six remaining stacks empty.

Lean: `CoinModuloMachine.run_source`.

The loader's real call boundary supplies division's raw input. A checked label invariant keeps division outside the coin region; it returns through a separate halt label. The proof derives each division charge and the common clock, using the actual response-word length. Numeric observation of the produced digits equals the original `sampleFairBitModulo` query tree, with its coin events mapped to the raw coin port. Modulo is executed here; observation only interprets digits already produced by instructions.

Lean: `CoinModuloMachine.sampler_value`.

Five kernel controls retain full states and unused coin suffixes: $6 \bmod 5 = 1$, nonpalindromic $2 \bmod 3 = 2$, unit-modulus zero, empty-width no-query execution, and the live loader-to-division handoff. Independent composed interpreters pass 2,032 exhaustive cases (width 0–6, $q = 1$ –16) and 750 seeded cases (7/19/41; width 0–64, $q = 1$ –4096), detecting wrong-input-port and restart-loader mutants; failures and gaveUp are zero. Fixture charges use the proved division upper bound, not a measurement of exact branch costs.

Lean: `CoinModuloMachineControls.input_order`.

Lean: `CoinModuloMachineControls.live_handoff`.

Width and modulus remain loaded operands. The following result delivers the produced digits to the existing scalar-prefix writer. Serialized preparation is connected in Section 1.103. Physical cache-caller composition, protocol/reduction efficiency and intended standard-PPT coverage are outside this component's guarantee.

1.99 Historical component: Serialization of the executed scalar

The existing prefix writer now reads the sampler's remainder port directly. It reuses the checked framing algorithm through finite coordinate changes; no host operation decodes the remainder and reloads its digits. Define $E(n) = 1^{\text{size}(n)} 0 \text{bits}(n)$, with digits in little-endian order. On canonical

input digits, the writer takes exactly $3 \text{size}(n) + 3$ transitions, emits $E(n)$ before an arbitrary suffix, clears its work ports, and preserves the modulus and two extra ports.

Lean: `ScalarWriteCode.run_digits`.

Lean: `ScalarWriteCode.local_cost`.

Theorem 1.101 (Two-phase scalar serialization; machine-checked). *Let W be any width-token word, $w = |W|$, and $q > 0$. Run the preceding sampler from its specified initial state for $T(w, q)$ transitions. Pass all eight result stacks and local memory unchanged to the writer, setting only its entry label, and run it for $D(q) = 3 \text{size}(q - 1) + 3$ transitions. There is a derived charge function c such that, for every response word b of length w ,*

$$c(b) \leq B(w, q) + 32D(q).$$

This two-phase computation has exactly the oracle query tree that queries w chronological coins and returns charge $c(b)$ with the halted writer configuration: memory 2, $E(\text{bitsValue}(b) \bmod q)$ on port five, canonical q digits retained on port one, and every other port empty.

Lean: `ScalarWriteCode.from_sampler`.

Lean: `ScalarWriteCode.sampleWord_source`.

Observation of port five is the existing finite modulo sampler followed by E , with its coin events mapped to the raw port, before choosing any probability handler. The bound uses $n < q$ for the actual computed remainder; it does not assume a typed scalar or an encoding correspondence.

Lean: `ScalarWriteCode.sampleWord_value`.

Six kernel-checked controls cover the complete frame, wrong-endian rejection, zero's required delimiter, and sampled two/zero with untouched future coins. Independent fixtures pass 3,840 exhaustive and 750 seeded cases (7/19/41, scalar widths up to 256 bits), detecting reversed digits and omitted delimiters; failures and gaveUp are zero.

Lean: `ScalarWriteCodeControls.sampled_two`.

Lean: `ScalarWriteCodeControls.omitted_zero_fails`.

The theorem concerns the explicit two-phase driver. The next result compiles its boundary into one uninterrupted program. The following sections connect its encoded output to the actual adaptive caller and execute serialized width/modulus preparation. Complete physical cache-caller composition and the all-PPT endpoint are outside this component's guarantee; the sampling loss is unchanged.

1.100 Historical component: Uninterrupted coin-to-prefix execution

The fixed program now contains the sampler's 20 labels followed by the existing writer's eight labels, on the same eight stacks and three local-memory values. The sampler's terminal instruction jumps to writer entry 25; the writer's halt is final. No host resumption occurs between the algorithms.

Theorem 1.102 (Uninterrupted scalar sampler; machine-checked). *For any width-token word W , $w = |W|$, and $q > 0$, run the fixed program for $T(w, q) + D(q)$ transitions from label 15, memory zero, width W on port two, canonical q digits on port one, and every other stack empty. Its complete oracle tree and accumulated charge equal those of Theorem 1.101, with the final label representation changed to this program's label type. Consequently there is a derived c with*

$$c(b) \leq B(w, q) + 32D(q) \quad (|b| = w),$$

and the run queries w chronological coins and returns $c(b)$ and the halted configuration with memory 2, $E(\text{bitsValue}(b) \bmod q)$ on port five, q digits on port one, and every other stack empty.

Lean: [CoinScalarMachine.run_eq](#).

Lean: [CoinScalarMachine.run_source](#).

Port-five observation equals the existing modulo sampler followed by E , before any probability handler is chosen. All code is fixed independently of W and q ; their preparation remains outside this loaded-operand theorem.

Lean: [CoinScalarMachine.sampler_value](#).

The return-link proof proceeds through actual oracle transitions and returns as soon as the source halts. It derives source-leaf termination from the sampler theorem and continuation termination from the writer theorem, retaining complete configurations and charges. Only final halted configurations are padded.

Lean: [BitOracleReturnLink.run](#).

Why continuation termination is necessary. A retained checked counterexample starts with a one-step halt and redirects it to a looping coin query. A source clock of two and continuation clock of one produce one query in the staged execution but two in the uninterrupted execution. Thus removing the continuation-halt condition invalidates the composition claim.

Lean: [CoinScalarMachineControls.live_padding_fails](#).

Six kernel controls cover sampled two/one, empty-width zero, the actual live return, final halted padding, and that counterexample. Independent stack traces pass 2,032 exhaustive and 750 seeded cases (7/19/41, widths 0–64, $q = 1$ –4096), detecting a cleared remainder at entry, reversed writer digits and an extra query after halt. Failures and gaveUp are zero. The complete prepared sampler is connected in Section 1.103. The surrounding physical cache caller and protocol efficiency/PPT coverage remain separate obligations.

1.101 Historical component: Cache requests with executed sampling

A word-facing handler now probes the existing encoded cache. On a hit, its parsed scalar digits go directly to the existing prefix writer. On a miss, `CoinScalarMachine` supplies the encoded scalar word directly to the existing insertion and chronological log-append routines. It returns the encoded answer, cache and log. Probe, insertion or append failure remains explicit as an absent result; the following theorem concerns canonical source inputs. For a scalar $a \in \mathbb{Z}_q$, write $E(a)$ for $E(a.val)$. The reference exact-uniform handler is retained.

Lean: [CacheHashCoins.sample_eq](#).

Lean: [CacheHashCoins.writeDigits_encode](#).

Theorem 1.103 (Executed cache sampling and full-state error; machine-checked). *Let $p, q > 0$, let K be any typed hash key in the existing public group, and let (C, L) be any finite cache and chronological miss log. Supply their original encodings to the word-facing handler, with a width-token word W , $w = |W|$. If $C(K) = a$, it returns $(E(a), \text{enc}(C), \text{enc}(L))$ without a coin query. If K is absent, its exact oracle tree samples the existing width- w finite modulo sampler, obtains scalar a , and returns*

$$(E(a), \text{enc}(C[K \mapsto a]), \text{enc}(L \mathbin{++} [K])).$$

All these canonical results are present. For W consisting of $\text{size}(q) + s$ tokens, the total variation distance between the complete encoded handler output and the encoded output of the original finite logged source, interpreted with its canonical exact-uniform scalar sampler, is at most 2^{-s} .

Lean: [CacheHashCoins.hash_hit](#).

Lean: [CacheHashCoins.hash_miss](#).

Lean: `CacheHashCoins.hash_distance`.

The comparison is tied to `ballotFiniteLoggedImpl`, including its retained state. Its exact challenge interpreter is the pinned library's actual `uniformSample` on $\mathbb{Z}_q$. The proof uses contraction of variation distance through the common answer/cache/log continuation; it does not assert exact uniformity of fixed-width modulo sampling.

Lean: `CacheHashCoins.hash_source`.

Lean: `CacheHashCoins.exactChallenge_value`.

Five kernel controls cover a late hit, sampled six with a nonempty old log, zero-width sampling and delimiter insertion, exclusion of eager hit sampling, and a concrete full-output error bound. Independent interpreters pass 576 exhaustive cases at $q = 11$, four coin bits and up to two existing entries, detecting hit resampling, insertion of a wrong scalar word and log prepending; failures and gaveUp are zero.

Lean: `CacheHashCoinsControls.miss_control`.

This is a one-request word-facing driver composed from the executed routines. It still loads operands and sequences those routines through host control. The complete adaptive host caller is connected in the next result. Physical cache-caller port transfers and whole-caller runtime are outside this component's guarantee. Serialized operand preparation and the physical sampler are connected below.

1.102 Historical component: Complete adaptive callers with executed sampling

The caller now passes each hash to the word-facing handler and decodes its actual returned scalar prefix for the next source continuation. Ordinary uniform-range requests execute the original upper bound's increment and use the resulting range in the same scalar sampler. A malformed result remains explicit failure. Every later request sees the returned cache and chronological log. The existing codecs, source-caller induction and adaptive sampling-error theorem supply the connection. Lean: [CacheCallerCoins.sampleUpper_eq](#).

For positive p, q , let P be any existing ballot-oracle computation returning an arbitrary type A , and let $S = (C, L)$ be any finite cache and miss log. Define $J(P, S)$ to be the original `runBallotFiniteLogged` interpreted with `CacheHashCoins.exactChallenge`, including its original exact uniform scalar sampler. Let R_s be the existing `runFairBitUniform` transformation with slack s , and let $\mathcal{E}$ encode the final cache and log while retaining the return value. The implemented caller H_s supplies width tokens of length $\text{size}(r) + s$ for each requested positive range r .

Theorem 1.104 (Complete caller correspondence and error; machine-checked). *For every s, P, S , the entire coin-query trees satisfy*

$$H_s(P, \text{enc}(C), \text{enc}(L)) = (\text{some } \circ \mathcal{E})_* R_s(J(P, S)).$$

No successor-state or source-correspondence premise is supplied. If the original caller has total query bound B , then $J(P, S)$ has total range-query bound B , and the complete observed distributions satisfy

$$\text{TV}(\text{Law}(H_s(P, \text{enc}(C), \text{enc}(L))), (\text{some } \circ \mathcal{E})_* \text{Law}(J(P, S))) \leq B 2^{-s}.$$

Here `Law` interprets sampling probabilistically. Query bounds constrain all source continuations; they are not runtime bounds.

Lean: [CacheCallerCoins.run_eq](#).

Lean: [CacheCallerCoins.source_bound](#).

Lean: [CacheCallerCoins.distance](#).

Five kernel controls cover an adaptive miss/hit/miss trace, all-hit continuations, a unit range, exclusion of a changed repeated answer, and full-caller error. Independent interpreters pass 6,144 exhaustive and 300 seeded cases (seeds 7/19/41, modulus 11, slack 0–3), including later keys chosen from earlier answers. Hit-resampling, cache-reset and dropped-log mutants are detected; failures and `gaveUp` are zero.

Lean: [CacheCallerCoinsControls.adaptive_control](#).

Lean: [CacheCallerCoinsControls.hits_control](#).

Component boundary. Its Helios dependencies are the ballot-key/scalar codecs and original finite logged handler; prepared-election and replay-source computations are potential callers. Decoding and invoking typed continuations remain host operations, as do constructing and loading routine input records. The sampler now executes serialized width/modulus preparation as described below. This result connects complete adaptive behavior. It does not compile the caller to one physical machine or derive the combined execution cost.

1.103 Historical component: Serialized operands and the complete physical sampler

The sampler's local administrative input is $I(s, n) = 1^s 0 E(n)$, using the existing natural-prefix format E . Two finite entry labels preserve the two source interfaces. Direct entry reads modulus

$q = n > 0$; uniform entry reads the original inclusive upper bound n and executes binary increment to $q = n + 1$. The code parses E , counts modulus digits while restoring them, and adds the slack tokens. A common gate checks successful preparation and complete input consumption before entering the coin sampler. These local input records add no public election observations.

Theorem 1.105 (Executed operand preparation; machine-checked). *Start with $I(s, n)$ followed by any suffix, empty work ports and arbitrary private frame ports. The corresponding entry prepares literal q -digits and exactly $\text{size}(q) + s$ width tokens, preserves the suffix and private frame, and clears scratch. The following clocks bound the actual preparation:*

entry	q	$P(s, n)$
direct	$n > 0$	$s + 5 \text{size}(n) + 8$
uniform	$n + 1$	$s + 5 \text{size}(n) + 2 \text{size}(n + 1) + 10$.

The compiled preparation has charge at most $32P(s, n)$. Direct modulus zero reaches explicit failure; uniform upper bound zero prepares range one.

Lean: `SamplerOperands.run`.

Lean: `SamplerOperands.uniform_run`.

Lean: `SamplerOperands.zero_rejected`.

Lean: `SamplerOperands.charged`.

Write $w = \text{size}(q) + s$. The already checked sampler clocks and charges used in the composition are

$$\begin{aligned} S(w, q) &= (4w + 2) + (w(8 \text{size}(q) + 13) + 3) + 3 \text{size}(q - 1) + 3, \\ C(w, q) &= 15w + 9 + 32(w(8 \text{size}(q) + 13) + 3) \\ &\quad + 32(3 \text{size}(q - 1) + 3). \end{aligned}$$

These are source execution bounds. The following composition also derives the physical input-loading and compiler obligations.

Theorem 1.106 (Prepared sampler and physical execution; machine-checked). *For either entry with $q > 0$, supply exactly $I(s, n)$ on its raw input port and leave work storage blank. One fixed program executes preparation, the gate, coin loading, division and scalar-prefix writing. At clock $P(s, n) + 1 + S(w, q)$, its entire query tree draws w chronological coins b and returns $E(\text{value}(b) \bmod q)$, retaining the modulus and clearing scratch. Every branch has charge at most*

$$B = 32P(s, n) + 5 + C(w, q).$$

The encoded observation equals the original width- w fair-bit range sampler. Starting from a physical raw input tape and blank work storage, there exists a startup duration $u \leq 4|I(s, n)| + 8$ such that the same complete query-tree observation is obtained at physical clock

$$u + 650(|I(s, n)| + B + 1)^2 BG,$$

where G is the fixed compiled-program factor. Termination, source cost, input loading and correspondence are derived for this component. The bounded oracle adapter may use any limit: the source tree contains only coin requests.

Lean: `PreparedScalarMachine.run_source`.

Lean: `PreparedScalarMachine.sampler_value`.

Lean: `PreparedScalarMachine.within`.

Lean: `PreparedScalarMachine.physical_run`.

Caller connection and controls. The actual adaptive caller now uses `hashPrepared` on hash misses and `sampleUpper` on ordinary range requests. Their equality laws preserve its existing complete return/cache/log correspondence and sampling-error bound. The prior arbitrary-width sampler and cache handler remain checked references.

Lean: `CacheHashCoins.hashPrepared_eq`.

Lean: `CacheCallerCoins.sampleUpper_eq`.

Eight kernel controls cover literal widths and frames, carry growth, the unit range, direct zero rejection, malformed inputs, sampled output, pre-coin gating and physical unit-range execution retaining its coin query. Independent operand checks pass 6,966 canonical, 4,094 raw and 750 seeded cases. Complete sampler traces pass 3,059 exhaustive, 10 directed malformed and 300 seeded cases, using the actual prepared ports without numeric modulus reloading. Updated adaptive caller checks also pass. The campaigns use seeds 7/19/41; failures and gaveUp are zero. They detect skipped increment, lost slack, reversed digits, bypassed gating, overwritten modulus and extra coins, in addition to the retained caller mutations. Runtime checks use derived upper bounds, not exact measured charges.

Lean: `SamplerOperandsControls.physical_control`.

Lean: `SamplerOperandsControls.gate_control`.

Reuse and remaining boundary. The preparation interface depends on the existing natural format and uniform upper-bound convention; its potential callers are cache misses and adaptive range requests. It reuses the prefix parser, incrementer, stack frame/return interfaces, scalar sampler and physical loader/compiler. The only new data loop constructs width tokens while restoring modulus digits. The combined physical sampler is a component result. The surrounding cache caller still constructs its input records, transports routine arguments/results, decodes and invokes typed continuations through host code.

1.104 Historical component: Sampler execution from retained caller storage

The cache caller uses eight work ports and four retained ports: key on 8, cache on 9, chronological log on 10 and sampler operand record on 11. A frame translation maps the existing deterministic instructions using their verified interface and relocates oracle request/reply ports. Every extra word is retained, including across coin and hash responses. For every code, initial configuration, frame and finite clock, the complete query tree commutes with this translation; every leaf has the translated configuration and the exact original charge. This is an execution theorem, with no termination or response-size premise.

Lean: `BitOracleStackFrame.local_cost`.

Lean: `BitOracleStackFrame.step`.

Lean: `BitOracleStackFrame.run`.

Theorem 1.107 (Sampler branch with executed caller transfer; machine-checked). *Let $q > 0$, $s \in \mathbb{N}$ and $R = I(s, q)$. Start with R only on retained port 11, arbitrary key/cache/log words on ports 8–10, and all eight work ports empty. A fixed program copies R to input port 4 through scratch port 0, restores port 11, and enters the preceding prepared sampler through an actual return label. If its preceding clock and charge bound are T_p and B_p , the combined clock and charge bound are*

$$T_c = 2|R| + 3 + T_p, \quad B_c = 10|R| + 12 + B_p.$$

The complete query tree draws $w = \text{size}(q) + s$ chronological coins b , returns $E(\text{bitsValue}(b) \bmod q)$ on port 5, retains modulus digits on 1, clears the remaining work ports, and preserves all four retained records. Each branch has its actual accumulated charge, bounded by B_c . The returned scalar word has the original fair-bit range-sampler observation.

Lean: `CacheHashMachine.sample_run`.

Lean: `CacheHashMachine.sample_value`.

Both return-link termination premises follow from the executed copy and sampler results. The actual adaptive caller’s miss branch uses this program via `preparedHash`; its existing complete return/cache/log correspondence and sampling-error bound are unchanged. Initial record construction and surrounding routine dispatch still use host operations.

Lean: `CacheHashCoins.preparedHash_eq`.

Deriving the next routine’s workspace. The complete cache-read post-state now exposes empty work on a miss. A hit instead retains the unread suffix and a nonzero counter, as well as parsed scalar digits and the original query/cache. The caller must clear or move these actual words before entering a routine that requires empty scratch. A kernel control proves nonempty counter storage even for a one-entry hit.

Lean: `CacheReadMachine.miss_run`.

Lean: `CacheReadMachine.hit_run`.

Lean: `CacheHashMachineControls.hit_counter_control`.

Seven kernel controls retain the input-placement, live-copy, full sampled state, zero-failure and permuted-oracle boundaries; a misrouted coin destroys a literal private word. Independent frame tests pass 2,268 exhaustive and 600 seeded cases; complete caller-copy sampling passes 1,421 exhaustive and 300 seeded cases, with seeds 7/19/41 and the documented routing, replacement, record-loss and cache-loss mutations detected. Updated adaptive caller fixtures also pass. Failures and gaveUp are zero; Python charge checks use derived upper bounds, not measured exact charges.

Lean: `CacheHashMachineControls.sampled_control`.

Lean: `CacheHashMachineControls.hash_frame_control`.

Lean: `CacheHashMachineControls.misroute_control`.

Interface and remaining boundary. The frame lemma depends only on the existing finite oracle-machine interface. Its immediate caller is this sampler branch; cache/replay routines can use it when suspending with private storage. The concrete branch uses the existing Helios natural record and cache-caller layout. Its input-transfer and sampling charge are derived, but the complete probe/hit/insert/append dispatcher, its workspace transfers and its loaded physical execution cost are discharged in Section 1.105. Section 1.106 adds serialized request input loading.

1.105 Historical component: Complete cache requests in finite control

The complete request now executes the probe, hit writer or miss sampler, cache insertion and chronological log append in one fixed program. The theorem covers canonical records of every typed finite cache, key and log. It derives the intermediate workspace, return flags, termination and combined charge. The enclosing input link is proved in Section 1.106. Effective caller continuations are outside this component's guarantee. General raw-input cache validation is not claimed here.

Complete caller states. Write C, K, L, V for the original cache, key, log and scalar codecs, and let $R = I(s, q)$ be the retained sampler record of Section 1.103. The following notation compresses the finite label and memory types without changing any data. The entry has label 0, memory 0 and stacks

$$[C, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, K, \epsilon, L, R].$$

Write $F(K, C, L, R, V)$ for the halted state with memory 2 and stacks

$$[\epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, V, K, C, L, R].$$

Its request observation is (V, C, L) ; retaining the other words in a proof does not add a protocol disclosure. The code and port layout are fixed independently of p, q, s and the input sizes.

Source specification and derived bounds. Assume $p, q > 0$, a typed finite cache c , key k , typed log ℓ and slack $s \in \mathbb{N}$. Put $w = \text{size}(q) + s$ and let W_w be the raw chronological tree of w fair-bit requests. For a leaf b , let $a_b = \text{bitsValue}(b) \bmod q$, regarded as an element of $\mathbb{Z}_q$. The complete request specification is

$$Q(c, k, \ell, s) = \begin{cases} \text{pure } F(K(k), C(c), L(\ell), R, V(a)), & c[k] = a, \\ (b \mapsto F(K(k), C(c[k \mapsto a_b]), L(\ell \mathrel{++} [(((), k)]), R, V(a_b)))_* W_w, & c[k] \text{ absent.} \end{cases}$$

The hit branch makes no oracle request. The miss branch preserves the full tree, not merely its tally or scalar marginal.

Here is a compressed transcription of the input-derived bounds. Let T_r be the existing probe bound, T_s, B_s the copied-sampler clock and charge, T_i the insertion bound and T_a the append/transfer bound. All refer to the actual original routines on these encoded inputs. Set

$$\begin{aligned} T_a &= 1 + \text{AppendCost}(|\ell|, \text{keyBound}(p), |L(\ell)|) + 1 + 3|L(\ell \mathrel{++} [(((), k)])| + 3, \\ T_m &= 3(2 \text{size}(q - 1) + 1) + \text{size}(q) + 5 + 3|C(c)| + 3 + T_i + T_a, \\ H &= \text{height}(\text{entry}) + T_r A_r, \quad T_h = 2(H + 1) + 3 \text{size}(q - 1) + 3. \end{aligned}$$

A_r is the fixed probe code-access bound. Existing deterministic stack growth derives H , including the unread suffix and nonzero hit counter. The common request clock and charge are

$$\begin{aligned} T &= \max\{1 + T_r + 2 + T_s + T_m, 1 + T_r + 1 + T_h\}, \\ B &= \max\{2 + 32T_r + 6 + B_s + 32T_m, 2 + 32T_r + 4 + 32T_h\}. \end{aligned}$$

These are definitions computed from the inputs and fixed routine bounds; they are not hypotheses asserting the missing execution theorem.

Theorem 1.108 (Complete loaded-request execution). *For the inputs above, let E be the actual charged execution of `CacheHashDispatch.code` for clock T from the entry state. Then*

$$\text{fst}_* E = Q(c, k, \ell, s), \quad \forall (x, d) \in \text{support}(E), \quad x.\text{label} = \text{none} \wedge d \leq B.$$

No lookup outcome, freshness witness, intermediate presentation or cost certificate is an additional premise.

Lean: `CacheHashDispatch.request_run`.

Lean: `CacheHashDispatch.miss_run`.

Lean: `CacheHashDispatch.hit_run`.

Why the composition closes. The probe’s miss state supplies empty work storage and the actual absence condition needed by guarded insertion. Its hit state supplies parsed digits, but also retained suffix/counter words; both are cleared by executed loops. The miss path executes answer and cache copies, insertion, chronological append and a final log transfer. Every routine’s complete post-state establishes its successor’s entry. Final common-clock padding is applied only after halt. The earlier live-return padding counterexample remains valid.

Lean: `CacheRoutineCode.probe_miss`.

Lean: `CacheRoutineCode.probe_hit`.

Lean: `CacheHashDispatch.sampled_tail`.

Lean: `CacheHashDispatch.hit_tail`.

Theorem 1.109 (Physical execution and existing caller observation). *The actual request discharges the existing physical compiler’s *Within* contract for every bounded-oracle policy. From the loaded caller layout, the compiled primitive program agrees with Q at clock*

$$\text{unitCost}(\text{height}(\text{entry}) + B) B G,$$

where G is the fixed compiler factor of this code. Projecting the request observation gives exactly the existing adaptive caller’s `CacheHashCoins.hashPrepared` tree, with coins lifted to raw requests.

Lean: `CacheHashDispatch.within`.

Lean: `CacheHashDispatch.physical_run`.

Lean: `CacheHashDispatch.readout_source`.

Lean: `CacheHashDispatch.execution_source`.

Controls and reuse boundary. Seven kernel controls retain a hit’s unconsumed entropy, literal values three, six and zero, chronological append, nonconstant sampling and private storage. The actual-code native harness passes 324 directed cases for moduli 1, 3, 11, slack 0, 2, three caches, three keys, two logs and three coin tapes. It detects always-sample, skipped-append and wrong-port cleanup mutations. This finite campaign has no random seeds or discarded cases; its 20,000-step fixture cap is not a proved runtime bound. The general theorem supplies that evidence.

Lean: [CacheHashDispatchControls.hit_control](#).

Lean: [CacheHashDispatchControls.miss_control](#).

Lean: [CacheHashDispatchControls.zero_control](#).

Lean: [CacheHashDispatchControls.log_not_prepend](#).

Lean: [CacheHashDispatchControls.miss_frame_control](#).

The implementation reuses Mathlib’s explicit-list finite equivalences and the project’s coordinate, frame, return-link and physical-compiler interfaces. The label tables, original codecs and twelve-port layout are specific to this Helios caller; no new generic compiler or Lake dependency is introduced. The theorem supplies a proved request component for the adaptive caller. That caller still constructs its enclosing input, decodes returned words and invokes typed host continuations. The following result supplies request input loading and width bounds.

1.106 Historical component: Serialized input to a complete cache request

One fixed program now consumes a serialized request on a conventional input tape, parses its four fields, checks successful completion, and executes the complete request above. This removes the assumed loaded-port entry for this component. The four-field packaging is local machine input; it changes neither the historical protocol nor its public observations.

Loader interface. For arbitrary bit words K, C, L, R , let

$$J = \text{bitFieldsEncode}[K, C, L, R], \quad f(n) = 3 \text{size}(n) + 7 + n(2 \text{size}(n) + 3),$$

and set $A = 13 + f(|K|) + f(|C|) + f(|L|) + f(|R|)$. The loader entry has label 0, memory 0, word J on port 6, and every other port empty. Its successful final state, denoted $U(K, C, L, R)$, has halted control, memory 2 and stacks

$$[C, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, K, \epsilon, L, R].$$

The fixed outer header is 1110001, the original encoding of four. Each field uses the existing length-prefix parser. Previously read fields are outside the next parser’s local ports; its complete post-state restores the field and clears scratch. The final guard requires exhausted input.

Theorem 1.110 (Complete request-input parsing). *For every four bit words, there are $u \leq A$ and $d \leq 32u$ such that the actual charged loader run for u steps returns exactly $\text{pure}(U(K, C, L, R), d)$.*

Lean: [CacheRequestInput.load_request](#).

Lean: [CacheRequestInput.clock_lengths](#).

Lean: [CacheRequestInput.field_run](#).

This is canonical-input completeness. Payloads are opaque to the loader; general malformed-input rejection and inner Helios validity are not conclusions of this theorem. It also does not execute construction of J by an outer caller.

The guarded combined run. Use the typed inputs, positive p, q , canonical codecs and slack s from Section 1.105, with $R = I(s, q)$. The new program returns the loader to a guard: memory 2 executes a reset to 0 and enters the dispatcher; every other memory value executes rejection. This transition costs three. Let T, B, Q be the preceding request clock, charge and complete query tree, and let ι rename its control labels into the combined program, preserving halted control, memory and all stacks. Define

$$T_J = A + 1 + T, \quad B_J = 32A + 3 + B.$$

Theorem 1.111 (Complete serialized-request execution). *From J on port 6 and otherwise blank work, the actual combined execution E_J for T_J steps satisfies*

$$\text{fst}_* E_J = \iota_* Q, \quad \forall (x, d) \in \text{support}(E_J), \quad x.\text{label} = \text{none} \wedge d \leq B_J.$$

The success guard, complete request and final-halt padding are executed or derived. No loaded configuration, intermediate return, or cost certificate is a caller-supplied premise.

Lean: `CacheRequestMachine.request_run`.

Lean: `CacheRequestMachine.enter_request`.

Width interface. The request bound now eliminates typed cache/log counts using their encoded lengths. Write $c = |C|$, $l = |L|$, $k_p = \text{keyBound}(p)$, $r = s + 2 \text{size}(q) + 2$, and let a_r be the fixed probe access bound. The following is a transcription of `CacheHashDispatch.loadedClock`; the named routine costs and `bitListSize` are the existing proved interfaces:

$$\begin{aligned} P &= \text{ReadCost}(p, q, c, c), \quad J_l = \text{bitListSize}(k_p, l + 1), \\ E &= 1 + \text{AppendCost}(l, k_p, l) + 1 + (3J_l + 3), \\ D &= 3(2 \text{size}(q - 1) + 1) + \text{size}(q) + 5 + 3c + 3 + \text{InsertCost}(p, q, c, c) + E, \\ H &= k_p + c + l + r, \\ \Lambda &= \max\{1 + P + 2 + T_s + D, \\ &\quad 1 + P + 1 + 2(H + Pa_r + 1) + 3 \text{size}(q - 1) + 3\}. \end{aligned}$$

Then $T \leq \Lambda$ and $B \leq 32\Lambda$. Consequently $T_J \leq A + 1 + \Lambda$ and $B_J \leq 32(A + 1 + \Lambda)$. The public p width remains explicit: a short encoded identity does not bound the ambient group width. The retained sampler record contributes to H . Appending a log entry may enlarge its outer count header; its length bound uses J_l , not merely the old length plus a field.

Lean: `CacheHashDispatch.request_clock_le_loaded`.

Lean: `CacheHashDispatch.request_cost_le_loaded`.

Lean: `CacheHashDispatch.private_record_omission_control`.

Lean: `CacheHashDispatch.append_header_control`.

Lean: `CacheRequestMachine.clock_le_loaded`.

Lean: `CacheRequestMachine.cost_le_loaded`.

Theorem 1.112 (Physical input and request execution). *Let H_J be the height of the one-port source entry and G_J the fixed primitive compiler factor for the combined code. For every bounded-oracle policy there is a startup length $v \leq 4|J| + 8$ such that, from blank physical work storage and conventional input tape J , execution for*

$$v + \text{unitCost}(H_J + B_J)B_JG_J$$

has observed query tree $\text{some}_ \iota_* Q$, under that same policy. The source termination/charge contract is derived from the combined run.*

Lean: `CacheRequestMachine.within`.

Lean: `CacheRequestMachine.physical_run`.

Independent controls and remaining scope. The actual loader passes 3,260 independently encoded native fixtures: 1,009 accept and 2,251 reject. Seeds are 118, 20260914 and 198, with 128 generated valid records per seed, plus directed and exhaustive short-word cases. Three mutants are detected: swapped key/log ports, skipped fourth-field guard and ignored suffix. Their locally minimized witnesses are respectively 11100010010100, 1110001000 and 111000100000. The combined program also passes 127 cases at $p = 23, q = 11$: 36 hits, 72 misses and 19 malformed framing cases. A guard bypass is detected by appending one false bit to a typed hit input. All final ports and remaining coins are checked. Eleven kernel controls retain the hit/miss/zero values, coin consumption, log order, private frame and rejection gate. Two minimized framing failures are checked on the actual combined run: missing fourth field in 36 steps with charge 136, and trailing suffix in 42 steps with charge 161.

Lean: `CacheRequestMachineControls.hit_control`.

Lean: `CacheRequestMachineControls.miss_control`.

Lean: `CacheRequestMachineControls.rejected_gate_not_dispatched`.

Lean: `CacheRequestMachineControls.missing_fourth_field_control`.

Lean: `CacheRequestMachineControls.trailing_suffix_control`.

No case is discarded or times out; these finite campaigns do not prove a runtime bound or general rejection. Their caps are 20,000 and 100,000 driver steps, respectively.

The implementation reuses the existing parser, framing, coordinate, return-link and physical-input/compiler proofs. Its four-field grammar, destination ports and dispatcher choice are specific to this caller; possible callers are the actual Helios cache/sampling reduction paths. It assumes canonical typed payloads only for the request theorem and adds no cryptographic assumption. Neither this component nor its width bounds establish all-PPT computational secrecy.

1.107 Historical component: Bounds at original request boundaries

The request component now has a polynomial bound specialized to reachable historical source states. This closes its width/budget obligation for the request component; it does not execute the enclosing adaptive caller.

Original-source observations. For a finite logged source P and initial state $s = (c, \ell)$, write $\mathcal{R}(P, s)$ for its original interpreter. The analysis interpreter $\widehat{\mathcal{R}}(P, s, E)$ also carries a list of events, each containing an original request and its input state. At request t with continuation k , it runs the unchanged original handler to obtain (a, s') , then continues as

$$\widehat{\mathcal{R}}(k(a), s', E ++ [(t, s)]).$$

Thus cache hits and uniform requests are retained even when they generate no fresh hash challenge. These events are proof instrumentation: they add neither public observations nor a claimed executed storage cost.

Theorem 1.113 (Capture and source-derived storage). *Erasing the event list from $\widehat{\mathcal{R}}(P, s, E)$ gives exactly $\mathcal{R}(P, s)$ as an oracle query tree. Every supported original result has a supported instrumented result with that same answer and final state. Existing events remain an ordered prefix through every*

supported continuation. If P has hash-query budget n and the initial event list is empty, every recorded event $(t, (c', \ell'))$ satisfies

$$|c'| \leq |c| + n, \quad |\ell'| \leq |\ell| + n.$$

Lean: `CacheRequestPrefixes.run_eq.`

Lean: `CacheRequestPrefixes.supported_lift.`

Lean: `CacheRequestPrefixes.query_run.`

Lean: `CacheRequestPrefixes.events_retained.`

Lean: `CacheRequestPrefixes.request_captured.`

Lean: `CacheRequestPrefixes.prestates_bound.`

The generic observer needs decidable key equality, with no cryptographic or sampling-law premise. Each original handler step grows cache and log by at most one on a hash request and by zero on a uniform. Induction consumes the source's hash budget at those original boundaries. Final-state storage bounds alone would not establish this result.

Polynomial request bound. Let a_r again be the fixed probe access bound and put $M = 10^6(a_r + 1)$. For the complete loaded clock Λ_J from the previous section, arbitrary natural parameters satisfy

$$\Lambda_J(p, q, K, C, L, s) \leq M(\text{size}(p) + \text{size}(q) + K + C + L + s + 1)^3.$$

Here K, C, L are payload lengths, not entry counts. The theorem covers zero arithmetic parameters; validity of the cryptographic request still requires positive moduli. Polynomial bounds on the six displayed inputs give an explicit polynomial bound on Λ_J .

Lean: `CacheRequestMachine.loadedClock_le_cubic.`

Lean: `CacheRequestMachine.loadedClock_polynomial.`

For positive p, q , let k_p be the existing key codec bound, $C_{p,q}(c)$ the complete cache codec bound and $L_p(l)$ the complete log codec bound, including their outer count headers. Define

$$\mathcal{B}(p, q, c, l, s) = M(\text{size}(p) + \text{size}(q) + k_p + C_{p,q}(c) + L_p(l) + s + 1)^3.$$

Theorem 1.114 (Executed request bounds on historical source states). *At every captured prestate of a source with initial counts (c, l) and hash budget n , the complete request clock for any typed hash key is at most $\mathcal{B}(p, q, c + n, l + n, s)$. If the captured request is that hash query, every supported leaf of its actual request-machine execution halts and has charge at most $32\mathcal{B}(p, q, c + n, l + n, s)$.*

For the historical explicit-nonce repaired-submission source, initialized with empty cache and log, the previously proved attacker hash budget n yields the bound $\mathcal{B}(p, q, n + 15, n + 15, s)$ at every captured request state. Polynomial bounds on public modulus widths, the count budgets and slack give an explicit polynomial bound on $\mathcal{B}$; codec/header length bounds are derived rather than assumed.

Lean: `CacheRequestPrefixes.encoded_prestates_bound.`

Lean: `CacheRequestPrefixes.request_clock_bound.`

Lean: `CacheRequestPrefixes.hash_request_execution_bound.`

Lean: `CacheRequestPrefixes.repaired_source_request_clock_bound.`

Lean: `CacheRequestPrefixes.requestBound_polynomial.`

The clock/charge assertions use the actual request program and its established run theorem. They are not cost certificates for a missing caller correspondence. Arbitrary uniform request ranges are still outside this hash-width bound.

Independent controls and remaining connection. The arithmetic gate evaluates the actual Lean clock on 788 fixtures; Python independently computes the proposed cap. Seeds 606, 20260914 and 118 contribute 200 cases each, alongside directed boundary/unbalanced cases. Omitting either loaded key length or public group width is refuted, with kernel witnesses. The public-width mutant initially survived smaller inputs; increasing unbalanced widths exposed it. This was insufficient test strength, not evidence of validity.

Lean: `CacheRequestMachine.cubic_zero_control`.

Lean: `CacheRequestMachine.cubic_loaded_control`.

Lean: `CacheRequestMachine.key_length_omission_control`.

Lean: `CacheRequestMachine.public_width_omission_control`.

The observer gate checks 1,278 typed executions against independently computed answers, cache order, log, prestates and unused randomness: all operation words of length at most three across three caches and two logs, plus 256 cases per seed 118, 198 and 20260914. Mutants omitting hits, omitting uniforms or recording poststates are all detected. Neither campaign discards a case or gives up. Kernel controls retain a five-request trace with answers [3, 2, 3, 6, 6], initial cache/log counts (1, 2), final counts (2, 3) and exactly two consumed answers.

Lean: `CacheRequestPrefixControls.complete_trace_control`.

Lean: `CacheRequestPrefixControls.offset_control`.

Lean: `CacheRequestPrefixControls.hits_not_omitted`.

Lean: `CacheRequestPrefixControls.uniform_not_omitted`.

Lean: `CacheRequestPrefixControls.miss_not_poststate`.

The observer reuses the finite logged handler, state projection and source query-bound induction. The width adapter depends on the original Helios codecs and request program; its potential callers are the actual source/replay paths with their own proved budgets. No general compiler or new dependency is added. The semantic caller's arbitrary Lean continuations have no assumed compiler. No all-PPT secrecy or reusable-framework claim follows from this increment.

1.108 Historical component: Resident caller entry and guarded return

The next caller region retains cache and log on the dispatcher's existing ports. It executes the transfers needed for another request, including clearing an old answer, and preserves a private saved-data word. This avoids serializing and immediately reparsing the same resident state.

Entry interface. Let K, C, L, R, V, S be key, cache, chronological log, sampler record, previous answer and saved data. The initial local stacks are

$$[\epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, V, K, C, L, R],$$

with one extra port holding S . The fixed code clears port 7, executes the original cache copy from 9 to 0 via scratch 2, clears 9, and enters the original dispatcher. Its request return goes through a guard: memory 2 succeeds; memory 0 or 1 rejects. The extra saved port is framed across both deterministic operations and oracle responses.

Put

$$\begin{aligned} U &= (|V| + 1) + (2|C| + 2) + (|C| + 1), \\ D &= 4(|V| + 1) + 5(2|C| + 2) + 4(|C| + 1). \end{aligned}$$

Theorem 1.115 (Executed resident entry). *For arbitrary resident words, actual execution for exactly U ticks reaches the live dispatcher entry with its required stacks $[C, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, \epsilon, K, \epsilon, L, R]$*

and unchanged saved word S . Its charge is at most D . No intermediate scratch, transfer or first-return certificate is assumed.

Lean: `CacheCallerMachine.load_request`.

The live boundary uses an exact recurrence for the existing copy instructions. The previously retained counterexample to padding a live return still applies; no such padding is used here.

Theorem 1.116 (Resident request and return). *Take positive p, q , typed canonical key/cache/log and $R = I(s, q)$. Let F, B, Q be the original dispatcher's request clock, charge bound and full oracle tree, and let ι_S rename its result labels and add saved port S . The actual resident caller execution for $U + F + 1$ ticks has projected tree $\iota_{S*}Q$. Every supported leaf halts with charge at most $D + B + 3$. The complete result and saved data are retained, and the return guard is executed.*

Lean: `CacheCallerMachine.request_run`.

Lean: `CacheCallerMachine.execution_source`.

The observation theorem matches the existing host caller's complete cache request interface. It does not compile that caller's arbitrary Lean continuation.

Cost and physical execution. The combined charge is at most $32(U + F + 1)$. At a captured original-source state with initial cache/log counts (c, l) and hash budget n , the resident clock is bounded by

$$|V| + 3C_{p,q}(c + n) + 5 + \mathcal{B}(p, q, c + n, l + n, s),$$

using the source and codec bounds from the preceding section. Old-answer length remains explicit until actual source decoding/production is connected. Let H be the full resident stack height, including S , let $B_r = D + B + 3$, and let G_r be the fixed primitive compiler factor. The existing compiler executes the resident request at time

$$\text{unitCost}(H + B_r)B_rG_r$$

from its lowered resident configuration, with the same bounded-oracle policy on both sides and observed tree some $_{*} \iota_{S*}Q$. Its required termination/charge contract is derived from the actual run.

Lean: `CacheCallerMachine.cost_le_clock`.

Lean: `CacheCallerMachine.source_clock_bound`.

Lean: `CacheCallerMachine.within`.

Lean: `CacheCallerMachine.physical_run`.

This is physical execution from resident storage. It does not assume polynomial saved-context width or execution of an enclosing source initialization.

Independent controls and boundary. The native campaign checks all thirteen ports, return flags, halting and exact coin consumption on 525 cases: 324 directed, 192 seeded (64 each for 118, 198, 20260914) and nine return-gate cases. Its 200 hits and 316 misses include nonempty cache/log, old answers and saved data. All four actual-code mutations are identified: skipped answer cleanup, skipped cache cleanup, corrupted saved port and forced-success return. Single-bit deletion shrinks old/saved words while holding typed request/cache/log fixed; no global minimality is claimed. There are no discarded cases or gaveUp results. The observed maximum of 17,471 ticks under the 100,000 fixture cap is not the general time theorem. Nine kernel controls retain stored three, sampled six, the complete layouts, unused randomness, nonempty saved data and the return policy. A second invocation reuses the actual returned configuration with only control/memory

reset; it hits the inserted key without another coin or log append. This is an entry control, not a compiled general continuation.

Lean: `CacheCallerMachineControls.hit_control`.

Lean: `CacheCallerMachineControls.miss_control`.

Lean: `CacheCallerMachineControls.old_answer_not_appended`.

Lean: `CacheCallerMachineControls.saved_frame_control`.

Lean: `CacheCallerMachineControls.repeated_entry_control`.

Lean: `CacheCallerMachineControls.failed_return_control`.

Lean: `CacheCallerMachineControls.nonzero_failure_control`.

Lean: `CacheCallerMachineControls.successful_return_control`.

Lean: `CacheCallerMachineControls.failed_return_not_success`.

The implementation reuses the original dispatcher/copy, stack frame, return linker and primitive compiler. Its specific ports and codecs are Helios interfaces; potential callers are concrete source/replay call sites. Saved data is not a new public protocol observation. Actual operand production, uniform range policy, saved-context bounds, decoding/continuations and the combined adaptive cost are outside this component's guarantee.

1.109 Historical component: Executed nonzero-nonce response

The historical nonce sampler draws $i \in \text{Fin}(q-1)$ and returns $i+1$ in $\mathbb{Z}_q$. Its response continuation is now an actual finite-control program. It takes the canonical natural prefix $E(i)$ followed by arbitrary suffix T , with three opaque saved words S_0, S_1, S_2 :

$$[\epsilon, \epsilon, \epsilon, \epsilon, E(i)T, S_0, S_1, S_2].$$

The program pushes the zero-slack delimiter, reuses the existing uniform operand preparation (which parses and increments), clears its width markers, and runs the existing scalar writer under port permutation $[1, 5, 2, 3, 0, 4, 6, 7]$. Only fixed control and coordinates are added.

Theorem 1.117 (Executed historical nonce successor). *For every natural n , suffix T and saved frame S , actual execution at clock*

$$B(n) = 1 + (5 \text{ size}(n) + 2 \text{ size}(n+1) + 10) + (\text{size}(n+1) + 1) + (3 \text{ size}(n+1) + 3) + 1$$

halts with success memory 2 and complete stack vector

$$[\epsilon, \epsilon, \epsilon, \epsilon, E(n+1)T, S_0, S_1, S_2].$$

There exists its actual charged execution value $c \leq 32B(n)$, with no coin or hash requests. For prime q and $i \in \text{Fin}(q-1)$, the output prefix is precisely $\text{scalarEncode}(\text{primeNonceValue}(i))$.

Lean: `PrimeNonceMachine.run_nat`.

Lean: `PrimeNonceMachine.successor_run`.

Lean: `PrimeNonceMachine.local_cost`.

Lean: `PrimeNonceMachine.charged`.

The return linker derives actual subroutine durations within their bounds; padding occurs only after the complete successor halts. Canonical input is explicit: this is not a malformed-input rejection theorem. The following section derives that input from the actual sampler.

Controls and remaining connection. Six kernel controls retain $0 \mapsto 1$ (the $q = 2$ boundary), $3 \mapsto 4$ with a suffix and distinct saved words, nonidentity, and counterexamples to bypassed increment, skipped width cleanup and corrupted saved storage.

Lean: `PrimeNonceMachineControls.zero_to_one.`

Lean: `PrimeNonceMachineControls.carry_three_to_four.`

Lean: `PrimeNonceMachineControls.zero_not_identity.`

Lean: `PrimeNonceMachineControls.bypass_increment_counterexample.`

Lean: `PrimeNonceMachineControls.skip_clear_counterexample.`

Lean: `PrimeNonceMachineControls.corrupt_frame_counterexample.`

The independent Python integer/prefix oracle checks all eight ports, success, halt and charge on 1,024 native cases: indices 0–255, 384 power-of-two boundaries through width 128, and 384 seeded cases (118, 606, 20260914) up to width 256 with arbitrary suffix/frame words. No cases are discarded or gaveUp. All three injected defects are detected and shrink to index zero with empty suffix/frame. These finite tests supplement the general execution theorem.

The interface requires only a canonical index and explicit saved storage. The successor routine itself is independent of groups and cryptographic assumptions; its specialization uses prime q and the historical nonce map. Immediate potential callers are the two nonce draws in honest ballot construction. The following section closes public $q \mapsto q - 1$ preparation and sampling composition. Subsequent group/scalar operations and the complete adaptive caller cost are outside this component's guarantee.

1.110 Historical component: Complete historical nonzero-nonce sampling

The original sampler chooses uniformly in $\text{Fin}(q - 1)$ and maps its index to the nonzero residue $i + 1$. The actual implementation now starts with the public record $I(s, q) = 1^s 0 E(q)$ and three saved words. It parses q , executes binary predecessor, collects width from $q - 1$, samples and reduces coins, then executes the preceding response continuation.

Preparation and saved storage. For $q \geq 2$, `NonceOperands` returns modulus $(q - 1).bits$ and $(\text{size}(q - 1) + s)$ width markers, with clock

$$P(s, q) = s + 5 \text{size}(q) + 2 \text{size}(q - 1) + 9$$

and charge at most $32P(s, q)$. Its general canonical-input theorem retains suffix and saved words. The original parser and predecessor supply their execution proofs; a closed-region invariant transfers the unchanged width code. Only the shared layout and existing width theorem are exposed.

Lean: `NonceOperands.run.`

Lean: `NonceOperands.local_cost.`

Lean: `NonceOperands.charged.`

Lean: `SamplerOperands.ready_run.`

The sampler uses all eight local ports, so its three saved words occupy external frame ports 8, 9, 10. The successor consumes the returned index on port 5 directly, retaining modulus port 1 and the external saved words. Input must be exhausted before sampling; both inter-region guards execute. Saved words describe machine state, not additional public protocol observations.

Put $m = q - 1$, $w = \text{size}(m) + s$. Let $U(w, m)$ and $D(w, m)$ be the existing scalar sampler's clock and charge, and $N = B(m - 1)$ the preceding successor clock. Define

$$T = P(s, q) + 1 + U(w, m) + 1 + N, \quad C = 32P(s, q) + 5 + D(w, m) + 3 + 32N.$$

For saved frame S , write $R(a)$ for the halted success configuration with stack vector

$$[\epsilon, m.\text{bits}, \epsilon, \epsilon, \epsilon, \text{scalarEncode}(a), \epsilon, \epsilon, S_0, S_1, S_2].$$

Theorem 1.118 (Complete executed nonce sampler). *For $q \geq 2$, from $I(s, q)$ on input port 4, blank work and saved frame S , actual execution at clock T has exactly the chronological w -coin tree. For each coin word b , the output is the complete success configuration with nonce prefix $E(\text{bitsValue}(b) \bmod m + 1)$, retained modulus and saved frame. Its actual charge is at most C . All subroutine halt, canonical index and scratch/frame premises are derived from the executed routines.*

Lean: [PrimeNonceMachine.tail_source](#).

Lean: [PrimeNonceMachine.sample_source](#).

Theorem 1.119 (Historical source and sampling loss). *For prime q , projecting the complete result of actual execution gives exactly*

$$\text{liftCoins}(R_* \text{runFairBitUniform}(s, \text{samplePrimeNonzero}))$$

as a raw oracle query tree. Interpreting its coins as fair yields an actual machine experiment whose complete result law has total variation distance at most 2^{-s} from $R_ \text{samplePrimeNonzero}$. The reference on the right uses the original exact-uniform nonzero sampler.*

Lean: [PrimeNonceMachine.sample_execution_source](#).

Lean: [PrimeNonceMachine.sample_experiment_source](#).

Lean: [PrimeNonceMachine.sample_loss](#).

This preserves the selected sampling loss; it does not assert exact uniformity of finite modulo sampling. The original Fin sampler's syntax is unfolded explicitly, so no distribution-only correspondence substitutes for query-tree agreement.

Physical execution. The complete run derives the existing primitive compiler contract. From the resident canonical input/frame, let H be actual initial stack height and G the fixed compiler factor for this program. Physical execution at

$$\text{unitCost}(H + C) C G$$

has the same complete source observation under the same bounded-handler policy on both sides. Saved-context width remains explicit; no polynomial width or missing runtime certificate is assumed. This component does not initialize an arbitrary enclosing caller.

Lean: [PrimeNonceMachine.sample_within](#).

Lean: [PrimeNonceMachine.sample_physical_run](#).

Independent controls and remaining caller. The preparation gate checks 282 cases, including $q = 0, 1$ rejection, borrow and width boundaries, suffix/frame and 192 seeded cases (118, 198, 20260914). Eight kernel controls include $q = 2$, $q = 8$, retained data, wrong width, and replacing predecessor by increment: $q = 2$ then produces modulus three.

Lean: [NonceOperandsControls.two_control](#).

Lean: [NonceOperandsControls.borrow_control](#).

Lean: [NonceOperandsControls.suffix_frame_control](#).

Lean: [NonceOperandsControls.old_width_not_retained](#).

Lean: [NonceOperandsControls.zero_rejected](#).

Lean: `NonceOperandsControls.one_rejected`.

Lean: `NonceOperandsControls.increment_mutant_control`.

Lean: `NonceOperandsControls.increment_mutant_not_predecessor`.

The full sampler gate checks 128 cases: 32 directed and 96 seeded (118,606,20260914), with independent integer/prefix expectations, complete states, charge, coin consumption and remaining tapes. Its four actual-code defects are detected at $q = 2$: omitted decrement, omitted successor, corrupted saved data and stale width. Stale width can return the correct nonce while consuming an extra coin. Neither campaign discards cases or gives up.

Lean: `PrimeNonceSamplingControls.two_control`.

Lean: `PrimeNonceSamplingControls.upper_control`.

Lean: `PrimeNonceSamplingControls.frame_control`.

Lean: `PrimeNonceSamplingControls.zero_not_returned`.

Lean: `PrimeNonceSamplingControls.omitted_decrement_counterexample`.

Lean: `PrimeNonceSamplingControls.omitted_successor_counterexample`.

Lean: `PrimeNonceSamplingControls.corrupted_frame_counterexample`.

Lean: `PrimeNonceSamplingControls.stale_width_counterexample`.

The following section executes `drawPrimeNoncePair`, retaining the first response and consuming successive independent coin words. The subsequent honest constructor needs executed modular exponentiation, multiplication and scalar addition. Existing coordinate equations specify those operations but do not execute them.

1.111 Historical component: Executing the historical nonce-pair caller

Interface and source. The actual `drawPrimeNoncePair` source binds two nonzero nonce samples in order. The finite controller now invokes the preceding sampler twice, saving the first answer before reentry. Its resident input has empty work ports, a canonical public sampler record on port 8, an empty first-answer slot 9 and arbitrary private context on port 10. These saved words are machine state, not additional public protocol observations.

The scoped transport copies record 8 to input 4. Between calls it copies the actual answer from 5 to 9, clears answer 5 and modulus 1, and reloads input 4 from the retained record. It executes the existing copy algorithm under fixed coordinate maps; no new arithmetic algorithm is used. Both transport entries preserve complete configurations and derive their own clocks and charges. Their interface accepts arbitrary resident words; the pair theorem below supplies canonical sampler inputs and derives the nonce-width bound.

Lean: `PrimeNoncePairTransport.entry_run`.

Lean: `PrimeNoncePairTransport.between_run`.

Combined execution. Write s for slack, $q \geq 2$, $m = q - 1$, $w = \text{size}(m) + s$, $N = 2 \text{size}(m) + 1$, and r for the canonical sampler record. Let $R = |r|$, $M = \text{size}(m)$, $E = 2R + 3$ and $F = 9(R + 1) + 2$. Let S and B be the preceding single-nonce clock and charge bound. Transcribing the controller definitions and collecting terms gives

$$\begin{aligned} T &= 2E + 2S + 3N + M + 5, \\ C &= 2F + 2B + 13(N + 1) + 4(M + 1) + 3. \end{aligned}$$

These are bounds derived from the executed instructions, including the success guard after the first sampler. The initial context contributes to physical storage cost below. For words a, b and context

h , the result is

$$\text{PairOut}(a, b, h) = (\text{halt}, 2, [\], \text{bits}(m), [\], [\], [\], b, [\], [\], r, a, h).$$

Theorem 1.120 (Complete pair execution, machine-checked). *For every s, q with $q \geq 2$ and every context h , there is a charge function $c(u, v)$ such that $|u| = |v| = w$ implies $c(u, v) \leq C$, and the actual run from the resident input at clock T is the following complete raw query tree:*

$$\begin{aligned} u &\leftarrow \text{word}(w); & v &\leftarrow \text{word}(w); \\ \text{return} &(\text{PairOut}(E_n(\text{value}(u) \bmod m + 1), E_n(\text{value}(v) \bmod m + 1), h), c(u, v)). \end{aligned}$$

Here E_n is the existing canonical natural-prefix encoding. The two word queries are sequential binds. Full state preservation, termination and charge are conclusions; no intermediate frame or cost certificate is a premise.

Lean: [PrimeNoncePairMachine.pair_run](#).

Only the first encoded nonce's length varies between calls; it is at most N . The proof composes its actual transport and second sampler before applying clock padding. Thus padding occurs at the final halt, never at a live return.

Theorem 1.121 (Joint historical source and physical execution, machine-checked). *If q is prime, discarding the private charge from the complete run gives exactly the lifted query tree of*

$$(a, b) \leftarrow \text{runFairBitUniform}_s(\text{drawPrimeNoncePair}); \quad \text{return PairOut}(E_q(a), E_q(b), h),$$

where E_q is the canonical scalar encoding. Under fair coins, the complete result distribution has total-variation distance at most $2 \cdot 2^{-s}$ from the encoded original exact-uniform pair. For actual resident height H and fixed primitive compiler factor G , physical execution at $\text{unitCost}(H + C)CG$ has this same source observation under the same bounded-handler policy on both sides.

Lean: [PrimeNoncePairMachine.pair_execution_source](#).

Lean: [PrimeNoncePairMachine.pair_experiment_source](#).

Lean: [PrimeNoncePairMachine.pair_loss](#).

Lean: [PrimeNoncePairMachine.pair_within](#).

Lean: [PrimeNoncePairMachine.pair_physical_run](#).

The original source's two-query bound is derived from its two binds. The physical theorem derives the compiler contract from actual branchwise halt and charge. It keeps context height explicit and does not initialize an arbitrary enclosing caller or assume that its context is polynomially bounded.

Independent controls and remaining dependency. At $q = 5, s = 0$, words 000 then 111 produce nonces one then four; reversing the words reverses the two saved nonces. Both traces preserve all eleven ports and trailing coins 101. At $q = 2$, both outputs are one but two coins are still consumed. Kernel controls exclude copied first answers and incomplete coin consumption. Four actual-code counterexamples retain omitted second invocation, corrupted context, fixed second coins and omitted reentry.

Lean: [PrimeNoncePairControls.low_high_control](#).

Lean: [PrimeNoncePairControls.high_low_control](#).

Lean: [PrimeNoncePairControls.singleton_control](#).

Lean: [PrimeNoncePairControls.second_not_first](#).

Lean: `PrimeNoncePairControls.both_words_consumed`.

Lean: `PrimeNoncePairControls.omitted_second_counterexample`.

Lean: `PrimeNoncePairControls.corrupted_context_counterexample`.

Lean: `PrimeNoncePairControls.fixed_second_coins_counterexample`.

Lean: `PrimeNoncePairControls.omitted_reentry_counterexample`.

The complete native gate checks 160 cases: 64 directed and 96 seeded (118,606,20260914), with independent integer/prefix expectations, both nonce outputs, all saved data, coin consumption and combined bounds. All four mutations are detected before candidate testing; no cases are discarded. Transport has separate full-state controls and a 138-case native gate.

Lean: `PrimeNoncePairTransport.entry_control`.

Lean: `PrimeNoncePairTransport.between_control`.

Lean: `PrimeNoncePairTransport.omission_control`.

Lean: `PrimeNoncePairTransport.omission_not_correct`.

This closes the actual pair invocation and storage obligation. These components provide scoped interfaces for the concrete caller; framework reuse remains unclaimed pending a second application.

1.112 Historical component: Loaded public-coordinate multiplication

Required source operation. The next honest proof request computes `honestProofStatement`, which uses `encryptWith`. Its public coordinates require powers and multiplication modulo p ; q is the scalar order. The checked coordinate equations specify these operations but supply no execution cost. The loaded multiplier below is the arithmetic prerequisite for square-and-multiply. It reuses the existing guarded subtraction, remainder update, copying and frame/return interfaces. No full-product or separate carry-adder library is introduced.

Reduced addition and its preparation. For $x, r < p$, write $c = p - x$. The inner controller computes

$$\text{add}_p(x, r) = \begin{cases} r - c, & c \leq r, \\ p - (c - r), & r < c. \end{cases} \quad \text{add}_p(x, r) = (r + x) \bmod p < p.$$

It first executes guarded $r - c$. Underflow preserves r and selects two further actual subtractions, $c - r$ and $p - (c - r)$, with copies that preserve c and p . Every subtraction intermediate fits the modulus width. The inner interface takes canonical r, c, p digits; it returns the canonical sum on port 0, retains c on 1 and p on 6, and clears every other local port. Its executed clock is bounded by $26 \text{size}(p) + 24$; its charge is bounded by 32 times that clock. Only the final halt is padded.

Lean: `BinaryModMultiplyArithmetic.reducedAdd_eq_mod`.

Lean: `BinaryModMultiplyArithmetic.reducedAdd_lt`.

Lean: `BinaryModMultiplyArithmetic.operand_bounds`.

Lean: `BinaryModMultiplyArithmetic.underflow_bounds`.

Lean: `BinaryModAddMachine.used_run`.

Lean: `BinaryModAddMachine.run`.

Lean: `BinaryModAddMachine.local_cost`.

Lean: `BinaryModAddMachine.charged`.

The outer multiplier does not receive a prepared complement. It copies its original modulus and executes $p - x$, reaching the complete loop-entry layout within $8 \text{size}(p) + 8$ ticks. This discharges the inner interface's preparation premise using actual instructions.

Lean: `BinaryModMultiply.prepare`.

Multiplier interface and invariant. The input layout has modulus digits on 6, little-endian multiplier word w on 8, multiplicand digits on 10 and all other ports empty. For each multiplier bit, read from most significant to least after an executed reversal, the reduced accumulator is doubled modulo p and conditionally increased by x using the inner controller. The arithmetic step

$$a \mapsto ((2a \bmod p) + (\text{if } b \text{ then } x \text{ else } 0)) \bmod p$$

preserves reduction. Starting from zero, folding over $\text{reverse}(w)$ gives $x \text{ value}(w) \bmod p$. This includes empty words and high zero padding. These arithmetic identities are separate from the execution proof.

Lean: [BinaryModMultiplyArithmetic.step_lt](#).

Lean: [BinaryModMultiplyArithmetic.step_reducedAdd](#).

Lean: [BinaryModMultiplyArithmetic.step_mul_bit](#).

Lean: [BinaryModMultiplyArithmetic.fold_reverse_zero](#).

Theorem 1.122 (Executed loaded multiplication, machine-checked). *For every p, x with $x < p$ and every loaded word w , the actual multiplier halts within*

$$T = 9 \text{size}(p) + 11 + |w|(34 \text{size}(p) + 38) \leq 38(|w| + 1)(\text{size}(p) + 1)$$

transitions. Its full result has success memory 2 and stacks

$$[\text{bits}(x \text{ value}(w) \bmod p), [], [], [], [], [], \text{bits}(p), [], [], [], \text{bits}(x)].$$

The code executes complement preparation, reversal, both conditional loop routes and final complement cleanup. It derives a total charge bound $B = 32T$. No intermediate frame, width invariant or cost certificate is a caller premise.

Lean: [BinaryModMultiply.run](#).

Lean: [BinaryModMultiply.padded_run](#).

Lean: [BinaryModMultiply.local_cost](#).

Lean: [BinaryModMultiply.charged](#).

Lean: [BinaryModMultiply.clock_le_product](#).

Theorem 1.123 (Loaded group coordinates and physical execution, machine-checked). *For $p \neq 0$ and $a, b \in \text{PrimeGroup}(p, q)$, the same actual multiplier, loaded with their public coordinate digits, returns the coordinate digits of $a+b$ in the historical additive group notation. Its underlying numeric operation is*

$$(\text{coord}(a + b)).\text{val} = ((\text{coord}(a)).\text{val}(\text{coord}(b)).\text{val}) \bmod p.$$

For any loaded input covered above, let H be its actual resident height and G the fixed primitive compiler factor. Physical execution from the lowered ready state with empty auxiliary oracle tapes, at $\text{unitCost}(H + B)BG$ returns the same complete result under the bounded-handler policy. Its halt/charge contract is derived from execution.

Lean: [BinaryModMultiply.group_coordinates](#).

Lean: [BinaryModMultiply.within](#).

Lean: [BinaryModMultiply.physical_run](#).

These statements start at resident digits. Parsing public records, routing actual constructor operands, exponentiation and final proof-request encoding remain separate caller obligations. They neither expose discrete logarithms nor establish the complete constructor's runtime.

Independent controls. The arithmetic gate checks 11,952 additions and 9,216 folds, with minimized strict-threshold and reversed-subtraction counterexamples. The actual inner controller gate checks 1,808 full-state cases and detects four code mutations. The outer gate checks 792 loaded products, including high zero padding and borrow/width boundaries, and detects five code mutations. All campaigns have zero discards or gave-up outcomes; none substitutes for the run theorem.

Lean: `BinaryModMultiplyArithmetic.wrap_control`.

Lean: `BinaryModMultiplyArithmetic.no_wrap_control`.

Lean: `BinaryModMultiplyArithmetic.product_control`.

Lean: `BinaryModMultiplyArithmetic.strict_threshold_counterexample`.

Lean: `BinaryModMultiplyArithmetic.reversed_operands_counterexample`.

Lean: `BinaryModAddMachine.wrap_control`.

Lean: `BinaryModAddMachine.underflow_control`.

Lean: `BinaryModAddMachine.skipped_underflow_counterexample`.

For the actual outer controller, $p = 23, x = 4, w = 1011$ gives six and clears every work port. Reading the same bits in the wrong order gives 21; omitting doubling gives 12; retaining the complement leaves a correct product with incorrect workspace. Separate controls cover zero, empty words, high zero padding and using the wrong scalar modulus.

Lean: `BinaryModMultiplyControls.product_control`.

Lean: `BinaryModMultiplyControls.padded_control`.

Lean: `BinaryModMultiplyControls.empty_control`.

Lean: `BinaryModMultiplyControls.modulus_one_control`.

Lean: `BinaryModMultiplyControls.not_scalar_modulus`.

Lean: `BinaryModMultiplyControls.omitted_preparation_counterexample`.

Lean: `BinaryModMultiplyControls.omitted_addition_counterexample`.

Lean: `BinaryModMultiplyControls.omitted_doubling_counterexample`.

Lean: `BinaryModMultiplyControls.retained_complement_counterexample`.

Lean: `BinaryModMultiplyControls.bit_order_counterexample`.

1.113 Historical component: Loaded exponentiation for encryption

Required source operation. The honest ciphertext coordinates require g^r and pk^r modulo p . The exponent uses the scalar's canonical value modulo q . The loaded controller calls the preceding multiplier; it scans exponent bits rather than iterating once per exponent value. Its original base, exponent and opaque caller data are retained so the encryption continuation can compute both coordinates.

Interface and invariant. Assume $2 \leq p, x < p$, an arbitrary little-endian exponent word w , and saved word s . The initial layout has canonical p on 6, canonical x on 11, w on 13 and s on 14; every other port is empty. The controller copies and reverses the exponent into work port 12 and initializes the accumulator on 0 to one. Each iteration first checks for a pending bit, executes a square using the checked multiplier, then pops that bit and conditionally multiplies by the retained base. This ordering keeps the bit in the saved frame while the multiplier uses its local memory. The arithmetic invariant follows the exponent prefix n :

$$a = x^n \bmod p, \quad a \mapsto \begin{cases} (a^2 \bmod p)x \bmod p, & b = 1, \\ a^2 \bmod p, & b = 0 \end{cases} = x^{2n+b} \bmod p.$$

Both branches remain reduced, so every multiplication operand has at most $\text{size}(p)$ digits. Folding over the reversed word from one gives $x^{\text{value}(w)} \bmod p$, including empty and padded exponents. The modulus premise justifies literal initialization to one; historical prime p implies it.

Lean: [BinaryModPowerArithmetic.step_lt](#).

Lean: [BinaryModPowerArithmetic.step_pow](#).

Lean: [BinaryModPowerArithmetic.fold_reverse_one](#).

Theorem 1.124 (Executed loaded power, machine-checked). *Under the preceding input conditions, the actual controller halts with success memory 2 and complete stacks*

$$[\text{bits}(x^{\text{value}(w)} \bmod p), [], [], [], [], [], \text{bits}(p), [], [], [], \text{bits}(x), [], w, s].$$

Writing $W = \text{size}(p)$, $L = |w|$ and $M = 9W + 11 + W(34W + 38)$ for the existing multiplier bound, its executed clock is bounded by

$$T = 3L + 5 + L(2M + 12W + 16) \leq 80(L + 1)(W + 1)^2.$$

Its total charge is at most $32T$. Original operand retention, clean call/return states and the bounds of all copied intermediate words are derived from the actual instructions. Only the final halted result is padded.

Lean: [BinaryModPower.run](#).

Lean: [BinaryModPower.padded_run](#).

Lean: [BinaryModPower.local_cost](#).

Lean: [BinaryModPower.charged](#).

Lean: [BinaryModPower.clock_le_product](#).

Theorem 1.125 (Historical scalar action and physical execution, machine-checked). *Let p be prime, $q \neq 0$, $r \in \mathbb{Z}_q$ and $a \in \text{PrimeGroup}(p, q)$. Loading the public coordinate digits of a and canonical digits of $r.\text{val}$ into the same controller returns*

$$\text{bits}((\text{coord}(r \bullet a)).\text{val}) = \text{bits}((\text{coord}(a)).\text{val}^{r.\text{val}} \bmod p),$$

with original operands, exponent and saved data retained as above. For any loaded input covered by the run theorem, put $B = 32T$ and let H be its actual resident height, including the saved data. From the lowered ready state with empty auxiliary oracle tapes, the primitive execution at $\text{unitCost}(H + B)BG$, with fixed compiler factor G , has the same complete result under the bounded-handler policy. The required halt/charge contract is derived from the executed power program.

Lean: [BinaryModPower.group_coordinates](#).

Lean: [BinaryModPower.within](#).

Lean: [BinaryModPower.physical_run](#).

The physical observation is a proof-side readout. The next section executes first-coordinate storage and the second power call. The initialized controller in Section 1.115 closes typed input parsing and first-nonce routing; proof-request construction is outside this component's guarantee.

Independent controls. The arithmetic gate checks 35,437 cases and detects wrong order, omitted squaring and omitted multiplication. The actual controller gate checks 451 complete states and cost bounds against independent integer modular powers; six code mutations are detected before the candidate campaign. Both gates have zero discards or gave-up outcomes. These bounded experiments accompany the execution theorem, rather than establish it.

Lean: `BinaryModPowerArithmetic.empty_control`.
 Lean: `BinaryModPowerArithmetic.power_control`.
 Lean: `BinaryModPowerArithmetic.padded_control`.
 Lean: `BinaryModPowerArithmetic.not_scalar_modulus`.
 Lean: `BinaryModPowerArithmetic.wrong_order_counterexample`.
 Lean: `BinaryModPowerArithmetic.omitted_square_counterexample`.
 Lean: `BinaryModPowerArithmetic.omitted_multiply_counterexample`.
 Lean: `BinaryModPowerControls.power_control`.
 Lean: `BinaryModPowerControls.empty_control`.
 Lean: `BinaryModPowerControls.zero_initialization_counterexample`.
 Lean: `BinaryModPowerControls.omitted_square_counterexample`.
 Lean: `BinaryModPowerControls.omitted_multiply_counterexample`.
 Lean: `BinaryModPowerControls.corrupted_exponent_counterexample`.
 Lean: `BinaryModPowerControls.wrong_order_counterexample`.
 Lean: `BinaryModPowerControls.corrupted_context_counterexample`.
 Lean: `BinaryModPowerControls.generator_full_control`.
 Lean: `BinaryModPowerControls.key_full_control`.
 Lean: `BinaryModPowerControls.padded_full_control`.

1.114 Historical component: Executing both ciphertext coordinates

Actual source and resident interface. The next honest proof request uses the existing `encryptWith` construction. The concrete controller invokes the preceding power routine twice, saves the first coordinate, and conditionally multiplies the second by the generator for a true vote. For natural coordinates $g, k < p$, $p \geq 2$, little-endian nonce word w , vote v and saved context s , the initial layout has canonical digits of p, k, g on ports 6,15,16, respectively, with nonce 13, context 14 and the one-bit vote 18; all other ports are empty. The mathematical outputs are

$$\alpha = g^{\text{value}(w)} \bmod p, \quad \beta = ((k^{\text{value}(w)} \bmod p)(\text{if } v \text{ then } g \text{ else } 1)) \bmod p.$$

This is a loaded input interface. The actual parser and routing from the prior nonce sampler are separate caller operations.

Lean: `PrimeEncryptMachine.program`.

Theorem 1.126 (Executed loaded ciphertext, machine-checked). *Under the preceding conditions, the actual controller returns α on port 17 and β on 0, both in canonical digits. It retains every original input port and clears work ports 1–5 and 7–12. Its full final configuration has halt label and success memory 2. Put $W = \text{size}(p)$, $L = |w|$, let P be the preceding power bound and $M = 9W + 11 + W(34W + 38)$. The actual execution has clock at most*

$$E = 2P + M + 15W + 20 \leq 220(L + 1)(W + 1)^2$$

and charge at most $32E$. Both power inputs, storage of the first result, public-key reload, vote branch, multiplier input, success guards and all cleanup execute in the same fixed controller. Their frames, reduced operands and width bounds are derived; no intermediate state or cost certificate is a caller premise. Only the final halted state is padded.

Lean: `PrimeEncryptMachine.run`.

Lean: `PrimeEncryptMachine.padded_run`.

Lean: `PrimeEncryptMachine.local_cost`.

Lean: `PrimeEncryptMachine.charged`.

Lean: `PrimeEncryptMachine.clock_le_product`.

Theorem 1.127 (Historical encryption and physical execution, machine-checked). *Let p, q be prime, $g, pk \in \text{PrimeGroup}(p, q)$, $r \in \mathbb{Z}_q$ and v be a vote. Loading their public coordinates and the canonical digits of $r.val$ into the same controller returns exactly both public coordinates of*

$$\text{encryptWith}(g, pk, r, \text{voteScalar}(v)),$$

with the complete retained-input and cleaned-work configuration above. For any loaded input satisfying the execution theorem, write $B = 32E$ and let H be the actual resident height, including the saved context and nonce. From the lowered ready state with empty auxiliary oracle tapes, primitive execution at $\text{unitCost}(H + B)BG$, with fixed compiler factor G , has the same complete result under the bounded-handler policy. Its required halt/charge contract is derived from this controller's run.

Lean: `PrimeEncryptMachine.source`.

Lean: `PrimeEncryptMachine.within`.

Lean: `PrimeEncryptMachine.physical_run`.

The physical observation is a proof-side readout. The next subsection closes source parsing and transfer from the actual nonce-pair layout. Proof construction and complete adaptive caller cost are outside this component's guarantee. This loaded encryption theorem does not itself establish ballot secrecy or full honest-ballot construction.

Independent controls. At $p = 23, g = 2, pk = 4, r = 3$, the expected ciphertexts are (8, 18) for false and (8, 13) for true. The 210-case native campaign checks both coordinates, all nineteen ports and candidate cost bounds against independent integer modular arithmetic. Seven actual-code defects are detected before testing the correct candidate: either missing power, using the generator in place of the key, omitting the vote product, and corrupting the first coordinate, retained nonce or saved context. Zero cases are discarded or give up. Separate kernel controls retain both complete positive states and all seven complete mutant states.

Lean: `PrimeEncryptMachineControls.true_vote_control`.

Lean: `PrimeEncryptMachineControls.false_vote_control`.

Lean: `PrimeEncryptMachineControls.omitted_first_power_counterexample`.

Lean: `PrimeEncryptMachineControls.omitted_second_power_counterexample`.

Lean: `PrimeEncryptMachineControls.reused_generator_counterexample`.

Lean: `PrimeEncryptMachineControls.omitted_vote_product_counterexample`.

Lean: `PrimeEncryptMachineControls.corrupted_alpha_counterexample`.

Lean: `PrimeEncryptMachineControls.corrupted_nonce_counterexample`.

Lean: `PrimeEncryptMachineControls.corrupted_context_counterexample`.

1.115 Historical component: From raw constructor input to the first ciphertext

Source boundary and input. Fix primes p, q , $g, pk \in \text{PrimeGroup}(p, q)$, sampling slack $s \in \mathbb{N}$, vote v and saved word d . The parameters g, pk, v are direct submission source arguments; s selects the fair-bit implementation and d is an uninterpreted private saved word. Let x be the existing five-field bit encoding of

$$[U(p), \text{ciphertextCodec}(g, pk), \text{samplerInput}(s, q, []), [v], d], \quad N = |x|,$$

where U is the natural-prefix encoding and the ciphertext codec contains the two group-coordinate encodings. The initial controller state has x on port 7 and every other port empty. An executed initializer copies x into private context 14, checks the fixed field arities and singleton vote, extracts canonical p, g, pk digits and the sampler record, and clears its workspace. It derives its complete result and charge at most $32I$, where $I = 14N^2 + 54N + 83$. It does not validate primality or arbitrary group membership: the theorem specializes the actual typed input. Parsing a larger public-key record would not establish key generation.

Lean: `PrimeHonestInputMachine.input`.

Lean: `PrimeHonestInputMachine.run`.

Lean: `PrimeHonestInputMachine.charged`.

Theorem 1.128 (Initialized first-ciphertext execution, machine-checked). *Under the preceding hypotheses, the fixed outer controller executes this initializer, its success/reset guard, fair-bit implementations of both historical nonce draws, parsing of the first retained scalar prefix, and the preceding ciphertext construction. Write J, K for the nonce-pair clock and charge bounds, $R = 7 \text{size}(q-1) + 9$ for routing, and E for the preceding encryption clock at nonce width $\text{size}(q-1)$. Then its source clock and charge bound are*

$$F = I + 1 + J + R + 1 + E, \quad B = 32I + 3 + K + 6R + 3 + 32E.$$

The charged execution is exactly two consecutive coin-word draws of width $W = \text{size}(q-1) + s$, followed by a pure complete result and a derived branch charge at most B . For each word a , the scalar is $\text{value}(a) \bmod (q-1) + 1$; equal draws are permitted. After forgetting the charge, the entire result tree equals the existing fair-bit implementation of `drawPrimeNoncePair`, mapped to the following full halted-success state. For its pair (r_0, r_1) and $(\alpha, \beta) = \text{encryptWith}(g, pk, r_0, \text{voteScalar}(v))$:

Ports	Final contents
0,17	Canonical public coordinates of β, α
6,15,16,18	Original p, pk, g digits and singleton vote
13,14	Canonical digits of $r_0.\text{val}$; original private input x
19,20,21,22	Sampler record; $U(r_0.\text{val})$; $U(r_1.\text{val})$; $(q-1)$ digits
All other ports	Empty

Every intermediate frame, success guard, operand width, halt and charge premise follows from the actual component executions. Padding occurs only after the final halt.

Lean: `PrimeNonceCiphertextMachine.charged_bounded`.

Lean: `PrimeNonceCiphertextMachine.scalar_clock_le`.

Lean: `PrimeNonceCiphertextCaller.caller_run`.

Lean: `PrimeNonceCiphertextCaller.execution_source`.

Lean: `PrimeHonestCiphertextMachine.linked_run`.

Lean: `PrimeHonestCiphertextMachine.charged`.

Lean: `PrimeHonestCiphertextMachine.execution_source`.

Theorem 1.129 (Physical execution from raw input, machine-checked). *For every bounded-oracle adapter limit, the actual physical machine starts with the ordinary raw input tape x and blank workspace. The source configuration reached by loading has stack height N . Let G be the fixed compiler factor and $T = \text{unitCost}(N+B)BG$. There is a derived startup time $u \leq 4N + 8$ such that execution for $u+T$ returns, under the adapter, exactly **some** of the complete source result above, with the same joint coin tree. The required termination/charge contract follows from the charged execution; neither a successful-loader premise nor a runtime certificate is supplied.*

Lean: `PrimeHonestCiphertextMachine.start_source`.

Lean: `PrimeHonestCiphertextMachine.start_height`.

Lean: `PrimeHonestCiphertextMachine.within`.

Lean: `PrimeHonestCiphertextMachine.physical_run`.

The complete configuration is a proof-side observation. Keeping the vote and original input privately does not add a protocol publication.

Independent controls and remaining boundary. The outer gate checks twelve independently encoded full-state cases, both votes and nonce orders, the order-two singleton case, and three fixed seeds. Five actual-link mutations are detected. Bypassing the initializer failure guard consumes eight coins on a malformed input before a later rejection; the baseline rejects before any draw. The component gates separately check 810 routing cases, 28 initializer cases and 60 combined caller cases. Kernel controls retain the full state and query/residual witnesses. The general charged theorem supplies aggregate cost for the caller campaign, whose per-fixture kernel checks omit that cost. A redundant interpreter campaign was deliberately interrupted after 48 of those 60 positives; it is not a completed campaign. No successful gate establishes secrecy.

Lean: `PrimeHonestCiphertextControls.canonical_0`.

Lean: `PrimeHonestCiphertextControls.canonical_3`.

This result stops at the first ciphertext. The actual reduction source lowers honest proof requests through `ballotFiniteProgrammedImpl`: it next draws four full-field simulator scalars and updates the programmed cache. This is distinct from executing the historical prover's three nonzero proof coins. The next section executes those four full-field draws. Commitment arithmetic, programmed-state updates, subsequent ciphertexts, key generation, effective attacker coverage and complete adaptive cost are outside this component's guarantee.

1.116 Historical component: Four programmed-transcript scalar draws

The selected reduction uses `ballotFiniteProgrammedImpl` for honest proof requests. Its `ballotFullSimTranscript` samples $c, e, z_0, z_1 \in \mathbb{Z}_q$ in that order, computes `ballotSimCommit`, then applies the programmed-cache update keyed by the statement and simulated commitment. The following prefix executes those four samples and their storage. Zero is permitted in each full-field sample. The historical real prover's nonzero proof coins belong to a separate source path; the existing simulation and sampling losses remain necessary.

Lean: `PrimeFullFieldSource.drawTranscriptScalars`.

Lean: `PrimeFullFieldSource.ballotFullSimTranscript_factor`.

Interface and reuse. The local sampler accepts a canonical resident record for s, q and arbitrary retained ciphertext, nonce, input-context and parameter words. Its entry has success memory and blank sampler workspace. It reuses the existing full-field cache sampler through a fixed port map. Between samples, an executed consuming transfer saves the current scalar, clears the old modulus workspace and returns to the same sampler. Its general interface accepts any $q > 0$; historical group and encryption assumptions enter through the preceding caller. The four simulator requests are its current callers; another full-field request needs a checked adapter to this record and frame layout. No general protocol compiler is provided.

Lean: `PrimeFullFieldSource.scalar_word_source`.

Lean: `PrimeHonestTranscriptMachine.charged`.

Lean: `PrimeHonestTranscriptMachine.execution_source`.

Lean: [TranscriptScalarSave.run](#).

Lean: [TranscriptScalarSave.charged_bounded](#).

Theorem 1.130 (Four-draw execution and source law, machine-checked). *Fix $q > 0$, slack s , and the retained words in the preceding interface. Let A, D be the existing guarded single-sample clock and charge bounds and set*

$$L = 2 \text{size}(q - 1) + 1, \quad S = 3L + \text{size}(q) + 7, \quad F_4 = 4A + 3S, \quad B_4 = 4D + 15S.$$

The actual controller at clock F_4 executes four successive coin-word draws, each of width $\text{size}(q) + s$, followed by a complete halted-success state and a derived charge at most B_4 on every branch. Each word w becomes $U(\text{value}(w) \bmod q)$, whose length is at most L . The three save destinations are proved empty when entered, and the modulus workspace is cleared before every repeated sample. The full final state retains the prior ciphertext, both nonce prefixes and input context, with

$$(\text{port}_{10}, \text{port}_{11}, \text{port}_{12}, \text{port}_7) = (U(c.\text{val}), U(e.\text{val}), U(z_0.\text{val}), U(z_1.\text{val})), \quad \text{port}_2 = \text{bits}(q).$$

After forgetting charges, the entire query tree equals the existing fair-bit implementation of `drawTranscriptScalars`, mapped to this full state. The corresponding bounded-adaptor termination/charge contract is derived.

Lean: [PrimeTranscriptDraws.charged](#).

Lean: [PrimeTranscriptDraws.execution_source](#).

Lean: [PrimeTranscriptDraws.within](#).

Controls and evidence limits. Independent full-state validation checks eleven local-sample cases and eight four-draw cases, detecting eight and four actual instruction mutations respectively. It checks fresh-query counts, remaining coins and independently computed clock/charge caps. Omitting modulus clearing consumes too many coins and remains live at the mutant's finite cap; this is a bounded counterexample, not a proof of eventual nontermination. The earlier local full-trace kernel campaign exhausted interpreter memory and remains unpassed. The successful interpreter campaign uses checked state-snapshot and program-table identities with the original instructions. Separate kernel controls derive literal full positive states from the general run theorem and retain paired instruction boundary counterexamples. These evidence scopes are distinct.

Lean: [PrimeTranscriptDrawsControls.four_zero_scalars_full_state](#).

Theorem 1.131 (Joint raw-input source and physical execution, machine-checked). *Under the prime-group, input and slack hypotheses of Section 1.115, the linked controller starts from the same raw word x and blank work. It executes the first-ciphertext prefix and then all four simulator draws. If F, B are the preceding prefix bounds, its source clock is $F + F_4$ and every actual branch charge is at most $B + B_4$. After forgetting charges, its complete query tree is*

$$\text{liftCoins} \left(\text{result } \langle \$ \rangle \text{runFairBitUniform}_s \left(\begin{array}{l} (r_0, r_1) \leftarrow \text{drawPrimeNoncePair}; \\ (c, e, z_0, z_1) \leftarrow \text{drawTranscriptScalars}; \\ \text{return}((r_0, r_1), (c, e, z_0, z_1)) \end{array} \right) \right).$$

Here `result` preserves the preceding full ciphertext/nonce/input state and stores the four scalars as above. All extraction, canonical-record, return and halt conditions follow from the reached first-ciphertext state. For every bounded-adaptor limit, let

$$T' = \text{unitCost}(N + B + B_4)(B + B_4)G',$$

where G' is this controller's fixed compiler factor. There is a derived startup time $u \leq 4N + 8$ such that physical execution for $u + T'$ returns *some* of that same complete source result under the adapter. The loaded source height is N ; no prepared caller state or cost certificate is assumed.

Lean: `PrimeHonestTranscriptCaller.linked_run.`

Lean: `PrimeHonestTranscriptCaller.start_source.`

Lean: `PrimeHonestTranscriptCaller.start_height.`

Lean: `PrimeHonestTranscriptCaller.within.`

Lean: `PrimeHonestTranscriptCaller.drawSource.`

Lean: `PrimeHonestTranscriptCaller.execution_source.`

Lean: `PrimeHonestTranscriptCaller.physical_run.`

The bounded raw-input campaign passes two smaller $p = 3, q = 2$ fixtures with both votes and both nonce coin-word orders, and detects the same two actual mutations: skipping all transcript draws and corrupting private context 14. It checks every port, coin/hash count, remaining stream and machine bounds. The initial 90-second response timeout and the larger $q = 11$ fixture's 300-second host timeout returned no mismatch and remain unpassed attempts. The smaller campaign uses the same machine bounds and original instructions. Its singleton nonzero nonce space does not test distinct nonce values; the separate four-draw campaign retains the larger reached-state fixtures. Kernel controls separately check typed/literal entry and paired full-frame instruction boundaries.

Lean: `PrimeHonestTranscriptCallerControls.typed_prefix_entry.`

Lean: `PrimeHonestTranscriptCallerControls.skipped_all_draws_boundary.`

Lean: `PrimeHonestTranscriptCallerControls.corrupted_context_boundary.`

The complete configuration remains a proof-side observation; private vote and input words are not additional public outputs. Equality is to the stated fair-bit implementation, and does not remove its ideal-uniform approximation loss. Commitment arithmetic, statement encoding, programmed-cache updates, subsequent ballot components and whole-election runtime are outside this component's guarantee. The saved scalars overlap the current power workspace; the next subsection supplies an explicit retained-data frame for the first commitment coordinate.

1.117 Historical component: First simulated commitment from raw input

Let g, α denote the public residue coordinates of the generator and first ciphertext component. Write e, z_0 for the canonical natural representatives of the reached scalars in $\mathbb{Z}_q$, and define

$$A(p, q, g, \alpha, e, z_0) = ((g^{z_0} \bmod p)(\alpha^{q-e} \bmod p)) \bmod p.$$

For a typed statement over `PrimeGroup`(p, q), $p \neq 0$ and prime q , this is exactly the value of the first coordinate of `ballotSimCommit`($stmt, c, (e, z_0, z_1)$). Subgroup membership supplies $\alpha^q \equiv 1 \pmod{p}$: at $e = 0$, the executed exponent is q , whereas canonical scalar negation has value zero. This choice avoids an extra zero branch and uses the existing raw-exponent power routine. It requires neither a new field inverse nor an exact-order assumption. The historical raw caller below retains prime p and q .

Lean: `PrimeSimCommitSource.first_coordinate_value.`

Lean: `PrimeSimCommitSource.complement_action.`

Lean: `PrimeSimCommitSource.complement_width.`

Theorem 1.132 (Complete first-coordinate execution, machine-checked). *Assume $p \geq 2, g, \alpha < p$ and $e, z_0 < q$. Let w be the full reached operand presentation `inputWords`, with an arbitrary retained*

eleven-word frame. The controller appends fifteen blank private work ports to the original 23 ports. Write $\ell(n)$ for bit length, $F_{\text{pow}}, F_{\text{mul}}$ for the existing power/product clocks and

$$\begin{aligned} F_{\text{comp}}(q) &= 10\ell(q-1) + 5\ell(q) + 17, \\ F_A(p, q) &= F_{\text{comp}}(q) + 2F_{\text{pow}}(p, \ell(q)) + F_{\text{mul}}(p, \ell(p)) \\ &\quad + 22\ell(p) + 13\ell(q) + 54, \quad B_A = 32F_A. \end{aligned}$$

There is $u \leq F_A$ such that the actual run from w halts successfully with $\text{bits}(A)$ on port 3. Every other original port is preserved and all fifteen appended ports are empty. Padded execution for F_A has this same result and an actual charge at most B_A ; it makes no oracle queries. The run executes both scalar-prefix reads, $q - e$ subtraction, two modular powers, their product, all operand transfers and final cleanup.

Lean: `ScalarComplementMachine.run`.

Lean: `ScalarComplementMachine.charged`.

Lean: `PrimeSimCommitMachine.inputWords_source`.

Lean: `PrimeSimCommitMachine.run`.

Lean: `PrimeSimCommitMachine.charged`.

Lean: `PrimeSimCommitMachine.source`.

Lean: `PrimeSimCommitMachine.charged_source`.

Theorem 1.133 (Raw source and physical execution, machine-checked). *Under the prime-group and raw-input hypotheses of Section 1.116, let x be that original input, $N = |x|$, and F, B the complete four-draw prefix bounds. The combined controller executes for source clock $F + F_A$, with actual charge at most $B' = B + B_A$. Its full query tree, after forgetting charges, is the same fair-bit drawSource tree of Section 1.116, mapped to the complete preceding state with the actual first commitment coordinate on port 3 and blank appended workspace. All operand, frame and halt conditions are derived from supported executions of that original prefix.*

For every bounded-adaptor limit, let

$$T = \text{unitCost}(N + B')B'G_A,$$

where G_A is the combined controller's own compiler factor. There is a startup time $s \leq 4N + 8$ such that physical execution for $s + T$, from x and blank work, observes *some* of that same complete source result under that same bounded adaptor.

Lean: `PrimeHonestTranscriptCaller.run_support`.

Lean: `PrimeSimCommitCaller.linked_run`.

Lean: `PrimeSimCommitCaller.run_support`.

Lean: `PrimeSimCommitCaller.within`.

Lean: `PrimeSimCommitCaller.execution_source`.

Lean: `PrimeSimCommitCaller.physical_run`.

Independent integer fixtures check the complete local state on nine cases and detect eight actual instruction mutations, including omitted powers/product, a positive exponent and damaged private data or cleanup. The raw $p = 3, q = 2$ fixture checks the full state and detects skipped commitment execution; its nonzero nonce space is a singleton. Both campaigns check query counts, remaining coins and the original clock/charge caps, with no discards or `gaveUp`. Kernel controls separately retain full positive states, instruction-boundary mutations, the zero-complement distinction and failure of the exponent identity outside the subgroup.

The extra workspace is private. It resolves the concrete overlap between saved transcript scalars and the existing power workspace; a smaller overlap layout would need additional scheduling, while packing would require executed serialization and restoration. This proof reuses the existing frame and arithmetic routines. The complete state is a proof-side observation, and fair-bit sampling loss remains. The other three commitment coordinates, statement encoding, programmed-cache updates, later ballot components and complete adaptive execution/cost are outside this component's guarantee.

1.118 Historical component: Zero-branch commitment pair from raw input

The second zero-branch coordinate uses the actual public key and second ciphertext component, rather than constructing a different statement by exchanging the generator and public key. For typed subgroup operands, prime q and $p \neq 0$, let $e, z_0 \in \mathbb{Z}_q$ be typed scalars, with canonical natural representatives $\bar{e}, \bar{z}_0$. Then

$$\begin{aligned} (\text{ballotSimCommit}(\text{stmt}, c, (e, z_0, z_1)))_{0,2} &= z_0 \cdot pk - e \cdot \beta, \\ B(p, q, pk, \beta, \bar{e}, \bar{z}_0) &= ((pk^{\bar{z}_0} \bmod p)(\beta^{q-\bar{e}} \bmod p)) \bmod p \end{aligned}$$

is its public residue value. The first line uses the group's additive notation; the second uses natural residue coordinates. The zero-message offset vanishes in this branch. As before, subgroup membership justifies exponent q when $e = 0$ without identifying it with canonical scalar negation.

Lean: `PrimeSimCommitSecondSource.second_group`.

Lean: `PrimeSimCommitSecondSource.second_coordinate_value`.

The existing numeric controller is reused unchanged through a fixed port relocation. Local ports 0/17 and 15/16 exchange roles to supply β and pk ; local scratch/output 3 goes to new port 38. Original port 3, which holds the first coordinate, is retained in a singleton frame. Every other original port keeps its role. The verified frame library preserves the exact execution tree and charge under this relocation, avoiding a new copying or arithmetic routine.

Lean: `PrimeSimCommitSecondMachine.code_frame`.

Lean: `PrimeSimCommitSecondMachine.source_inputs`.

Lean: `PrimeSimCommitSecondMachine.source_outputs`.

Theorem 1.134 (Actual zero-branch pair and execution cost, machine-checked). *Assume prime p, q , typed g, pk , a Boolean vote, explicit slack and a saved private word. Let w be the actual complete first-coordinate source result of Section 1.117, for any nonce/transcript source tuple. The relocated controller starts from its 38 words and one new blank word. For the same F_A, B_A as that section, its actual charged run for F_A is a pure successful result with charge at most B_A . It preserves every word of w , including the first coordinate on port 3, and places the actual second coordinate B on port 38. All operand, reduced-value and scratch conditions are derived from w .*

From the original raw input x and blank work, the combined caller executes both zero-branch coordinates. Let $F_{\text{pre}}, B_{\text{pre}}$ be the complete raw four-draw prefix bounds and $N = |x|$. The source clock and actual charge bound are

$$F_{\text{pair}} = F_{\text{pre}} + 2F_A, \quad B_{\text{pair}} = B_{\text{pre}} + 2B_A.$$

After forgetting charges, its complete query tree is the unchanged fair-bit drawSource tree, mapped to the full result with both coordinates on ports 3/38 and all work ports 23–37 empty. For every bounded-adaptor limit, physical execution with the same adaptor on both sides returns some of this source result after a derived startup $u \leq 4N + 8$ and

$$T = \text{unitCost}(N + B_{\text{pair}})B_{\text{pair}}G_{\text{pair}}$$

steps, where G_{pair} is the combined controller's own compiler factor. The theorem assumes no supplied execution, source-correspondence or cost certificate.

Lean: `PrimeSimCommitSecondMachine.charged_source`.

Lean: `PrimeSimCommitSecondMachine.source`.

Lean: `PrimeSimCommitPairCaller.linked_run`.

Lean: `PrimeSimCommitPairCaller.run_support`.

Lean: `PrimeSimCommitPairCaller.within`.

Lean: `PrimeSimCommitPairCaller.execution_source`.

Lean: `PrimeSimCommitPairCaller.physical_run`.

Seven directed full-state fixtures include both votes, zero scalars, distinct coordinate values and reversed nonce order. Seven actual mutations test wrong bases, overwritten first output, omitted second computation, wrong destination, private-context damage and incomplete cleanup. The raw $p = 3, q = 2$ fixture and skip-suffix mutation test the original-input connection separately; the raw nonzero nonce space remains a singleton. These are bounded directed campaigns, with no randomized coverage claim. Kernel source and full-state controls retain their separate scope. Sampling loss and the public-observation policy are unchanged; complete private configurations remain proof-side observations.

The one-branch pair uses canonical $d = c - e$, response z_1 and the adjusted ciphertext component $\beta - g$. At $e = 0$, treating $q - e$ as a reduced field operand violates the modular-addition routine's premise. The retained $q - 1$ word and existing prefix writer avoid new exponent construction and serialization for those interfaces. Sections 1.119–1.121 execute these operands and complete both remaining coordinates. Statement encoding, programmed-cache updates, later ballots and complete adaptive cost are outside this component's guarantee.

1.119 Historical component: Executed canonical challenge difference

The one-message branch uses $d = c - e$ in $\mathbb{Z}_q$. Its canonical natural representative is $(\bar{c} + q - \bar{e}) \bmod q$; truncated natural subtraction would give the wrong result when $\bar{c} < \bar{e}$. The existing modular-addition controller already executes this difference from the direct digit words $\bar{c}$, $\bar{e}$ and q . For $e \neq 0$, its earlier theorem applies with $x = q - \bar{e}$. At $e = 0$, the claim $q - \bar{e} < q$ is false. A separate proof of the same program's subtraction-success and restoration path closes that endpoint without changing an instruction or asserting the false premise.

Lean: `BinaryModAddMachine.difference_run`.

Lean: `BinaryModAddMachine.difference_charged_source`.

Theorem 1.135 (Encoded scalar difference, machine-checked). *For $q \neq 0$ and $c, e \in \mathbb{Z}_q$, start with the canonical words $q.\text{bits}$, $U(\bar{c})$, $U(\bar{e})$ and blank output/work ports. A fixed controller copies and parses both prefixes, executes the existing difference routine, clears the retained temporary e digits and writes $U(\bar{c} - e)$ with the existing prefix writer. It retains all three original inputs and clears every temporary port. Its exact charged run is a pure successful result with actual charge at most $B_D = 32F_D$, where*

$$F_D(q) = 18 \text{size}(q - 1) + 26 \text{size}(q) + 47.$$

No parsing, reduced-operand, source or runtime certificate is a premise.

Lean: `ScalarDifferenceMachine.run`.

Lean: `ScalarDifferenceMachine.charged_source`.

Theorem 1.136 (Actual raw-input challenge difference, machine-checked). *Assume prime p, q , typed $g, pk : \text{PrimeGroup}(p, q)$, a Boolean vote, explicit slack and a saved private word. From the actual zero-branch pair result of Section 1.118, a fixed relocation places $U(\overline{c - e})$ in previously empty port1. Both commitment coordinates and every other original word are retained; all temporary work is restored empty. The source itself derives both input prefixes, their canonical bounds and every frame condition.*

*From the original raw input x , let $N = |x|$ and let $F_{\text{pair}}, B_{\text{pair}}$ be the complete prefix bounds. The combined caller has clock $F = F_{\text{pair}} + F_D$ and derived actual charge at most $B = B_{\text{pair}} + B_D$. After forgetting charges, the complete query tree is the unchanged fair-bit nonce/transcript source, mapped to this full result. For each bounded-adapter limit, the same adapter on both sides gives physical equality to **some** of the source result, with a derived startup $u \leq 4N + 8$ and*

$$T = \text{unitCost}(N + B)BG_D$$

execution steps, using the combined caller's own compiler factor G_D .

Lean: `PrimeSimDifferenceMachine.source_inputs.`
 Lean: `PrimeSimDifferenceMachine.source_outputs.`
 Lean: `PrimeSimDifferenceMachine.charged_source.`
 Lean: `PrimeSimDifferenceCaller.linked_run.`
 Lean: `PrimeSimDifferenceCaller.run_support.`
 Lean: `PrimeSimDifferenceCaller.within.`
 Lean: `PrimeSimDifferenceCaller.execution_source.`
 Lean: `PrimeSimDifferenceCaller.physical_run.`

The arithmetic gate covers every pair of residues for $1 \leq q \leq 16$, directed boundaries and three seeded campaigns; it detects four actual instruction defects, including restoration failure at $e = 0$. The encoded controller gate independently checks full states and six actual mutations. The raw-input fixture and skip-difference mutation retain the earlier $p = 3, q = 2$ scope, with a singleton nonzero nonce space. Kernel controls preserve their separate source/full-state/instruction evidence. Sampling loss and the public observation policy are unchanged. The following subsection covers adjusted-base execution from the reached state. Section 1.121 completes the one-branch commitments and raw composition. Statement construction, programming and complete adaptive cost remain separate obligations.

1.120 Historical component: Executed adjusted ciphertext operand

The remaining second commitment uses $\gamma = \beta - g$. For typed subgroup elements, prime q and $p \neq 0$, its canonical public residue is

$$\bar{\gamma} = (\overline{\beta}(\overline{g}^{q-1} \bmod p)) \bmod p.$$

Subgroup membership justifies this inverse representative; exact order is not an additional premise of this identity. The actual one-branch commitments are $z_1 \cdot g - d \cdot \alpha$ and $z_1 \cdot pk - d \cdot \gamma$.

Lean: `PrimeSimCommitOneSource.adjusted_beta_value.`
 Lean: `PrimeSimCommitOneSource.first_coordinate_value.`
 Lean: `PrimeSimCommitOneSource.second_coordinate_value.`

Theorem 1.137 (Adjusted-base execution, machine-checked). *For numeric $p \geq 2$ and reduced $g, \beta < p$, start from their canonical words, the retained $(q - 1)$ digits, blank work and an arbitrary*

retained frame. One fixed relocation of the existing power routine computes $g^{q-1} \bmod p$; a second relocation lets the existing multiplier consume that inverse with β . Two actual success/reset guards connect the routines. The complete charged run preserves all original words, clears all temporary work and writes $(\beta(g^{q-1} \bmod p)) \bmod p$ on a new output. Its clock and charge bound are

$$\begin{aligned} F_\gamma(p, q) &= F_{\text{pow}}(p, \text{size}(q-1)) + F_{\text{mul}}(p, \text{size}(p)) + 2, \\ B_\gamma(p, q) &= 32F_\gamma(p, q). \end{aligned}$$

The numeric theorem needs no primality or subgroup assumption.

For prime p, q and $g, pk : \text{PrimeGroup}(p, q)$, for any Boolean vote, slack, saved private word and nonce/transcript tuple, the actual reached challenge-difference result supplies every numeric operand, the $(q-1)$ word and blank work. The same run therefore produces the actual $\beta - g$, retaining d 's prefix, both preceding commitment coordinates and all original state. The enclosing source/frame conditions are derived from that reached result.

Lean: `PrimeAdjustedBetaMachine.charged.`

Lean: `PrimeAdjustedBetaMachine.padded_run.`

Lean: `PrimeAdjustedBetaMachine.inputWords_source.`

Lean: `PrimeAdjustedBetaMachine.charged_source.`

Lean: `PrimeAdjustedBetaMachine.source.`

This controller reads the retained operands directly through fixed frame maps; it needs no extra copying or constant-prefix construction. Eight directed full-state fixtures and seven actual instruction mutations check the arithmetic, preserved nonempty context and consumed inverse. A guard-rejection control checks unsuccessful entry. These are bounded tests, with no random-coverage claim. Separate source and kernel controls retain the wrong unadjusted-beta and truncated-subtraction counterexamples. Section 1.121 connects this reached-state result to the raw caller and executes both one-branch commitments.

1.121 Historical component: All simulator commitments from raw input

The remaining coordinates reuse the original numeric commitment controller through fixed port maps. With $d = c - e$ and $\gamma = \beta - g$, the complete commitment has group coordinates

$$\begin{pmatrix} z_0 \cdot g - e \cdot \alpha & z_0 \cdot pk - e \cdot \beta \\ z_1 \cdot g - d \cdot \alpha & z_1 \cdot pk - d \cdot \gamma \end{pmatrix}.$$

Actual reached-state presentations supply d, z_1, g, α for the first remaining coordinate and d, z_1, pk, γ for the second. Retained frames keep all earlier outputs outside scratch. Each run has the original F_A, B_A bounds, with no new arithmetic or cryptographic premise.

Lean: `PrimeSimOneFirstMachine.source_inputs.`

Lean: `PrimeSimOneFirstMachine.source_outputs.`

Lean: `PrimeSimOneFirstMachine.charged_source.`

Lean: `PrimeSimOneSecondMachine.source_inputs.`

Lean: `PrimeSimOneSecondMachine.source_outputs.`

Lean: `PrimeSimOneSecondMachine.charged_source.`

Theorem 1.138 (Complete raw commitment construction, machine-checked). *Assume prime p, q , $g, pk : \text{PrimeGroup}(p, q)$, a Boolean vote, slack and a saved private word. From the original canonical raw input and blank work, one finite controller executes both nonce draws, the first ciphertext, all four*

simulator scalar draws, canonical $d = c - e$, adjusted $\gamma = \beta - g$ and all four actual `ballotSimCommit` coordinates. Its complete source query tree, after forgetting charges, is the unchanged fair-bit nonce/transcript tree mapped to the full final state. The four output words at ports 3/38/40/42 are exactly the canonical coordinate digits of the original simulator result. The intermediate d prefix and γ value remain on 1/39; temporary work returns empty, including scratch 41. Original inputs and saved state are retained.

Let $F_{\text{diff}}, B_{\text{diff}}$ be the complete raw challenge-difference bounds from Section 1.119. The full clock and derived charge bound are

$$F_{\text{all}} = F_{\text{diff}} + F_{\gamma} + 2F_A, \quad B_{\text{all}} = B_{\text{diff}} + B_{\gamma} + 2B_A.$$

Each return derives the complete next state from the actual preceding source result. For every bounded-adapter limit, physical execution with the same adapter on both sides gives *some* of this full source result after startup $u \leq 4N + 8$ and

$$T = \text{unitCost}(N + B_{\text{all}})B_{\text{all}}G_{\text{all}}$$

steps, where N is the original input length and G_{all} is this combined controller's own compiler factor. Source correspondence, presentation, width and runtime certificates are derived rather than supplied as premises.

Lean: `PrimeSimOneTail.beta_return.`

Lean: `PrimeSimOneTail.first_return.`

Lean: `PrimeSimOneTail.charged_source.`

Lean: `PrimeSimAllCommitCaller.prefix_return.`

Lean: `PrimeSimAllCommitCaller.linked_run.`

Lean: `PrimeSimAllCommitCaller.run_support.`

Lean: `PrimeSimAllCommitCaller.within.`

Lean: `PrimeSimAllCommitCaller.source_commitments.`

Lean: `PrimeSimAllCommitCaller.execution_source.`

Lean: `PrimeSimAllCommitCaller.physical_run.`

Each remaining-coordinate gate checks four independent full-state fixtures and six actual instruction mutations, including wrong operands, destination and preserved-data/cleanup failures. Complete final-state controls retain both votes and all four outputs, d, γ and the private frame. The final finite composition uses checked label identities and complete return states; no additional full raw-native campaign is claimed. The raw subtraction campaign passed its canonical and mutation cases with 600 seconds of host response allowance; its earlier 300-second timeout remains a resource failure without a returned mismatch. Both attempts use the same modeled clock and charge limits. Private configurations remain proof-side observations, and fair-bit sampling loss is unchanged.

This completes execution of the first simulated ballot's commitment values. Section 1.122 executes its flat hash-key encoding. Programmed-cache insertion with its sticky collision flag and duplicate history, remaining ballot construction, effective attacker continuations and complete adaptive cost are outside this component's guarantee.

1.122 Historical component: Executed simulator hash-key serialization

The original hash key contains the complete statement and both commitment pairs. Its group-coordinate order is

$$(g, pk, \alpha, \beta, A, B, C, D).$$

The preceding raw constructor retains their canonical residue digits on ports 16, 15, 17, 0, 3, 38, 40, 42. These words are not yet the key codec. Writing $U(n)$ for the existing canonical natural prefix, one encoded field is

$$E(n, w) = U(|U(n)|) ++ U(n) ++ w.$$

The complete flat record is $U(8)$ followed by the eight framed coordinate prefixes. In particular, coordinates use the field modulus p , not the scalar modulus q . Actual-source presentation proves their origins and width bounds; no new group operation is required.

Lean: `PrimeSimKeySource.source_digits`.

Lean: `PrimeSimKeySource.encoding`.

Lean: `PrimeSimKeySource.source_width`.

Lean: `PrimeSimKeySource.source_work`.

Theorem 1.139 (One preserved coordinate field, machine-checked). *For every natural n and arbitrary suffix w , a fixed controller starts with n 's canonical digits, w and blank work. It preserves the original digits, returns $E(n, w)$ and clears all five work ports. It invokes the existing copy, prefix and field writers. With $s = \text{size}(n)$, its clock and actual charge bound are*

$$F_E(n) = 5s + 8 + \text{FrameWriteMachine.cost}(2s + 1), \quad B_E(n) = 32F_E(n).$$

The complete charged run equals a pure successful result with derived charge at most $B_E(n)$. The clock is monotone in n ; neither a precomputed prefix nor an encoding-execution certificate is supplied.

Lean: `NatFieldWriterMachine.run`.

Lean: `NatFieldWriterMachine.padded_run`.

Lean: `NatFieldWriterMachine.charged`.

Lean: `NatFieldWriterMachine.clock_mono`.

Theorem 1.140 (Full key from the actual commitment state, machine-checked). *Assume prime p, q , typed $g, pk : \text{PrimeGroup}(p, q)$, a Boolean vote, slack, a saved private word and the original nonce/transcript tuple. Starting from the actual complete commitment result and one new blank output port, eight fixed writer invocations and a fixed count-prefix step produce exactly `ballotKeyBitCodec`'s encoding of the original statement and simulated commitment. All 43 preceding words are retained; reused work ports 23–27 are empty again. The new key occupies port 43.*

The controller writes fields in reverse order because each invocation prepends to its existing suffix. Typed residue bounds imply $n_i \leq p - 1$ for every coordinate, giving the complete clock and derived charge bound

$$F_K(p) = 8F_E(p - 1) + 1, \quad B_K(p) = 32F_K(p).$$

Every source, workspace and reduced-operand premise of the numeric execution lemma is discharged from the actual preceding result.

Lean: `PrimeSimKeyMachine.field_run`.

Lean: `PrimeSimKeyMachine.run`.

Lean: `PrimeSimKeyMachine.charged`.

Lean: `PrimeSimKeyMachine.charged_source`.

Lean: `PrimeSimKeyMachine.source`.

Theorem 1.141 (Original raw-input key execution, machine-checked). *Under the same prime-group, vote, slack and saved-word assumptions, let $F_{\text{all}}, B_{\text{all}}$ denote the complete preceding raw commitment bounds. The combined caller derives*

$$F_{\text{key}} = F_{\text{all}} + F_K(p), \quad B_{\text{key}} = B_{\text{all}} + B_K(p).$$

After forgetting charges, its complete query tree is the unchanged fair-bit nonce/transcript source mapped to the full result with the exact key on port 43. Each of the original 43 words is preserved. For every bounded-adapter limit, physical execution from the original raw input and blank work, using the same adapter on both sides, returns some of this source result after derived startup $u \leq 4N + 8$ and

$$T = \text{unitCost}(N + B_{\text{key}})B_{\text{key}}G_{\text{key}}$$

steps. Here N is the original raw-input length and G_{key} is the combined controller's own compiler factor. Source correspondence and costs are proved from the executed code.

Lean: `PrimeSimKeyCaller.prefix_return.`

Lean: `PrimeSimKeyCaller.linked_run.`

Lean: `PrimeSimKeyCaller.run_support.`

Lean: `PrimeSimKeyCaller.within.`

Lean: `PrimeSimKeyCaller.source_key.`

Lean: `PrimeSimKeyCaller.source_retained.`

Lean: `PrimeSimKeyCaller.execution_source.`

Lean: `PrimeSimKeyCaller.physical_run.`

The single-field gate covers 33 independently generated cases, including zero, nonempty suffixes and width boundaries, plus five actual instruction mutations. The key gate covers five source-consistent states and three additional seeded canonical-digit states, seven actual mutations and a rejecting entry guard. Every comparison includes full retained state, queries, residual tape and the original clock/charge bounds. Separate kernel controls pin source coordinates and literal bytes, including an eight-distinct-coordinate example. These are bounded campaigns; the final raw composition uses checked finite-code and complete return-state identities, with no additional full raw-native campaign.

This closes the first simulated ballot's flat hash-key construction. Its programmed-statement history has a different nested-pair codec. Executing that history update, sticky collision status, shadow/live cache handling, later ballots, effective attacker continuations and complete adaptive cost are outside this component's guarantee. Private configurations remain proof-side observations and fair-bit sampling loss is unchanged.

1.123 Historical component: Recovering the original programmed state

The next constructor boundary needs the shadow cache S , separate live cache L , sticky collision flag b and ordered statement history H . The original saved-field codec is

$$F[F[S, F[[b], H]], L], \quad F[\dots] = \text{bitFieldsEncode}[\dots].$$

Here S, L, H denote their existing encoded words. Repetitions and history order are retained. An arbitrary saved word does not establish this presentation.

Theorem 1.142 (Exact first programming request). *For prime p, q , typed group elements g, pk , a vote, any original submission attacker continuation, and arbitrary finite initial shadow state s*

and live cache L , the complete source factors exactly as follows: draw the original nonzero nonce pair r , draw the four original simulator scalars u , apply $s.\text{program}$ to the actual first statement and transcript, then run the original remaining continuation from that result and unchanged L . The two remaining Alice proofs, verification, Bob, attacker submission and final verification remain in the continuation. Empty initialization is a specialization.

Lean: `PrimeFirstProgramSource.first_program_bind.`

Lean: `PrimeFirstProgramSource.source_eq.`

Lean: `PrimeFirstProgramSource.submission_run.`

Lean: `PrimeFirstProgramSource.submission_empty.`

This is exact semantic source equality. It does not execute saved-state serialization or the subsequent programming transition.

Theorem 1.143 (Complete state extraction from the retained input). *Let the first four raw fields be arbitrary words and let the fifth have the nested form above, with singleton Boolean flag $[b]$. Suppose port 14 contains this raw record of length N and work ports 23–27 are empty. Execution starts from $\text{start}(\text{old})$: the fixed entry label, success memory 2, and four appended blank output words. The extractor returns $S, L, [b], H$ on ports 44–47, preserves all 44 previous words, and restores the work ports. It uses at most*

$$T(N) = 11(3 \text{size}(N) + 7 + N(2 \text{size}(N) + 3)) + 6N + 41$$

local steps and charge at most $32T(N)$. Each field length is bounded by the actual raw record; no independent field-size or cost certificate is assumed.

Lean: `PrimeProgrammedStateInputMachine.run.`

Lean: `PrimeProgrammedStateInputMachine.charged.`

Lean: `PrimeProgrammedStateSource.raw_shape.`

Lean: `PrimeProgrammedStateSource.source_context.`

Lean: `PrimeProgrammedStateSource.source_work.`

Lean: `PrimeProgrammedStateInputMachine.charged_source.`

The source specialization derives every presentation and work premise from the actual key-caller result with saved field `ballotBothCachesBitCodec.encode(s, L)`. The complete raw caller retains the same fair-bit nonce/transcript query tree. Its total charge is bounded by the key-caller charge bound plus $32T(N)$; physical execution uses this combined program's own compiler factor and the same bounded adapter on both sides.

Lean: `PrimeProgrammedStateCaller.linked_run.`

Lean: `PrimeProgrammedStateCaller.run_support.`

Lean: `PrimeProgrammedStateCaller.source_state.`

Lean: `PrimeProgrammedStateCaller.source_retained.`

Lean: `PrimeProgrammedStateCaller.execution_source.`

Lean: `PrimeProgrammedStateCaller.physical_run.`

Example 1.144 (Occupied insertion does not clear its work). *On an occupied canonical key, the unchanged insertion routine retains an unread tail, a positive binary counter and the old encoded answer. It preserves the original cache and proposed answer and returns the collision result. The derived occupied endpoint has a nonempty counter. Thus a surrounding caller must capture the collision flag and clear insertion work ports 0, 4 and 5 before reuse. At the source level, even an agreeing old and proposed challenge sets the sticky flag. Section 1.124 executes that update and cleanup.*

Lean: `CacheInsertMachine.occupied_full_run.`

Lean: `CacheInsertMachine.occupied_charged_run.`

Lean: `CacheInsertMachine.occupied_counter_not_empty.`

Lean: `CacheInsertOccupiedControls.canonical_occupied_counter.`

Lean: `PrimeFirstProgramSource.control_collision.`

The extractor reuses the existing copy and field-parser library interfaces. It checks framing and the singleton flag; it does not validate arbitrary cache contents. Typed source codecs establish that interpretation. Execution from this canonical raw input does not itself prove that an enclosing predecessor serialized its state. Section 1.124 executes collision cleanup and the flag/history update. Enclosing serialization, repacking and successor continuation are outside this component's guarantee. The full election also has an auxiliary-domain cache outside this ballot-state codec; the enclosing caller must preserve it. Private configuration observations and sampling losses are unchanged.

1.124 Historical component: Executing the first programmed-state update

Let $s = (S, b, H)$ be the original finite shadow state, L its separate live cache, x the actual first simulator statement, $k = (x, t)$ its full hash key and c the sampled challenge; t denotes the four-coordinate commitment. The original source operation has successor

$$S' = \begin{cases} S[k \mapsto c] & S(k) = \text{none}, \\ S & S(k) = \text{some}(v), \end{cases} \quad b' = b \vee (S(k) \neq \text{none}), \quad H' = x :: H.$$

Any occupied key sets the flag, including an old answer equal to c . History prepends the statement and preserves repeated entries.

Lean: `PrimeProgrammedInsertSource.program_fresh.`

Lean: `PrimeProgrammedInsertSource.program_occupied.`

Theorem 1.145 (Executed cache update and collision cleanup). *For nonzero moduli p, q , a typed finite cache, typed key and challenge, arbitrary live/history words and Boolean flag, initialize the local insertion wrapper with success memory, the canonical cache/challenge/key words and empty work. Its complete execution returns the encoded S' , singleton $[b']$, unchanged challenge, key, live word and history word, with every insertion work word empty. Let I bound the existing insertion cost on the actual cache and D bound $1 + |S| + |c| + |k| + |L| + |H|$, where bars denote lengths of the existing encoded bit words. The wrapper halts within*

$$T_{\text{ins}}(I, D) = I + 3(D + 5I) + 5, \quad \text{charge} \leq 32T_{\text{ins}}(I, D).$$

The factor 5 is the maximum stack access count of the unchanged insertion code, kernel-checked by the durable insertion gate driver. Occupied residue lengths follow from that code's actual execution growth. The source specialization derives I, D and every initialization premise.

Lean: `CacheProgrammedInsertMachine.run.`

Lean: `CacheProgrammedInsertMachine.charged_bounded.`

Lean: `PrimeProgrammedInsertInputMachine.charged_bounded.`

Lean: `PrimeProgrammedInsertMachine.source_return.`

Lean: `PrimeProgrammedInsertMachine.charged_source.`

Lean: `PrimeProgrammedInsertMachine.source_state.`

The surrounding preparation copies shadow port 44 to work port 23, clears 44, and enters the existing local insertion layout. The wrapper captures the return status before clearing residue. Its

full result changes only ports 44 and 46; live 45, ordered history 47, challenge 10, key 43 and the remaining frame persist. For original raw length N , the actual source supplies

$$I = \text{CacheInsertMachine.cost}(p, q, N, N),$$

$$D = 1 + 3N + \text{keyRecordBitBound}(p) + \text{groupRecordBitBound}(q).$$

These bounds use cache entry count at most encoded cache length at most N . They are uniform over the original sampled tuple.

Lean: `PrimeProgrammedInsertSource.source_entries_le`.

Lean: `PrimeProgrammedInsertSource.source_payload_lengths`.

Lean: `PrimeProgrammedInsertSource.source_input_bound`.

Theorem 1.146 (Nested statement encoding and ordered prepend). *Suppose ports 0, 17, 15, 16 respectively contain canonical binary digits of $\beta, \alpha, pk, g < p$, ports 23–29 are empty and history 47 is the canonical counted list $F[H_0, \dots, H_{n-1}]$. From its entry with success memory, the history routine updates only port 47 to*

$$F[F[F[U(g), U(pk)], F[U(\alpha), U(\beta)]], H_0, \dots, H_{n-1}].$$

It preserves all other words and clears its work. Its clock is the sum of four existing natural-field writers, two pair-field writers, one statement-field writer, the derived history count parsing/increment/prefix budget, and nine fixed control steps. Charge is at most 32 times that clock. For an enclosing length bound N , its padded clock requires the actual old history length to be at most N .

Lean: `PrimeProgrammedHistoryMachine.run`.

Lean: `PrimeProgrammedHistoryMachine.charged_bounded`.

Lean: `PrimeProgrammedHistoryMachine.charged_source`.

Lean: `PrimeProgrammedHistoryMachine.source_state`.

The typed specialization derives the four values, their modulus bounds, blank work and history-length bound from the executed insertion successor. The statement uses its original nested codec, rather than a prefix of the flat key.

Theorem 1.147 (Original raw execution through the resident update). *For prime p, q , typed g, pk , vote v , arbitrary finite initial s, L and sampler slack, start with only the canonical original raw input and blank work. The composed caller executes the original nonce/transcript draw tree and reaches `sourceResult(s, L, out)`, whose ports 44, 46, 47 encode S', b', H' above and whose port 45 still encodes L . It preserves all other resident words from the preceding state extractor. The full query-tree equality uses the same fair-bit sampler and introduces no additional queries. Its charge bound adds the extractor caller, preparation, insertion and history bounds. The physical execution theorem derives startup at most $4N + 8$ and uses the composed program's own compiler factor; the same bounded adapter occurs on both sides of that equality.*

Lean: `PrimeProgramMachine.charged_source`.

Lean: `PrimeProgramMachine.source_state`.

Lean: `PrimeProgramCaller.linked_run`.

Lean: `PrimeProgramCaller.run_support`.

Lean: `PrimeProgramCaller.source_state`.

Lean: `PrimeProgramCaller.source_retained`.

Lean: `PrimeProgramCaller.execution_source`.

Lean: `PrimeProgramCaller.physical_run`.

Independent actual-code gates exercise fresh/occupied flags, agreeing collisions, nonempty residue, nested field order and repeated history, and detect altered instructions. Their finite samples validate only the recorded cases; the general results above come from Lean proofs. Full-state kernel controls retain both positive endpoints and nearby wrong results. Private configuration observations are proof observations; they add no public leakage to the protocol.

Component boundary. Port 14 still contains the *original* raw input with the old saved state. Re-extracting it after this update would restore stale data. The next subsection executes repacking and returned-proof encoding with enlarged storage bounds. Remaining proof requests, ballot assembly, verification and effective attacker continuations still need their execution proofs. Preserve the enclosing auxiliary-domain cache.

1.125 Historical component: Encoding the returned proof and saved state

The first proof request returns a proof as well as the updated state. For commitments $(A, B), (C, D)$ and the original sampled scalars c, e, z_0, z_1 , its two branches are

$$P_0 = (A, B, e, z_0), \quad P_1 = (C, D, c - e, z_1),$$

where subtraction is in $\mathbb{Z}_q$. The prior execution already computed the canonical prefix for $c - e$. Encoding uses that resident value; it does not replace modular subtraction by natural truncated subtraction or by c .

Lean: `PrimeProgramProofSource.program_return`.

Lean: `PrimeProgramProofSource.scalar_values`.

Lean: `PrimeProgramProofSource.encoding_resident`.

The original proof codec has seven nested pairs:

$$F[F[F[U(A), U(B)], F[U(e), U(z_0)]], F[F[U(C), U(D)], F[U(c - e), U(z_1)]]].$$

The updated saved state has three:

$$F[F[S', F[[b'], H']], L].$$

Here group coordinates use modulus p , scalars use modulus q , and S', H', L stand for their existing encoded words. The two cache roles remain distinct. The source interfaces derive all resident operands and blank work from the preceding executed programming result.

Lean: `PrimeProgramProofSource.source_groups`.

Lean: `PrimeProgramProofSource.source_scalars`.

Lean: `PrimeProgramProofSource.source_work`.

Lean: `PrimeProgramRepackSource.source_shape`.

Theorem 1.148 (Preserving pair construction). *For arbitrary words a, b , initialize the pair writer at its entry with success memory, inputs a, b , blank output and four blank work words. It returns $F[a, b]$, retains both inputs, restores all work and halts with success memory. For $|a| \leq L$ and $|b| \leq R$, it executes within*

$$\begin{aligned} T_{\text{pair}}(L, R) &= 2(L + R) + \text{FrameWrite.cost}(L) \\ &\quad + \text{FrameWrite.cost}(R) + 7, \\ \text{charge} &\leq 32T_{\text{pair}}(L, R). \end{aligned}$$

These length premises concern the actual words; the typed callers derive them.

Lean: `BitPairWriterMachine.run`.

Lean: `BitPairWriterMachine.charged_bounded`.

The writer copies the right input into temporary storage, writes its field, then does the same for the left input and writes the fixed two-field count. It reuses the existing copy and field routines. Arbitrary-word construction is not a claim that every constructed record decodes as a cryptographic object.

Theorem 1.149 (Updated storage bounds and stale-record exclusion). *For prime p, q and any typed initial shadow state and live cache, let N be the original raw input length. After the first programming request, the shadow cache and ordered history each have at most $N + 1$ entries; the live cache has at most N entries. Existing codec bounds therefore bound their enlarged encoded records. The new saved-state encoding differs from the old one on every call: even an occupied key prepends a history entry.*

Lean: `PrimeProgramRepackSource.source_counts`.

Lean: `PrimeProgramRepackSource.source_lengths`.

Lean: `PrimeProgramRepackSource.saved_length`.

Lean: `PrimeProgramRepackSource.saved_changed`.

The retained live word also keeps its original direct length bound N . No old-payload bound is silently reused for the enlarged shadow cache or history.

Theorem 1.150 (Original raw execution through encoded return). *For prime p, q , typed g, pk , a vote, arbitrary finite initial shadow state and live cache, and sampler slack, the composed caller starts from the canonical original raw input with blank work. It executes the same fair-bit nonce and transcript draw tree as the preceding caller. For every sampled tuple, its result appends the original returned-proof encoding on port 48 and updated saved-state encoding on port 49, preserving all 48 preceding resident words. All numeric operands, empty workspace and enlarged record bounds are derived from that preceding execution. No source presentation or cost certificate is supplied by the caller.*

The deterministic encoding clock adds four natural-field writer bounds, eight pair-writer bounds, the eight temporary-word length bounds and eleven fixed steps. Encoding charge is at most 32 times that clock. The raw caller adds the preceding caller's charge. Its physical theorem derives startup at most $4N + 8$ and uses the composed code's own compiler factor, with the same bounded oracle adapter on both sides.

Lean: `PrimeProgramOutputMachine.run`.

Lean: `PrimeProgramOutputMachine.charged`.

Lean: `PrimeProgramOutputMachine.charged_source`.

Lean: `PrimeProgramOutputMachine.source_outputs`.

Lean: `PrimeProgramOutputMachine.source_retained`.

Lean: `PrimeProgramOutputMachine.source_saved_changed`.

Lean: `PrimeProgramOutputCaller.linked_run`.

Lean: `PrimeProgramOutputCaller.run_support`.

Lean: `PrimeProgramOutputCaller.execution_source`.

Lean: `PrimeProgramOutputCaller.physical_run`.

Independent controller tests cover six numeric/canonical-shaped boundary cases, ten actual instruction mutations and one failed-entry rejection, with seeds 118, 606 and 20260914, no discards and no gaveUp. Every run respects the original modeled caps, issues no queries and preserves the unused coin suffix. These fixtures include arbitrary surrounding words; reachable typed origins

follow from the separate source proof. The preserving pair writer also has its own independent word fixtures and mutations. Typed endpoint controls retain fresh, occupied, agreeing-challenge and sticky-flag cases, as well as nearby wrong proofs and saved states. These configurations are private proof observations; the public election view is unchanged.

Continuation interface. The new saved word is available, but port 14 still contains the original raw input with its old saved state. Reentering the whole prefix would also resample the nonces and repeat the first proof. The next actual continuation must retain those nonces and the first proof, execute the other-component request at (false, r_2) and the overall request at $(v, r_1 + r_2)$, and allow the modular sum to be zero. Later ballot assembly, Bob’s construction, verification, attacker execution and complete adaptive costs are outside this component’s guarantee. The enclosing auxiliary-domain cache must remain framed.

1.126 Historical component: Continuing with the retained nonce pair

After the first proof p_0 , the actual source requests the other component and then the overall proof at witnesses

$$(\text{false}, r_2), \quad (v, r_1 + r_2).$$

It preserves p_0 and threads the updated programmed state through these requests. Neither request resamples Alice’s nonce pair. Let D_q be the original four-full-field-draw transcript distribution. For an honest statement w and draw tuple c , write $T(w, c)$ for the existing simulator transcript at w .

Theorem 1.151 (The exact remaining proof requests). *For prime p, q , typed g, pk , arbitrary nonce pair (r_1, r_2) , first proof p_0 , finite initial programmed state s , live cache L and continuation K , the original two-request computation has exactly this source tree:*

$$\begin{aligned} c_1 &\leftarrow D_q, & (p_1, s_1) &= s.\text{program}(w_1, T(w_1, c_1)), \\ c_t &\leftarrow D_q, & (p_t, s_t) &= s_1.\text{program}(w_t, T(w_t, c_t)), \\ & & &\text{run}(K(p_0, p_1, p_t), s_t, L). \end{aligned}$$

Here w_1, w_t abbreviate the statements of the two witnesses above. Specializing K to the original post-proof body retains Alice’s ballot assembly and verification, Bob’s constructor, both honest decisions, the attacker and final submission, including every rejection branch.

Lean: `PrimeRemainingProgramSource.remaining_program_bind.`

Lean: `PrimeRemainingProgramSource.continuation_eq.`

Lean: `PrimeRemainingProgramSource.continuation_run.`

This is an equality of the original probabilistic source computations. It does not itself execute the remaining body in the finite machine.

Example 1.152 (Nonzero components can have zero total nonce). *For $p = 7, q = 3, g = 2, pk = 4$, choose $r_1 = 1, r_2 = 2$ and $v = \text{true}$. Both component nonces are nonzero, but $r_1 + r_2 = 0$ in $\mathbb{Z}_3$. The other-component ciphertext is $(4, 2)$; the overall ciphertext is $(1, 2)$. A positive-nonce precondition on the overall-proof entry would exclude this actual source case.*

Lean: `PrimeRemainingProgramSourceControls.exact_remaining_statements.`

Lean: `PrimeRemainingProgramSourceControls.wrong_nonce_distinguished.`

Lean: `PrimeRemainingProgramSourceControls.threaded_successor.`

Lean: `PrimeRemainingProgramSourceControls.stale_state_distinguished.`

Lean: `PrimeRemainingProgramSourceControls.first_proof_retained.`

Lean: `PrimeRemainingProgramSourceControls.occupied_request_keeps_live.`

The resident operand interface selects the original second prefix, shares the existing public parameters and sampler record, and uses the updated saved word as context. The first proof, original input and both nonce prefixes remain in the frame. Six previously blank work words suffice for the reused local routines; separate outputs retain the new ciphertext, nonce digits, false vote and four transcript scalars. The entry directly shares the existing saved-state context and public-parameter words.

Lean: `PrimeSecondTranscriptSource.source_inputs.`

Lean: `PrimeSecondTranscriptSource.source_work.`

Lean: `PrimeSecondTranscriptSource.source_presentation.`

Lean: `PrimeSecondTranscriptSource.source_lengths.`

Lean: `PrimeSecondTranscriptSource.saved_not_stale.`

Theorem 1.153 (The second ciphertext and fresh transcript prefix). *Let p, q be prime, g, pk typed group inputs, v a vote, s, L arbitrary finite initial programmed and live states, and σ the sampling slack. Let $I = \text{input}(g, pk, \sigma, v, \text{saved}(s, L))$ be the original `PrimeHonestInputMachine.input` encoding, A_q the existing first nonce/transcript tuple source, and F_σ the existing fair-bit modulo interpretation. Write $o = ((r_1, r_2), c_0)$ for the first source tuple. The combined caller satisfies*

$$\text{fst} < \$ > \text{run}(M, \text{clock}(I, \sigma, p, q), \text{start}(I)) = \text{liftCoins} \left(\begin{array}{l} o \leftarrow F_\sigma(A_q); \\ c \leftarrow F_\sigma(D_q); \\ \text{return } R(o, c). \end{array} \right).$$

The result $R(o, c)$ retains every word of the first returned-proof state and adds the ciphertext `encryptWith( $g, pk, r_2, 0$ )`, its nonce digits, the false component vote, four canonically encoded scalars and the scalar modulus. Every supported machine result halts with charge at most

$$B = B_0 + 6R_{\text{route}} + 32E_{\text{encrypt}} + B_{\text{draws}} + 6.$$

Here B_0 is the preceding raw caller's bound, R_{route} uses $q - 1$, E_{encrypt} uses the derived nonce width $\text{size}(q - 1)$, and B_{draws} is the existing four-draw bound. The two fixed control transitions contribute the final six units.

Lean: `PrimeSecondTranscriptMachine.charged.`

Lean: `PrimeSecondTranscriptMachine.charged_source.`

Lean: `PrimeSecondTranscriptMachine.source_fields.`

Lean: `PrimeSecondTranscriptMachine.source_retained.`

Lean: `PrimeSecondTranscriptMachine.execution_source.`

Lean: `PrimeSecondTranscriptMachine.run_support_source.`

Lean: `PrimeSecondTranscriptCaller.linked_run.`

Lean: `PrimeSecondTranscriptCaller.run_support.`

Lean: `PrimeSecondTranscriptCaller.execution_source.`

Every supported source result also determines both sampled tuples, supplying the typed return interface for the next resident call.

Lean: `PrimeSecondTranscriptCaller.source_support.`

R observes the internal configuration for this execution proof. This is not an additional public election observation, and the theorem does not establish the remaining protocol's privacy.

The physical corollary derives startup at most $4|I| + 8$ and remaining execution bound $\text{unitCost}(|I| + B) B \text{globalFactor}(M)$ from the same raw input and the combined code. Both sides use the same bounded oracle adapter. This statement preserves the fair-bit modulo tree; it does not assert exact ideal-uniform sampling.

Lean: [PrimeSecondTranscriptCaller.physical_run](#).

The endpoint precedes the second proof's commitments, programming update and returned-proof encoding. The semantic factorization above identifies the remaining source; it does not assume an execution theorem for that remainder. The overall nonce sum, subsequent ballots, verification, attacker execution and complete adaptive cost are outside this component's guarantee.

The independent instruction gate covers eight distinct numeric fixtures, nine mutations and one entry rejection across two runs. A 120-second host timeout on seed 606 interrupted the first run after all directed cases and mutations had passed. The isolated seeded replay passes seeds 118, 606 and 20260914 with a 300-second host allowance; controller and driver hashes, fixtures and modeled cost caps are identical. The earlier timeout remains an incomplete campaign. Seven fixtures match typed source layouts with selected finite initial states; the zero-nonce fixture tests the numeric boundary only. Every completed case checks all stored words, exact coin consumption, unused tape suffix and zero hash queries. There are no discards or gaveUp. Independently specified typed endpoint controls distinguish wrong nonces/votes, stale saved state, lost first proof and reused scalar outputs. No extra whole-raw native campaign is claimed for the checked caller composition.

Lean: [PrimeSecondTranscriptControls.full_state_presentation](#).

Lean: [PrimeSecondTranscriptControls.wrong_nonce_and_vote_excluded](#).

Lean: [PrimeSecondTranscriptControls.prior_proof_and_saved_retained](#).

Lean: [PrimeSecondTranscriptControls.stale_saved_and_zero_proof_excluded](#).

Lean: [PrimeSecondTranscriptControls.fresh_scalar_outputs](#).

1.127 Historical component: The second proof's first commitment

Keep the preceding notation $o = ((r_1, r_2), c_0)$ and write the fresh tuple as $c_1 = (c, e, z_0, z_1)$. The next statement is $w_1 = \text{honestProofStatement}(g, pk, (\text{false}, r_2))$. Its first simulator commitment has public coordinate

$$A_1 = (g^{z_0} \alpha_1^{q-e}) \bmod p, \quad \alpha_1 = g^{r_2} \bmod p.$$

Here g, α_1 denote their canonical public residues and e, z_0 their canonical scalar representatives. When $e = 0$, the executed exponent is q ; the typed subgroup law justifies this representative. The power bound uses $\text{size}(q)$.

Lean: [PrimeSecondCommitSource.answer_source](#).

Lean: [PrimeSecondCommitSource.source_complement_width](#).

The unchanged numeric commitment controller reads the resident second ciphertext and fresh scalar prefixes through a fixed port map. Its operands, blank work and complete retained frame are derived from the preceding source result. The earlier proof and updated saved state stay resident; the old raw input is not reparsed. Let $R_A(o, c_1)$ retain all 59 words of $R(o, c_1)$, append the canonical digits of A_1 on port 60, and leave the other five new words blank. Both results use successful halted control state.

Lean: [PrimeSecondCommitSource.source_inputs_embed](#).

Lean: [PrimeSecondCommitSource.source_outputs_embed](#).

Lean: [PrimeSecondCommitSource.source_bounds](#).

Theorem 1.154 (Resident and raw execution of the first p1 commitment). *Let p, q be prime, g, pk typed group inputs, v a Boolean vote, s, L arbitrary finite initial programmed and live states, and*

$\sigma \in \mathbb{N}$. For every typed o, c_1 , the relocated local controller N satisfies

$$\exists a \leq 32K(p, q), \quad \text{run}(N, K(p, q), \text{start}(R(o, c_1).\text{stk})) = \text{pure}(R_A(o, c_1), a),$$

where K is the existing `PrimeSimCommitMachine.clock`. Its bound includes complement preparation, both powers, multiplication, transfers and cleanup. For the original input I and fair-bit sources A_q, D_q defined above, the combined raw caller M_A has the same source-tree equality as the preceding theorem with result R_A . Every supported result halts with charge at most

$$B_A = B + 32K(p, q).$$

The physical corollary derives startup at most $4|I| + 8$ and execution bound $\text{unitCost}(|I| + B_A) B_A \text{globalFactor}(M_A)$, using the same bounded oracle adapter on both sides.

Lean: `PrimeSecondCommitMachine.charged_source`.

Lean: `PrimeSecondCommitMachine.execution_source`.

Lean: `PrimeSecondCommitMachine.source_retained`.

Lean: `PrimeSecondCommitMachine.source_coordinate`.

Lean: `PrimeSecondCommitCaller.linked_run`.

Lean: `PrimeSecondCommitCaller.run_support`.

Lean: `PrimeSecondCommitCaller.execution_source`.

Lean: `PrimeSecondCommitCaller.physical_run`.

The raw theorem derives the preceding tuples from supported execution rather than requiring their reconstruction as a premise.

Lean: `PrimeSecondCommitCaller.source_support`.

Arithmetic adds no oracle queries. Internal configurations remain proof-side observations, and the fair-bit modulo interpretation is unchanged.

The independent instruction gate passes nine canonical fixtures, eight actual instruction mutations and one guard rejection, with seeds 118, 606 and 20260914 and no discards or gaveUp. Eight fixtures use source-shaped layouts with selected finite initial states; one tests numeric nonce zero, outside the original nonzero sampler's support. Every case checks the full state, zero queries, unchanged residual tape and original modeled bounds. The host allowance is 300 seconds per response. Independent typed controls give $A_1 = 6$ for $p = 23, q = 11, g = 2, r_2 = 1, e = 1, z_0 = 10$, whereas using the old challenge gives 12 and the preceding proof's commitment is 16. The zero-challenge case gives 12 while retaining all preceding words.

Lean: `PrimeSecondCommitControls.full_state_presentation`.

Lean: `PrimeSecondCommitControls.stale_commitment_and_scalar_excluded`.

Lean: `PrimeSecondCommitControls.prior_proof_saved_and_context_retained`.

Lean: `PrimeSecondCommitControls.zero_challenge_boundary`.

This component stops before the other three p1 commitment coordinates, programming and proof return. Subsequent subsections compose these operations.

1.128 Historical component: Completing the second proof's commitment computation

Retain o, c_1, w_1 from Section 1.127. Write β_1 for the second coordinate of w_1 's ciphertext and let $d = (c - e) \bmod q$ be the canonical scalar representative. The remaining simulator coordinates are

$$\begin{aligned} B_1 &= pk^{z_0} \beta_1^{q-e} \bmod p, \\ \gamma_1 &= \beta_1 g^{q-1} \bmod p, \\ C_1 &= g^{z_1} \alpha_1^{q-d} \bmod p, \\ D_1 &= pk^{z_1} \gamma_1^{q-d} \bmod p. \end{aligned}$$

These are the original group expressions $z_0 pk - e\beta_1$, $\beta_1 - g$, $z_1 g - d\alpha_1$ and $z_1 pk - d(\beta_1 - g)$ in the additive source notation. Subtraction in $\mathbb{Z}_q$ is canonicalized before exponentiation; the controller computes $(c+q-e) \bmod q$ on canonical natural representatives, rather than the truncated difference $\max(c-e, 0)$. The zero values $e = 0$ or $d = 0$ use exponent q , justified by the typed subgroup law.

Lean: `PrimeSecondCommitBSource.answer_source`.

Lean: `PrimeSecondOneFirstMachine.answer_source`.

Lean: `PrimeSecondOneSecondMachine.answer_source`.

Each stage runs existing arithmetic code under a fixed frame. Actual source values supply its public bases, scalar prefixes, bounds and blank workspace. The adjusted-base routine shares the still-resident $q-1$ digits; the original raw word is retained as data and is not reparsed during these arithmetic stages. The successful returns feed the next entries with unchanged resident words and blank new outputs.

Lean: `PrimeSecondDifferenceMachine.charged_source`.

Lean: `PrimeSecondAdjustedMachine.charged_source`.

Lean: `PrimeSecondCommitTail.b_return`.

Lean: `PrimeSecondCommitTail.difference_return`.

Lean: `PrimeSecondCommitTail.adjusted_return`.

Lean: `PrimeSecondCommitTail.first_return`.

Let $R_4(o, c_1)$ preserve all 65 words of the previous first-coordinate result $R_A(o, c_1)$ and append B_1 , the scalar prefix of d , γ_1 , C_1 and D_1 on ports 65 through 69. The four original simulator coordinates therefore occupy ports 60, 65, 68 and 69. All group outputs use canonical public-coordinate digits. The result has successful halted control state.

Lean: `PrimeSecondCommitTail.source_commitments`.

Lean: `PrimeSecondCommitTail.source_preparation`.

Lean: `PrimeSecondCommitTail.source_retained`.

Theorem 1.155 (All p1 commitments from the original input). *Let p, q be prime, g, pk typed group inputs, v a Boolean vote, s, L arbitrary finite initial programmed and live states, and $\sigma \in \mathbb{N}$. Let $I = \text{input}(g, pk, \sigma, v, \text{saved}(s, L))$. For every typed o, c_1 , the resident continuation executes from $R_A(o, c_1).stk$ to $R_4(o, c_1)$ without oracle queries, with charge at most $96K + S + 32G$. Here K is `PrimeSimCommitMachine.clock` at (p, q) , S is `ScalarDifferenceMachine.cost` at q , and G is `PrimeAdjustedBetaMachine.clock` at (p, q) . The combined raw caller M_4 satisfies*

$$\text{fst} < \$ > \text{run}(M_4, \text{clock}(I, \sigma, p, q), \text{start}(I)) = \text{liftCoins} \left(\begin{array}{l} o \leftarrow F_\sigma(A_q); \\ c_1 \leftarrow F_\sigma(D_q); \\ \text{return } R_4(o, c_1). \end{array} \right).$$

Every supported result halts with charge at most $B_4 = B_A + 96K + S + 32G$. For every bounded-adapter limit, physical execution has derived startup at most $4|I| + 8$ and remaining bound $\text{unitCost}(|I| + B_4) B_4 \text{globalFactor}(M_4)$, with the same bounded oracle adapter on both sides.

Lean: [PrimeSecondCommitTail.charged_source](#).

Lean: [PrimeSecondCommitTail.execution_source](#).

Lean: [PrimeSecondAllCommitCaller.linked_run](#).

Lean: [PrimeSecondAllCommitCaller.run_support](#).

Lean: [PrimeSecondAllCommitCaller.execution_source](#).

Lean: [PrimeSecondAllCommitCaller.physical_run](#).

This source-tree equality supplies the original sampling order; arbitrary endpoint parameters alone do not establish a fresh-draw distribution. All configurations here are internal execution observations. The public election observation and fair-bit modulo interpretation are unchanged.

The component instruction gates and independent literal endpoint controls are separate from the general execution theorem. Zero and underflow differences are included. For $p = 23, q = 11, g = 2, pk = 4, r_2 = 1$ and $(c, e, z_0, z_1) = (0, 1, 10, 0)$, the complete endpoint has

$$(A_1, B_1, d, \gamma_1, C_1, D_1) = (6, 13, 10, 2, 2, 2).$$

The controls retain the earlier proof, both nonces and saved state across all four selected occupied/bad combinations. Component gates test actual operand, output, cleanup and retained-state mutations against full states and original bounds; the composed links have their own checked instruction/return identities. No additional whole-row native campaign is claimed for this composition.

Lean: [PrimeSecondOneSecondControls.execution_full_state](#).

Lean: [PrimeSecondOneSecondControls.underflow_not_truncated](#).

Lean: [PrimeSecondOneSecondControls.adjusted_gamma_not_beta](#).

Lean: [PrimeSecondOneSecondControls.actual_saved_corruption](#).

The next sections execute the original p1 key serialization, current-state programming update and returned-proof encoding, then the overall proof using the same resident executor. Later election execution is outside this component's guarantee. All four commitment coordinates do not by themselves constitute a returned proof or computational ballot-secrecy theorem.

1.129 Historical component: The second proof's original hash key

Let w_1, c_1, R_4 be as in Section 1.128. The next source operation programs the key

$$k_1 = (w_1, \text{ballotSimCommit}(w_1, c, (e, z_0, z_1))).$$

Its serialized form is the original flat record of eight canonical group coordinates, in the order

$$(g, pk, \alpha_1, \beta_1, A_1, B_1, C_1, D_1).$$

Each coordinate is first encoded as a canonical natural prefix, then framed by its encoded length; the whole record begins with its field count eight. This is the cache-key codec from Section 1.122, not the nested statement codec used by the history update.

Lean: [PrimeSecondKeyMachine.key](#).

Lean: [PrimeSecondKeyMachine.source_encoding](#).

The existing numeric key serializer executes unchanged under a fixed frame. Its operands now come from ports 16, 15, 50, 51, 60, 65, 68 and 69. The source theorem derives their canonical digits,

bounds below p and empty work storage. Its output adds the original key encoding on port 70 and retains every old word, including the first proof, nonce pair and current saved state.

Lean: [PrimeSecondKeyMachine.source_values](#).

Lean: [PrimeSecondKeyMachine.source_width](#).

Lean: [PrimeSecondKeyMachine.source_work](#).

Lean: [PrimeSecondKeyMachine.source_retained](#).

Lean: [PrimeSecondKeyMachine.source_key](#).

Theorem 1.156 (Original raw execution through the second key). *Let p, q be prime, g, pk typed group inputs, v a Boolean vote, s, L arbitrary finite initial programmed and live states, and $\sigma \in \mathbb{N}$. Let $I = \text{input}(g, pk, \sigma, v, \text{saved}(s, L))$. For every typed o, c_1 , key serialization executes from $R_4(o, c_1)$ to $R_K(o, c_1)$, preserving all 70 preceding words and writing $\text{ballotKeyBitCodec}(k_1)$ on port 70, without oracle queries. Its charge is at most $32T_K(p)$, where*

$$T_K(p) = 8 \text{NatFieldWriterMachine.clock}(p - 1) + 1.$$

The original raw caller has the same source-tree equality as Theorem 14.155, with R_K in place of R_4 and its own actual code and clock. Every supported result halts with charge at most $B_K = B_4 + 32T_K(p)$. For every bounded-adaptor limit, physical execution from the original raw input and blank work has startup at most $4|I| + 8$ and remaining bound

$$\text{unitCost}(|I| + B_K) B_K \text{globalFactor}(M_K),$$

using the same bounded oracle adapter on both sides.

Lean: [PrimeSecondKeyMachine.charged_source](#).

Lean: [PrimeSecondKeyCaller.linked_run](#).

Lean: [PrimeSecondKeyCaller.run_support](#).

Lean: [PrimeSecondKeyCaller.execution_source](#).

Lean: [PrimeSecondKeyCaller.physical_run](#).

For the independently computed $p = 23, q = 11$ fixture of the preceding section, the eight fields are $(2, 4, 2, 4, 6, 13, 2, 2)$. The actual instruction gate compares the complete resulting storage and query behavior against an independent record encoder. Its mutations test stale operands, wrong order/arity, omitted or truncated fields, altered saved data, wrong output and guard bypass. This finite validation is separate from the general source and execution theorem.

Lean: [PrimeSecondKeyControls.execution_full_state](#).

Lean: [PrimeSecondKeyControls.stale_operands_excluded](#).

Lean: [PrimeSecondKeyControls.flat_order_and_count](#).

Lean: [PrimeSecondKeyControls.actual_saved_corruption](#).

The following programming call must use the state left by the first call. Resident ports 44–47 already hold that cache, separate live cache, sticky flag and ordered history. With $N = |I|$, the cache and history counts are at most $N + 1$. Their encoded payload bounds follow from the existing codec-size laws; the first call’s payload bound by N must not be reused for the enlarged state. Original raw port 14 still describes the initial state. These source facts prepare the next call. The resident executor below supplies the repeated state update and proof return; later election execution is outside this component’s guarantee.

Lean: [PrimeSecondProgramSource.source_fields](#).

Lean: [PrimeSecondProgramSource.source_counts](#).

Lean: [PrimeSecondProgramSource.source_lengths](#).

Lean: [PrimeSecondProgramSource.source_context](#).

1.130 Historical component: One complete programmed proof request

This component executes the second and overall historical requests with one proved procedure, its reentry routine and a composition of the two distinct source requests. The remaining election continuation is outside this component's guarantee.

Let p, q be prime, $g, pk \in \text{PrimeGroup}(p, q)$, v a Boolean, $r \in \mathbb{Z}_q$, and s, L finite programmed and live states. No nonzero premise is imposed on r . Define the statement and sampled transcript by

$$S = (g, pk, \text{encryptWith}(g, pk, r, \text{voteScalar}(v))), \\ \tau_c = (\text{ballotSimCommit}(S, c, e, z_0, z_1), c, (e, z_0, z_1)).$$

Here (c, e, z_0, z_1) is the original full-field transcript draw. The notation compresses the nested tuples in `PrimeProofRequest.transcript`. The initial configuration I is the canonical resident encoding of these inputs, with blank working/output slots and two arbitrary retained byte words. It is specified by `PrimeProofRequest.start`; raw-input preparation is not part of this entry contract.

Theorem 1.157 (Complete resident request execution). *For every such input and slack $\sigma \in \mathbb{N}$, the fixed finite code M satisfies*

$$\pi_{\text{cfg}}(\text{run}(M, T, I)) = \text{simulateQ}(\text{liftCoins}) (R < \$ > \text{runFairBitUniform}_\sigma(\text{drawTranscriptScalars}(q))).$$

Every supported run result halts and has charge at most B . The configuration $R(c, e, z_0, z_1)$ contains the original encoded proof returned by $s.\text{program}(S, \tau_c)$ and the encoding of its successor state paired with the unchanged live cache L .

Lean: `PrimeProofRequest.execution_source`.

Lean: `PrimeProofRequest.run_support`.

Lean: `PrimeProofRequest.fullResult_return`.

The clock T and charge bound B are sums of the existing draw/encryption, commitment/key, programming and output-encoding clocks and charges. The latter two use the current state's derived bounds

$$N_i = \max\{|\text{enc}(s.\text{cache})|, |\text{enc}(L)|, |\text{enc}(s.\text{programmed})|\}, \\ N_o = \max\{|s.\text{cache}|, |s.\text{programmed}|, |\text{enc}(L)|\}.$$

No execution or cost certificate is supplied by the caller. Literal state identities connect the stages; semantic realization equality is not used to infer a structural presentation. Successor-size bounds follow from actual cache insertion and history growth, including occupied-key and sticky-bad behavior.

Lean: `PrimeProofRequest.charged`.

Lean: `PrimeProofRequest.draw_commit_boundary`.

Lean: `PrimeProofRequest.commit_program_boundary`.

Lean: `PrimeProofRequest.next_inputSize`.

A bounded actual-code integration test uses $p = 3, q = 2, r = 0$, true vote and empty caches/history. It checks the complete state, fresh coin consumption and independently encoded proof/state, and detects a broken return to the output writer. This test is distinct from the general theorem. The checked reentry and source presentations below supply reset, nonce-sum preparation and preservation of earlier proofs and ciphertexts.

The equality concerns the fair-bit programmed simulator; comparison with ideal uniform sampling uses the separate sampling-loss transport. The bounds here are source-machine clocks and charges. A physical execution bound must also account for the sizes of retained words.

1.131 Historical component: Historical reentry and the architecture assessment

The bounded assessment asks whether one complete request implementation removes repeated proof work. Its acceptance test is the actual second component request at (false, r_2) followed by the overall request at $(v, r_1 + r_2)$, with current state and prior proofs retained. The complete combined run/source/cost theorem now checks, alongside the component theorems and independent bounded test below. The assessment ends before further protocol construction.

Let $U(n)$ denote the existing scalar-prefix encoding. Reentry takes a mode t , natural $r_1, r_2 < q$, a vote v and 58 resident words W . Its entry premises are

$$W_2 = \text{bits}(q), \quad W_{20} = U(r_1), \quad W_{21} = U(r_2), \quad W_{18} = [v], \quad W_{23}, \dots, W_{29}, W_{37} = \epsilon.$$

Set $N = \max_i |W_i|$. Define W' by clearing fields

$$\{1, 2, 3, 7, 10\text{--}13, 23\text{--}29, 37\text{--}40, 42, 43, 49, 50, 51\},$$

preserving all other fields, and then writing

$$(W'_{50}, W'_{51}) = \begin{cases} (U((r_1 + r_2) \bmod q), [v]) & t = \text{true}, \\ (U(r_2), [\text{false}]) & t = \text{false}. \end{cases}$$

Theorem 1.158 (Executed historical reentry). *Under these entry conditions the fixed reentry code starts at `PrimeProofRequestReentry.start`(t, W), with memory 2 and entry label 1 for total or 0 for p1. It runs for its defined clock $T_R(q, N)$ to the halted configuration with memory 2 and words W' . Its actual charge is at most $32T_R(q, N)$; it makes no oracle requests.*

Lean: `PrimeProofRequestReentry.charged`.

The statement compresses the literal endpoint and clock definitions. Both historical endpoints derive the displayed entry premises; full configuration identities present the next request's canonical start. The nonce sum may be zero. Earlier component ciphertexts/proofs and original nonces/vote survive the total request's defined endpoint.

Lean: `PrimeRemainingProofRequests.initial_reentry_inputs`.

Lean: `PrimeRemainingProofRequests.p1_reentry_inputs`.

Lean: `PrimeRemainingProofRequests.p1_start`.

Lean: `PrimeRemainingProofRequests.total_start`.

Lean: `PrimeRemainingProofRequests.original_retained`.

Lean: `PrimeRemainingProofRequests.p1_retained`.

Lean: `PrimeRemainingProofRequests.total_statement`.

Example 1.159 (A rejected operand map). *An earlier layout supplied the intended one-bit nonce sum to the prefix writer's input instead of its counter. Three actual writer steps then emitted $U(0)$ and left the intended value 1 untouched. The corrected counter layout emits $U(1)$. A zero-sum case alone cannot distinguish these layouts. The checked negative and positive controls retain this representation defect.*

Lean: `PrimeProofRequestReentry.Controls.wrong_input_one`.

Lean: `PrimeProofRequestReentry.Controls.wrong_input_not_one`.

Lean: `PrimeProofRequestReentry.Controls.correct_counter_one`.

The combined controller consists of the original raw first-proof prefix, reentry, a framed copy of the complete executor, reentry, and a second framed copy of the same executor definition. This

shares code and proof definitions; it is not a general call stack or one shared physical instruction block. Its independent native fixture uses $p = 3, q = 2, g = 2, pk = 1$, true vote, $r_1 = r_2 = 1$ and total nonce zero. All resident words, state/history, 26 consumed coins and residual tape are checked. Mutations break the total return or omit the second component's state update. The identity public key is a valid typed constructor input, not an honest nonzero-secret key-generation trace. This bounded validation is independent of the general run theorem below.

Theorem 1.160 (Two distinct requests from original raw input). *Let p, q be prime, g, pk typed group inputs, v Boolean, s, L arbitrary finite initial programmed/live states and $\sigma \in \mathbb{N}$. Let $I = \text{input}(g, pk, \sigma, v, \text{saved}(s, L))$. Write T_0, B_0 for the existing first-proof raw caller's clock and charge bound on I , and $T_Q(N), B_Q(N)$ for the complete resident request bounds with both state-size parameters set to N . Define*

$$\begin{aligned} H_0 &= |I| + B_0, \\ H_1 &= H_0 + B_R(H_0) + B_Q(H_0), \\ T &= T_0 + T_R(H_0) + T_Q(H_0) + T_R(H_1) + T_Q(H_1), \\ B &= B_0 + B_R(H_0) + B_Q(H_0) + B_R(H_1) + B_Q(H_1), \end{aligned}$$

where $T_R(N) = T_R(q, N)$ and $B_R(N) = 32T_R(q, N)$. The fixed combined controller M satisfies

$$\pi_{\text{cfg}}(\text{run}(M, T, \text{start}(I))) = \text{simulateQ}(\text{liftCoins})(R < \$ > \text{runFairBitUniform}_\sigma(D)).$$

Here D draws the original nonce-pair/first-proof tuple, then fresh (c, e, z_0, z_1) for $p1$, then a fresh such tuple for the total proof, in that order. R is the full configuration defined by *PrimeRemainingProofCaller.sourceResult*. Every supported result halts and has actual charge at most B .

Lean: [PrimeRemainingProofCaller.execution_source](#).

Lean: [PrimeRemainingProofCaller.run_support](#).

Lean: [PrimeRemainingProofCaller.linked_run](#).

The display compresses the clock/cost definitions and the nested tuple in `drawSource`. Both resident bounds follow from actual supported executions; no frame, source-correspondence or cost certificate is supplied as a premise. The original raw-input start has blank work and memory 0. Reentry executes both nonce presentations, including a zero sum, and retains the current cache, sticky flag, duplicate-preserving history and separate live state.

Specializing the original two `ballotProofQuery` calls to return their proofs gives exactly the same two-transcript source body and successor state. Final ports 48, 52 and 53 encode the original `p0`, `p1` and total proofs; port 49 encodes that successor state paired with unchanged live cache. Earlier ciphertexts and nonces are retained. These claims use the original proof and state codecs.

Lean: [PrimeRemainingProofSource.source_run](#).

Lean: [PrimeRemainingProofSource.endpoint_outputs](#).

For every bounded-adapter limit, the same controller has physical startup at most $4|I| + 8$ and remaining clock $\text{unitCost}(|I| + B) B \text{globalFactor}(M)$. The physical observation matches the same full source under the same bounded adapter on both sides. This is the existing physical correspondence interface, not standard-PPT coverage of all election attackers.

Lean: [PrimeRemainingProofCaller.physical_run](#).

This component leaves the original `afterAliceProofs` continuation explicit and unexecuted: it does not establish runtime bounds for Alice verification/submission, Bob construction, attacker execution or final submission. Its fair-bit sampling loss is unchanged. The formal reference states the complete reduction theorem under external efficiency assumptions.

Limits of the historical implementation strategy. The historical implementation used the finite-machine representation and resident executor. The request's arithmetic, commitment, programming and serialization proofs are reused. These components do not establish a complete compiler or a polynomial-time implementation of the whole reduction.

For two fixed calls, bounding successor tape height by current height plus paid execution cost is sufficient. Iterating a polynomial-in-height bound polynomially many times need not remain polynomial. Full reduction efficiency requires structural cache/history/replay-state bounds. Query counts alone cannot bound arbitrary host computation. The next section records the historical 90-minute feasibility experiment on the native compilation boundary.

1.132 Historical component: A bounded native adaptive compilation test

The historical 90-minute experiment tested whether an actual finite native oracle program can be translated mechanically while preserving adaptive queries, independent private work and execution cost. A falsifier is a changed second query, a lost private cell, or a charge unsupported by the executed target.

The native program reads only finite labels and current tape cells. It hashes an input word x , branches on the leading bit of the reply a (empty means false), reads a coin c , and hashes

$$d(x, a, c) = [\text{headD}(a, \text{false}), c] ++ \text{drop}_2(x).$$

The branch bit is written to the query tape before the coin replaces the answer tape. The private work tape is untouched. The source halts after eight steps; the nine-step validation clock adds one absorbing halted step.

Lean: `NativeAdaptiveProbe.exact_run`.

Lean: `NativeAdaptiveProbe.padded_run`.

Write $Z(b, w)$ for a tape whose finite left and inclusive right halves are b, w , nearest cells first, with blanks beyond them. Write $T(x) = Z([], x)$. The mechanical translation $P = \text{code}(\text{NativeAdaptiveProbe.code})$ uses six stacks for the three pairs of halves. It reads all three heads before selecting the native command, then retains that finite selection while executing its actions. There is no separately supplied target implementation. Its loaded start $I(b, w, x, o)$ has memory 0 and stacks $[b, w, [], x, [], o]$. Let $R(b, w, x, a, c, z)$ be its successful halted state, memory 0, with stacks $[b, w, [], d(x, a, c), [], z]$.

Theorem 1.161 (Fixed-program native correspondence and exact charge). *For arbitrary Boolean words b, w, x, o , the generated execution is exactly*

$$\begin{aligned} \text{run}(P, 19, I(b, w, x, o)) &= \text{hash}(x) \gg= (a \mapsto \text{coin} \gg= (c \mapsto \\ &\quad \text{hash}(d(x, a, c)) \gg= (z \mapsto \text{pure}(R(b, w, x, a, c, z), \kappa))))), \\ \kappa &= 66 + |x| + |o| + 2|a| + |d(x, a, c)| + |z|. \end{aligned}$$

Observing only successful target halts gives exactly the native eight-step query tree and its complete final tapes $(Z(b, w), T(d(x, a, c)), T(z))$. Both hash replies are arbitrary words in this equality, including empty replies and their entire tails.

Lean: `NativeAdaptiveRun.exact_run`.

Lean: `NativeAdaptiveRun.native_correspondence`.

The display abbreviates the constructors of the exact oracle query tree. Observation returns none for a running state or rejection memory, so an unsupported or partial run cannot satisfy the displayed successful endpoint. The charge is the sum of actual generated steps, including replacement of the old answer. It is not a supplied certificate.

Theorem 1.162 (Bounded replies and physical execution). *For every $L \in \mathbb{N}$, all branches with $|a|, |z| \leq L$ halt and satisfy*

$$\kappa \leq B := 68 + 2|x| + |o| + 3L.$$

For any previous physical query word v and any previous answer tape, set

$$H = \text{height}(I(b, w, x, o)) + B + |v|, \quad F = \text{unitCost}(H) B \text{globalFactor}(P).$$

Starting from the existing primitive lowering of the resident ready state, physical execution at clock F has exactly the same query tree and observed target configurations under the same bounded-reply adapter on both sides.

Lean: `NativeAdaptiveCost.within.`

Lean: `NativeAdaptiveCost.physical_run.`

The adapter permits both coin values and every hash reply of length at most L ; it does not alter requests or accepted reply data. The physical start already contains the six resident words. Loading arbitrary external tape representations into that state is outside this theorem.

Validation and explicit boundary. Independent full-state/event/charge checks cover all input words of length at most 3, both reply words of length at most 1, both coins and three private frames: 810 cases, plus three directed cases and four compiled source-instruction mutations. The source also has three full-tape positives and four mutation controls. They distinguish wrong answer/work-head selection, omitted branching, lost coin dependence and work corruption. These bounded controls validate the stated fixtures; the general equality above is a separate proof.

The dense representation deliberately rejects blank writes and moves that would store a blank cell. Checked counterexamples execute native blank erasure or a move from a blank head successfully, while the generated target halts with rejection memory 26. Thus the translator is executable on arbitrary finite native code, but its correctness theorem here covers only the fixed adaptive program and derived reachable states.

Lean: `NativeWordCompilerBoundary.blank_write_native.`

Lean: `NativeWordCompilerBoundary.blank_write_target.`

Lean: `NativeWordCompilerBoundary.right_blank_native.`

Lean: `NativeWordCompilerBoundary.right_blank_target.`

Lean: `NativeWordCompilerBoundary.rejection_not_success.`

1.133 Historical component: General native query export

This historical experiment isolates the query-export operation needed beyond the dense fixed-program test. It uses the existing three-symbol tape representation: a cell is `none`, `some false` or `some true`, and its stored pair is `[isSome, getD false]`. Thus blank cells have an explicit payload bit and remain distinguishable from false cells. The earlier compiler's blank-operation counterexamples remain valid.

Let E encode a finite cell list by concatenating these pairs. For any head cell h , before/after lists b, a , and n arbitrary private Boolean words P_i , the resident exporter starts with register `cellCode(h)` and words

$$W_0 = E(a), \quad W_1 = W_2 = [], \quad W_3 = E(b), \quad W_{4+i} = P_i.$$

This is the existing head-register/cell-side representation with blank output and scratch, framed with independent private storage. Define $Q = \text{readWord}(\text{tape}(h, b, a))$, the original native query decoder.

Theorem 1.163 (General preserving native query export). *For every such h, b, a, P , the fixed exporter runs from this resident start at clock $2|a| + 6$ to a successful halt, with an actual charge $\chi \leq 12|a| + 36$. Its complete result has the original head register and all the original words, except $W_1 = Q$; scratch W_2 is empty. No restriction excludes interior or trailing blanks, and every private word is unchanged.*

Lean: `NativeQueryExportFrame.charged.`

Lean: `NativeQueryExportFrame.native_result.`

Lean: `NativeQueryExportFrame.native_storage.`

Lean: `NativeQueryExportFrame.private_retained.`

The head's two encoded bits are prepended by executed instructions, then removed to restore the register. The preserving core stops before the first blank, leaves that cell and the entire later suffix untouched, and restores the scanned prefix while producing the raw output. The native-decoder equality is proved for arbitrary finite cell representatives. The clock and charge follow from these actual instructions; neither correspondence nor cost is a premise.

Lean: `NativeQueryExportAdapter.run.`

Lean: `NativeQueryReadWord.readWord.`

Independent core and actual-head campaigns each cover 1,521 exhaustive cases and one longer case, with five and seven instruction mutations respectively. They check full storage, output, scratch, head restoration and charge. Checked literal controls distinguish false from blank and retain the postblank suffix. This general operation does not yet compile arbitrary adaptive native programs. The following subsection supplies reply import and the complete one-oracle-command composition. The theorem starts with resident encoded cells; it does not execute arbitrary external input packing.

Lean: `NativeQueryExportControls.ActualHead.false_head_is_bit.`

Lean: `NativeQueryExportControls.ActualHead.interior_blank_retains_suffix.`

1.134 Historical component: One general native oracle round trip

The next interface executes one oracle command of an arbitrary finite native program p . Let q be its current label and h its three head cells, with $p(q, h) = \text{oracle}(k, q')$, where k selects hash or coin. Each work, query and answer tape has arbitrary finite before/after cell lists b_i, a_i , including internal blanks. The resident start stores their lossless encodings in ports $2i, 2i + 1$, all three heads in finite memory, empty scratch ports 6, 7, and n arbitrary private words. Command selection uses the original heads before the exporter or importer changes local memory.

Write $Q = \text{readWord}(h_1, b_1, a_1)$ and let L bound hash-reply length; coins still admit both Boolean replies, including when $L = 0$. Define

$$E = 2|a_1| + 6, \quad I = 2|b_2| + |Q| + 2(L + 1) + 5,$$

$$F = E + I + 4, \quad C = 8 + 6E + (2 + |Q| + |\text{cells}(a_2)| + L + 1) + 5I.$$

The executor performs one actual entry step, then at most F further steps, stopping on the next native entry. This is a live code label, not a halt or a host callback; stopping there prevents padding from executing the continuation.

Theorem 1.164 (Full native oracle return, compressed interface). *For every such p, q, h, b, a, L and private family P , the projected full-state query tree of the framed executor equals*

$$(r \mapsto R(q', h, b, a, r, P)) < \$ > \text{boundedCall}_L(k, Q).$$

Every supported execution has accumulated instruction charge at most C . Here R is live at q' , preserves the complete work/query tapes and every private word, replaces the entire old answer by the native word tape of r , and clears both scratch ports and the local register. Its saved answer head is $r.\text{head?}$; the work/query heads are unchanged.

Lean: [NativeRoundTrip.framed_correspondence](#).

Lean: [NativeRoundTrip.framed_cost](#).

Lean: [NativeRoundTrip.native_correspondence](#).

The proof equates the exact oracle tree and establishes the native successor representation at every leaf. A support-only actual-prefix lemma would not establish this event correspondence. Independent campaigns check 6,481 whole calls with six mutations, and 2,206 importer cases with three mutations; 13 kernel controls and observer counterexamples supplement them. These bounded checks do not replace the quantified theorem. The interface can serve later native oracle calls with independent private storage.

References

- [1] V. Cortier and B. Smyth. *Attacking and fixing Helios: An analysis of ballot secrecy*. [Author preprint](#). Protocol and symbolic model: §5; replay and permutation attacks: §3.
- [2] B. Smyth. *Replay attacks that violate ballot secrecy in Helios*. [May 2012 manuscript](#).
- [3] V. Cheval, V. Cortier, A. Debant and F. Moser. *Simultaneously Proving Privacy and Verifiability: A ProVerif Framework for Internet Voting*. CSF 2026. [Publication page](#).
- [4] D. Bernhard, O. Pereira and B. Warinschi. *How not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios*. ASIACRYPT 2012; [ePrint version](#).